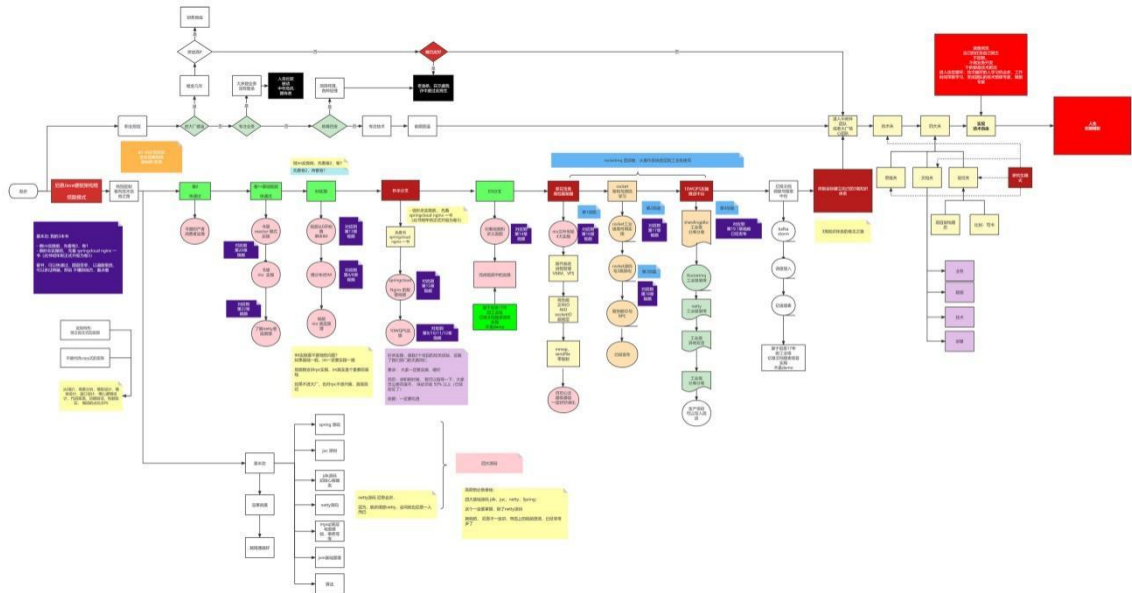


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

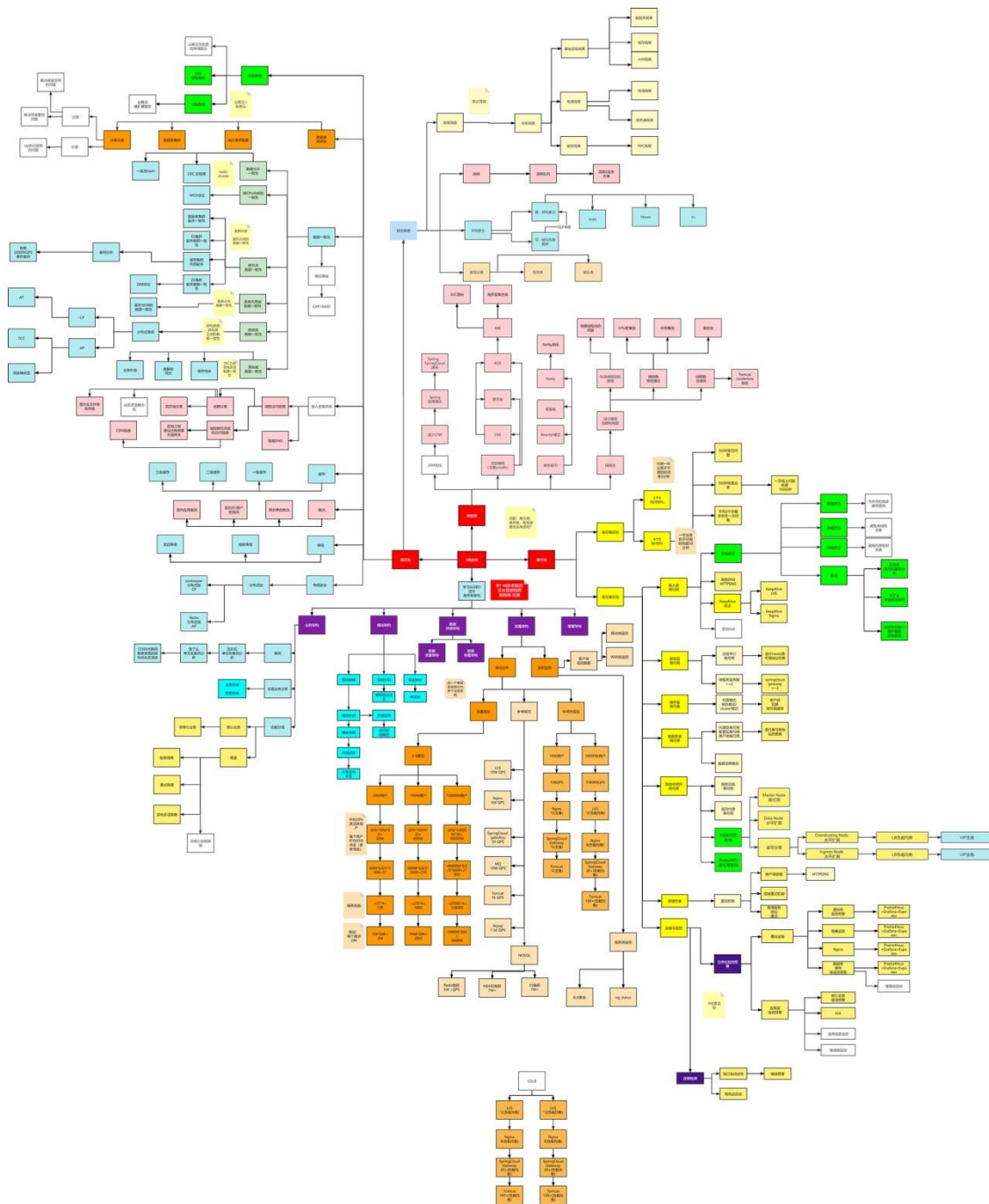
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

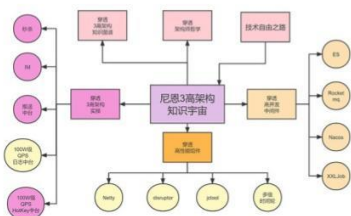
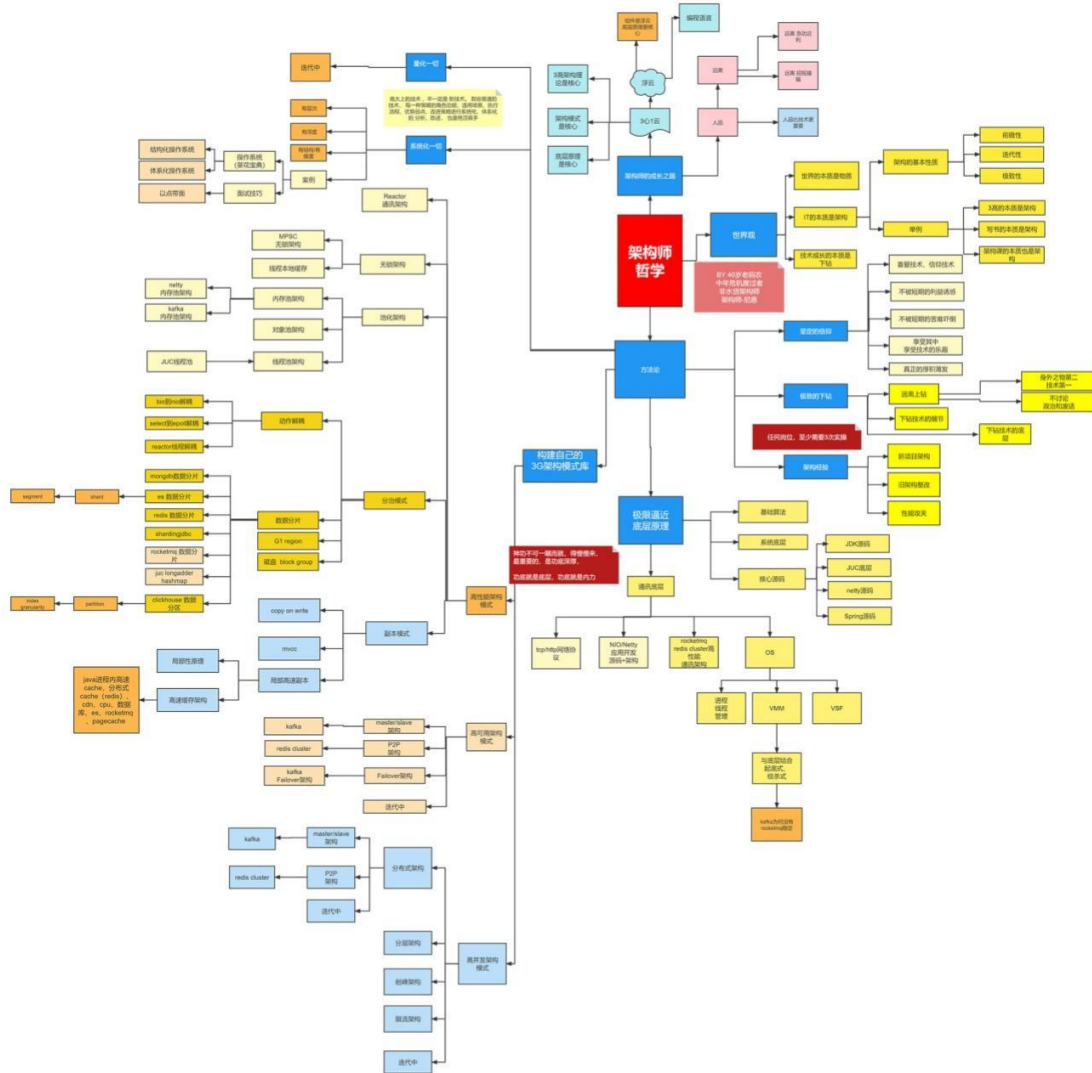
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



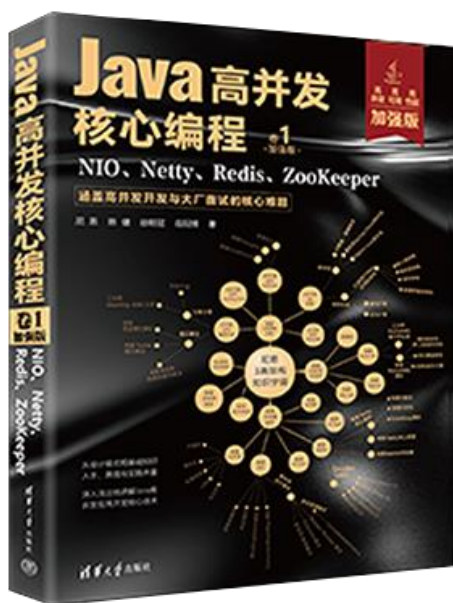
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



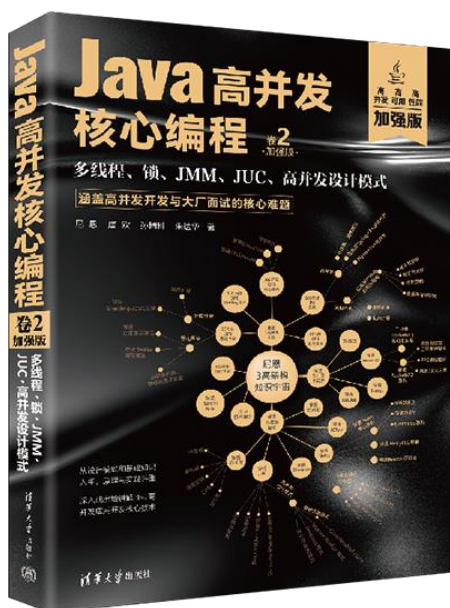
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



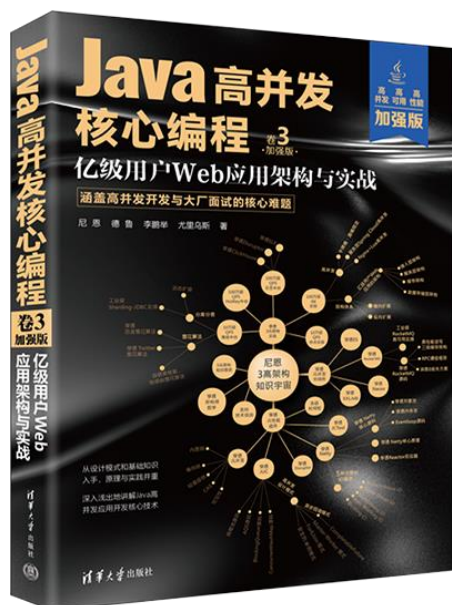
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

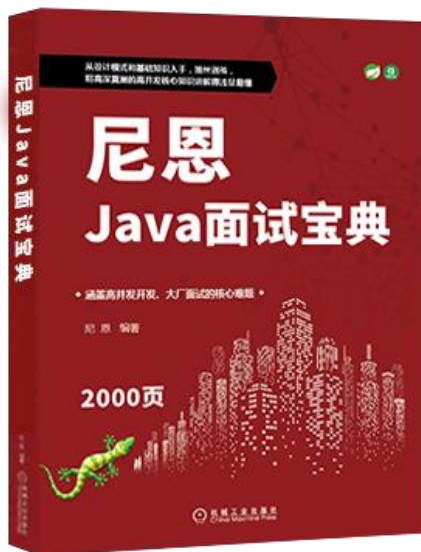
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

35 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称 专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V10-from-尼恩Java面试宝典-release.pdf 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf 专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书 -release.pdf 专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V5-from-Java面试红宝书-release.pdf 专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V11-from-Java面试红宝书-release.pdf 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf 专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf 专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf

专题40：操作面试题（史上最全、定期更新）

本文版本说明：V28

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《Java面试红宝书》，后面会不断升级，迭代。

本专题，作为 《Java面试红宝书》专题之一，《Java面试红宝书》一共**30个面试专题**，后续还会增加

《尼恩面试宝典》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，

入交流群，加尼恩微信即可，发送“**入群**”，

加尼恩的方式，请参考 本文末尾的 链接地址



本文目录

专题40：操作面试题（史上最全、定期更新）

本文版本说明：V28

《尼恩面试宝典》升级的规划为：

面试问题交流说明：

本文目录

操作系统基础篇

操作系统的特征

并发

共享

互斥共享方式

同时访问方式

虚拟

异步

[操作系统五大功能](#)

[操作系统分类](#)

[聊聊：什么是操作系统](#)

[聊聊：操作系统的主要功能](#)

[聊聊：软件访问硬件的几种方式](#)

[聊聊：操作系统的主要目的是什么](#)

[聊聊：操作系统的种类有哪些](#)

[聊聊：为什么 Linux 系统下的应用程序不能直接在 Windows 下运行](#)

[聊聊：操作系统结构](#)

[单体系统](#)

[分层系统](#)

[微内核](#)

[客户-服务器模式](#)

[聊聊：为什么称为陷入内核](#)

[聊聊：什么是用户态和内核态](#)

[聊聊：用户态和内核态是如何切换的？](#)

[聊聊：什么是内核](#)

[聊聊：什么是实时系统](#)

[聊聊：Linux 操作系统的启动过程](#)

进程线程协程篇

[系统调度](#)

[进程详解](#)

[线程详解](#)

[线程模型](#)

[线程实现在用户空间下](#)

[线程实现在操作系统内核中](#)

[使用用户线程加轻量级进程混合实现](#)

[Linux中的线程](#)

[内核线程](#)

[协程](#)

[Go协程模型](#)

[进程线程协程详解总结](#)

进程线程协程区别

[进程协程进程对比](#)

[进程概念](#)

[线程概念](#)

[协程概念](#)

[进程线程协程详解](#)

[进程](#)

[线程](#)

[协程](#)

[图解](#)

[进程与线程比较](#)

[协程与线程进行比较](#)

[进程线程协程区别总结](#)

[孤儿进程](#)

[孤儿进程教程](#)

[案例](#)

[创建孤儿进程](#)

[僵尸进程](#)

[僵尸进程教程](#)

[案例](#)

[创建僵尸进程](#)

[怎么避免僵尸进程](#)

[守护进程](#)

[守护进程教程](#)

[创建守护进程的步骤](#)

[案例](#)

- 创建守护进程
- 上下文切换
 - 进程切换
 - 线程切换
- 进程间通信方式
 - 概述
 - 进程通信的目的
 - Linux进程间通信（IPC）的发展
- Linux使用的进程间通信方式
 - 管道(pipe)
 - 信号量(semaphore)
 - 消息队列(message queue)
 - 信号 (singal)
 - 共享内存(shared memory)
 - 套接字(socket)
- 进程间通信各种方式效率比较
- 通信方式的比较和优缺点
- 进程间通信方式的选择
- 线程间通信方式
 - 消息队列
 - 全局变量
 - 使用事件
- 线程间同步方式
 - 线程间的同步方式有四种
 - 临界区
 - 互斥量
 - 信号量
 - 事件
- Linux进程状态
 - Linux进程状态教程
 - Linux 进程状态
 - Linux进程状态详解
 - R (TASK_RUNNING), 可执行状态
 - S (TASK_INTERRUPTIBLE) 可中断的睡眠状态
 - D (TASK_UNINTERRUPTIBLE) 不可中断的睡眠状态
 - T (TASK_STOPPED or TASK_TRACED), 暂停状态或跟踪状态
 - Z (TASK_DEAD - EXIT_ZOMBIE), 退出状态, 进程成为僵尸进程
 - X (TASK_DEAD - EXIT_DEAD), 退出状态, 进程即将被销毁
 - 进程的初始状态
 - 进程状态变迁
- 线程的几种状态
- 进程调度
- 进程调度的时机和方式
 - 时机
 - 方式
 - 补充
 - 调度队列
 - 调度程序
 - 上下文切换
- 处理机调度的三个层次
 - 定义
 - 三个层次
- 进程调度算法
 - 评价指标
 - 早期批处理系统的调度算法
 - 先来先服务调度算法（FCFS）
 - 短作业优先（SJF）调度算法
 - HRRN 算法

总结

交互式系统的调度算法

RR算法

优先级算法

多级反馈队列算法

总结

进程调度算法

先来先服务 (FCFS) 调度算法

短作业优先 (SJF) 调度算法

时间片轮转 (RR) 调度算法

高响应比优先 (Highest Response Ratio First, HRRF) 调度算法

多级反馈队列 (Multi-Level Feedback Queue) 调度算法

最高优先级优先调度算法

聊聊：什么是线程，线程和进程的区别

聊聊：有了进程为什么还要线程

聊聊：什么是进程和进程表

聊聊：并发和并行

聊聊：多处理系统的优势

聊聊：什么是上下文切换

聊聊：使用多线程的好处是什么

聊聊：进程终止的方式

进程的终止

正常退出

错误退出

严重错误

被其他进程杀死

聊聊：进程间的通信方式

聊聊：进程间状态模型

进程的三态模型

进程的五态模型

聊聊：什么是僵尸进程

聊聊：什么是守护、僵尸、孤儿进程

聊聊：Semaphore(信号量) Vs Mutex(互斥锁)

聊聊：进程调度策略有哪几种？

聊聊：进程有哪些状态？

死锁篇

聊聊：死锁是什么

聊聊：死锁产生条件

聊聊：解决死锁的基本方法

聊聊：死锁产生的原因

聊聊：死锁产生的必要条件

聊聊：死锁的恢复方式

通过抢占进行恢复

通过回滚进行恢复

杀死进程恢复

聊聊：如何破坏死锁

破坏互斥条件

破坏保持等待的条件

破坏不可抢占条件

破坏循环等待条件

聊聊：死锁类型

两阶段加锁

通信死锁

活锁

饥饿

聊聊：什么是临界区，如何解决冲突？

聊聊：什么是线程安全

聊聊：同步与异步

同步：

异步：

聊聊：同步与异步的优缺点：

基础知识：系统调用

系统调用概述

为什么需要系统调用

API/POSIX/C库的区别与联系

区别

联系

系统调用的实现原理

基本机制

响应函数sys_xxx

系统调用表与系统调用号==>数组与下标

进程的系统调用命令转换为INT 0x80中断的过程

聊聊：进程调度算法了解多少

聊聊：调度算法都有哪些

批处理中的调度

先来先服务

最短作业优先

最短剩余时间优先

交互式系统中的调度

轮询调度

优先级调度

最短进程优先

彩票调度

公平分享调度

聊聊：影响调度程序的指标是什么

聊聊：什么是 RR 调度算法

Copy-on-write (写时拷贝)

Linux中的fork()

fork()中的copy-on-write

Java中的CopyOnWrite容器

C++中的std::string

Redis中的COW

Copy On Write技术实现原理

Copy On Write技术好处

Copy On Write技术缺点

总结

动态库与静态库区别

动态库与静态库

动态链接

静态链接

静态库和动态库的区别

动态库

静态库

静态库与动态库优缺点

静态链接库的优点

动态链接库的优点

不足之处

内核态与用户态

概念

内核空间相关

用户空间相关

内核态和用户态的切换

系统调用

异常事件

外围设备的中断

用户态到内核态具体的切换步骤

虚拟内存篇

虚拟内存教程

第一层理解

第二层理解

虚拟内存总结

虚拟存储器

物理内存

物理内存的内核映射

物理内存管理机制

非连续内存区内存的分配

页面置换算法

最佳置换算法 (OPT, Optimal)

算法思想

举例说明

先进先出置换算法 (FIFO)

算法思想

举例说明

最近最久未使用置换算法 (LRU)

算法思想

实现方法

举例说明

时钟置换算法

算法思想

改进型的时钟置换算法

算法思想

算法规则

总结

软中断与硬中断

中断的基本概念

中断的分类与优先级

硬中断

软中断

硬中断与软中断之区别与联系

中断处理过程

中断与异常

中断

异常

中断面试题

聊聊：外中断和异常的区别

内存的管理策略

聊聊：中断的处理过程？

聊聊：中断和轮询有什么区别？

虚拟内存(Virtual Memory)

操作系统快表

什么是快表

快表 (TLB) 原理

TLB表项

全相连 - full associative

直接匹配

组相连 - set-associative

TLB表项更新

操作系统页表

什么是页表

页表作用

页表的实现

地址翻译过程如下

页表转化失败

TLB

页表格式

Present

Accessed

Dirty

Read/Write

User/Supervisor

Exec

页表项的创建和操作

多级页表

单级页表存在的问题

两级页表

虚拟存储技术

几个问题

使用二级页表的优势

总结

局部性原理

什么是局部性原理

示例

相关应用

CPU缓存

广义局部性

利弊结合

分段与分页机制

分段机制

什么是分段机制

什么是段

段的作用

段的存储地址

段选择符

逻辑地址到线性地址的变换过程

虚拟地址到物理地址的变换过程

分页机制

什么是分页机制

分页机制的存储

线性地址和物理地址之间的变换过程

聊聊：分段机制和分页机制的区别

聊聊：分段分页优缺点

优点

缺点

聊聊：什么是逻辑地址/线性地址/物理地址

逻辑地址 (Logical Address)

线性地址 (Linear Address)

物理地址 (Physical Address)

虚拟内存 (Virtual Memory)

聊聊：什么是虚拟内存？

聊聊：什么是分页？

聊聊：什么是分段？

聊聊：分页和分段有什么区别？

聊聊：为什么使用两级页表

聊聊：地址变换中，有快表和没快表的区别

聊聊：动态分区分配算法的了解

聊聊：几种典型的锁

聊聊：常见的几种磁盘调度算法

聊聊：什么叫抖动

聊聊：页面置换算法的了解

聊聊：页面替换算法有哪些？

聊聊：页面置换算法都有哪些

聊聊：什么是按需分页

聊聊：什么是虚拟内存
聊聊：虚拟内存的实现方式
聊聊：内存为什么要分段
聊聊：物理地址、逻辑地址、有效地址、线性地址、虚拟地址的区别
聊聊：空闲内存管理的方式
 使用位图进行管理
 使用空闲链表
聊聊：什么是交换空间？
聊聊：什么是缓冲区溢出？有什么危害？
聊聊：用户态（模式）与内核态（模式）
聊聊：用户态切换到内核态的3种方式

文件系统篇

聊聊：如何提高文件系统性能的方式
 高速缓存
 块提前读
 减少磁盘臂运动
 磁盘碎片整理
聊聊：磁盘调度算法
聊聊：RAID 的不同级别

IO 篇

聊聊：操作系统中的时钟是什么
 时钟硬件
聊聊：设备控制器的主要功能
聊聊：中断处理过程
聊聊：什么是设备驱动程序
聊聊：什么是 DMA
聊聊：直接内存访问的特点
聊聊：IO多路复用？
聊聊：硬链接和软链接有什么区别？
聊聊：大小端模式

操作系统基础篇

在信息化时代，软件被称为计算机系统的灵魂。而作为软件核心的操作系统，已经与现代计算机系统密不可分、融为一体。计算机系统自下而上可粗分为四个部分：硬件、操作系统、应用程序和用户。操作系统管理各种计算机硬件，为应用程序提供基础，并充当计算机硬件和用户的中介。

硬件，如中央处理器、内存、输入输出设备等，提供了基本的计算资源。应用程序，如字处理程序、电子制表软件、编译器、网络浏览器等，规定了按何种方式使用这些资源来解决用户的计算问题。操作系统控制和协调各用户的应用程序对硬件的使用。

综上所述，操作系统是指控制和管理整个计算机系统的硬件和软件资源，并合理的组织调度计算机的工作和资源的分配，以提供给用户和其他软件方便的接口和环境集合。计算机操作系统是随着计算机研究和应用的发展逐步形成并发展起来的，它是计算机系统中最基本的系统软件。

操作系统的特征

操作系统是一种系统软件，但与其他系统软件和应用软件有很大的不同，他有自己的特殊性即基本特征，操作系统的基本特征包括并发、共享、虚拟和异步。这些概念对理解和掌握操作系统的核心至关重要。

并发

并发是指两个或多个事件在同一时间间隔内发生，在多道程序环境下，一段时间内宏观上有多个程序在同时执行，而在同一时刻，单处理器环境下实际上只有一个程序在执行，故微观上这些程序还是在分时的交替进行。操作系统的并发是通过分时得以实现的。操作系统的并发性是指计算机系统中同时存在多个运行着的程序，因此它具有处理和调度多个程序同时执行的能力。在操作系统中，引入进程的目的在于使程序能并发执行。

共享

资源共享即共享，是指系统中的资源可供内存中多个并发执行的进程共同使用。共享可以分为以下两种资源共享方式。

互斥共享方式

系统中的某些资源，如打印机、磁带机，虽然他们可以提供给多个进程使用，但为使所打印的内容不致造成混淆，应规定在同一段时间内只允许一个进程访问该资源。

为此，当进程a访问某资源时，必须先提出请求，如果此时该资源空闲，系统便可将之分配给进程a使用，倘若若有其他进程也要访问该资源（只要a未用完）则必须等待。仅当进程a访问完并释放该资源后，才允许另一进程对该资源进行访问。计算机系统的大多数物理设备，以及某些软件中所用的栈、变量和表格，都属于临界资源，他们都要求被互斥的共享。

同时访问方式

系统中还有一种资源，允许在一段时间内由多个进程“同时”对它进行访问。这里所谓的“同时”往往是宏观上的，而在微观上，这些进程可能是交替的对该资源进行访问即“分时共享”。典型的可供多个进程同时访问的资源是磁盘设备，一些用重入码编写的文件也可以被“同时”共享，即若干个用户同时访问该文件。

并发和共享是操作系统两个最基本的特征，这两者之间又是互为存在条件的：1资源共享是以程序的并发为条件的，若系统不允许程序并发执行，则自然不存在资源共享的问题；2若系统不能对资源共享实施有效地管理，也必将影响到程序的并发执行，甚至根本无法并发执行。

虚拟

虚拟是指把一个物理上的实体变为若干个逻辑上的对应物。物理实体是实的，即实际存在的；而后者是虚的，是用户感觉上的事物。相应的，用于实现虚拟的技术，成为虚拟技术。在操作系统中利用了多种虚拟技术，分别用来实现虚拟处理器、虚拟内存和虚拟外部设备。

在虚拟处理器技术中，是通过多道程序设计技术，让多道程序并发执行的方法，来分时使用一台处理器的。此时，虽然只有一台处理器，但他能同时为多个用户服务，是每个终端用户都认为是有有一个中央处理器在为他服务。利用多道程序设计技术，把一台物理上的 CPU 虚拟为多台逻辑上的 CPU，称为虚拟处理器。

类似的，可以通过虚拟存储器技术，将一台机器的物理存储器变为虚拟存储器，一边从逻辑上来扩充存储器的容量。当然，这是用户所感觉到的内存容量是虚的，我们把用户所发哦绝倒的存储器程序虚拟存储器。

还可以通过虚拟设备技术，将一台物理IO设备虚拟为多台逻辑上的IO设备，并允许每个用户占用一台逻辑上的 IO 设备，这样便可使原来仅允许在一段时间内有一个用户访问的设备，变为在一段时间内允许多个用户同时访问的共享设备。

因此操作系统的虚拟技术可归纳为：时分复用技术和空分复用技术。

异步

在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底，而是走走停停，以不可预知的速度向前推进，这就是进程的异步性。

异步性使得操作系统运行在一种随机的环境下，可能导致进程产生于时间有关的错误。但是只要运行环境相同，操作系统必须保证多次运行进程，都获得相同的结果。

操作系统五大功能

一般来说，操作系统可以分为五大管理功能部分：

1. 设备管理：主要是负责内核与外围设备的数据交互，实质是对硬件设备的管理，包括对输入输出设备的分配，初始化，维护与回收等。例如管理音频输入输出。
2. 作业管理：这部分功能主要是负责人机交互，图形界面或者系统任务的管理。
3. 文件管理：这部分功能涉及文件的逻辑组织和物理组织，目录结构和管理等。从操作系统的角度来看，文件系统是系统对文件存储器的存储空间进行分配，维护和回收，同时负责文件的索引，共享和权限保护。而从用户的角度来说，文件系统是按照文件目录和文件名来进行存取的。
4. 进程管理：说明一个进程存在的唯一标志是 pcb（进程控制块），负责维护进程的信息和状态。进程管理实质上是系统采取某些进程调度算法来使处理合理的分配给每个任务使用。
5. 存储管理：数据的存储方式和组织结构。

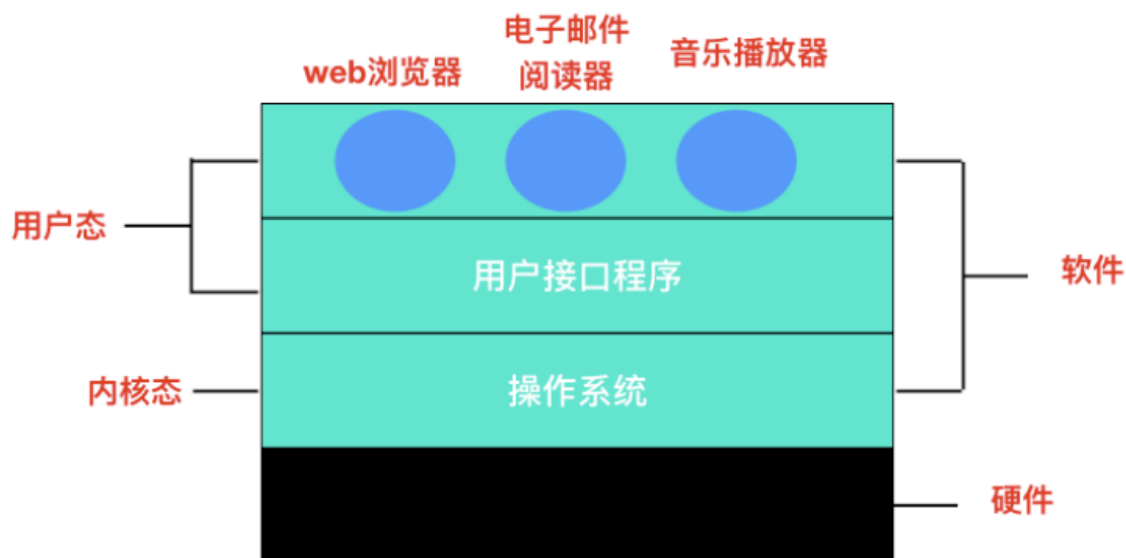
操作系统分类

操作系统的类型也可以分为几种：批处理系统，分时操作系统，实时操作系统，网络操作系统等。下面将简单的介绍他们各自的特点：

1. 批处理系统：首先，用户提交完作业后并在获得结果之前不会再与操作系统进行数据交互，用户提交的作业由系统外存储为后备作业；数据是成批处理的，有操作系统负责作业的自动完成；支持多道程序运行。
2. 分时操作系统：首先交互性方面，用户可以对程序动态运行时对其加以控制；支持多个用户登录终端，并且每个用户共享CPU和其他系统资源。
3. 实时操作系统：会有时钟管理，包括定时处理和延迟处理。实时性要求比较高，某些任务必须优先处理，而有些任务则会被延迟调度完成。
4. 网络操作系统：网络操作系统主要有几种基本功能
 1. 网络通信：负责在源主机与目标主机之间的数据的可靠通信，这是最基本的功能。
 2. 网络服务：系统支持一些电子邮件服务，文件传输，数据共享，设备共享等。
 3. 资源管理：对网络中共享的资源进行管理，例如设置权限以保证数据源的安全性。
 4. 网络管理：主要任务是实现安全管理，例如通过“存取控制”来确保数据的存取安全性，通过“容错性”来保障服务器故障时数据的安全性。
 5. 支持交互操作：在客户/服务器模型的LAN环境下，多种客户机和主机不仅能与服务器进行数连接通信，并且可以访问服务器的文件系统。

聊聊：什么是操作系统

操作系统是管理硬件和软件的一种应用程序。操作系统是运行在计算机上最重要的一种 **软件**，它管理计算机的资源和进程以及所有的硬件和软件。它为计算机硬件和软件提供了一种中间层，使应用软件和硬件进行分离，让我们无需关注硬件的实现，把关注点更多放在软件应用上。



通常情况下，计算机上会运行着许多应用程序，它们都需要对内存和 CPU 进行交互，操作系统的目的就是为了保证这些访问和交互能够准确无误的进行。

聊聊：操作系统的主要功能

一般来说，现代操作系统主要提供下面几种功能

- **进程管理**：进程管理的主要作用就是任务调度，在单核处理器下，操作系统会为每个进程分配一个任务，进程管理的工作十分简单；而在多核处理器下，操作系统除了要为进程分配任务外，还要解决处理器的调度、分配和回收等问题
- **内存管理**：内存管理主要是操作系统负责管理内存的分配、回收，在进程需要时分配内存以及在进程完成时回收内存，协调内存资源，通过合理的页面置换算法进行页面的换入换出
- **设备管理**：根据确定的设备分配原则对设备进行分配，使设备与主机能够并行工作，为用户提供良好的设备使用界面。
- **文件管理**：有效地管理文件的存储空间，合理地组织和管理文件系统，为文件访问和文件保护提供更有效的方法及手段。
- **提供用户接口**：操作系统提供了访问应用程序和硬件的接口，使用户能够通过应用程序发起系统调用从而操纵硬件，实现想要的功能。

聊聊：软件访问硬件的几种方式

软件访问硬件其实就是一种 IO 操作，软件访问硬件的方式，也就是 I/O 操作的方式有哪些。

硬件在 I/O 上大致分为**并行和串行**，同时也对应串行接口和并行接口。

随着计算机技术的发展，I/O 控制方式也在不断发展。选择和衡量 I/O 控制方式有如下三条原则

- (1) 数据传送速度足够快，能满足用户的需求但又不丢失数据；
- (2) 系统开销小，所需的处理控制程序少；
- (3) 能充分发挥硬件资源的能力，使 I/O 设备尽可能忙，而 CPU 等待时间尽可能少。

根据以上控制原则，I/O 操作可以分为四类

- **直接访问**：直接访问由用户进程直接控制主存或 CPU 和外围设备之间的信息传送。直接程序控制方式又称为忙/等待方式。
- **中断驱动**：为了减少程序直接控制方式下 CPU 的等待时间以及提高系统的并行程度，系统引入了中断机制。中断机制引入后，外围设备仅当操作正常结束或异常结束时才向 CPU 发出中断请求。

在 I/O 设备输入每个数据的过程中，由于无需 CPU 的干预，一定程度上实现了 CPU 与 I/O 设备的并行工作。

上述两种方法的特点都是以 CPU 为中心，数据传送通过一段程序来实现，软件的传送手段限制了数据传送的速度。接下来介绍的这两种 I/O 控制方式采用硬件的方法来显示 I/O 的控制

- **DMA 直接内存访问**：为了进一步减少 CPU 对 I/O 操作的干预，防止因并行操作设备过多使 CPU 来不及处理或因速度不匹配而造成的数据丢失现象，引入了 DMA 控制方式。
- **通道控制方式**：通道，独立于 CPU 的专门负责输入输出控制的处理机，它控制设备与内存直接进行数据交换。有自己的通道指令，这些指令由 CPU 启动，并在操作结束时向 CPU 发出中断信号。

聊聊：操作系统的主要目的是什么

操作系统是一种软件，它的主要目的有三种

- 管理计算机资源，这些资源包括 CPU、内存、磁盘驱动器、打印机等。
- 提供一种图形界面，就像我们前面描述的那样，它提供了用户和计算机之间的桥梁。
- 为其他软件提供服务，操作系统与软件进行交互，以便为其分配运行所需的任何必要资源。

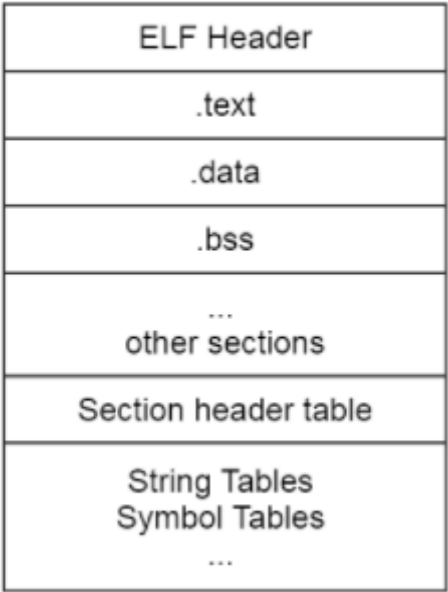
聊聊：操作系统的种类有哪些

操作系统通常预装在你购买计算机之前。大部分用户都会使用默认的操作系统，但是你也可以升级甚至更改操作系统。但是一般常见的操作系统只有三种：**Windows、macOS 和 Linux**。

聊聊：为什么 Linux 系统下的应用程序不能直接在 Windows 下运行

这是一个老生常谈的问题了，在这里给出具体的回答。

其中一点是因为 Linux 系统和 Windows 系统的格式不同，**格式就是协议**，就是在固定位置有意义的数
据。Linux 下的可执行程序文件格式是 `elf`，可以使用 `readelf` 命令查看 elf 文件头。



而 Windows 下的可执行程序是 **PE** 格式，它是一种可移植的可执行文件。

还有一点是因为 Linux 系统和 Windows 系统的 **API** 不同，这个 API 指的就是操作系统的 API，Linux 中的 API 被称为 **系统调用**，是通过 `int 0x80` 这个软中断实现的。而 Windows 中的 API 是放在动态链接库文件中的，也就是 Windows 开发人员所说的 **DLL**，这是一个库，里面包含代码和数据。Linux 中的可执行程序获得系统资源的方法和 Windows 不一样，所以显然是不能在 Windows 中运行的。

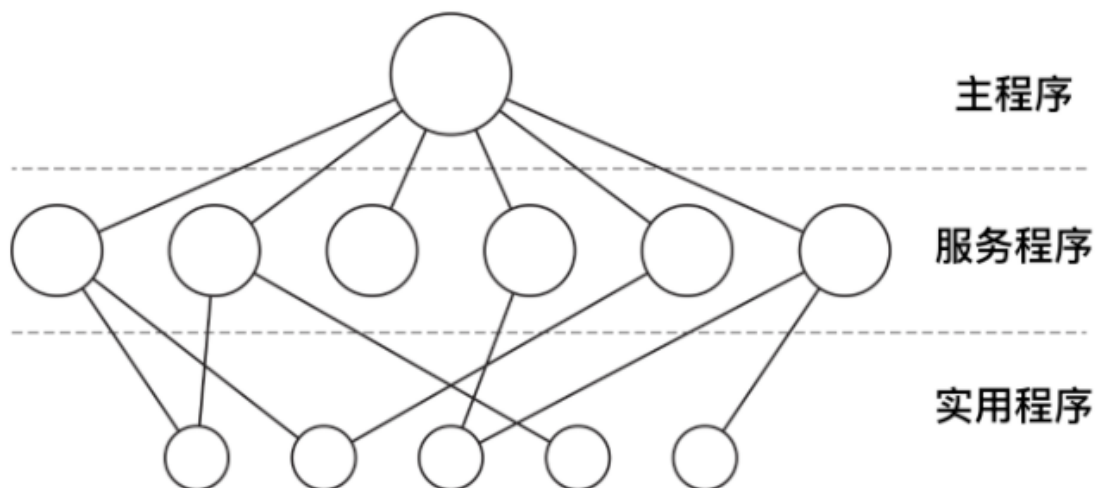
聊聊：操作系统结构

单体系统

在大多数系统中，整个系统在内核态以单一程序的方式运行。整个操作系统是以程序集合来编写的，链接在一块形成一个大的二进制可执行程序，这种系统称为单体系统。

在单体系统中构造实际目标程序时，会首先编译所有单个过程（或包含这些过程的文件），然后使用系统链接器将它们全部绑定到一个可执行文件中

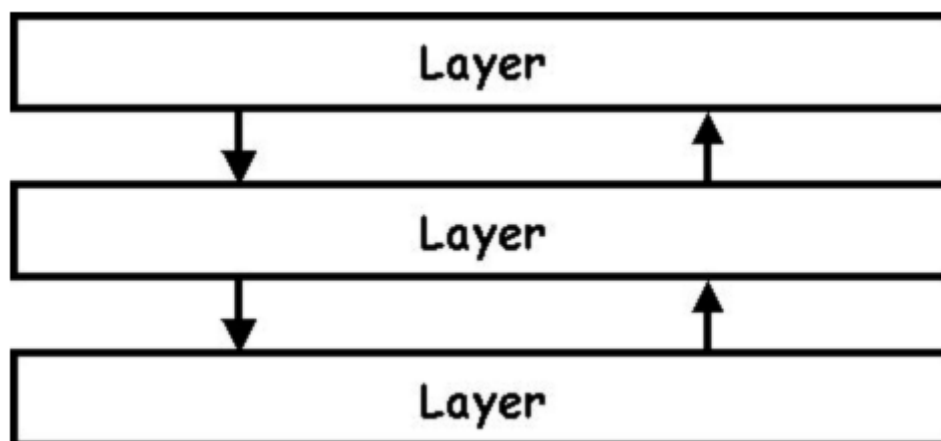
在单体系统中，对于每个系统调用都会有一个服务程序来保障和运行。需要一组实用程序来弥补服务程序需要的功能，例如从用户程序中获取数据。可将各种过程划分为一个三层模型



除了在计算机初启动时所装载的核心操作系统外，许多操作系统还支持额外的扩展。比如 I/O 设备驱动和文件系统。这些部件可以按需装载。在 UNIX 中把它们叫做 共享库(shared library)，在 Windows 中则被称为 动态链接库(Dynamic Link Library,DLL)。他们的扩展名为 .dll，在 C:\windows\system32 目录下存在 1000 多个 DLL 文件，所以不要轻易删除 C 盘文件，否则可能就炸了哦。

分层系统

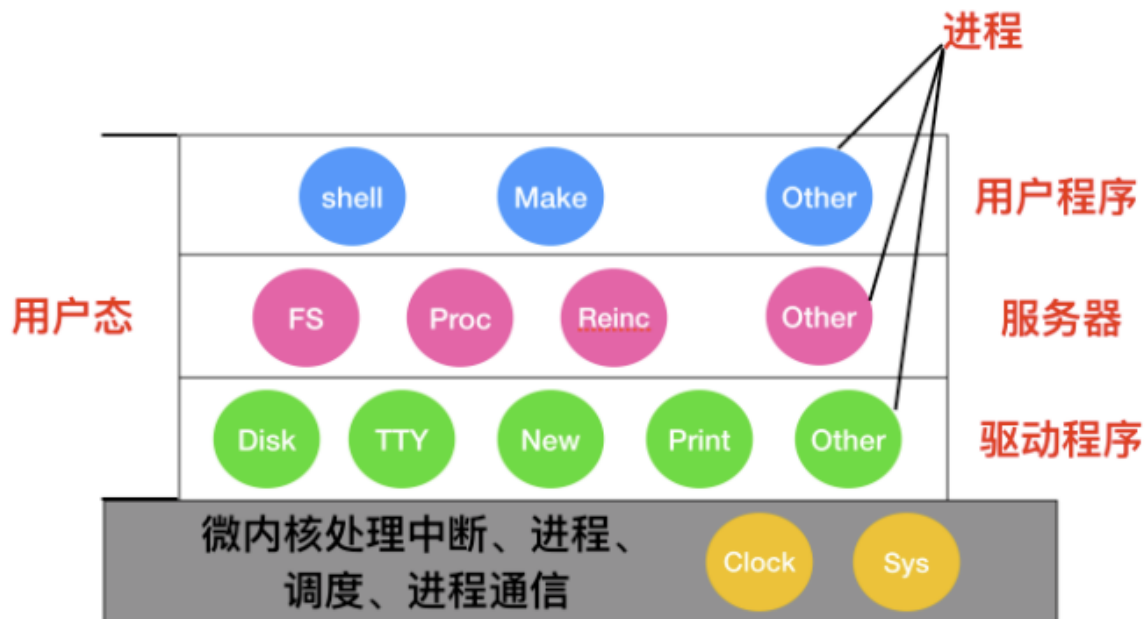
分层系统使用层来分隔不同的功能单元。每一层只与该层的上层和下层通信。每一层都使用下面的层来执行其功能。层之间的通信通过预定义的固定接口通信。



微内核

为了实现高可靠性，将操作系统划分成小的、层级之间能够更好定义的模块是很有必要的，只有一个模块 — 微内核 — 运行在内核态，其余模块可以作为普通用户进程运行。由于把每个设备驱动和文件系统分别作为普通用户进程，这些模块中的错误虽然会使这些模块崩溃，但是不会使整个系统死机。

MINIX 3 是微内核的代表作，它的具体结构如下

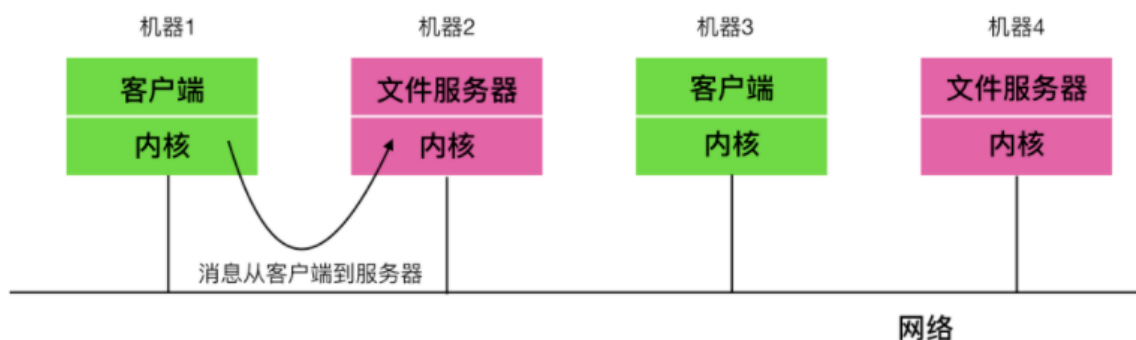


在内核的外部，系统的构造有三层，它们都在用户态下运行，最底层是设备驱动器。由于它们都在用户态下运行，所以不能物理的访问 I/O 端口空间，也不能直接发出 I/O 命令。相反，为了能够对 I/O 设备编程，驱动器构建一个结构，指明哪个参数值写到哪个 I/O 端口，并声称一个内核调用，这样就完成了一次调用过程。

客户-服务器模式

微内核思想的策略是把进程划分为两类：服务器，每个服务器用来提供服务；客户端，使用这些服务。这个模式就是所谓的 客户-服务器 模式。

客户-服务器模式会有两种载体，一种情况是一台计算机既是客户又是服务器，在这种方式下，操作系统会有某种优化；但是普遍情况下是客户端和服务端在不同的机器上，它们通过局域网或广域网连接。



客户通过发送消息与服务器通信，客户端并不需要知道这些消息是在本地机器上处理，还是通过网络被送到远程机器上处理。对于客户端而言，这两种情形是一样的：都是发送请求并得到回应。

聊聊：为什么称为陷入内核

如果把软件结构进行分层说明的话，应该是这个样子的，最外层是应用程序，里面是操作系统内核。



应用程序处于特权级 3，操作系统内核处于特权级 0。如果用户程序想要访问操作系统资源时，会发起系统调用，陷入内核，这样 CPU 就进入了内核态，执行内核代码。至于为什么是陷入，我们看图，内核是一个凹陷的构造，有陷下去的感觉，所以称为陷入。

聊聊：什么是用户态和内核态

用户态和内核态是操作系统的两种运行状态。

- **内核态**：处于内核态的 CPU 可以访问任意的数据，包括外围设备，比如网卡、硬盘等，处于内核态的 CPU 可以从一个程序切换到另外一个程序，并且占用 CPU 不会发生抢占情况，一般处于特权级 0 的状态我们称之为内核态。
- **用户态**：处于用户态的 CPU 只能受限的访问内存，并且不允许访问外围设备，用户态下的 CPU 不允许独占，也就是说 CPU 能够被其他程序获取。

那么为什么要有用户态和内核态呢？

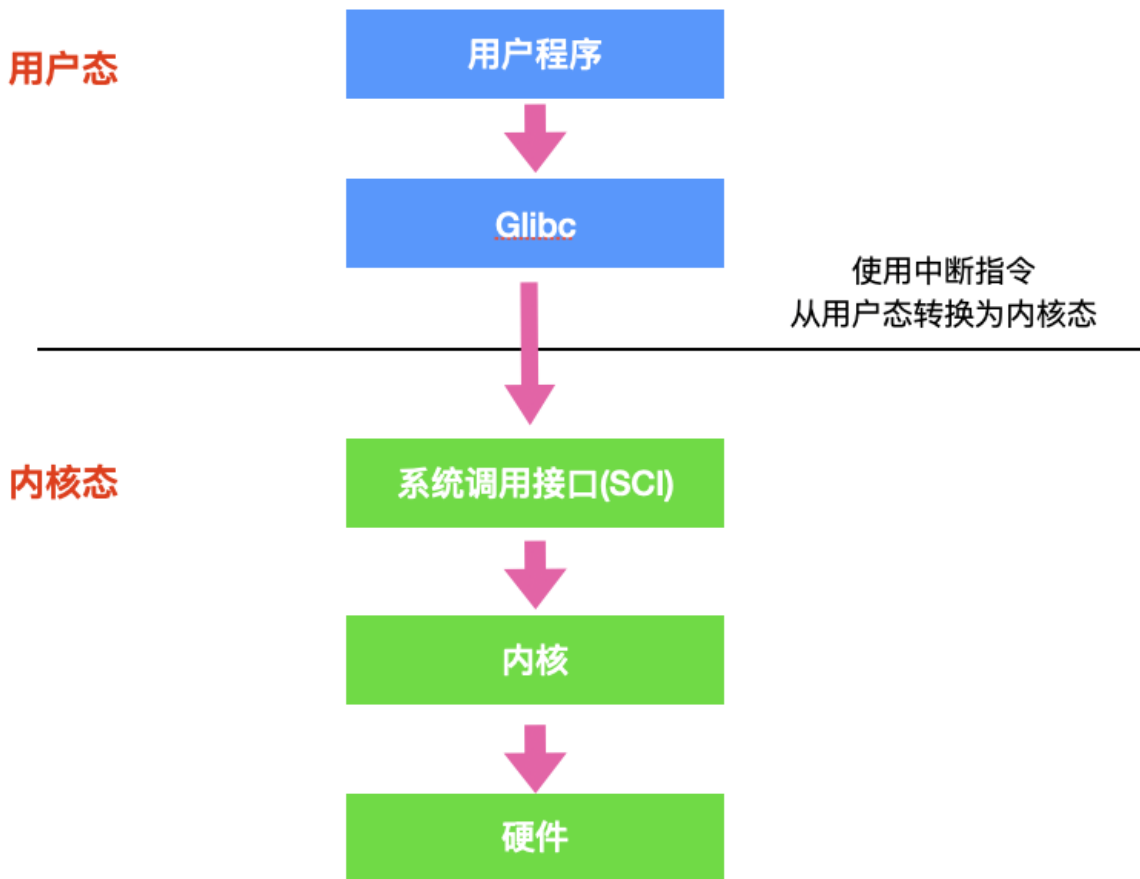
这个主要是访问能力的限制的考量，计算机中有一些比较危险的操作，比如设置时钟、内存清理，这些都需要在内核态下完成，如果随意进行这些操作，那你的系统得崩溃多少次。

聊聊：用户态和内核态是如何切换的？

所有的用户进程都是运行在用户态的，但是我们上面也说了，用户程序的访问能力有限，一些比较重要的比如从硬盘读取数据，从键盘获取数据的操作则是内核态才能做的事情，而这些数据却又对用户程序来说非常重要。所以就涉及到两种模式下的转换，即**用户态 -> 内核态 -> 用户态**，而唯一能够做这些操作的只有 **系统调用**，而能够执行系统调用的就只有 **操作系统**。

一般用户态 -> 内核态的转换我们都称之为 trap 进内核，也被称之为 **陷阱指令(trap instruction)**。

他们的工作流程如下：



- 首先用户程序会调用 `glibc` 库，`glibc` 是一个标准库，同时也是一套核心库，库中定义了很多关键 API。
- `glibc` 库知道针对不同体系结构调用系统调用的正确方法，它会根据体系结构应用程序的二进制接口设置用户进程传递的参数，来准备系统调用。
- 然后，`glibc` 库调用 软件中断指令(SWI)，这个指令通过更新 `CPSR` 寄存器将模式改为超级用户模式，然后跳转到地址 `0x08` 处。
- 到目前为止，整个过程仍处于用户态下，在执行 SWI 指令后，允许进程执行内核代码，MMU 现在允许内核虚拟内存访问
- 从地址 `0x08` 开始，进程执行加载并跳转到中断处理程序，这个程序就是 ARM 中的 `vector_swi()`。
- 在 `vector_swi()` 处，从 SWI 指令中提取系统调用号 `SCNO`，然后使用 `SCNO` 作为系统调用表 `sys_call_table` 的索引，调转到系统调用函数。
- 执行系统调用完成后，将还原用户模式寄存器，然后再以用户模式执行。

聊聊：什么是内核

在计算机中，内核是一个计算机程序，它是操作系统的核心，可以控制操作系统中所有的内容。内核通常是在 boot loader 装载程序之前加载的第一个程序。

这里还需要了解一下什么是 `boot loader`。

`boot loader` 又被称为引导加载程序，能够将计算机的操作系统放入内存中。在电源通电或者计算机重启时，BIOS 会执行一些初始测试，然后将控制权转移到引导加载程序所在的主引导记录 (MBR)。

聊聊：什么是实时系统

实时操作系统对时间做出了严格的要求，实时操作系统分为两种：**硬实时和软实时**

硬实时操作系统 规定某个动作必须在规定的时刻内完成或发生，比如汽车生产车间，焊接机器必须在某一时刻内完成焊接，焊接的太早或者太晚都会对汽车造成永久性伤害。

软实时操作系统 虽然不希望偶尔违反最终的时限要求，但是仍然可以接受。并且不会引起任何永久性伤害。比如数字音频、多媒体、手机都是属于软实时操作系统。

你可以简单理解硬实时和软实时的两个指标：**是否在时刻内必须完成以及是否造成严重损害**。

聊聊：Linux 操作系统的启动过程

当计算机电源通电后，BIOS 会进行 开机自检(Power-On-Self-Test, POST)，对硬件进行检测和初始化。因为操作系统的启动会使用到磁盘、屏幕、键盘、鼠标等设备。

下一步，磁盘中的第一个分区，也被称为 MBR(Master Boot Record) 主引导记录，被读入到一个固定的内存区域并执行。这个分区中有一个非常小的，只有 512 字节的程序。程序从磁盘调入 boot 独立程序，boot 程序将自身复制到高位地址的内存从而为操作系统释放低位地址的内存。

复制完成后，boot 程序读取启动设备的根目录。boot 程序要理解文件系统和目录格式。然后 boot 程序被调入内核，把控制权移交给内核。直到这里，boot 完成了它的工作。系统内核开始运行。

内核启动代码是使用 汇编语言 完成的，主要包括创建内核堆栈、识别 CPU 类型、计算内存、禁用中断、启动内存管理单元等，然后调用 C 语言的 main 函数执行操作系统部分。

这部分也会做很多事情，首先会分配一个消息缓冲区来存放调试出现的问题，调试信息会写入缓冲区。如果调试出现错误，这些信息可以通过诊断程序调出来。

然后操作系统会进行自动配置，检测设备，加载配置文件，被检测设备如果做出响应，就会被添加到已链接的设备表中，如果没有相应，就归为未连接直接忽略。

配置完所有硬件后，接下来要做的就是仔细手工处理进程0，设置其堆栈，然后运行它，执行初始化、配置时钟、挂载文件系统。创建 **init 进程(进程 1)** 和 **守护进程(进程 2)**。

init 进程会检测它的标志以确定它是否为单用户还是多用户服务。在前一种情况中，它会调用 fork 函数创建一个 shell 进程，并且等待这个进程结束。后一种情况调用 fork 函数创建一个运行系统初始化的 shell 脚本（即 /etc/rc）的进程，这个进程可以进行文件系统一致性检测、挂载文件系统、开启守护进程等。

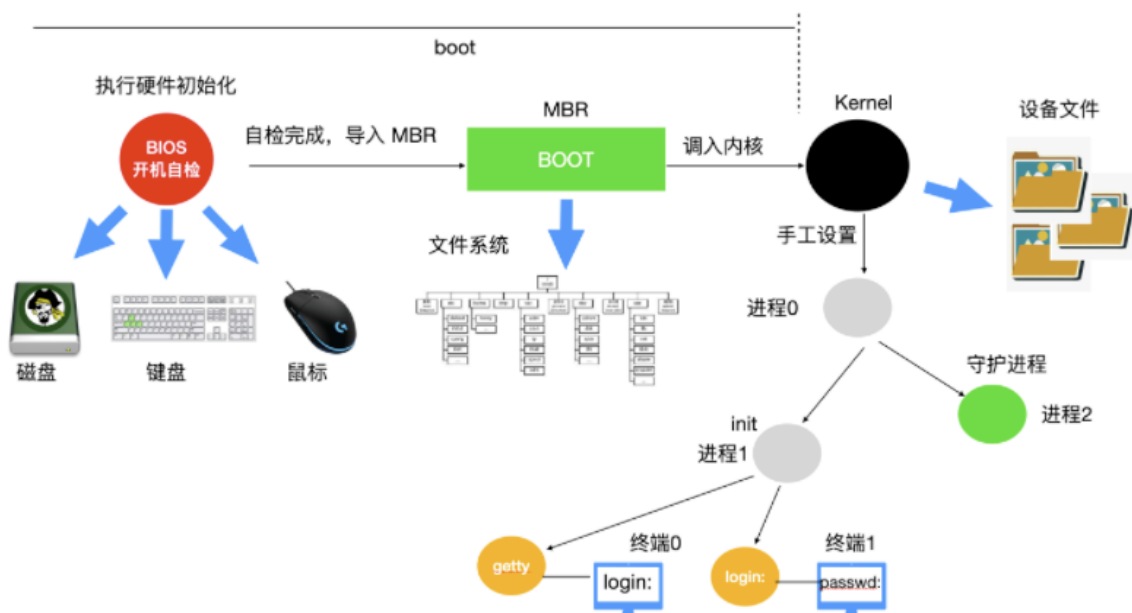
然后 /etc/rc 这个进程会从 /etc/ttytys 中读取数据，/etc/ttytys 列出了所有的终端和属性。对于每一个启用的终端，这个进程调用 fork 函数创建一个自身的副本，进行内部处理并运行一个名为 **getty** 的程序。

getty 程序会在终端上输入

```
login:
```

等待用户输入用户名，在输入用户名后，getty 程序结束，登陆程序 **/bin/login** 开始运行。login 程序需要输入密码，并与保存在 **/etc/passwd** 中的密码进行对比，如果输入正确，login 程序以用户 shell 程序替换自身，等待第一个命令。如果不正确，login 程序要求输入另一个用户名。

整个系统启动过程如下



##

进程线程协程篇

系统调度

在未配置 OS 的系统中，程序的执行方式是顺序执行，即必须在一个程序执行完后，才允许另一个程序执行；在多道程序环境下，则允许多个程序并发执行。程序的这两种执行方式间有着显著的不同。也正是程序并发执行时的这种特征，才导致了在操作系统中引入进程的概念。进程是资源分配的基本单位，线程是资源调度的基本单位。

应用启动体现的就是静态指令加载进内存，进而进入 CPU 运算，操作系统在内存开辟了一段栈内存用来存放指令和变量值，从而形成了进程。早期的操作系统基于进程来调度 CPU，不同进程间是不共享内存空间的，所以进程要做任务切换就要切换内存映射地址。由于进程的上下文关联的变量，引用，计数器等现场数据占用了打段的内存空间，所以频繁切换进程需要整理一大段内存空间来保存未执行完的进程现场，等下次轮到 CPU 时间片再恢复现场进行运算。

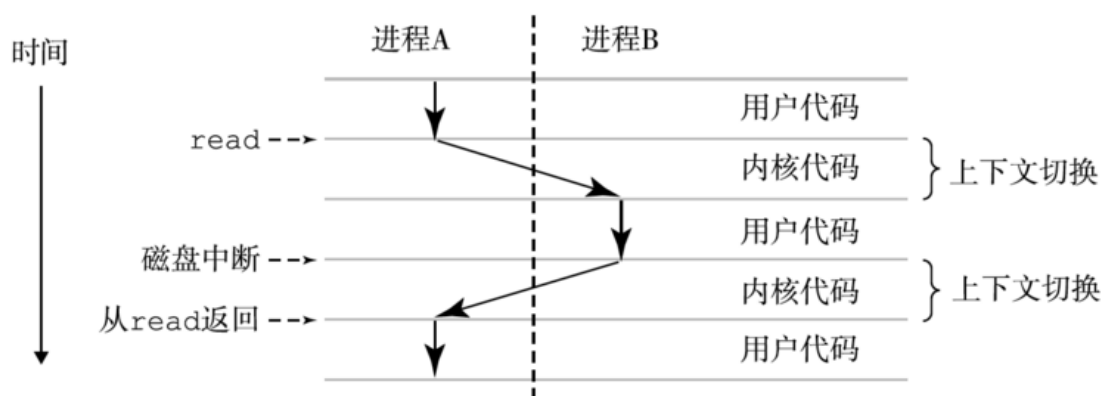
这样既耗费时间又浪费空间，所以我们才要研究多线程。一个进程创建的所有线程，都是共享一个内存空间的，所以线程做任务切换成本就很低了。现代的操作系统都基于更轻量的线程来调度，现在我们提到的“任务切换”都是指“线程切换”。

进程详解

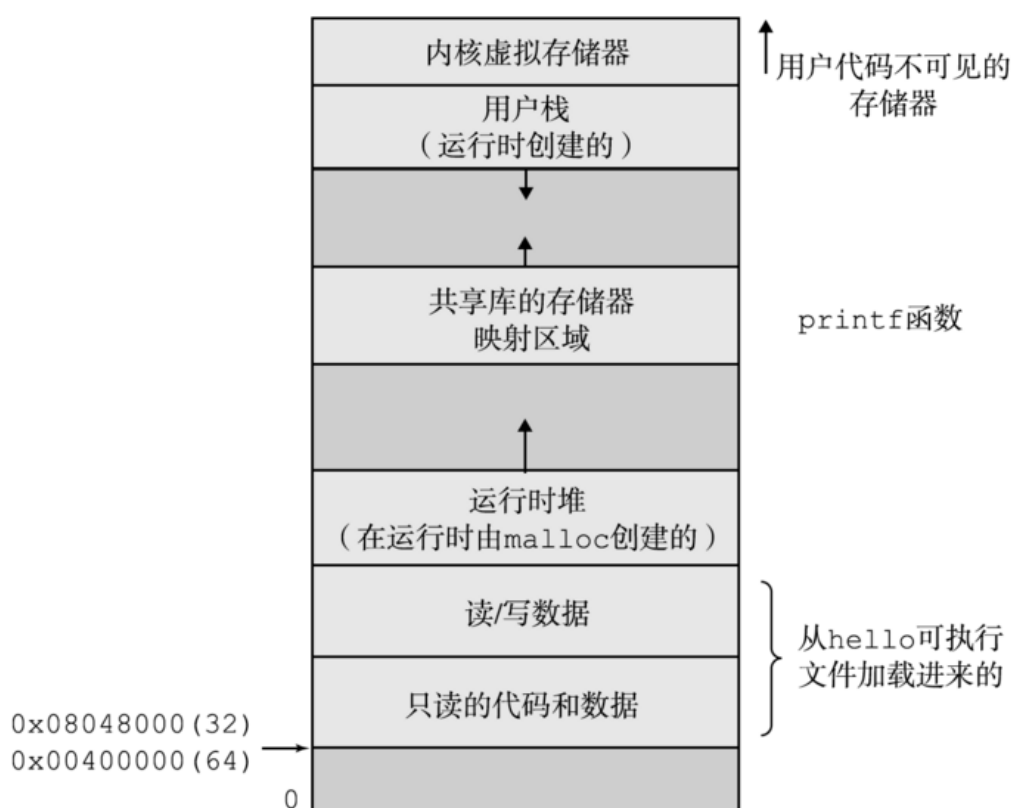
进程是操作系统对一个正在运行的程序的一种抽象，在一个系统上可以同时运行多个进程，而每个进程都好像在独占地使用硬件。所谓的并发运行，则是说一个进程的指令和另一个进程的指令是交错执行的。无论是在单核还是多核系统中，可以通过处理器在进程间切换，来实现单个 CPU 看上去像是在并发地执行多个进程。操作系统实现这种交错执行的机制称为上下文切换。

操作系统保持跟踪进程运行所需的所有状态信息。这种状态，也就是上下文，它包括许多信息，例如 PC 和寄存器文件的当前值，以及主存的内容。在任何一个时刻，单处理器系统都只能执行一个进程的代码。当操作系统决定要把控制权从当前进程转移到某个新进程时，就会进行上下文切换，即保存当前进程的上下文、恢复新进程的上下文，然后将控制权传递到新进程。新进程就会从上次停止的地方开

始。



操作系统为每个进程提供了一个假象，即每个进程都在独占地使用主存。每个进程看到的是一致的存储器，称为虚拟地址空间。其虚拟地址空间最上面的区域是为操作系统中的代码和数据保留的，这对所有进程来说都是一样的；地址空间的底部区域存放用户进程定义的代码和数据。



程序代码和数据：对于所有的进程来说，代码是从同一固定地址开始，直接按照可执行目标文件的内容初始化。

堆：代码和数据区后紧随着的是运行时堆。代码和数据区是在进程一开始运行时就被规定了大小，与此不同，当调用如 malloc 和 free 这样的 C 语言 标准库函数时，堆可以在运行时动态地扩展和收缩。

共享库：大约在地址空间的中间部分是一块用来存放像 C 标准库和数学库这样共享库的代码和数据区域。

栈：位于用户虚拟地址空间顶部的是用户栈，编译器用它来实现函数调用。和堆一样，用户栈在程序执行期间可以动态地扩展和收缩。

内核虚拟存储器：内核总是驻留在内存中，是操作系统的一部分。地址空间顶部的区域是为内核保留的，不允许应用程序读写这个区域的内容或者直接调用内核代码定义的函数。

线程详解

在现代系统中，一个进程实际上可以由多个称为线程的执行单元组成，每个线程都运行在进程的上下文中，并共享同样的代码和全局数据。进程的个体间是完全独立的，而线程间是彼此依存的。多进程环境中，任何一个进程的终止，不会影响到其他进程。而多线程环境中，父线程终止，全部子线程被迫终止(没有了资源)。

而任何一个子线程终止一般不会影响其他线程，除非子线程执行了 `exit()` 系统调用。任何一个子线程执行 `exit()`，全部线程同时灭亡。多线程程序中至少有一个主线程，而这个主线程其实就是有 `main` 函数的进程。它是整个程序的进程，所有线程都是它的子线程；我们通常把具有多线程的主进程称为主线程。

线程共享的环境包括：进程代码段、进程的公有数据、进程打开的文件描述符、信号的处理程序、进程的当前目录、进程用户 ID 与进程组 ID 等，利用这些共享的数据，线程很容易的实现相互之间的通讯。线程拥有这许多共性的同时，还拥有自己的个性，并以此实现并发性：

线程 ID：每个线程都有自己的线程 ID，这个 ID 在本进程中是唯一的。进程用此来标识线程。

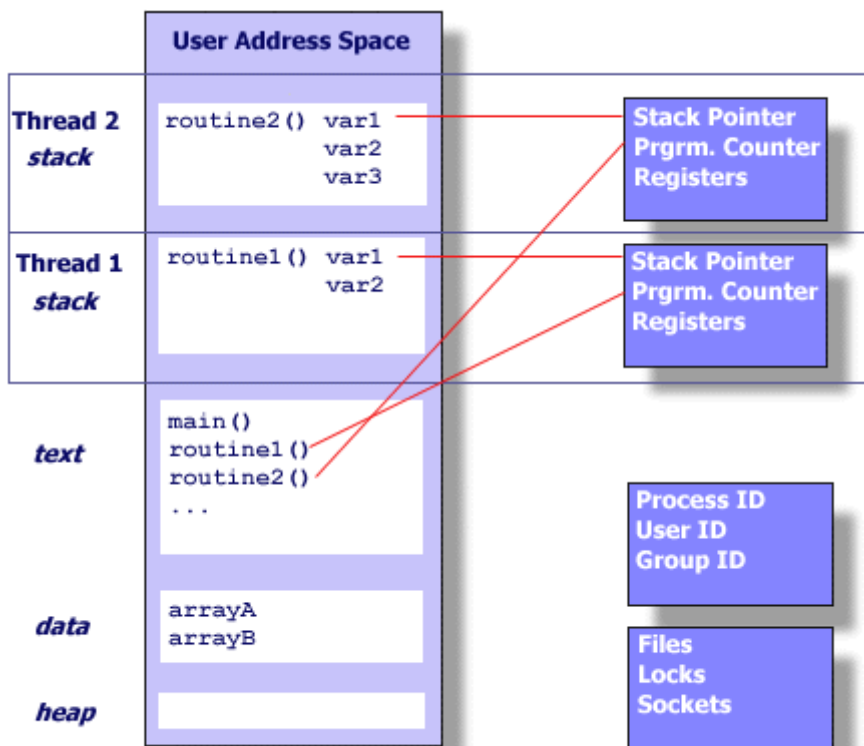
寄存器组的值：由于线程间是并发运行的，每个线程有自己不同的运行线索，当从一个线程切换到另一个线程上时，必须将原有的线程的寄存器集合的状态保存，以便将来该线程在被重新切换到时能得以恢复。

线程的堆栈：堆栈是保证线程独立运行所必须的。线程函数可以调用函数，而被调用函数中又是可以层层嵌套的，所以线程必须拥有自己的函数堆栈，使得函数调用可以正常执行，不受其他线程的影响。

错误返回码：由于同一个进程中有很多个线程在同时运行，可能某个线程进行系统调用后设置了 `errno` 值，而在该线程还没有处理这个错误，另外一个线程就在此时被调度器投入运行，这样错误值就有可能被修改。所以，不同的线程应该拥有自己的错误返回码变量。

线程的信号屏蔽码：由于每个线程所感兴趣的信号不同，所以线程的信号屏蔽码应该由线程自己管理。但所有的线程都共享同样的信号处理器。

线程的优先级：由于线程需要像进程那样能够被调度，那么就必须要要有可供调度使用的参数，这个参数就是线程的优先级。

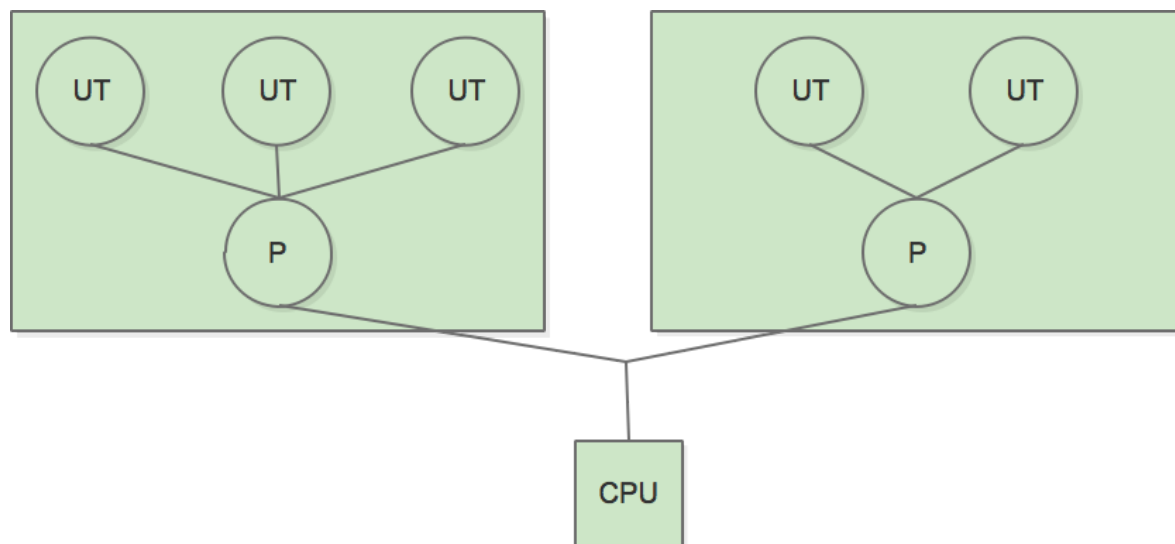


线程模型

线程实现在用户空间下

当线程在用户空间下实现时，操作系统对线程的存在一无所知，操作系统只能看到进程，而不能看到线程。所有的线程都是在用户空间实现。在操作系统看来，每一个进程只有一个线程。过去的操作系统大部分是这种实现方式，这种方式的好处之一就是即使操作系统不支持线程，也可以通过库函数来支持线程。

在这在模型下，程序员需要自己实现线程的数据结构、创建销毁和调度维护。也就相当于需要实现一个自己的线程调度内核，而同时这些线程运行在操作系统的一个进程内，最后操作系统直接对进程进行调度。



这样做有一些优点，首先就是确实在操作系统中实现了真实的多线程，其次就是线程的调度只是在用户态，减少了操作系统从内核态到用户态的切换开销。这种模式最致命的缺点也是由于操作系统不知道线程的存在，因此当一个进程中的某一个线程进行系统调用时，比如缺页中断而导致线程阻塞，此时操作系统会阻塞整个进程，即使这个进程中其它线程还在工作。还有一个问题是假如进程中一个线程长时间不释放 CPU，因为用户空间并没有时钟中断机制，会导致此进程中的其它线程得不到 CPU 而持续等待。

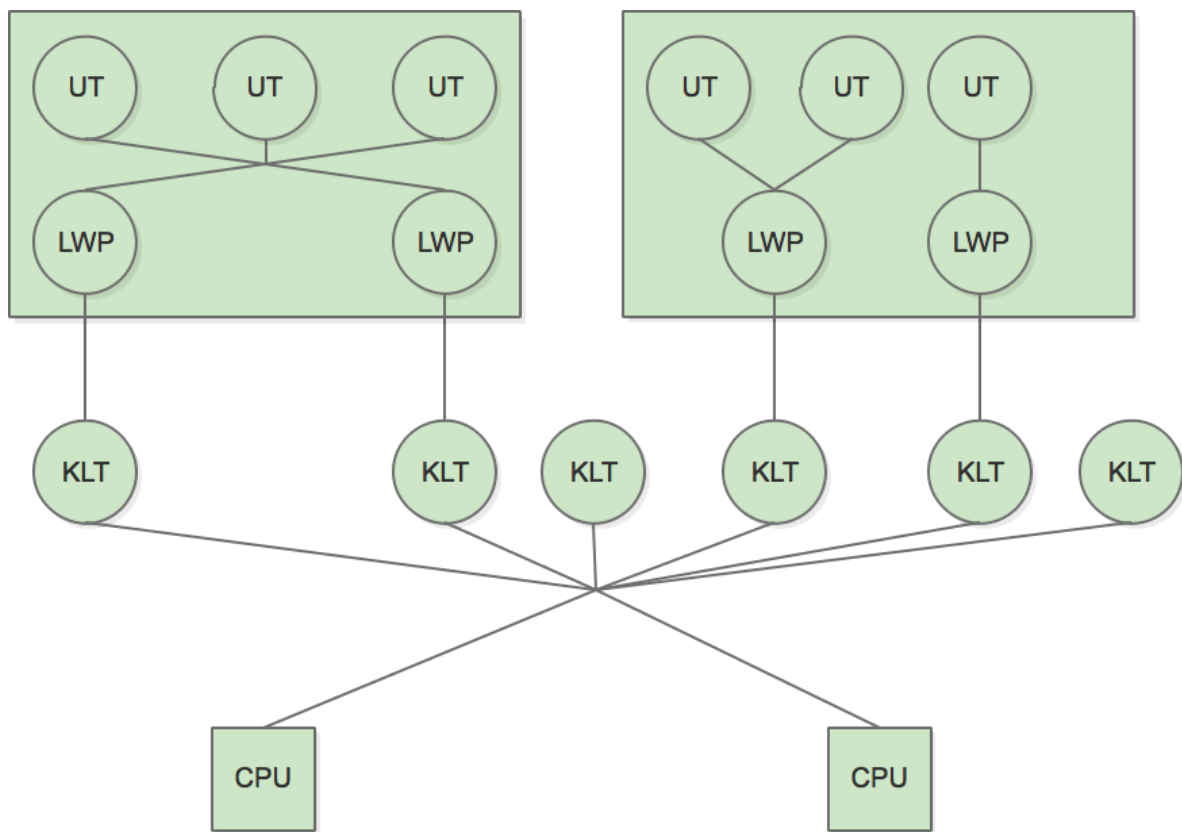
线程实现在操作系统内核中

内核线程就是直接由操作系统内核（Kernel）支持的线程，这种线程由内核来完成线程切换，内核通过操纵调度器（Scheduler）对线程进行调度，并负责将线程的任务映射到各个处理器上。每个内核线程可以视为内核的一个分身，这样操作系统就有能力同时处理多件事情，支持多线程的内核就叫做多线程内核（Multi-Threads Kernel）。

程序员直接使用操作系统中已经实现的线程，而线程的创建、销毁、调度和维护，都是靠操作系统（准确的说是内核）来实现，程序员只需要使用系统调用，而不需要自己设计线程的调度算法和线程对 CPU 资源的抢占使用。

使用用户线程加轻量级进程混合实现

在这种混合实现下，即存在用户线程，也存在轻量级进程。用户线程还是完全建立在用户空间中，因此用户线程的创建、切换、析构等操作依然廉价，并且可以支持大规模的用户线程并发。而操作系统提供支持的轻量级进程则作为用户线程和内核线程之间的桥梁，这样可以使用内核提供的线程调度功能及处理器映射，并且用户线程的系统调用要通过轻量级进程来完成，大大降低了整个进程被完全阻塞的风险。在这种混合模式中，用户线程与轻量级进程的数量比是不定的，即为 N:M 的关系：



* UT: 用户线程 (User Thread)

* LWP: 轻量级进程 (Light Weight Process, LWP)

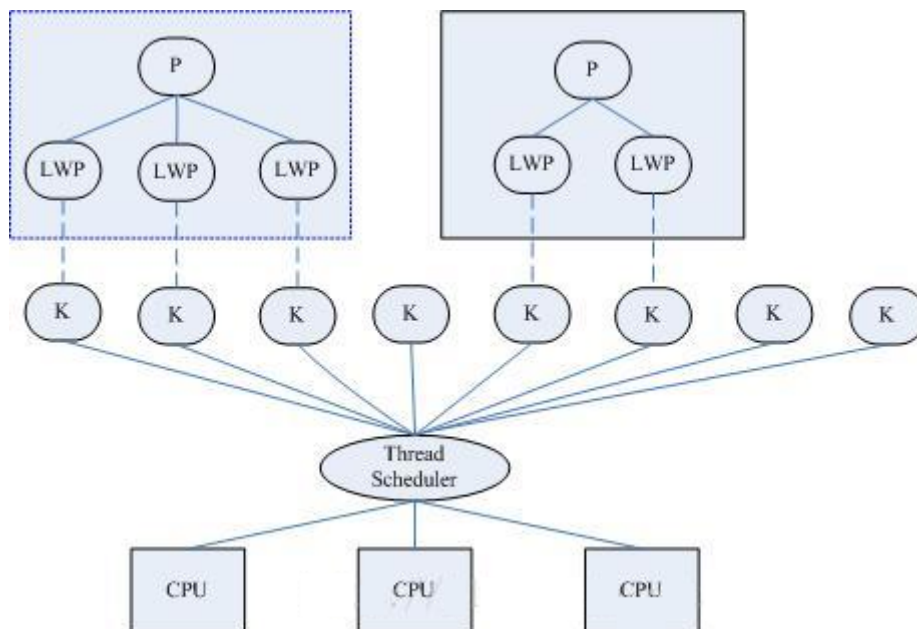
* KLT: 内核线程 (Kernel-Level Thread, KLT)

GoLang 的协程就是使用了这种模型，在用户态，协程能快速的切换，避免了线程调度的 CPU 开销问题，协程相当于线程的线程。

Linux中的线程

在 Linux 2.4 版以前，线程的实现和管理方式就是完全按照进程方式实现的；在 Linux 2.6 之前，内核并不支持线程的概念，仅通过轻量级进程 (Lightweight Process) 模拟线程；轻量级进程是建立在内核之上并由内核支持的用户线程，它是内核线程的高度抽象，每一个轻量级进程都与一个特定的内核线程关联。内核线程只能由内核管理并像普通进程一样被调度。这种模型最大的特点是线程调度由内核完成了，而其他线程操作 (同步、取消) 等都是核外的线程库 (Linux Thread) 函数完成的。

为了完全兼容 Posix 标准，Linux 2.6 首先对内核进行了改进，引入了线程组的概念 (仍然用轻量级进程表示线程)，有了这个概念就可以将一组线程组织称为一个进程，不过内核并没有准备特别的调度算法或是定义特别的数据结构来表征线程；相反，线程仅仅被视为一个与其他进程 (概念上应该是线程) 共享某些资源的进程 (概念上应该是线程)。在实现上主要的改变就是在 `task_struct` 中加入 `tgid` 字段，这个字段就是用于表示线程组 id 的字段。在用户线程库方面，也使用 NPTL 代替 Linux Thread，不同调度模型上仍然采用 1 对 1 模型。



进程的实现是调用 fork 系统调用：`pid_t fork(void);`，线程的实现是调用 clone 系统调用：`int clone(int (*fn)(void *), void *child_stack, int flags, void *arg, ...)`。与标准 `fork()` 相比，线程带来的开销非常小，内核无需单独复制进程的内存空间或文件描写叙述符等等。这就节省了大量的 CPU 时间，使得线程创建比新进程创建快上十到一百倍，能够大量使用线程而无需太过于操心带来的 CPU 或内存不足。无论是 `fork`、`vfork`、`kthread_create` 最后都是要调用 `do_fork`，而 `do_fork` 就是根据不同的函数参数，对一个进程所需的资源进行分配。

内核线程

内核线程是由内核自己创建的线程，也叫做守护线程（Deamon），在终端上用命令 `ps -Al` 列出的所有进程中，名字以 `k` 开关以 `d` 结尾的往往都是内核线程，比如 `kthreadd`、`kswapd` 等。与用户线程相比，它们都由 `do_fork()` 创建，每个线程都有独立的 `task_struct` 和内核栈；也都参与调度，内核线程也有优先级，会被调度器平等地换入换出。二者的不同之处在于，内核线程只工作在内核态中；而用户线程则既可以运行在内核态（执行系统调用时），也可以运行在用户态；内核线程没有用户空间，所以对于一个内核线程来说，它的 `0~3G` 的内存空间是空白的，它的 `current->mm` 是空的，与内核使用同一张页表；而用户线程则可以看到完整的 `0~4G` 内存空间。

在 Linux 内核启动的最后阶段，系统会创建两个内核线程，一个是 `init`，一个是 `kthreadd`。其中 `init` 线程的作用是运行文件系统上的一系列“init”脚本，并启动 `shell` 进程，所以 `init` 线程称得上是系统中所有用户进程的祖先，它的 `pid` 是 1。`kthreadd` 线程是内核的守护线程，在内核正常工作时，它永远不退出，是一个死循环，它的 `pid` 是 2。

协程

协程是用户模式下的轻量级线程，最准确的名字应该叫用户空间线程（User Space Thread），在不同的领域中也有不同的叫法，譬如纤程（Fiber）、绿色线程（Green Thread）等等。操作系统内核对协程一无所知，协程的调度完全有应用程序来控制，操作系统不管这部分的调度；一个线程可以包含一个或多个协程，协程拥有自己的寄存器上下文和栈，协程调度切换时，将寄存器上细纹和栈保存起来，在切换回来时恢复先前保运的寄存上下文和栈。

协程的优势如下：

- 节省内存，每个线程需要分配一段栈内存，以及内核里的一些资源
- 节省分配线程的开销（创建和销毁线程要各做一次 `syscall`）
- 节省大量线程切换带来的开销
- 与 `NIO` 配合实现非阻塞的编程，提高系统的吞吐

比如 Golang 里的 go 关键字其实就是负责开启一个 Fiber，让 func 逻辑跑在上面。而这一切都是发生的用户态上，没有发生在内核态上，也就是说没有 ContextSwitch 上的开销。

Go协程模型

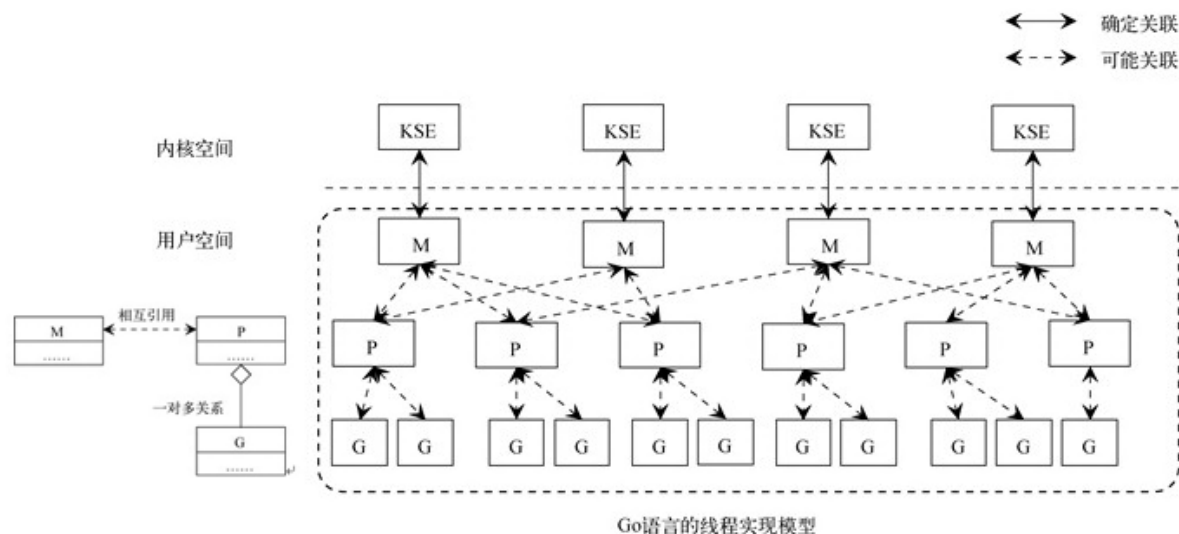
Go 线程模型属于多对多线程模型，在操作系统提供的内核线程之上，Go 搭建了一个特有的两级线程模型。Go 中使用 Go 语句创建的 Goroutine 可以认为是轻量级的用户线程，Go 线程模型包含三个概念：

G: 表示 Goroutine，每个 Goroutine 对应一个 G 结构体，G 存储 Goroutine 的运行堆栈、状态以及任务函数，可重用。G 并非执行体，每个 G 需要绑定到 P 才能被调度执行。

P: Processor，表示逻辑处理器，对 G 来说，P 相当于 CPU 核，G 只有绑定到 P(在 P 的 local runq 中)才能被调度。对 M 来说，P 提供了相关的执行环境 (Context)，如内存分配状态 (mcache)，任务队列 (G) 等，P 的数量决定了系统内最大可并行的 G 的数量 (物理 CPU 核数 $\geq$ P 的数量)，P 的数量由用户设置的 GOMAXPROCS 决定，但是不论 GOMAXPROCS 设置为多大，P 的数量最大为 256。

M: Machine，OS 线程抽象，代表着真正执行计算的资源，在绑定有效的 P 后，进入 schedule 循环；M 的数量是不定的，由 Go Runtime 调整，为了防止创建过多 OS 线程导致系统调度不过来，目前默认最大限制为 10000 个。

在 Go 中每个逻辑处理器会绑定到某一个内核线程上，每个逻辑处理器 (P) 内有一个本地队列，用来存放 Go 运行时分配的 goroutine。多对多线程模型中是操作系统调度线程在物理 CPU 上运行，在 Go 中则是 Go 的运行时调度 Goroutine 在逻辑处理器 (P) 上运行。

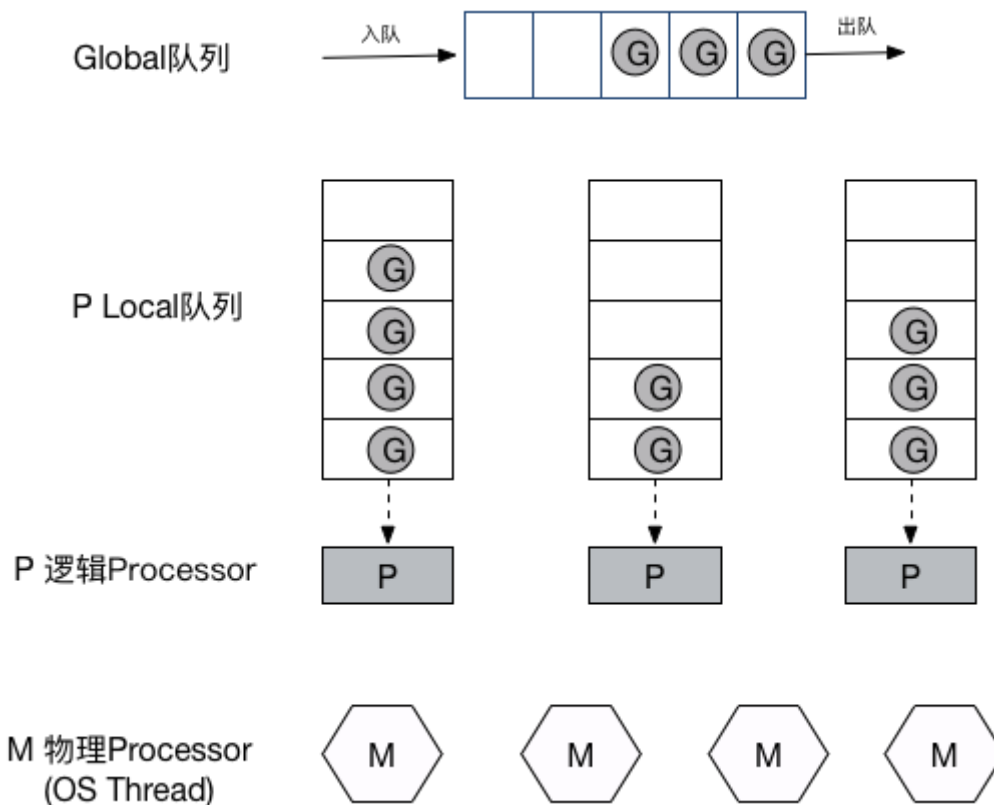


Go 的栈是动态分配大小的，随着存储数据的数量而增长和收缩。每个新建的 Goroutine 只有大约 4KB 的栈。每个栈只有 4KB，那么在一个 1GB 的 RAM 上，我们就可以有 256 万个 Goroutine 了，相对于 Java 中每个线程的 1MB，这是巨大的提升。Golang 实现了自己的调度器，允许众多的 Goroutines 运行在相同的 OS 线程上。就算 Go 会运行与内核相同的上下文切换，但是它能够避免切换至 ring-0 以运行内核，然后再切换回来，这样就会节省大量的时间。

在 Go 中存在两级调度：

- 一级是操作系统的调度系统，该调度系统调度逻辑处理器占用 cpu 时间片运行；
- 一级是 Go 的运行时调度系统，该调度系统调度某个 Goroutine 在逻辑处理上运行。

使用 Go 语句创建一个 Goroutine 后，创建的 Goroutine 会被放入 Go 运行时调度器的全局运行队列中，然后 Go 运行时调度器会把全局队列中的 Goroutine 分配给不同的逻辑处理器 (P)，分配的 Goroutine 会被放到逻辑处理器 (P) 的本地队列中，当本地队列中某个 Goroutine 就绪后待分配到时间片后就可以在逻辑处理器上运行了。



进程线程协程详解总结

进程是操作系统对一个正在运行的程序的一种抽象，在一个系统上可以同时运行多个进程，而每个进程都好像在独占地使用硬件。

在现代系统中，一个进程实际上可以由多个称为线程的执行单元组成，每个线程都运行在进程的上下文中，并共享同样的代码和全局数据。

协程是用户模式下的轻量级线程，最准确的名字应该叫用户空间线程（User Space Thread）。

进程线程协程区别

进程协程进程对比

进程概念

进程是系统资源分配的最小单位, 系统由一个个进程(程序)组成 一般情况下, 包括文本区域 (text region)、数据区域 (data region) 和堆栈 (stack region)。

文本区域存储处理器执行的代码，数据区域存储变量和进程执行期间使用的动态分配的内存，堆栈区域存储着活动过程调用的指令和本地变量。

因此进程的创建和销毁都是相对于系统资源,所以是一种比较昂贵的操作。 进程有三个状态:

状态	描述
等待态	等待某个事件的完成
就绪态	等待系统分配处理器以便运行
运行态	占有处理器正在运行

进程是抢占式的争夺 CPU 运行自身,而 CPU 单核的情况下同一时间只能执行一个进程的代码,但是多进程的实现则是通过 CPU 飞快的切换不同进程,因此使得看上去就像是多个进程在同时进行。

通信问题:由于进程间是隔离的,各自拥有自己的内存资源,因此相对于线程比较安全,所以不同进程之间的数据只能通过 IPC(Inter-Process Communication) 进行通信共享。

线程概念

线程属于进程,线程共享进程的内存地址空间并且线程几乎不占有系统资源。

通信问题:进程相当于一个容器,而线程而是运行在容器里面的,因此对于容器内的东西,线程是共同享有的,因此线程间的通信可以直接通过全局变量进行通信。但是由此带来的例如多个线程读写同一个地址变量的时候则将带来不可预期的后果,因此这时候引入了各种锁的作用,例如互斥锁等。

同时多线程是不安全的,当一个线程崩溃了,会导致整个进程也崩溃了,即其他线程也挂了,但多进程而不会,一个进程挂了,另一个进程依然照样运行。

进程是系统分配资源的最小单位,线程是 CPU 调度的最小单位。由于默认进程内只有一个线程,所以多核 CPU 处理多进程就像是一个进程一个核心。

协程概念

协程是属于线程的。协程程序是在线程里面跑的,因此协程又称微线程和纤程等,协没有线程的上下文切换消耗。协程的调度切换是用户(程序员)手动切换的,因此更加灵活,因此又叫用户空间线程。

原子操作性。由于协程是用户调度的,所以不会出现执行一半的代码片段被强制中断了,因此无需原子操作锁。

进程线程协程详解

进程

进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动,进程是系统进行资源分配和调度的一个独立单位。每个进程都有自己的独立内存空间,不同进程通过进程间通信来通信。由于进程比较重量,占据独立的内存,所以上下文进程间的切换开销(栈、寄存器、虚拟内存、文件句柄等)比较大,但相对比较稳定安全。

线程

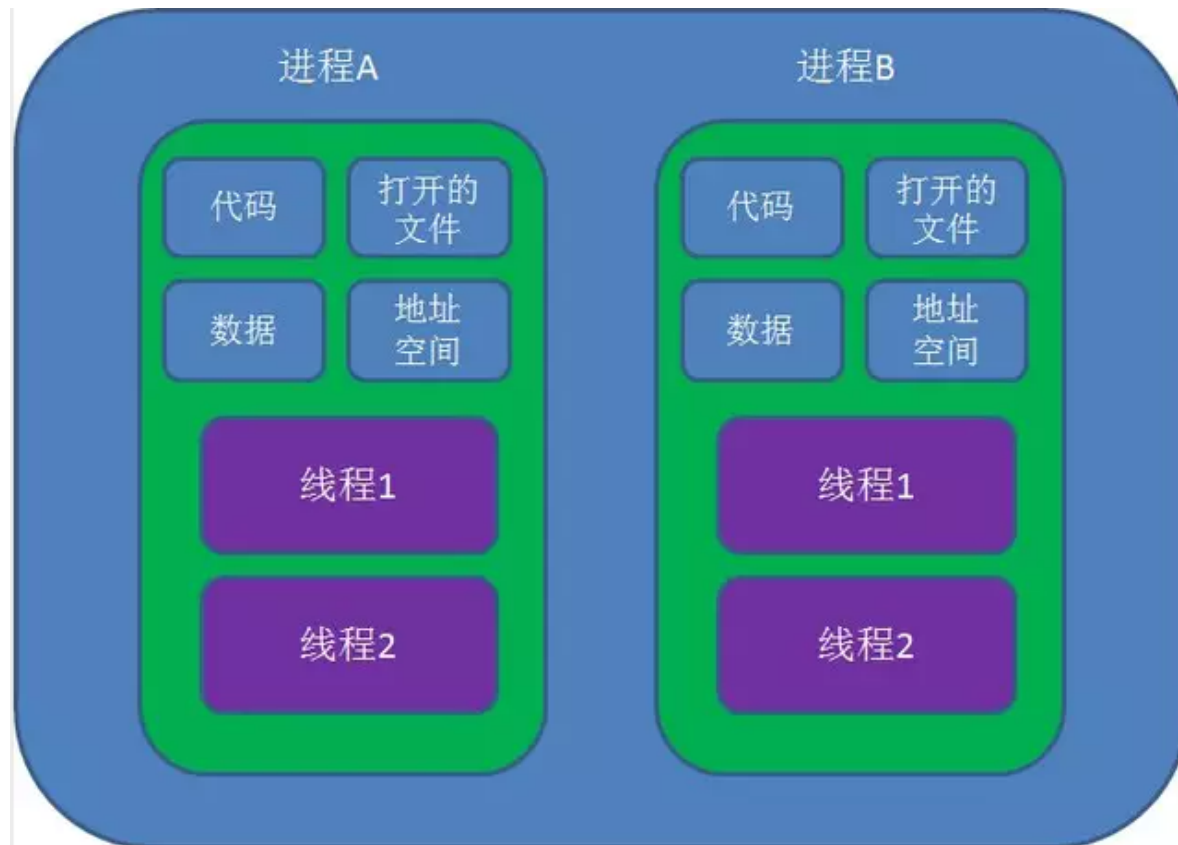
线程是进程的一个实体,是 CPU 调度和分派的基本单位,它是比进程更小的能独立运行的基本单位.线程自己基本上不拥有系统资源,只拥有一点在运行中必不可少的资源(如程序计数器,一组寄存器和栈),但是它可与同属一个进程的其他的线程共享进程所拥有的全部资源。线程间通信主要通过共享内存,上下文切换很快,资源开销较少,但相比进程不够稳定容易丢失数据。

协程

协程是一种用户态的轻量级线程，协程的调度完全由用户控制。协程拥有自己的寄存器上下文和栈。协程调度切换时，将寄存器上下文和栈保存到其他地方，在切回来的时候，恢复先前保存的寄存器上下文和栈，直接操作栈则基本没有内核切换的开销，可以不加锁的访问全局变量，所以上下文的切换非常快。

图解

线程图解如下：



协程图解如下：



进程与线程比较

1. 地址空间:线程是进程内的一个执行单元, 进程内至少有一个线程, 它们共享进程的地址空间, 而进程有自己独立的地址空间。
2. 资源拥有: 进程是资源分配和拥有的单位, 同一个进程内的线程共享进程的资源。
3. 线程是处理器调度的基本单位, 但进程不是。
4. 二者均可并发执行。
5. 每个独立的线程有一个程序运行的入口、顺序执行序列和程序的出口, 但是线程不能够独立执行, 必须依存在应用程序中, 由应用程序提供多个线程执行控制。

协程与线程进行比较

1. 一个线程可以多个协程, 一个进程也可以单独拥有多个协程。
2. 线程进程都是同步机制, 而协程则是异步。
3. 协程能保留上一次调用时的状态, 每次过程重入时, 就相当于进入上一次调用的状态。

进程线程协程区别总结

进程是系统资源分配的最小单位, 系统由一个个进程(程序)组成 一般情况下, 包括文本区域 (text region)、数据区域 (data region) 和堆栈 (stack region) 。

线程属于进程, 线程共享进程的内存地址空间并且线程几乎不占有系统资源。

协程是属于线程的。协程程序是在线程里面跑的, 因此协程又称微线程和纤程等, 协没有线程的上下文切换消耗。协程的调度切换是用户(程序员)手动切换的, 因此更加灵活, 因此又叫用户空间线程。

孤儿进程

孤儿进程教程

如果父进程先退出, 子进程还没退出那么子进程将被托孤给 init 进程, 这时子进程的父进程就是 init 进程 (1 号进程) 。

案例

创建孤儿进程

我们在 Linux 下使用 vim 新建一个 childprocess.c 的文件, 编写如下 C 语言 代码如下:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <errno.h>
#include <signal.h>
int main(void)
{
    pid_t pid ;
    signal(SIGCHLD, SIG_IGN);
```

```

printf("before fork pid:%d\n",getpid());
int abc = 10;
pid = fork();
if(pid == -1)
{
    perror("tile");
    return -1;
}
if(pid > 0)           //父进程先退出
{
    abc++;
    printf("parent:pid:%d \n",getpid());
    printf("abc:%d \n",abc);
    sleep(5);
}
else if(pid == 0)     //子进程后退出,被托付给init进程
{
    abc++;
    printf("child:%d,parent: %d\n",getpid(),getppid());
    printf("abc:%d",abc);
    sleep(100);
}
printf("fork after...\n");
}

```

我们使用 gcc 编译上述程序，具体命令如下：

```
gcc childprocess.c -ochildprocess
```

编译完成后，会在当前目录生成一个 childprocess 的二进制可执行文件，我们使用 ls 命令，查看，如下：

```

haicoder(www.haicoder.net)# ls
childprocess  childprocess.c
haicoder(www.haicoder.net)#

```

此时，我们直接运行该二进制文件，输入以下命令：

```
./childprocess
```

运行成功后，控制台输出如下：

```

haicoder(www.haicoder.net)# ./childprocess
before fork pid:24588
parent:pid:24588
abc:11
child:24589,parent: 24588
fork after...

```

此时，我们在另一终端，使用 ps 命令，查看当前进程的状态，具体命令如下：

```
ps -elf | grep childprocess
```

此时，运行后，控制台输出如下：

```

haicoder(www.haicoder.net)# ps -elf | grep childprocess
0 S root    24658 24248  0 80   0 - 1039 hrtime 10:24 pts/0    00:00:00 ./childprocess
1 S root    24659 24658  0 80   0 - 1039 hrtime 10:24 pts/0    00:00:00 ./childprocess
0 R root    24661 24488  0 80   0 - 28162 -      10:24 pts/1    00:00:00 grep --color=auto childprocess

```

此时，我们可以看到，有两个 childprocess 进程在运行，稍等一会，我们再次使用 ps 命令查看当前进程状态，此时，运行后，控制台输出如下：


```
haicoder(www.haicoder.net)# ps -elf | grep childprocess
1 S root      24659      1 0 80   0 - 1039 hrtim 10:24 pts/0    00:00:00 ./childprocess
0 R root      24665 24488  0 80   0 - 28162 -    10:24 pts/1    00:00:00 grep --color=auto childprocess
```

此时，我们可以看到，只有一个 childprocess 进程在运行了，而且此时的 childprocess 进程的父进程变成了 1，也就是我们说的 init 进程。

僵尸进程

僵尸进程教程

如果我们了解过 Linux 进程状态及转换关系，我们应该知道进程这么多状态中有一种状态是僵死状态，就是进程终止后进入僵死状态（zombie）等待告知父进程自己终止后才能完全消失。

但是如果一个进程已经终止了，但是其父进程还没有获取其状态，那么这个进程就称之为僵尸进程。

僵尸进程还会消耗一定的系统资源，并且还保留一些概要信息供父进程查询子进程的状态可以提供父进程想要的信息，一旦父进程得到想要的信息，僵尸进程就会结束。

案例

创建僵尸进程

我们在 Linux 下使用 vim 新建一个 zombie.c 的文件，编写如下 C 语言 代码如下：

```
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <errno.h>
int main(void)
{
    pid_t pid ;
    //signal(SIGCHLD,SIG_IGN);
    printf("before fork pid:%d\n",getpid());
    int abc = 10;
    pid = fork();
    if(pid == -1)
    {
        perror("tile");
        return -1;
    }
    if(pid > 0)
    {
        abc++;
        printf("parent:pid:%d \n",getpid());
        printf("abc:%d \n",abc);
        sleep(20);
    }
    else if(pid == 0)
    {
        abc++;
        printf("child:%d,parent: %d\n",getpid(),getppid());
        printf("abc:%d",abc);
        exit(0);
    }
}
```



```
}  
    printf("fork after...\n");  
}
```

我们使用 gcc 编译上述程序，具体命令如下：

```
gcc zombie.c -ozombie
```

编译完成后，会在当前目录生成一个 zombie 的二进制可执行文件，我们使用 ls 命令，查看，如下：

```
haicoder(www.haicoder.net)# ls  
zombie  zombie.c  
haicoder(www.haicoder.net)#
```

此时，我们直接运行该二进制文件，输入以下命令：

```
./zombie
```

运行成功后，控制台输出如下：

```
haicoder(www.haicoder.net)# ./zombie  
before fork pid:25025  
parent:pid:25025  
abc:11  
child:25026,parent: 25025  
abc:11
```

此时，我们在另一终端，使用 ps 命令，查看当前进程的状态，具体命令如下：

```
ps -elf | grep zombie
```

此时，运行后，控制台输出如下：

```
haicoder(www.haicoder.net)# ps -elf | grep zom  
0 S root      823 25451 0 80  0 - 1039 hrtim 12:08 pts/2    00:00:00 ./zombie  
1 Z root      824 823 0 80  0 - 0 exit  12:08 pts/2    00:00:00 [zombie] <defunct>  
0 R root      828 802 0 80  0 - 28162 - 12:08 pts/3    00:00:00 grep --color=auto zom  
haicoder(www.haicoder.net)#
```

此时，我们可以看到，zombie 进程后面的状态为 defunct，即此时的 zombie 进程即为僵尸进程。

怎么避免僵尸进程

看程序被注释的那句 `signal(SIGCHLD,SIG_IGN)`，加上就不会出现僵尸进程了。

这是 `signal()` 函数的声明 `sighandler_t signal(int signum, sighandler_t handler)`，我们可以得出 `signal` 函数的第一个函数是 Linux 支持的信号，第二个参数是对信号的操作，是系统默认还是忽略或捕获。

我们这就可以知道 `signal(SIGCHLD,SIG_IGN)` 是选择对子程序终止信号选择忽略，这是僵尸进程就是交个内核自己处理，并不会产生僵尸进程。

守护进程

守护进程教程

守护进程就是在后台运行，不与任何终端关联的进程。

通常情况下守护进程在系统启动时就在运行，它们以 root 用户或者其他特殊用户（apache 和 postfix）运行，并能处理一些系统级的任务。习惯上守护进程的名字通常以 d 结尾（sshd），但这些不是必须的。

创建守护进程的步骤

- 调用 fork(), 创建新进程，它会是将来的守护进程。
- 在父进程中调用 exit, 保证子进程不是进程组长。
- 调用 setsid() 创建新的会话区。
- 将当前目录改成跟目录（如果把当前目录作为守护进程的目录，当前目录不能被卸载他作为守护进程的工作目录）。
- 将标准输入，标注输出，标准错误重定向到 /dev/null。

案例

创建守护进程

我们在 Linux 下使用 vim 新建一个 daemon.c 的文件，编写如下 C 语言 代码如下：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <errno.h>
#include <signal.h>
#include <fcntl.h>
#include <unistd.h>
#include <linux/fs.h>
int main(void)
{
    pid_t pid;
    int i;
    pid = fork();    //创建一个新进程,将来会是守护进程
    if(pid == -1)
    {
        return -1;
    }
    else if(pid != 0)
    {
        //父进程调用exit,保证子进程不是进程组长
        exit(EXIT_SUCCESS);
    }
    if(setsid() == -1) //创建新的会话区
    {
        return -1;
    }
    if(chdir("/") == -1) //将当前目录改成根目录
    {
        return -1;
    }
    for(i = 0; i < 1024; i++)
    {
        close(i);
    }
}
```

```
}
open("/dev/null", O_RDWR); //重定向
dup(0);
dup(0);
return 0;
}
```

我们使用 gcc 编译上述程序，具体命令如下：

```
gcc daemon.c -odaemon
```

编译完成后，会在当前目录生成一个 daemon 的二进制可执行文件，我们使用 ls 命令，查看，如下：

```
haicoder(www.haicoder.net)# ls
daemon  daemon.c
haicoder(www.haicoder.net)#
```

此时，我们直接运行该二进制文件，输入以下命令：

```
./daemon
```

运行成功后，控制台输出如下：

```
haicoder(www.haicoder.net)# ./daemon
haicoder(www.haicoder.net)#
```

此时，我们的程序就是守护进程了。

上下文切换

进程切换

1. 切换页目录以使用新的地址空间
2. 切换内核栈
3. 切换硬件上下文

线程切换

1. 切换内核栈
2. 切换硬件上下文

进程间通信方式

概述

进程通信(Interprocess Communication, IPC)是一个进程与另一个进程间共享消息的一种通信方式。消息(message)是发送进程形成的一个消息块，将消息内容传送给接收进程。

IPC 机制是消息从一个进程的地址空间拷贝到另一个进程的地址空间。

进程通信的目的

数据传输：一个进程需要将其数据发送给另一进程，发送的数据量在一个字节到几 M 字节之间。

共享数据：多个进程操作共享数据。

事件通知：一个进程需要向另一个或一组进程发送消息，通知它（它们）发生了某种事件（如进程终止时要通知父进程）。

资源共享：多个进程之间共享同样的资源。为了作到这一点，需要内核提供锁和同步机制。

进程控制：有些进程希望完全控制另一个进程的执行（如 Debug 进程），此时控制进程希望能够拦截另一个进程的所有陷入和异常，并能够及时知道它的状态改变。

Linux进程间通信（IPC）的发展

Linux 下的进程通信手段基本上是从 Unix 平台上的进程通信手段继承而来的。而对 Unix 发展做出重大贡献的两大主力 AT&T 的贝尔实验室及 BSD（加州大学伯克利分校的伯克利软件发布中心）在进程间通信方面的侧重点有所不同。

前者对 Unix 早期的进程间通信手段进行了系统的改进和扩充，形成了“system V IPC”，通信进程局限在单个计算机内；后者则跳过了该限制，形成了基于套接口（socket）的进程间通信机制。Linux 则把两者继承了下来。

- 早期 UNIX 进程间通信
- 基于 System V 进程间通信
- 基于 Socket 进程间通信
- POSIX 进程间通信

UNIX 进程间通信方式包括：管道、FIFO、信号。

System V 进程间通信方式包括：System V 消息队列、System V 信号灯、System V 共享内存

POSIX 进程间通信包括：posix 消息队列、posix 信号灯、posix 共享内存。

由于 Unix 版本的多样性，电子电气工程协会（IEEE）开发了一个独立的 Unix 标准，这个新的 ANSI Unix 标准被称为计算机环境的可移植性操作系统界面（PSOIX）。现有大部分 Unix 和流行版本都是遵循 POSIX 标准的，而 Linux 从一开始就遵循 POSIX 标准。

BSD 并不是没有涉足单机内的进程间通信（socket 本身就可以用于单机内的进程间通信）。事实上，很多 Unix 版本的单机 IPC 留有 BSD 的痕迹，如 4.4BSD 支持的匿名内存映射、4.3+BSD 对可靠信号语义的实现等等。

Linux使用的进程间通信方式

1. 管道（pipe）,流管道(s_pipe)和有名管道（FIFO）
2. 信号（signal）
3. 消息队列
4. 共享内存
5. 信号量
6. 套接字（socket）

管道(pipe)

管道这种通讯方式有两种限制，一是半双工的通信，数据只能单向流动，二是只能在具有亲缘关系的进程间使用。进程的亲缘关系通常是指父子进程关系。

流管道 s_pipe: 去除了第一种限制,可以双向传输。

管道可用于具有亲缘关系进程间的通信，命名管道：name_pipe 克服了管道没有名字的限制，因此，除具有管道所具有的功能外，它还允许无亲缘关系进程间的通信。

信号量(semaphore)

信号量是一个计数器，可以用来控制多个进程对共享资源的访问。它常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。因此，主要作为进程间以及同一进程内不同线程之间的同步手段。

信号是比较复杂的通信方式，用于通知接受进程有某种事件发生，除了用于进程间通信外，进程还可以发送信号给进程本身；linux 除了支持 Unix 早期信号语义函数 signal 外，还支持语义符合 Posix.1 标准的信号函数 sigaction（实际上，该函数是基于 BSD 的，BSD 为了实现可靠信号机制，又能够统一对外接口，用 sigaction 函数重新实现了 signal 函数）；

消息队列(message queue)

消息队列是由消息的链表，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺点。

消息队列是消息的链接表，包括 Posix 消息队列 system V 消息队列。有足够权限的进程可以向队列中添加消息，被赋予读权限的进程则可以读走队列中的消息。消息队列克服了信号承载信息量少，管道只能承载无格式字节流以及缓冲区大小受限等缺点。

信号 (singal)

信号是一种比较复杂的通信方式，用于通知接收进程某个事件已经发生。

主要作为进程间以及同一进程不同线程之间的同步手段。

共享内存(shared memory)

共享内存就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的 IPC 方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号量，配合使用，来实现进程间的同步和通信。

使得多个进程可以访问同一块内存空间，是最快的可用 IPC 形式。是针对其他通信机制运行效率较低而设计的。往往与其它通信机制，如信号量结合使用，来达到进程间的同步及互斥。

套接字(socket)

套接口也是一种进程间通信机制，与其他通信机制不同的是，它可用于不同机器间的进程通信

更为一般的进程间通信机制，可用于不同机器之间的进程间通信。起初是由 Unix 系统的 BSD 分支开发出来的，但现在一般可以移植到其它类 Unix 系统上：Linux 和 System V 的变种都支持套接字。

进程间通信各种方式效率比较

类型	无连接	可靠	流控制	优先级
普通PIPE	N	Y	Y	N
流PIPE	N	Y	Y	N
命名PIPE(FIFO)	N	Y	Y	N
消息队列	N	Y	Y	Y
信号量	N	Y	Y	Y
共享存储	N	Y	Y	Y
UNIX流SOCKET	N	Y	Y	N
UNIX数据包SOCKET	Y	Y	N	N

注:无连接: 指无需调用某种形式的OPEN,就有发送消息的能力流控制, 如果系统资源短缺或者不能接收更多消息,则发送进程能进行流量控制。

通信方式的比较和优缺点

1. 管道: 速度慢, 容量有限, 只有父子进程能通讯
2. FIFO: 任何进程间都能通讯, 但速度慢
3. 消息队列: 容量受到系统限制, 且要注意第一次读的时候, 要考虑上一次没有读完数据的问题
4. 信号量: 不能传递复杂消息, 只能用来同步
5. 共享内存区: 能够很容易控制容量, 速度快, 但要保持同步, 比如一个进程在写的时候, 另一个进程要注意读写的问题, 相当于线程中的线程安全, 当然, 共享内存区同样可以用作线程间通讯, 不过没这个必要, 线程间本来就已经共享了同一进程内的一块内存

如果用户传递的信息较少或是需要通过信号来触发某些行为, 前文提到的软中断信号机制不失为一种简洁有效的进程间通信方式。

但若是进程间要求传递的信息量比较大或者进程间存在交换数据的要求, 那就需要考虑别的通信方式了。

无名管道简单方便, 但局限于单向通信的工作方式, 并且只能在创建它的进程及其子孙进程之间实现管道的共享。

有名管道虽然可以提供给任意关系的进程使用, 但是由于其长期存在于系统之中, 使用不当容易出错, 所以普通用户一般不建议使用。

消息缓冲可以不再局限于父子进程, 而允许任意进程通过共享消息队列来实现进程间通信, 并由系统调用函数来实现消息发送和接收之间的同步, 从而使得用户在使用消息缓冲进行通信时不再需要考虑同步问题, 使用方便, 但是信息的复制需要额外消耗 CPU 的时间, 不适宜于信息量大或操作频繁的场所。

共享内存针对消息缓冲的缺点改而利用内存缓冲区直接交换信息, 无须复制, 快捷、信息量大是其优点。

但是共享内存的通信方式是通过将共享的内存缓冲区直接附加到进程的虚拟地址空间中来实现的, 因此, 这些进程之间的读写操作的同步问题操作系统无法实现。必须由各进程利用其他同步工具解决。另外, 由于内存实体存在于计算机系统中, 所以只能由处于同一个计算机系统内的诸进程共享。不方便网络通信。

共享内存块提供了在任意数量的进程之间进行高效双向通信的机制。每个使用者都可以读取写入数据, 但是所有程序之间必须达成并遵守一定的协议, 以防止诸如在读取信息之前覆写内存空间等竞争状态的出现。

不幸的是，Linux 无法严格保证提供对共享内存块的独占访问，甚至是在您通过使用 IPC_PRIVATE 创建新的共享内存块的时候也不能保证访问的独占性。同时，多个使用共享内存块的进程之间必须协调使用同一个键值。

进程间通信方式的选择

- PIPE 和 FIFO(有名管道)用来实现进程间相互发送非常短小的、频率很高的消息，这两种方式通常适用于两个进程间的通信。
- 共享内存用来实现进程间共享的、非常庞大的、读写操作频率很高的数据；这种方法适用于多进程间的通信。
- 其他考虑用 socket。主要应用在分布式开发中。

线程间通信方式

线程间通信方式主要包括消息队列、使用全局变量和使用事件。

消息队列

消息队列，是最常用的一种，也是最灵活的一种，通过自定义数据结构，可以传输复杂和简单的数据结构。

在 Windows 程序设计中，每一个线程都可以拥有自己的消息队列（UI 线程默认自带消息队列和消息循环，工作线程需要手动实现消息循环），因此可以采用消息进行线程间通信 `SendMessage`，`PostMessage`。

1. 定义消息 `#define WM_THREAD_SENDMSG=WM_USER+20;`
2. 添加消息函数声明 `afx_msg int OnTSendmsg();`
3. 添加消息映射 `ON_MESSAGE(WM_THREAD_SENDMSG, OnTSM);`
4. 添加 `OnTSM()` 的实现函数；
5. 在线程函数中添加 `PostMessage` 消息 `Post` 函数。

全局变量

进程中的线程间内存共享，这是比较常用的通信方式和交互方式。

注：定义全局变量时最好使用 `volatile` 来定义，以防编译器对此变量进行优化。

使用事件

使用事件 `CEvent` 类实现线程间通信，Event 对象有两种状态：有信号和无信号，线程可以监视处于有信号状态的事件，以便在适当的时候执行对事件的操作。

1. 创建一个 `CEvent` 类的对象：`CEvent threadStart;` 它默认处在未通信状态；
2. `threadStart.SetEvent();` 使其处于通信状态；
3. 调用 `WaitForSingleObject()` 来监视 `CEvent` 对象。

线程间同步方式

各个线程可以访问进程中的公共变量，资源，所以使用多线程的过程中需要注意的问题是如何防止两个或两个以上的线程同时访问同一个数据，以免破坏数据的完整性。数据之间的相互制约包括：

1. 直接制约关系，即一个线程的处理结果，为另一个线程的输入，因此线程之间直接制约着，这种关系可以称之为同步关系。
2. 间接制约关系，即两个线程需要访问同一资源，该资源在同一时刻只能被一个线程访问，这种关系称之为线程间对资源的互斥访问，某种意义上说互斥是一种制约关系更小的同步。

线程间的同步方式有四种

临界区

临界区对应着一个 CcriticalSection 对象，当线程需要访问保护数据时，调用 EnterCriticalSection 函数；当对保护数据的操作完成之后，调用 LeaveCriticalSection 函数释放对临界区对象的拥有权，以使另一个线程可以夺取临界区对象并访问受保护的数据。

PS: 关键段对象会记录拥有该对象的线程句柄即其具有“线程所有权”概念，即进入代码段的线程在 leave 之前，可以重复进入关键代码区域。所以关键段可以用于线程间的互斥，但不可以用于同步（同步需要在一个线程进入，在另一个线程 leave）

互斥量

互斥与临界区很相似，但是使用时相对复杂一些（互斥量为内核对象），不仅可以在同一应用程序的线程间实现同步，还可以在不同的进程间实现同步，从而实现资源的安全共享。

PS:

1. 互斥量由于也有线程所有权的概念，故也只能进行线程间的资源互斥访问，不能由于线程同步；
2. 由于互斥量是内核对象，因此其可以进行进程间通信，同时还具有一个很好的特性，就是在进程间通信时完美的解决了“遗弃”问题。

信号量

信号量的用法和互斥的用法很相似，不同的是它可以同一时刻允许多个线程访问同一个资源，PV 操作

PS: 事件可以完美解决线程间的同步问题，同时信号量也属于内核对象，可用于进程间的通信。

事件

事件分为手动置位事件和自动置位事件。事件 Event 内部它包含一个使用计数（所有内核对象都有），一个布尔值表示是手动置位事件还是自动置位事件，另一个布尔值用来表示事件有无触发。由 SetEvent() 来触发，由 ResetEvent() 来设成未触发。

PS: 事件是内核对象,可以解决线程间同步问题，因此也能解决互斥问题。

Linux进程状态

Linux进程状态教程

Linux 是一个多用户，多任务的系统，可以同时运行多个用户的多个程序，就必然会产生很多的进程，而每个进程会有不同的状态。

Linux 进程状态

状态	状态全称	描述
<i>R</i>	TASK_RUNNING	可执行状态
<i>S</i>	TASK_INTERRUPTIBLE	可中断的睡眠状态
<i>D</i>	TASK_UNINTERRUPTIBLE	不可中断的睡眠状态
<i>T</i>	TASK_STOPPED or TASK_TRACED	暂停状态或跟踪状态
<i>Z</i>	TASK_DEAD - EXIT_ZOMBIE	退出状态，进程成为僵尸进程
<i>X</i>	TASK_DEAD - EXIT_DEAD	退出状态，进程即将被销毁

Linux进程状态详解

R (TASK_RUNNING), 可执行状态

只有在该状态的进程才可能在 CPU 上运行。

而同一时刻可能有多个进程处于可执行状态，这些进程的 task_struct 结构（进程控制块）被放入对应 CPU 的可执行队列中（一个进程最多只能出现在一个 CPU 的可执行队列中）。

进程调度器的任务就是从各个 CPU 的可执行队列中分别选择一个进程在该 CPU 上运行。

S (TASK_INTERRUPTIBLE) 可中断的睡眠状态

处于这个状态的进程因为等待某某事件的发生（比如等待 socket 连接、等待信号量），而被挂起。

这些进程的 task_struct 结构被放入对应事件的等待队列中。当这些事件发生时（由外部中断触发、或由其他进程触发），对应的等待队列中的一个或多个进程将被唤醒。

通过 ps 命令我们会看到，一般情况下，进程列表中的绝大多数进程都处于 TASK_INTERRUPTIBLE 状态（除非机器的负载很高）。毕竟 CPU 就这么一两个，进程动辄几十上百个，如果不是绝大多数进程都在睡眠，CPU 又怎么响应得过来。

D (TASK_UNINTERRUPTIBLE) 不可中断的睡眠状态

与 TASK_INTERRUPTIBLE 状态类似，进程处于睡眠状态，但是此刻进程是不可中断的。不可中断，指的并不是 CPU 不响应外部硬件的中断，而是指进程不响应异步信号。

绝大多数情况下，进程处在睡眠状态时，总是应该能够响应异步信号的。否则你将惊奇的发现，kill -9 竟然杀不死一个正在睡眠的进程了！于是我们也很好理解，为什么 ps 命令看到的进程几乎不会出现 TASK_UNINTERRUPTIBLE 状态，而总是 TASK_INTERRUPTIBLE 状态。

而 TASK_UNINTERRUPTIBLE 状态存在的意义就在于，内核的某些处理流程是不能被打断的。如果响应异步信号，程序的执行流程中就会被插入一段用于处理异步信号的流程（这个插入的流程可能只存在于内核态，也可能延伸到用户态），于是原有的流程就被中断了。

在进程对某些硬件进行操作时（比如进程调用 read 系统调用对某个设备文件进行读操作，而 read 系统调用最终执行到对应设备驱动的代码，并与对应的物理设备进行交互），可能需要使用 TASK_UNINTERRUPTIBLE 状态对进程进行保护，以避免进程与设备交互的过程被打断，造成设备陷入不可控的状态。这种情况下的 TASK_UNINTERRUPTIBLE 状态总是非常短暂的，通过 ps 命令基本上不可能捕捉到。

然后我们可以试验一下 TASK_UNINTERRUPTIBLE 状态的威力。不管 kill 还是 kill -9，这个 TASK_UNINTERRUPTIBLE 状态的父进程依然屹立不倒。

T (TASK_STOPPED or TASK_TRACED), 暂停状态或跟踪状态

向进程发送一个 SIGSTOP 信号，它就会因响应该信号而进入 TASK_STOPPED 状态（除非该进程本身处于 TASK_UNINTERRUPTIBLE 状态而不响应信号）。（SIGSTOP 与 SIGKILL 信号一样，是非常强制的。不允许用户进程通过 signal 系列的系统调用重新设置对应的信号处理函数。）

向进程发送一个 SIGCONT 信号，可以让其从 TASK_STOPPED 状态恢复到 TASK_RUNNING 状态。

当进程正在被跟踪时，它处于 TASK_TRACED 这个特殊的状态。“正在被跟踪”指的是进程暂停下来，等待跟踪它的进程对它进行操作。比如在 gdb 中对被跟踪的进程下一个断点，进程在断点处停下来时就处于 TASK_TRACED 状态。而在其他时候，被跟踪的进程还是处于前面提到的那些状态。

对于进程本身来说，TASK_STOPPED 和 TASK_TRACED 状态很类似，都是表示进程暂停下来。

而 TASK_TRACED 状态相当于在 TASK_STOPPED 之上多了一层保护，处于 TASK_TRACED 状态的进程不能响应 SIGCONT 信号而被唤醒。只能等到调试进程通过 ptrace 系统调用执行 PTRACE_CONT、PTRACE_DETACH 等操作（通过 ptrace 系统调用的参数指定操作），或调试进程退出，被调试的进程才能恢复 TASK_RUNNING 状态。

Z (TASK_DEAD - EXIT_ZOMBIE)，退出状态，进程成为僵尸进程

进程在退出的过程中，处于 TASK_DEAD 状态。

在这个退出过程中，进程占有的所有资源将被回收，除了 task_struct 结构（以及少数资源）以外。于是进程就只剩下 task_struct 这么个空壳，故称为僵尸。之所以保留 task_struct，是因为 task_struct 里面保存了进程的退出码、以及一些统计信息。而其父进程很可能会关心这些信息。比如在 shell 中，\$? 变量 就保存了最后一个退出的前台进程的退出码，而这个退出码往往被作为 if 语句的判断条件。

当然，内核也可以将这些信息保存在别的地方，而将 task_struct 结构释放掉，以节省一些空间。但是使用 task_struct 结构更为方便，因为在内核中已经建立了从 pid 到 task_struct 查找关系，还有进程间的父子关系。释放掉 task_struct，则需要建立一些新的数据结构，以便让父进程找到它的子进程的退出信息。

父进程可以通过 wait 系列的系统调用（如wait4、waitid）来等待某个或某些子进程的退出，并获取它的退出信息。然后 wait 系列的系统调用会顺便将子进程的尸体（task_struct）也释放掉。

子进程在退出的过程中，内核会向其父进程发送一个信号，通知父进程来“收尸”。这个信号默认是 SIGCHLD，但是在通过 clone 系统调用创建子进程时，可以设置这个信号。

只要父进程不退出，这个僵尸状态的子进程就一直存在。那么如果父进程退出了呢，谁又来给子进程“收尸”？

当进程退出的时候，会将它的所有子进程都托管给别的进程（使之成为别的进程的子进程）。托管给谁呢？可能是退出进程所在进程组的下一个进程（如果存在的话），或者是 1 号进程。所以每个进程、每时每刻都有父进程存在。除非它是 1 号进程。1 号进程，pid 为 1 的进程，又称 init 进程。

Linux 系统启动后，第一个被创建的用户态进程就是 init 进程。它有两项使命：

- 执行系统初始化脚本，创建一系列的进程（它们都是 init 进程的子孙）；
- 在一个死循环中等待其子进程的退出事件，并调用 waitpid 系统调用来完成“收尸”工作；

init 进程不会被暂停、也不会被杀死（这是由内核来保证的）。它在等待子进程退出的过程中处于 TASK_INTERRUPTIBLE 状态，“收尸”过程中则处于 TASK_RUNNING 状态。

X (TASK_DEAD - EXIT_DEAD)，退出状态，进程即将被销毁

而进程在退出过程中也可能不会保留它的 task_struct。比如这个进程是多线程程序中被 detach 过的进程（进程？线程？参见《linux线程浅析》）。或者父进程通过设置 SIGCHLD 信号的 handler 为 SIG_IGN，显式的忽略了 SIGCHLD 信号。（这是 posix 的规定，尽管子进程的退出信号可以被设置为 SIGCHLD 以外的其他信号。）

此时，进程将被置于 EXIT_DEAD 退出状态，这意味着接下来的代码立即就会将该进程彻底释放。所以 EXIT_DEAD 状态是非常短暂的，几乎不可能通过 ps 命令捕捉到。

进程的初始状态

进程是通过 fork 系列的系统调用 (fork、clone、vfork) 来创建的，内核（或内核模块）也可以通过 kernel_thread 函数创建内核进程。

这些创建子进程的函数本质上都完成了相同的功能——将调用进程复制一份，得到子进程。（可以通过选项参数来决定各种资源是共享、还是私有。）

那么既然调用进程处于 TASK_RUNNING 状态（否则，它若不是正在运行，又怎么进行调用？），则子进程默认也处于 TASK_RUNNING 状态。

另外，在系统调用调用 clone 和内核函数 kernel_thread 也接受 CLONE_STOPPED 选项，从而将子进程的初始状态置为 TASK_STOPPED。

进程状态变迁

进程自创建以后，状态可能发生一系列的变化，直到进程退出。而尽管进程状态有好几种，但是进程状态的变迁却只有两个方向——从 TASK_RUNNING 状态变为非 TASK_RUNNING 状态、或者从非 TASK_RUNNING 状态变为 TASK_RUNNING 状态。

也就是说，如果给一个 TASK_INTERRUPTIBLE 状态的进程发送 SIGKILL 信号，这个进程将先被唤醒（进入 TASK_RUNNING 状态），然后再响应 SIGKILL 信号而退出（变为 TASK_DEAD 状态）。并不会从 TASK_INTERRUPTIBLE 状态直接退出。

进程从非 TASK_RUNNING 状态变为 TASK_RUNNING 状态，是由别的进程（也可能是中断处理程序）执行唤醒操作来实现的。执行唤醒的进程设置被唤醒进程的状态为 TASK_RUNNING，然后将其 task_struct 结构加入到某个 CPU 的可执行队列中。于是被唤醒的进程将有机会被调度执行。

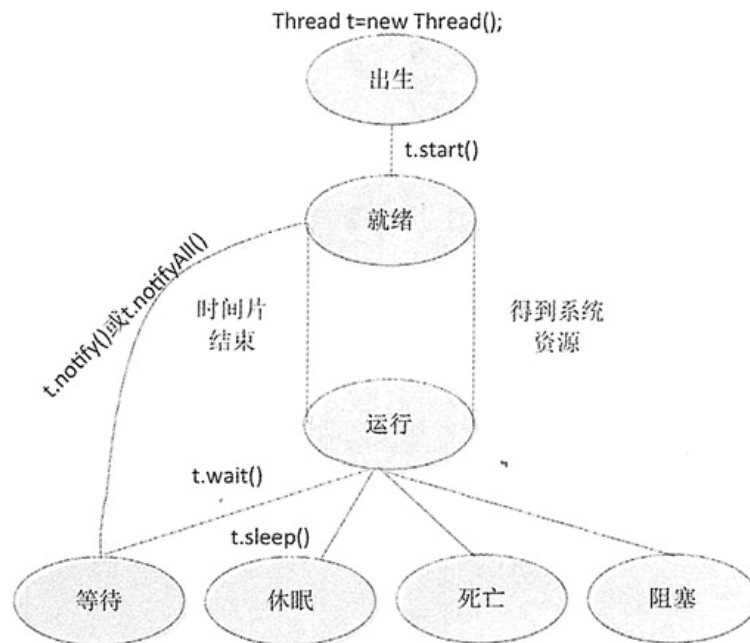
而进程从 TASK_RUNNING 状态变为非 TASK_RUNNING 状态，则有两种途径：

- 响应信号而进入 TASK_STOPPED 状态、或 TASK_DEAD 状态；
- 执行系统调用主动进入 TASK_INTERRUPTIBLE 状态（如 nanosleep 系统调用）、或 TASK_DEAD 状态（如 exit 系统调用）；或由于执行系统调用需要的资源得不到满足，而进入 TASK_INTERRUPTIBLE 状态或 TASK_UNINTERRUPTIBLE 状态（如 select 系统调用）。

显然，这两种情况都只能发生在进程正在 CPU 上执行的情况下。

线程的几种状态

线程也具有生命周期，主要包括 7 种状态，分别是出生状态、就绪状态、运行状态、等待状态、休眠状态、阻塞状态和死亡状态，如下图所示：



下面对线程生命周期中的 7 种状态做说明：

- 出生状态：用户在创建线程时所处的状态，在用户使用该线程实例调用 start() 方法之前，线程都处于出生状态。
- 就绪状态：也称可执行状态，当用户调用 start() 方法之后，线程处于就绪状态。
- 运行状态：当线程得到系统资源后进入运行状态。
- 等待状态：当处于运行状态下的线程调用 Thread 类的 wait() 方法时，该线程就会进入等待状态。进入等待状态的线程必须调用 Thread 类的 notify() 方法才能被唤醒。notifyAll() 方法是将所有处于等待状态下的线程唤醒。
- 休眠状态：当线程调用 Thread 类中的 sleep() 方法时，则会进入休眠状态。
- 阻塞状态：如果一个线程在运行状态下发出输入/输出请求，该线程将进入阻塞状态，在其等待输入/输出结束时，线程进入就绪状态。对阻塞的线程来说，即使系统资源关闭，线程依然不能回到运行状态。
- 死亡状态：当线程的 run() 方法执行完毕，线程进入死亡状态。

提示：一旦线程进入可执行状态，它会在就绪状态与运行状态下辗转，同时也可能进入等待状态、休眠状态、阻塞状态或死亡状态。

根据上图所示，可以总结出使线程处于就绪状态有如下几种方法：

- 调用 sleep() 方法。
- 调用 wait() 方法。
- 等待输入和输出完成。

当线程处于就绪状态后，可以用如下几种方法使线程再次进入运行状态：

- 线程调用 notify() 方法。
- 线程调用 notifyAll() 方法。
- 线程调用 interrupt() 方法。
- 线程的休眠时间结束。
- 输入或者输出结束。

进程调度

多道程序设计的目标是，无论何时都有进程运行，从而最大化 CPU 利用率。'

分时系统的目的是在进程之间快速切换 CPU，以便用户在程序运行时能与其交互。

为了满足这些目标，进程调度器选择一个可用进程（可能从多个可用进程集合中）到 CPU 上执行。

如果有多个进程，那么余下的需要等待 CPU 空闲并能重新调度。

进程调度的时机和方式

时机

进程调度的时机是什么呢？

也就是说，什么时候会从就绪队列中选取一个进程，分配处理机给它呢？

分为两种情况：当前进程主动放弃处理机 以及 当前进程被动放弃处理机。

- 当前进程主动放弃处理机：比如进程正常终止、运行过程中发生异常而终止、进程主动请求阻塞（如等待 I/O）
- 当前进程被动放弃处理机：比如进程的时间片用完、有更紧急的事需要处理（如 I/O 中断）、有更高优先级的进程进入就绪队列

方式

根据进程运行的过程中，处理机能否被其它进程抢占，将调度分为两种方式：

- **非抢占式**：“非抢占”即“不能抢占”。一旦把处理机分配给某个进程后，除非该进程终止或者主动要求进入阻塞态，否则会一直运行下去，不允许其它进程抢占自己占有的处理机。
- **抢占式**：把处理机分配给某个进程 A 后，如果有一个更重要、更紧急的进程 B 需要用到处理机，那么进程 A 会立即暂停，把处理机交给进程 B。

补充

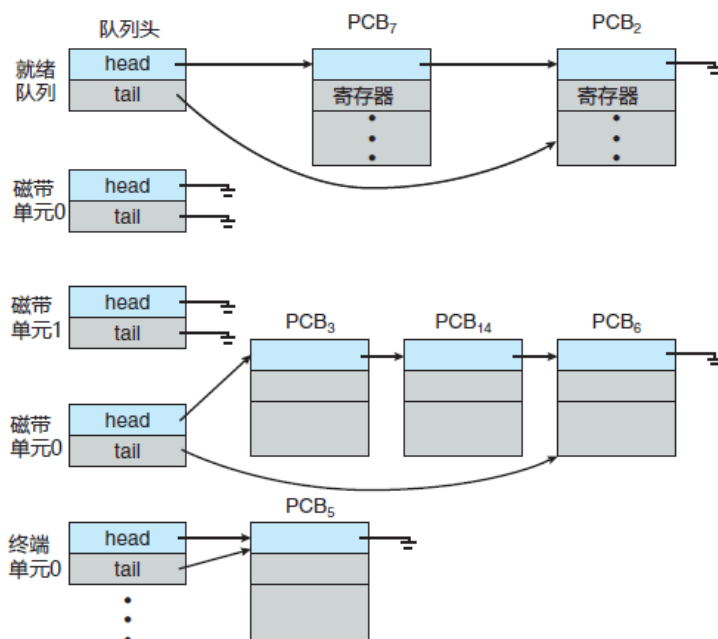
以下情况不会发生进程调度：

- 处理中断的时候：由于中断处理过程复杂，与硬件密切相关，很难做到在中断处理过程中进行进程切换。
- 进程在操作系统内核程序临界区的时候：注意是内核程序的临界区。普通临界区依然是有可能发生进程调度的。
- 进行原子操作的时候。

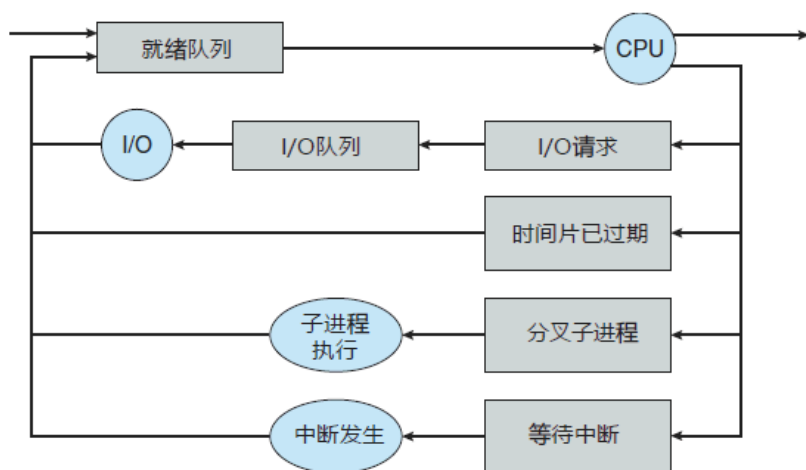
调度队列

进程在进入系统时，会被加到作业队列，这个队列包括系统内的所有进程。驻留在内存中的、就绪的、等待运行的进程保存在就绪队列上。就绪队列通常用链表实现；其头节点有两个指针，用于指向链表的第一个和最后一个 PCB 块；每个 PCB 还包括一个指针，指向就绪队列的下一个 PCB，如下图。

系统还有其他队列。当一个进程被分配了 CPU 后，它执行一段时间，最终退出，或被中断，或等待特定事件发生如 I/O 请求的完成。假设进程向一个共享设备如磁盘发出 I/O 请求。由于系统具有许多进程，磁盘可能忙于其他进程的 I/O 请求，因此该进程可能需要等待磁盘。等待特定 I/O 设备的进程列表，称为设备队列。每个设备都有自己的设备队列。



进程调度通常用队列图来表示，如下图所示。每个矩形框代表一个队列；这里具有两种队列：就绪队列和设备队列。圆圈表示服务队列的资源；箭头表示系统内的进程流向。



最初，新进程被加到就绪队列；它在就绪队列中等待，直到被选中执行或被分派。当该进程分配到 CPU 并执行时，以下事件可能发生：

- 进程可能发出 I/O 请求，并被放到 I/O 队列。
- 进程可能创建一个新的子进程，并等待其终止。
- 进程可能由于中断而被强制释放 CPU，并被放回到就绪队列。

对于前面两种情况，进程最终从等待状态切换到就绪状态，并放回到就绪队列。进程重复这一循环直到终止；然后它会从所有队列中删除，其 PCB 和资源也被释放。

调度程序

进程在整个生命周期中，会在各种调度队列之间迁移。操作系统为了调度必须按一定方式从这些队列中选择进程。进程选择通过适当调度器或调度程序来执行。

通常，对于批处理系统，提交的进程多于可以立即执行的。这些进程会被保存到大容量存储设备（通常为磁盘）的缓冲池，以便以后执行。长期调度程序（或作业调度程序）从该池中选择进程，加到内存，以便执行。短期调度程序（或 CPU 调度程序）从准备执行的进程中选择进程，并分配 CPU。

两种调度程序的主要区别是执行频率：

- 短期调度程序必须经常为 CPU 选择新的进程。进程可能执行几毫秒，就会等待 I/O 请求。通常，短期调度程序每 100ms 至少执行一次。由于执行之间的时间短，短期调度程序必须快速。如果花

费 10ms 来确定执行一个运行 100ms 的进程，那么 $10/(100 + 10) = 9\%$ 的 CPU 时间会用（浪费）在调度工作上。

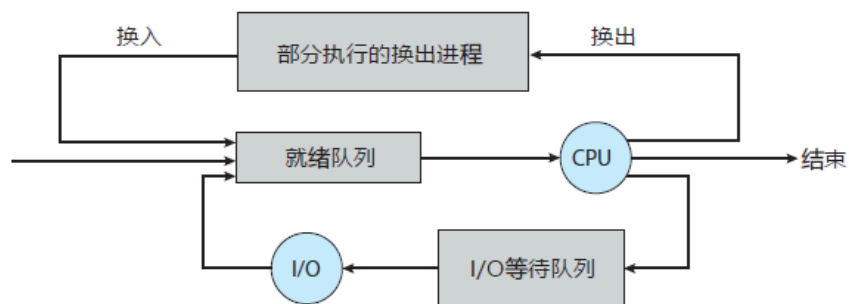
- 长期调度程序执行并不频繁；在新进程的创建之间，可能有几分钟间隔。长期调度程序控制多道程序程度（内存中的进程数量）。如果多道程序程度稳定，那么创建进程的平均速度必须等于进程离开系统的平均速度。因此，只有在进程离开系统时，才需要长期调度程序的调度。由于每次执行之间的更长时间间隔，长期调度程序可以负担得起更多时间，以便决定应该选择执行哪个进程。

重要的是，长期调度程序进行认真选择。通常，大多数进程可分为 I/O 为主或 CPU 为主，I/O 密集型进程执行 I/O 比执行计算需要花费更多时间。相反，CPU 密集型进程很少产生 I/O 请求，而是将更多时间用于执行计算。

长期调度程序应该选择 I/O 密集型和 CPU 密集型的合理进程组合。因为如果所有进程都是 I/O 密集型的，那么就绪队列几乎总是为空，从而短期调度程序没有什么可做。如果所有进程都是 CPU 密集型的，那么 I/O 等待队列几乎总是为空，从而设备没有得到使用，因而系统会不平衡。

有的系统，可能没有或极少采用长期调度程序。例如，UNIX 或微软 Windows 的分时系统通常没有长期调度程序，只是简单将所有新进程放于内存，以供短期调度程序使用。这些系统的稳定性取决于物理限制（如可用的终端数）或用户的自我调整。如果多用户系统性能下降到令人难以接受，那么有的用户就会退出。

有的操作系统如分时系统，可能引入一个额外的中期调度程序，如下图所示：



中期调度程序的核心思想是可将进程从内存（或从 CPU 竞争）中移出，从而降低多道程序程度。之后，进程可被重新调入内存，并从中断处继续执行。这种方案称为交换。

通过中期调度程序，进程可换出，并在后来可换入。为了改善进程组合，或者由于内存需求改变导致过度使用内存从而需要释放内存，就有必要使用交换。

上下文切换

前面提过，中断会导致 CPU 从执行当前任务改变到执行内核程序。这种操作在通用系统中经常发生。当中断发生时，系统需要保存当前运行在 CPU 上的进程的上下文，以便在处理后能够恢复上下文，即先挂起进程，再恢复进程。

切换 CPU 到另一个进程需要保存当前进程状态和恢复另一个进程的状态，这个任务称为上下文切换。

当进行上下文切换时，内核会将旧进程状态保存在其 PCB 中，然后加载经调度而要执行的新进程的上下文。上下文切换的时间是纯粹的开销，因为在切换时系统并没有做任何有用工作。上下文切换的速度因机器不同而有所不同，它依赖于内存速度、必须复制的寄存器数量、是否有特殊指令（如加载或存储所有寄存器的单个指令）。典型速度为几毫秒。

上下文切换的时间与硬件支持密切相关。例如，有的处理器（如 Sun UltraSPARC）提供了多个寄存器组，上下文切换只需简单改变当前寄存器组的指针。当然，如果活动进程数量超过寄存器的组数，那么系统需要像以前一样在寄存器与内存之间进行数据复制。

不仅如此，操作系统越复杂，上下文切换所要做的就越多，高级的内存管理技术在每次上下文切换时，所需切换的数据会更多。例如，在使用下一个进程的地址空间之前，需要保存当前进程的地址空间。如何保存地址空间，需要做什么才能保存等，取决于操作系统的内存管理方法。

处理机调度的三个层次

定义

调度研究的问题是：

面对有限的资源，如何处理任务执行的先后顺序。对于处理机调度来说，这个资源就是有限的处理机，而任务就是多个进程。

故处理机调度研究的问题是：面对有限的处理机，如何从就绪队列中按照一定的算法选择一个进程并将处理机分配给它运行，从而实现进程的并发执行。

处理机调度共有三个层次，这三个层次也是一个作业从提交开始到完成所经历的三个阶段。

三个层次

作业调度

作业调度也即高级调度，这个阶段可以看作是准备阶段。主要任务是按照一定的规则从外存上处于后备队列的作业中挑选一个或多个作业，为其分配内存，建立 PCB（进程）等，使它们具备竞争处理机的能力。

这个阶段进程的状态变化是：无 -> 创建态 -> 就绪态

内存调度

内存调度也即中级调度，这个阶段可以看作是优化阶段。主要任务是将暂时不能运行的进程对换到外存中，使它们挂起；而当挂起的进程具备运行条件时，它们会被重新对换回内存，得到激活。这个阶段的主要目的是提高内存利用率和系统吞吐量。

这个阶段进程的状态变化是：静止就绪态 -> 活动就绪态，静止阻塞态 -> 活动阻塞态

进程调度

进程调度即低级调度，这个阶段让进程真正运行起来。主要任务是按照某种算法，从就绪队列中选取一个进程，分配处理机给它。进程调度是最基本、次数最频繁的阶段。

这个阶段进程的状态变化是：就绪态 -> 活动态

进程调度算法

评价指标

- CPU 利用率：忙碌的时间 / 总时间
- 系统吞吐量：完成作业量 / 总时间
- 周转时间：作业完成时间 - 作业提交时间 = 作业实际运行的时间 + 等待时间
- 平均周转时间：各作业周转时间之和 / 作业数
- 带权周转时间：周转时间 / 作业实际运行的时间
- 平均带权周转时间：各作业带权周转时间之和 / 作业数
- 等待时间：进程或者作业处于等待处理机状态的时间之和，即 周转时间 - 作业实际运行的时间
 - 对于进程来说，等待时间指的是进程建立后等待被服务的时间之和（由于等待 I/O 完成的期间也属于被服务时间，所以这个时间不计入等待时间）
 - 对于作业来说，除了进程建立后的等待时间，还包括作业在外存后备队列中等待的时间

平均等待时间：各作业等待时间之和 / 作业数

响应时间：从用户提交请求到首次产生响应所用的时间

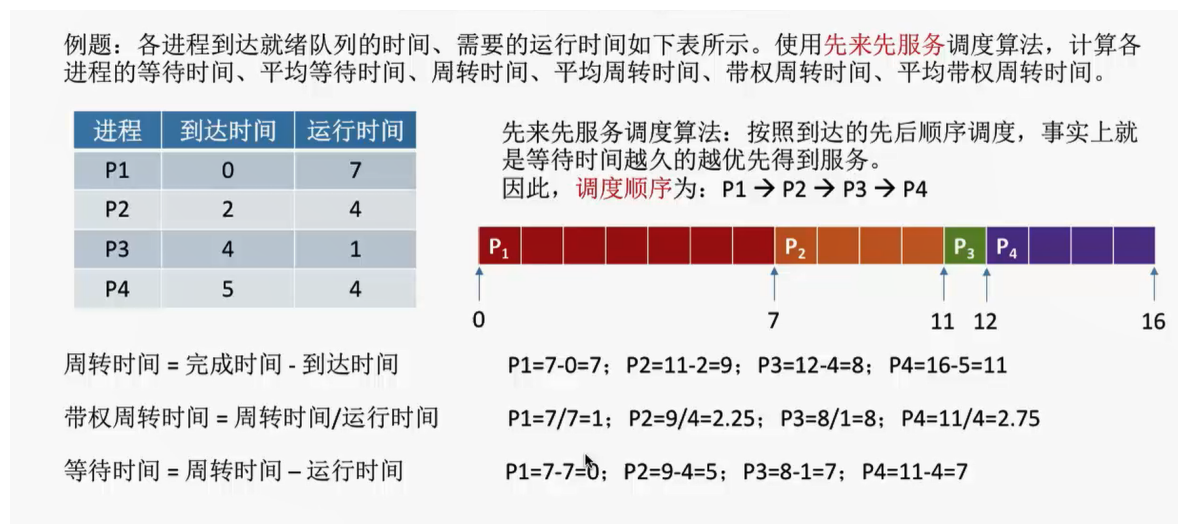
早期批处理系统的调度算法

先来先服务调度算法（FCFS）

FCFS 算法即“先来先服务”算法，类似于我们生活中的排队，谁先来，谁就先享受服务。

对于作业调度，它指的是谁先到达后备队列，谁就先出队，进而先被执行；对于进程调度，它指的是谁先到达就绪队列，谁就先出队，进而先被执行。

看下面的例子：



这个就是很自然的谁先到达，谁就先享受服务，所以顺序上就是从 P1 到 P4。注意这里的到达时间，就是前面说过的提交时间。这里不考虑等待 I/O 的情况，否则计算等待时间的时候还需要减去等待 I/O 的时间。

- FCFS 算法是非抢占式的算法，不存在某个进程在执行的时候被其它进程抢占处理机的情况。
- 它的优点是公平、算法实现简单，并且不会导致饥饿（不管等多久，所有进程最后都会运行，不存在某个进程永远得不到处理机的情况）
- 缺点是对长作业有利、对短作业不利 —— 对于长作业，如果它先到，那么它自然无需做过多的等待，而即使是后到，它等待短作业的时间也是不足挂齿的，所以长作业怎么都不亏；对于短作业，如果它先到，自然也无需做过多等待，但是如果它后到，那么它不得不花很长的时间去等待长作业完成，然而它自己运行所需的时间却是很短的，所以说这个算法对短作业不利。在这种情况下，短作业的带权周转时间会很大，也即周转时间远远大于实际运行时间，表示有大量时间用于等待。
- 有时候也说 FCFS 算法对 CPU 繁忙型作业有利，对 I/O 繁忙型作业不利。这是因为 CPU 繁忙型作业的特点是需要大量的 CPU 时间进行计算，而很少请求 I/O 操作，通常视作长作业。

短作业优先（SJF）调度算法

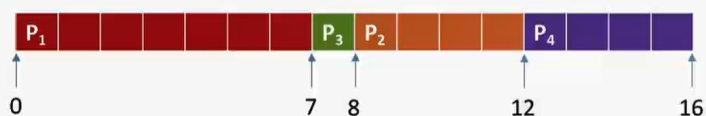
SJF 算法即“短作业优先”算法，前面的算法问题在于对短作业不利，所以 SJF 算法优先顾及短作业，让当前已到达并且运行时间最短的进程先执行。SJF 算法有非抢占式（默认）版本和抢占式版本，抢占式版本也叫做 SRTN 算法，即最短剩余时间优先算法。

先看非抢占式版本的例子：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**非抢占式**的**短作业优先**调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

短作业/进程优先调度算法：每次调度时选择**当前已到达且运行时间最短**的作业/进程。
因此，**调度顺序**为：P1 → P3 → P2 → P4



周转时间 = 完成时间 - 到达时间

P1=7-0=7; P3=8-4=4; P2=12-2=10; P4=16-5=11

带权周转时间 = 周转时间/运行时间

P1=7/7=1; P3=4/1=4; P2=10/4=2.5; P4=11/4=2.75

等待时间 = 周转时间 - 运行时间

P1=7-7=0; P3=4-1=3; P2=10-4=6; P4=11-4=7

平均周转时间 = (7+4+10+11)/4 = 8

平均带权周转时间 = (1+4+2.5+2.75)/4 = 2.56

平均等待时间 = (0+3+6+7)/4 = 4

8.75
3.5
4.75

对比FCFS算法的结果，显然SJF算法的平均等待/周转/带权周转时间都要更低

运行顺序的说明：

注意这里虽然 P1 不是运行时间最短的，但是它是 **当前最先到达且运行时间最短** 的进程，所以它首先运行，并且在运行过程中，P2, P3, P4 陆续到达就绪队列。在 P1 运行完之后就需调度了，这时候，就绪队列中满足“当前已到达且运行时间最短”的进程是 P3，所以 P3 运行；P3 运行完之后继续调度其它进程，P2 和 P4 运行时间都一样，不过 P2 首先到达，所以 P2 运行，最后再轮到 P4 运行。

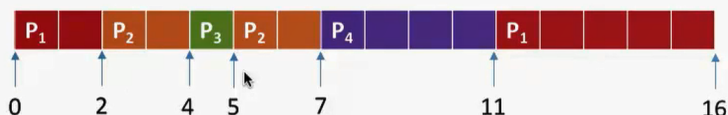
另外，由于这是非抢占式版本，所以除非进程终止或者其它原因，否则其它进程是无法与当前进程竞争处理机的。

接着看抢占式版本的例子：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**抢占式**的**短作业优先**调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

最短剩余时间优先算法：每当有进程加入**就绪队列改变时就需要调度**，如果新到达的进程**剩余时间**比当前运行的进程**剩余时间更短**，则由新进程**抢占**处理机，当前运行进程重新回到就绪队列。另外，当一个**进程完成时也需要调度**



需要注意的是，当有新进程到达时就绪队列就会改变，就要按照上述规则进行检查。以下 $P_n(m)$ 表示当前 P_n 进程剩余时间为 m 。各个时刻的情况如下：

0时刻（P1到达）：P1（7）

2时刻（P2到达）：P1（5）、P2（4）

4时刻（P3到达）：P1（5）、P2（2）、P3（1）

5时刻（P3完成且P4刚好到达）：P1（5）、P2（2）、P4（4）

7时刻（P2完成）：P1（5）、P4（4）

11时刻（P4完成）：P1（5）

多了一个调度条件：

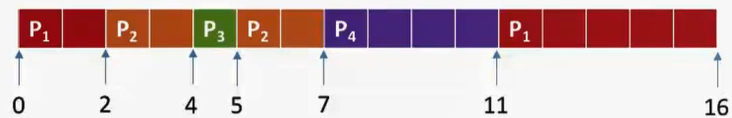
由于这是抢占式版本，所以存在着进程之间对于处理机的竞争。也就是说，除了进程正常终止会发生调度之外，每次有新进程进入就绪队列的时候，也可能发生调度。而具体谁会被调度并夺得处理机，则是比较新到达进程的剩余时间与正在运行进程的剩余时间，前者如果更短，那么它将夺得处理机。

下面是抢占式版本的相关指标计算：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**抢占式的短作业优先**调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

最短剩余时间优先算法：每当有进程加入就绪队列改变时就需要调度，如果新到达的进程**剩余时间**比当前运行的进程剩余时间**更短**，则由新进程**抢占**处理机，当前运行进程重新回到就绪队列。另外，当一个**进程完成时**也需要调度。



周转时间 = 完成时间 - 到达时间

$P1=16-0=16$; $P2=7-2=5$; $P3=5-4=1$; $P4=11-5=6$

带权周转时间 = 周转时间/运行时间

$P1=16/7=2.28$; $P2=5/4=1.25$; $P3=1/1=1$; $P4=6/4=1.5$

等待时间 = 周转时间 - 运行时间

$P1=16-7=9$; $P2=5-4=1$; $P3=1-1=0$; $P4=6-4=2$

平均周转时间 = $(16+5+1+6)/4 = 7$

平均带权周转时间 = $(2.28+1.25+1+1.5)/4 = 1.50$

平均等待时间 = $(9+1+0+2)/4 = 3$

8
2.56
4

对比非抢占式的短作业优先算法，显然抢占式的这几个指标又要更低

注意：

一般可以认为，SJF 算法的平均等待时间、平均周转时间都是最少的（相比于其它算法），但是更准确地说，其实它的抢占式版本，也即 SRTN 算法，各项指标要比 SJF 算法更低。

- SJF 算法的优点在于，它拥有“最短的”平均等待时间和平均周转时间
- 缺点在于，虽然这次顾及了短作业，但是没有顾及长作业，对长作业是不利的。因为一旦短作业源源不断进入，那么它们就会不断跑在长作业前面，导致长作业永远无法运行，产生“饥饿”甚至“饿死”现象。
- 另外一个缺点是，在实际实现中，要做到真正意义上的短作业优先，具有一定难度。

HRRN 算法

HRRN 算法即高响应比优先算法，它优先调度响应比高的进程。

响应比 = (等待时间 + 实际运行时间) / 实际运行时间 = 等待时间 / 实际运行时间 + 1

可以说它同时综合了 FCFS 算法和 SJF 算法的优点。为什么优先调度响应比高的进程？因为当两个进程的等待时间一样时，响应比越高的进程，它的实际运行时间越短，这一点类似于 SJF 算法，优先顾及运行时间短的进程；而当两个进程的实际运行时间一样时，响应比越高的进程，它的等待时间越长，等待时间越长说明该进程越先到达，这一点类似于 FCFS 算法，优先顾及先到达的进程。

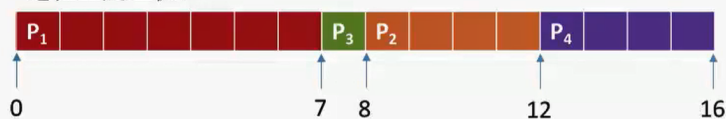
HRRN 是非抢占式的算法，因此只有当前运行进程正常放弃处理机的时候，才会计算哪个进程的响应比高，然后进行调度。

看下面的例子：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**高响应比优先**调度算法，计算各进程的等待时间、平均等待时间、周转时间、平均周转时间、带权周转时间、平均带权周转时间。

进程	到达时间	运行时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

高响应比优先算法：非抢占式的调度算法，只有当前运行的进程**主动放弃CPU**时（正常/异常完成，或主动阻塞），才需要进行调度，调度时**计算所有就绪进程的响应比**，**选响应比最高的进程**上处理机。



$$\text{响应比} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

P2和P4要求服务时间一样，但P2等待时间长，所以必然是P2响应比更大

0时刻：只有 P₁ 到达就绪队列，P₁ 上处理机

7时刻（P₁主动放弃CPU）：就绪队列中有 P₂ (响应比=(5+4)/4=2.25)、P₃ ((3+1)/1=3)、P₄ ((2+4)/4=1.5)，

8时刻（P₃完成）：P₂(2.5)、P₄(1.75)

12时刻（P₂完成）：就绪队列中只剩下 P₄

注意这里“要求服务的时间”就是实际需要运行的时间，等待时间则是从进程到达就绪队列的那一刻起，到发生进程调度这一段所花费的时间。

HRRN 算法的优点是综合考虑了等待时间和实际运行时间，而且也不会导致长作业饥饿的问题（因为长作业等待时间变长之后，它的响应比也会变高，增加了可以被调度的机会）。

总结

上面这几种算法主要关注对用户的公平性、平均周转时间、平均等待时间等评价系统整体性能的指标，但是不关心“响应时间”，也并不区分任务的紧急程度，因此对于用户来说，交互性很糟糕。

因此它们一般适合用于早期的批处理系统。下面介绍的算法则适合用于交互式系统。

交互式系统的调度算法

RR算法

RR 算法即时间片轮转算法。像前面的算法的话，通常都是非抢占式的，也就是说，一个进程正常运行完，另一个进程才有机会被调度，整体呈现出“顺序”的特点；而 RR 算法的特点则在于“公平分配”，按照进程到达就绪队列的顺序，轮流让每个进程执行一个相等长度的时间片，若在自己的时间片内没有执行完，则进程自动进入就绪队列队尾，并调度队头进程运行。整体呈现出“交替”的特点。因为进程即使没运行完也会发生调度，所以这是一个抢占式的算法。

看下面的例子：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用**时间片轮转**调度算法，分析时间片大小分别是2、5时的进程运行情况。

进程	到达时间	运行时间
P1	0	5
P2	2	4
P3	4	1
P4	5	6

时间片轮转调度算法：轮流让就绪队列中的进程依次执行一个时间片（每次选择的都是排在就绪队列队头的进程）

先来看时间片为 2 的情况：

0 时刻：此时就绪队列为 P1(5)，P1 上处理机运行

2 时刻：P2 到达就绪队列队头，同时 P1 时间片用完，到达就绪队列队尾。此时就绪队列为 P2(4) —— P1(3)，P2 被调度，上处理机运行。

4 时刻： P3 到达就绪队列队尾，同时 P2 时间片用完，进入就绪队列，紧挨在 P3 后面。此时就绪队列为 P1(3) —— P3(1) —— P2(2)，P1 被调度，上处理机运行。

5 时刻： P4 到达就绪队列队尾，P1 时间片还没用完，仍然在运行。此时就绪队列为 P3(1) —— P2(2) —— P4(6)

6 时刻： P1 时间片用完，进入就绪队列队尾，此时就绪队列为 P3(1) —— P2(2) —— P4(6) —— P1(1)。P3 被调度，上处理机运行。

7 时刻： 虽然 P3 有 2 个单位的时间片可用，但是它实际上只需要用到一个单位，所以 7 时刻的时候它正常运行完，轮到 P2 被调度。此时就绪队列为 P4(6) —— P1(1)。

9 时刻： P2 时间片用完，同时也正常运行结束。P4 被调度，上处理机运行。此时就绪队列为 P1(1)。

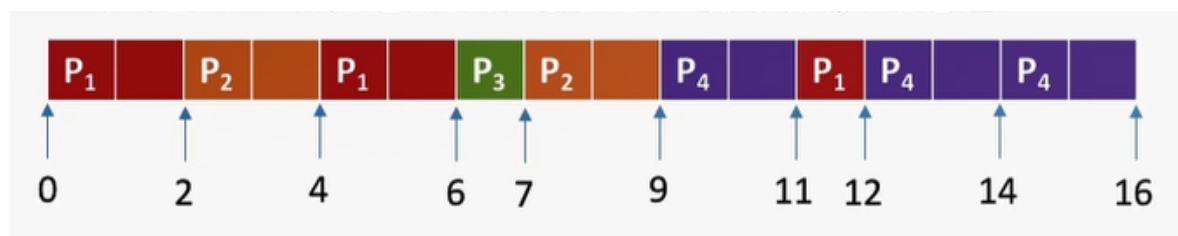
11 时刻： P4 时间片用完，到达就绪队列队尾。此时就绪队列为 P1(1) —— P4(4)。P1 被调度，上处理机运行。

12 时刻： 在 12 时刻的时候，P1 就已经运行结束。此时再次调度 P4 上处理机运行

14 时刻： P4 时间片用完，由于就绪队列中没有其它进程可供调度，所以让 P4 接着运行一个时间片

16 时刻： P4 正常运行结束。

整个过程如下图所示：



再来看时间片为 5 的情况：

0 时刻： 此时就绪队列为 P1(5)，P1 上处理机运行

2 时刻： P2 到达就绪队列队头，P1 仍在运行

4 时刻： P3 到达就绪队列队尾，P1 仍在运行

5 时刻： P4 到达就绪队列队尾。P1 正常运行结束，时间片刚好用完。此时就绪队列是 P2(4) —— P3(1) —— P4(6)，所以 P2 被调度上处理机

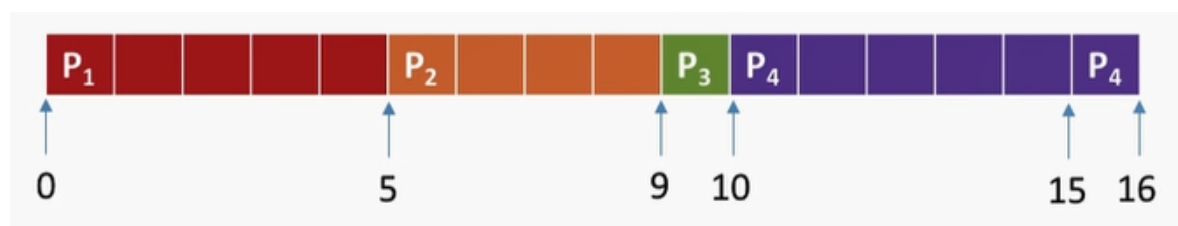
9 时刻： 尽管时间片没有用完，但是 P2 正常运行结束，所以 P3 会被调度上处理机

10 时刻： P3 正常运行结束，同样调度 P4

15 时刻： P4 时间片用完，但是就绪队列没有可供调度的进程，所以 P4 还得继续运行

16 时刻： P4 正常运行结束

整个过程如下图所示：



这里会发现，效果和使用 FCFS 算法是差不多的。实际上，如果时间片太大，那么 RR 算法会退化成 FCFS 算法，而且会增加进程响应时间，所以时间片应该设置得小一点；另一方面，时间片也不能设置得太小，否则进程切换会过于频繁，导致更多的时间用于切换而不是有效执行进程。

总的来说，RR 算法的优点是公平、响应快，适用于分时操作系统；缺点则是进程切换频率相比其他算法会高一点，因此有一定的开销。另外它不区分任务的紧急程度，再紧急的任务，如果某个运行进程的时间片还没用完，这个任务也不会被调度。

RR 算法不会导致饥饿，因为时间片一到自然会切换到其它进程，不存在某个进程永远无法被调度的情况。

优先级算法

优先级算法在某种程度上和 HRRN 算法很像，两者可以联系起来进行理解。

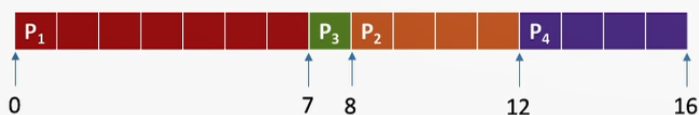
前面我们所讲的算法都无法区分进程紧急程度，而优先级算法弥补了这个问题。它会给每个进程一个优先级，调度时会选择当前已到达并且优先级最高的进程。和 HRRN 算法一样，它也有非抢占式和抢占式两个版本。

先看非抢占式版本：

例题：各进程到达就绪队列的时间、需要的运行时间、进程优先数如下表所示。使用**非抢占式**的**优先级**调度算法，分析进程运行情况。（注：优先数越大，优先级越高）

进程	到达时间	运行时间	优先数
P1	0	7	1
P2	2	4	2
P3	4	1	3
P4	5	4	2

非抢占式的优先级调度算法：每次调度时选择**当前已到达且优先级最高**的进程。当前进程**主动放弃处理机**时发生调度。



注：以下括号内表示当前处于就绪队列的进程

0时刻（P1）：只有P1到达，P1上处理机。

7时刻（P2、P3、P4）：P1运行完成主动放弃处理机，其余进程都已到达，P3优先级最高，P3上处理机。

8时刻（P2、P4）：P3完成，P2、P4优先级相同，由于P2先到达，因此P2优先上处理机

12时刻（P4）：P2完成，就绪队列只剩P4，P4上处理机。

16时刻（）：P4完成，所有进程都结束

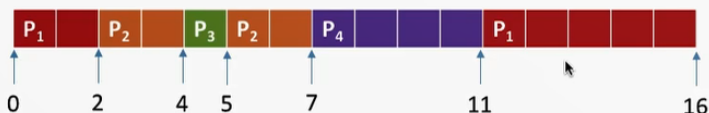
这里和 HRRN 算法是很像的，进程会正常运行，直到结束之后才发生调度，在调度的时候会选择队列中优先级最高的进程。

再看抢占式版本：

例题：各进程到达就绪队列的时间、需要的运行时间、进程优先数如下表所示。使用**抢占式**的**优先级**调度算法，分析进程运行情况。（注：优先数越大，优先级越高）

进程	到达时间	运行时间	优先数
P1	0	7	1
P2	2	4	2
P3	4	1	3
P4	5	4	2

抢占式的优先级调度算法：每次调度时选择**当前已到达且优先级最高**的进程。当前进程**主动放弃处理机**时发生调度。另外，当**就绪队列发生改变**时也需要检查是会发生抢占。



注：以下括号内表示当前处于就绪队列的进程

0时刻（P1）：只有P1到达，P1上处理机。

2时刻（P2）：P2到达就绪队列，优先级比P1更高，发生抢占。P1回到就绪队列，P2上处理机。

4时刻（P1、P3）：P3到达，优先级比P2更高，P2回到就绪队列，P3抢占处理机。

5时刻（P1、P2、P4）：P3完成，主动释放处理机，同时，P4也到达，由于P2比P4更先进入就绪队列，因此选择P2上处理机

7时刻（P1、P4）：P2完成，就绪队列只剩P1、P4，P4上处理机。

11时刻（P1）：P4完成，P1上处理机

这里同样和 HRRN 算法很像。除了正常运行结束会发生调度之外，每次就绪队列有新的进程到达时还会做一次检查，如果新到达进程优先级高于正在运行进程的优先级，那么新到达进程会抢占处理机。

PS：在优先级算法中，就绪队列可能不止有一个，可以按照不同优先级分成很多种队列。另外还要注意，有的地方规定优先数越小，优先级越高，具体看题目要求。

静态优先级和动态优先级：

优先级还包括静态优先级和动态优先级。上面所讲的属于静态优先级，指的是进程的优先级在它创建的时候就确定了，此后一直不会改变；动态优先级则相对灵活很多，它会根据具体情况动态调整进程的优先级。

- 对于静态优先级，一般认为系统进程优先级要高于用户进程优先级；前台进程优先级高于后台进程优先级；I/O 型进程优先级会比较高。
- 于动态优先级，会尽量遵循公平的原则。也就是说，如果某个进程实在等得太久，那么不妨提高它的优先级，让他有机会被调度；反之，如果某个进程占用处理机时间过长，那么就要考虑降低它的优先级，不要让他一直“霸占”处理机了。另外，之前说过 I/O 型进程的优先级会很高，所以如果某个进程频繁进行 I/O 操作，也可以考虑提高它的优先级。

总的来说，优先级算法的优点在于区分了各个进程的紧急程度，比较紧急重要的进程会优先得到处理，因此它适用于实时操作系统。另外，由于动态优先级的存在，使得它对进程的调度相对灵活很多。缺点则是，如果源源不断进来了一些高优先级的进程，那么优先级相对较低的进程可能一直无法执行，进而导致饥饿现象的发生。这点和 HRRN 算法也是很像的。（其实也可以把 HRRN 算法看作优先级算法的一种特殊情况，将响应比作为优先级评判的标准）

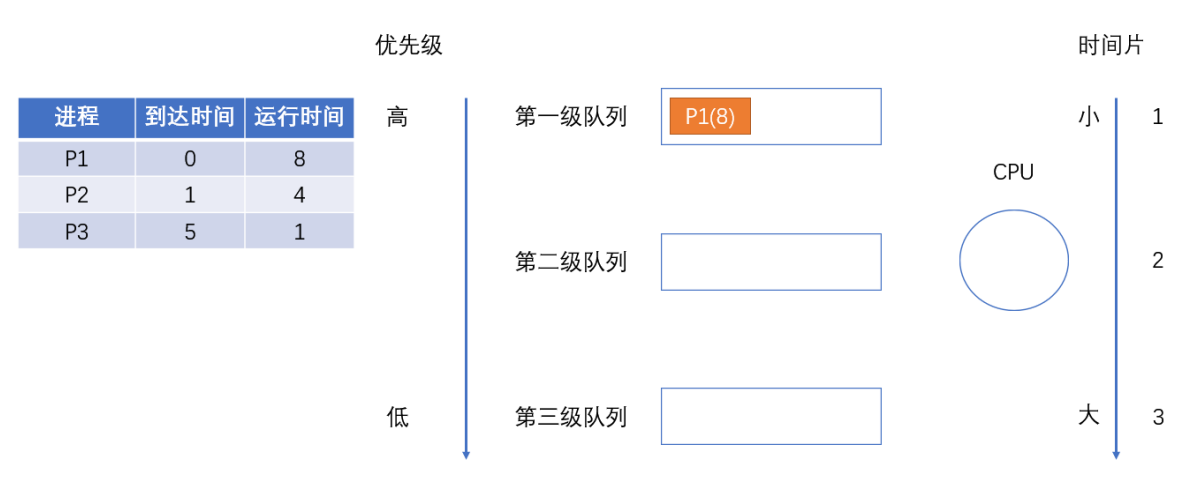
多级反馈队列算法

多级反馈队列算法是对其他调度算法的折中权衡，它的分析过程会复杂很多。下面我们先给定多级反馈队列算法的几个规则，再结合图片文字理一理具体的过程。

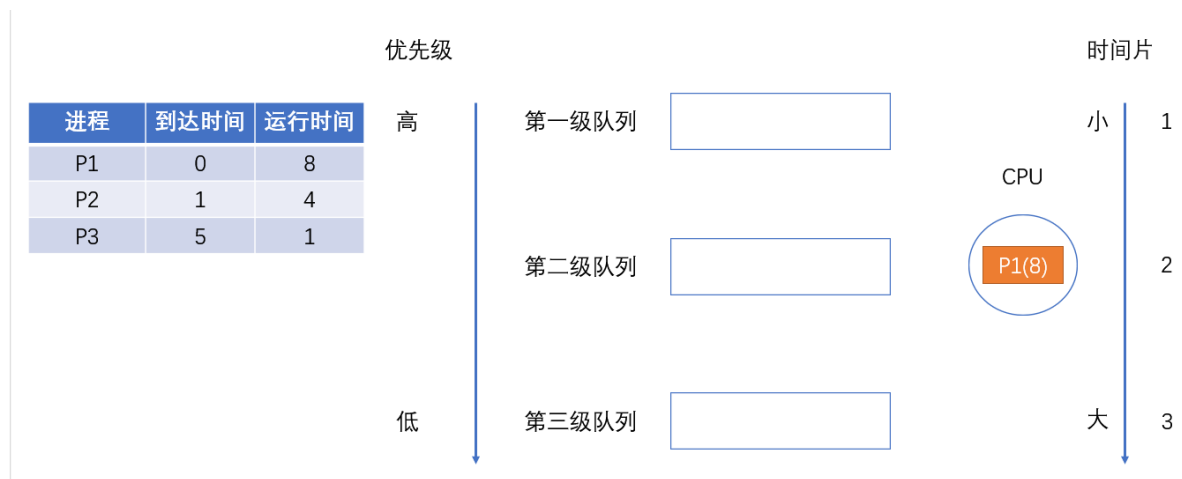
- 有多个级别的就绪队列，各级队列优先级从高到低，时间片从小到大
- 每次有新进程到达，都会首先进入第一级队列，并按 FCFS 算法被分配时间片。如果时间片用完了而进程还没执行完，那么该进程将被送到下一级队列队尾。如果当前已经是最后一级，则重新放回当前队列队尾
- 当且仅当上层级别的队列为空时，下一级队列的进程才有机会被调度
- 关于抢占：如果某个进程运行的时候，比它所在队列级别更高的队列中有新进程到达，则那个新进程会抢占处理机，而当前正在运行的进程会被送到当前队列队尾

下面我们结合图片来进行理解。

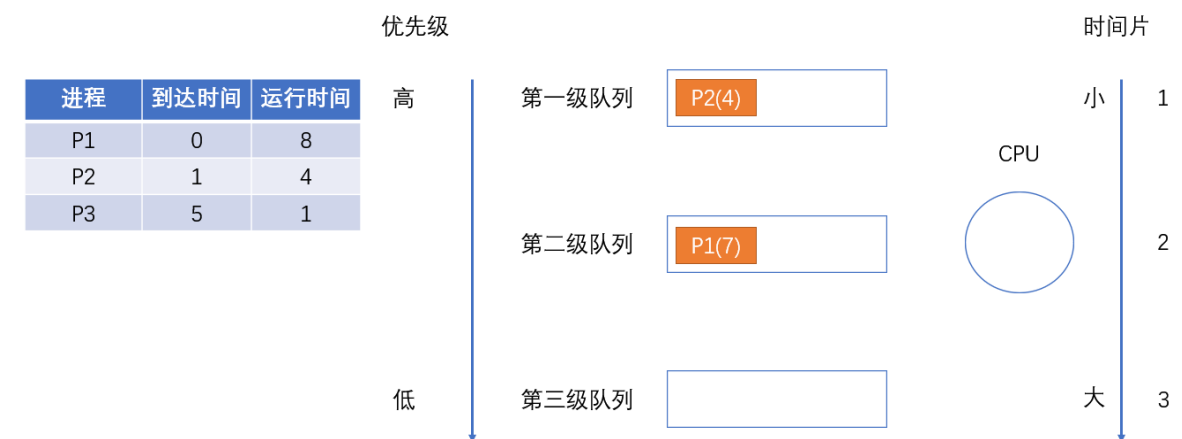
在 0 时刻，P1 首先到达第一级就绪队列



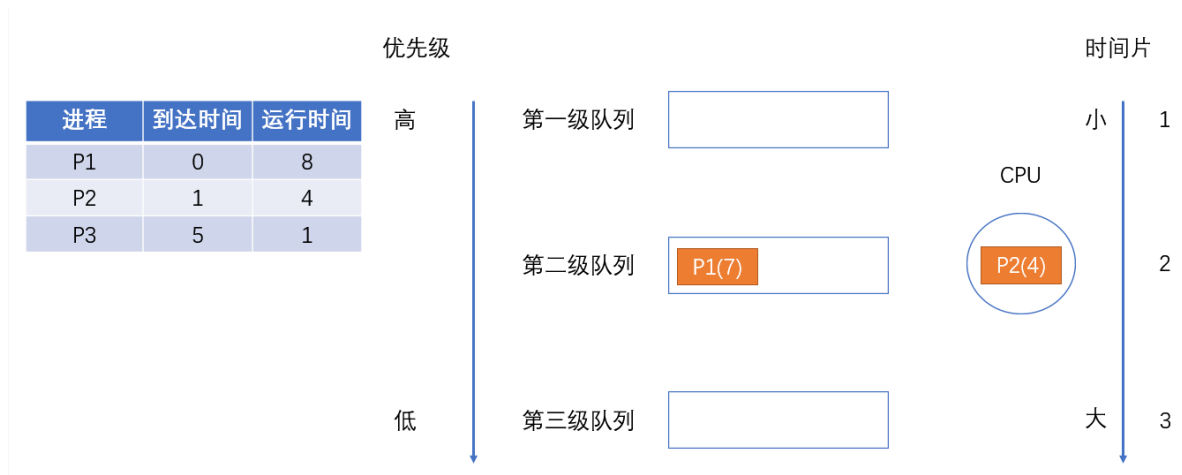
然后，它被调度，来到了处理机这里



在 1 时刻，P1 时间片已经用完，但是进程还没执行完，所以这时候 P1 “降级”进入第二级就绪队列。同时，P2 作为新进程进入第一级就绪队列



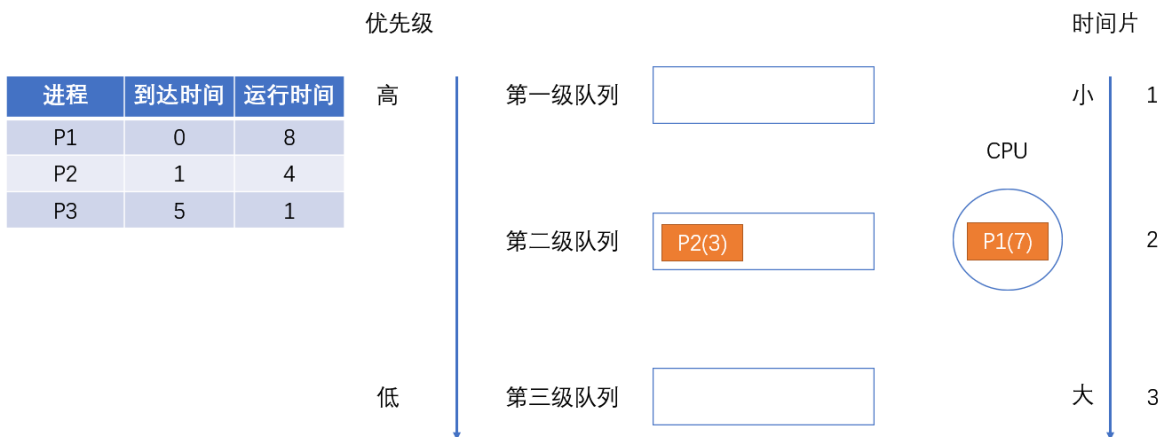
P2 被调度进入处理机



在 2 时刻，P2 时间片已经用完，但是进程还没执行完，所以这时候 P2 也“降级”进入第二级就绪队列



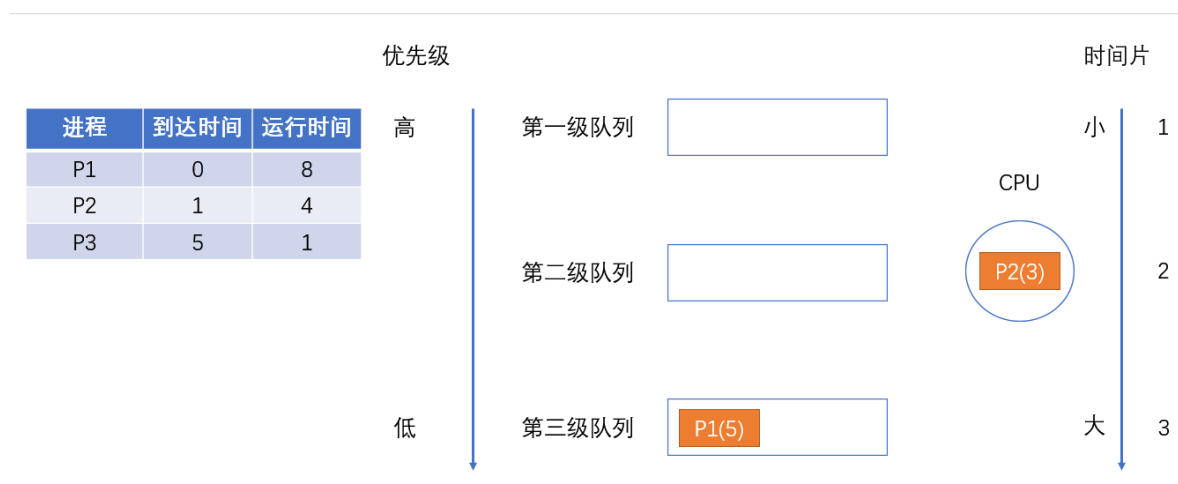
像前面所说的，“当且仅当上层级别的队列为空时，下一级队列的进程才有机会被调度”，此时第一级队列为空，所以开始调度第二级队列的进程。队头进程 P1 进入处理机



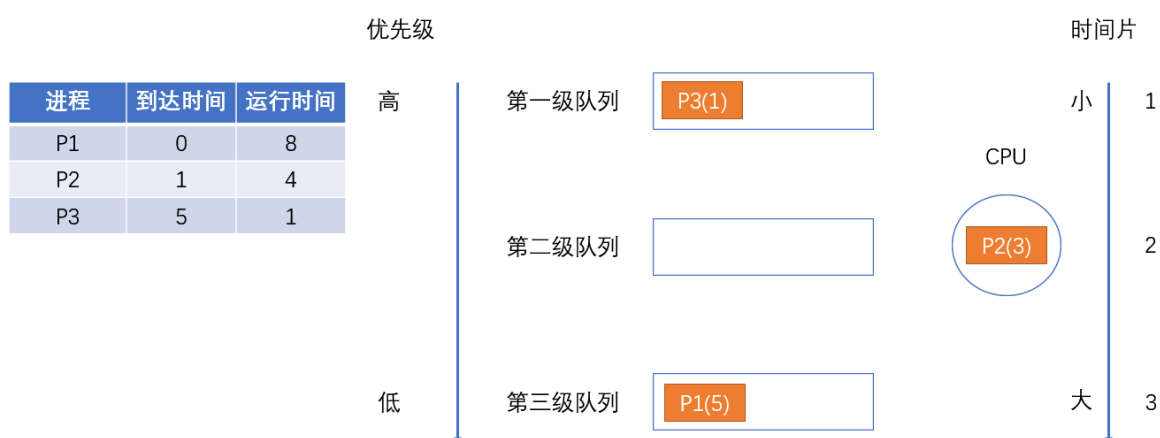
在 3 时刻，P1 时间片没用完，所以继续执行；在 4 时刻，P1 时间片用完，进程却还没执行完，所以再次“降级”来到第三级就绪队列。



此时，由于 P2 位于优先级更高的队列，所以 P2 被调度，来到处理机



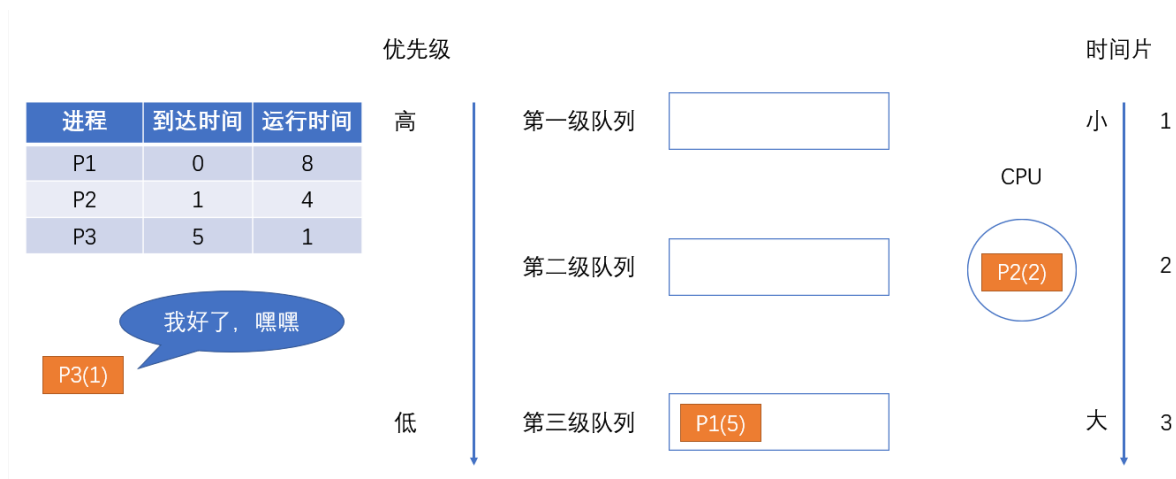
在 5 时刻，P2 时间片还没用完，所以还在正常执行。但是，P3 作为新进程到达了第一级就绪队列



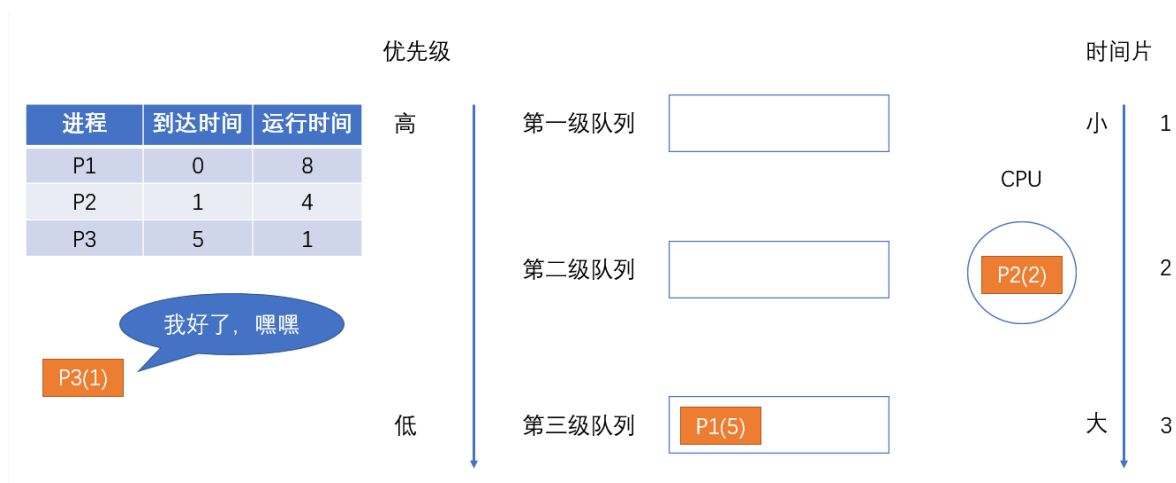
根据前面说的，“如果某个进程运行的时候，比它所在队列级别更高的队列中有新进程到达，则那个新进程会抢占处理机，而当前正在运行的进程会被送到当前队列队尾”，所以这时候 P3 抢占了处理机



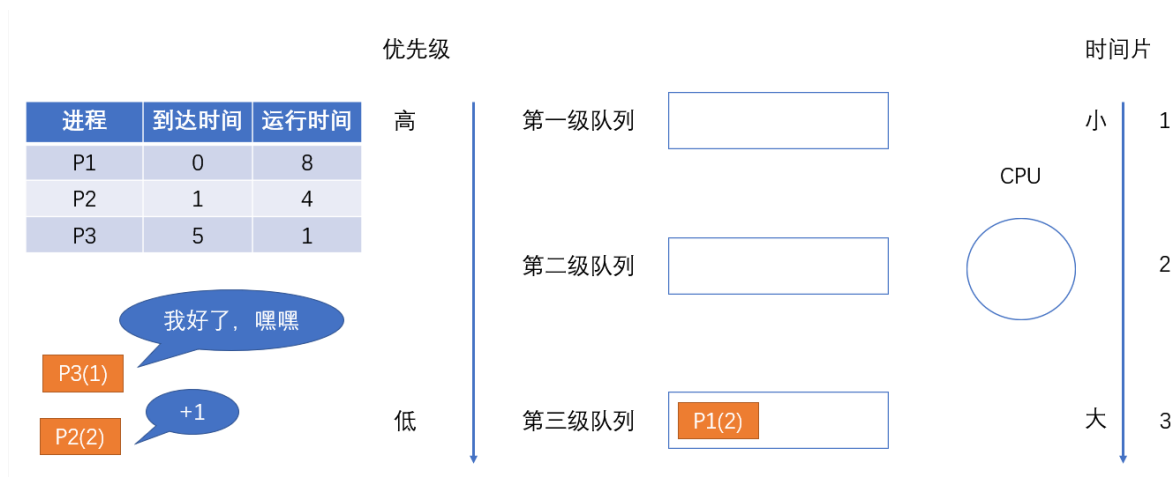
在 6 时刻，P3 时间片用完，且刚好进程也执行完了，所以这时候没有 P3 什么事了。由于 P2 所在队列优先级更高，所以此时 P2 被调度，来到处理机



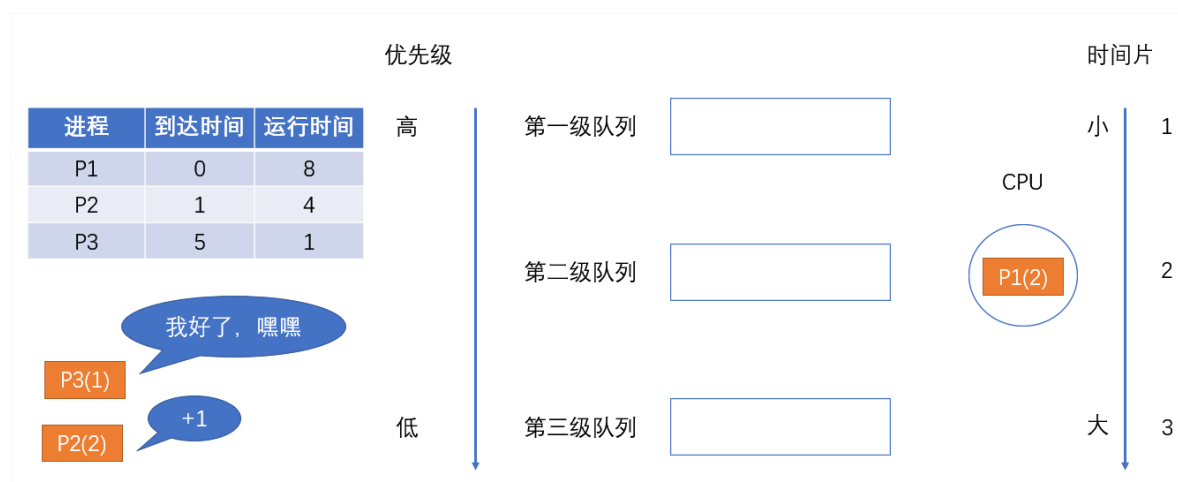
在 7 时刻，P2 时间片没用完，所以继续执行；在 8 时刻，P2 时间片用完了，且刚好进程也执行完了，所以这时候没有 P2 什么事了。此时还没完事的就剩下 P1 了，所以 P1 被调度



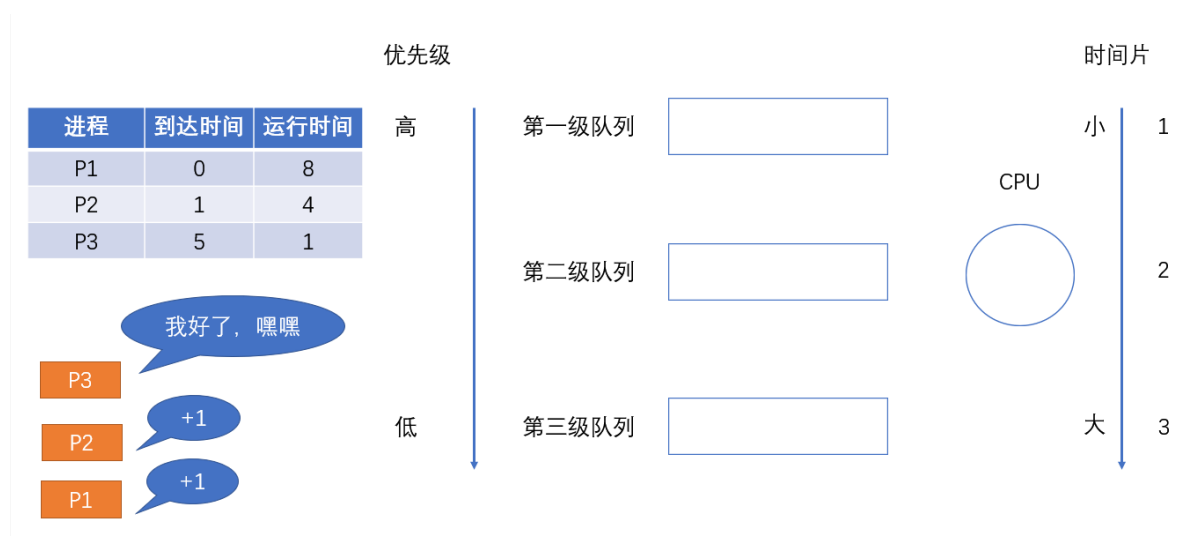
从 7 时刻被调度，一直到 10 时刻，P1 时间片用完了，但是进程还没执行完（剩下两个单位的时间），根据前面说的，“如果当前已经是最后一级，则重新放回当前队列队尾”，所以 P1 重新被送到第三级队列。



P1 作为唯一的进程再次被调度，来到处理机



从 10 时刻被调度到 12 时刻, P1 终于执行完毕



最后再做一下总结:

- 优点:
 - 对各类型进程相对公平 (FCFS 的优点): 谁先进来, 谁就会处于高级队列, 优先得到服务
 - 每个新到达的进程都可以很快就得到响应 (RR 的优点): 新到达的进程首先在高级队列, 可以很快得到响应
 - 短进程只用较少的时间就可完成 (SPF 的优点): 不需要经历过多的队列
 - 可灵活地调整对各类进程的偏好程度, 比如 CPU 密集型进程、I/O 密集型进程 (拓展: 可以将因 I/O 而阻塞的进程重新放回原队列, 这样 I/O 型进程就可以保持较高优先级)
 - 对各类型用户友好。

对于终端型用户来说, 他们提交的大多属于较小的交互型作业, 系统只要能使这些作业在第一队列所规定的时间片内完成, 便可使终端型作业用户都感到满意; 对短批处理作业用户来说, 只需在第一队列中执行一个时间片, 或至多在第二和第三队列中各执行一个时间片即可完成; 对长批处理作业用户来说, 只要让作业依次在第 1, 2, ..., n 个队列中运行, 然后再按轮转方式运行, 用户不必担心其作业长期得不到处理。

- 缺点: 可能会导致饥饿。若有源源不断的短进程到达第一队列, 那么这些进程会持续被调度, 使得下面一级的那些进程一直得不到调度, 导致饥饿现象的发生。

总结

比起早期的批处理操作系统来说, 由于计算机造价大幅降低,

因此之后出现的交互式操作系统（包括分时操作系统、实时操作系统等）更注重系统的响应时间、公平性、平衡性等指标。

而以上这三种算法恰好也能较好地满足交互式系统的需求。

因此这三种算法适合用于交互式系统。（比如 UNIX 使用的就是多级反馈队列调度算法）

进程调度算法

先来先服务（FCFS）调度算法

处于就绪态的进程按先后顺序链入到就绪队列中，而FCFS调度算法按就绪进程进入就绪队列的先后次序选择当前最先进入就绪队列的进程来执行，直到此进程阻塞或结束，才进行下一次的进程选择调度。

FCFS调度算法采用的是不可抢占的调度方式，一旦一个进程占有处理机，就一直运行下去，直到该进程完成其工作，或因等待某一事件而不能继续执行时，才释放处理机。

操作系统如果采用这种进程调度方式，则一个运行时间长且正在运行的进程会使很多晚到的且运行时间短的进程的等待时间过长。

短作业优先（SJF）调度算法

其实目前作业的提法越来越少，我们姑且把“作业”用“进程”来替换，改称为短进程优先调度算法，此算法选择就绪队列中确切（或估计）运行时间最短的进程进入执行。

它既可采用可抢占调度方式，也可采用不可抢占调度方式。可抢占的短进程优先调度算法通常也叫做最短剩余时间优先（Shortest Remaining Time First, SRTF）调度算法。短进程优先调度算法能有效地缩短进程的平均周转时间，提高系统的吞吐量，但不利于长进程的运行。

而且如果进程的运行时间是“估计”出来的话，会导致由于估计的运行时间不一定准确，而不能实际做到短作业优先。

时间片轮转（RR）调度算法

RR 调度算法与 FCFS 调度算法在选择进程上类似，但在调度的时机选择上不同。RR调度算法定义了一个的时间单元，称为时间片（或时间量）。一个时间片通常在1 ~ 100 ms之间。

当正在运行的进程用完了时间片后，即使此进程还要运行，操作系统也不让它继续运行，而是从就绪队列依次选择下一个处于就绪态的进程执行，而被剥夺CPU使用的进程返回到就绪队列的末尾，等待再次被调度。

时间片的大小可调整，如果时间片大到让一个进程足以完成其全部工作，这种算法就退化为FCFS调度算法；若时间片设置得很小，那么处理机在进程之间的进程上下文切换工作过于频繁，使得真正用于运行用户程序的时间减少。

时间片可以静态设置好，也可根据系统当前负载状况和运行情况动态调整，时间片大小的动态调整需要考虑就绪态进程个数、进程上下文切换开销、系统吞吐量、系统响应时间等多方面因素。

高响应比优先（Highest Response Ratio First, HRRF）调度算法

HRRF 调度算法是介于先来先服务算法与最短进程优先算法之间的一种折中算法。先来先服务算法只考虑进程的等待时间而忽视了进程的执行时间，而最短进程优先调度算法只考虑用户估计的进程的执行时间而忽视了就绪进程的等待时间。

HRRF调度算法二者兼顾，既考虑进程等待时间，又考虑进程的执行时间，为此定义了响应比（Rp）这个指标：

$$Rp = (\text{等待时间} + \text{预计执行时间}) / \text{执行时间} = \text{响应时间} / \text{执行时间}$$

上个表达式假设等待时间与预计执行时间之和等于响应时间。HRRF调度算法将选择 R_p 最大值的进程执行，这样既照顾了短进程又不使长进程的等待时间过长，改进了调度性能。

但HRRF调度算法需要每次计算各个进程的响应比 R_p ，这会带来较大的时间开销（特别是在就绪进程个数多的情况下）。

多级反馈队列（Multi-Level Feedback Queue）调度算法

在采用多级反馈队列调度算法的执行逻辑流程如下：

1. 设置多个就绪队列，并为各个队列赋予不同的优先级。第一个队列的优先级最高，第二队次之，其余队列优先级依次降低。仅当第 $1 \sim i-1$ 个队列均为空时，操作系统调度器才会调度第 i 个队列中的进程运行。赋予各个队列中进程执行时间片的大小也各不相同。在优先级越高的队列中，每个进程的执行时间片就越小或越大（Linux-2.4内核就是采用这种方式）。
2. 当一个就绪进程需要链入就绪队列时，操作系统首先将它放入第一队列的末尾，按FCFS的原则排队等待调度。若轮到该进程执行且在一个时间片结束时尚未完成，则操作系统调度器便将该进程转入第二队列的末尾，再同样按先来先服务原则等待调度执行。如此下去，当一个长进程从第一队列降到最后一个队列后，在最后一个队列中，可使用FCFS或RR调度算法来运行处于此队列中的进程。
3. 如果处理机正在第 i ($i > 1$) 队列中为某进程服务时，又有新进程进入第 k ($k < i$) 的队列，则新进程将抢占正在运行进程的处理机，即由调度程序把正在执行进程放回第 i 队列末尾，重新将处理机分配给处于第 k 队列的新进程。

从MLFQ调度算法可以看出长进程无法长期占用处理机，且系统的响应时间会缩短，吞吐量也不错（前提是没有频繁的短进程）。所以MLFQ调度算法是一种合适不同类型应用特征的综合进程调度算法。

最高优先级优先调度算法

进程的优先级用于表示进程的重要性及运行的优先性。一个进程的优先级可分为两种：静态优先级和动态优先级。静态优先级是在创建进程时确定的。一旦确定后，在整个进程运行期间不再改变。

静态优先级一般由用户依据包括进程的类型、进程所使用的资源、进程的估计运行时间等因素来设置。一般而言，若进程需要的资源越多、估计运行的时间越长，则进程的优先级越低；反之，对于I/O bounded的进程可以把优先级设置得高。

动态优先级是指在进程运行过程中，根据进程执行情况的变化来调整优先级。动态优先级一般根据进程占有CPU时间的长短、进程等待CPU时间的长短等因素确定。

占有处理机的时间越长，则优先级越低，等待时间越长，优先级越高。那么进程调度器将根据静态优先级和动态优先级的总和现在优先级最高的就绪进程执行。

聊聊：什么是线程，线程和进程的区别

这又是一道老生常谈的问题了，从操作系统的角度来回答一下吧。

我们上面说到进程是正在运行的程序的实例，而线程其实就是进程中的单条流向，因为线程具有进程中的某些属性，所以线程又被称为轻量级的进程。浏览器如果是一个进程的话，那么浏览器下面的每个tab页可以看作是一个个的线程。

下面是线程和进程持有资源的区别

每个进程中的内容	每个线程中的内容
地址空间	程序计数器
全局变量	寄存器
打开文件	堆栈
子进程	状态
即将发生的定时器	
信号与信号处理程序	
账户信息	

线程不像进程那样具有很强的独立性，线程之间会共享数据

创建线程的开销要比进程小很多，因为创建线程仅仅需要 `堆栈指针` 和 `程序计数器` 就可以了，而创建进程需要操作系统分配新的地址空间，数据资源等，这个开销比较大。

聊聊：有了进程为什么还要线程

不同进程之间切换实现并发，各自占有CPU实现并行

但是这些也会导致缺点，

一个进程只能做一件事，其他的进程来了会将其阻塞，为此引进了更小的粒度，

线程减少程序在并发执行时所付出的时间和空间开销，提高并发性能

聊聊：什么是进程和进程表

进程就是正在执行程序的实例，比如说 Web 程序就是一个进程，shell 也是一个进程，文章编辑器 typora 也是一个进程。

操作系统负责管理所有正在运行的进程，操作系统会为每个进程分配特定的时间来占用 CPU，操作系统还会为每个进程分配特定的资源。

操作系统为了跟踪每个进程的活动状态，维护了一个进程表。

在进程表的内部，列出了每个进程的状态以及每个进程使用的资源等。

聊聊：并发和并行

- 并发是指宏观上在一段时间内能同时运行多个程序
- 并行则指同一时刻能运行多个指令（需要硬件支持，如多流水线、多核处理器或者分布式计算系统）

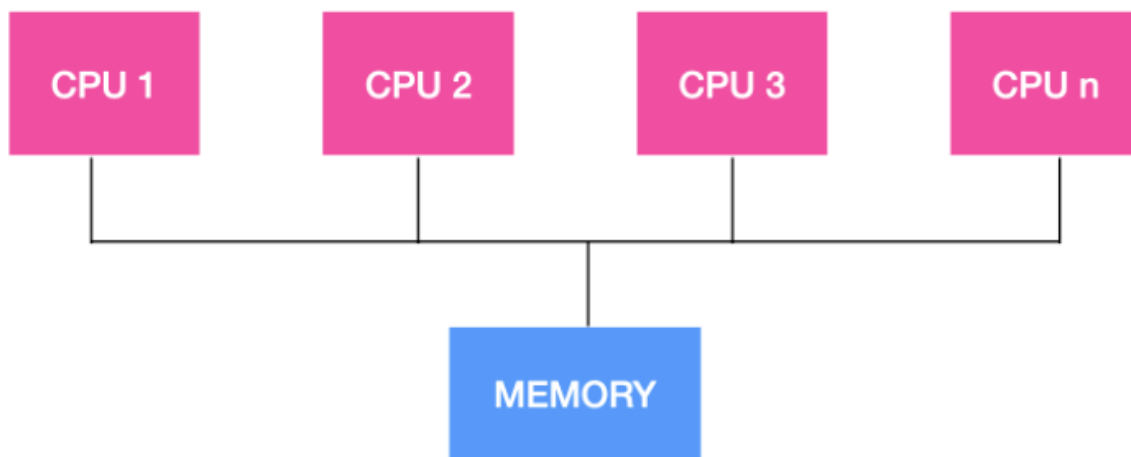
操作系统通过引入进程和线程，使得程序能够并发运行

- 并行是指两个或者多个事件在同一时刻发生；
- 而并发是指两个或多个事件在同一时间间隔发生

并行是在不同实体上的多个事件，并发是在同一实体上的多个事件；

聊聊：多处理系统的优势

随着处理器的不断增加，我们的计算机系统由单机系统变为了多处理系统，多处理系统的吞吐量比较高，多处理系统拥有多个并行的处理器，这些处理器共享时钟、内存、总线、外围设备等。



多处理系统由于可以共享资源，因此可以开源节流，省钱。整个系统的可靠性也随之提高。

聊聊：什么是上下文切换

对于单核单线程 CPU 而言，在某一时刻只能执行一条 CPU 指令。

上下文切换 (Context Switch) 是一种 **将 CPU 资源从一个进程分配给另一个进程的机制**。

从用户角度看，计算机能够并行运行多个进程，这恰恰是操作系统通过快速上下文切换造成的结果。

在切换的过程中，操作系统需要先存储当前进程的状态 (包括内存空间的指针，当前执行完的指令等等)，再读入下一个进程的状态，然后执行此进程。

聊聊：使用多线程的好处是什么

多线程是程序员不得不知的基本素养之一，所以，下面我们给出一些多线程编程的好处

- 能够提高对用户的响应顺序
- 在流程中的资源共享
- 比较经济适用
- 能够对多线程架构有深入的理解

聊聊：进程终止的方式

进程的终止

进程在创建之后，它就开始运行并做完成任务。然而，没有什么事儿是永不停歇的，包括进程也一样。进程早晚会发生终止，但是通常是由于以下情况触发的

- 正常退出(自愿的)
- 错误退出(自愿的)
- 严重错误(非自愿的)
- 被其他进程杀死(非自愿的)

正常退出

多数进程是由于完成了工作而终止。当编译器完成了所给定程序的编译之后，编译器会执行一个系统调用告诉操作系统它完成了工作。这个调用在 UNIX 中是 `exit`，在 Windows 中是 `ExitProcess`。面向屏幕中的软件也支持自愿终止操作。字处理软件、Internet 浏览器和类似的程序中总有一个供用户点击的图标或菜单项，用来通知进程删除它锁打开的任何临时文件，然后终止。

错误退出

进程发生终止的第二个原因是发现严重错误，例如，如果用户执行如下命令

```
cc foo.c
1
```

为了能够编译 `foo.c` 但是该文件不存在，于是编译器就会发出声明并退出。在给出了错误参数时，面向屏幕的交互式进程通常并不会直接退出，因为这从用户的角度来说并不合理，用户需要知道发生了什么并想要进行重试，所以这时候应用程序通常会弹出一个对话框告知用户发生了系统错误，是需要重试还是退出。

严重错误

进程终止的第三个原因是由进程引起的错误，通常是由于程序中的错误所导致的。例如，执行了一条非法指令，引用不存在的内存，或者除数是 0 等。在有些系统比如 UNIX 中，进程可以通知操作系统，它希望自行处理某种类型的错误，在这类错误中，进程会收到信号（中断），而不是在这类错误出现时直接终止进程。

被其他进程杀死

第四个终止进程的原因是，某个进程执行系统调用告诉操作系统杀死某个进程。在 UNIX 中，这个系统调用是 `kill`。在 Win32 中对应的函数是 `TerminateProcess`（注意不是系统调用）。

聊聊：进程间的通信方式

进程间的通信方式比较多，

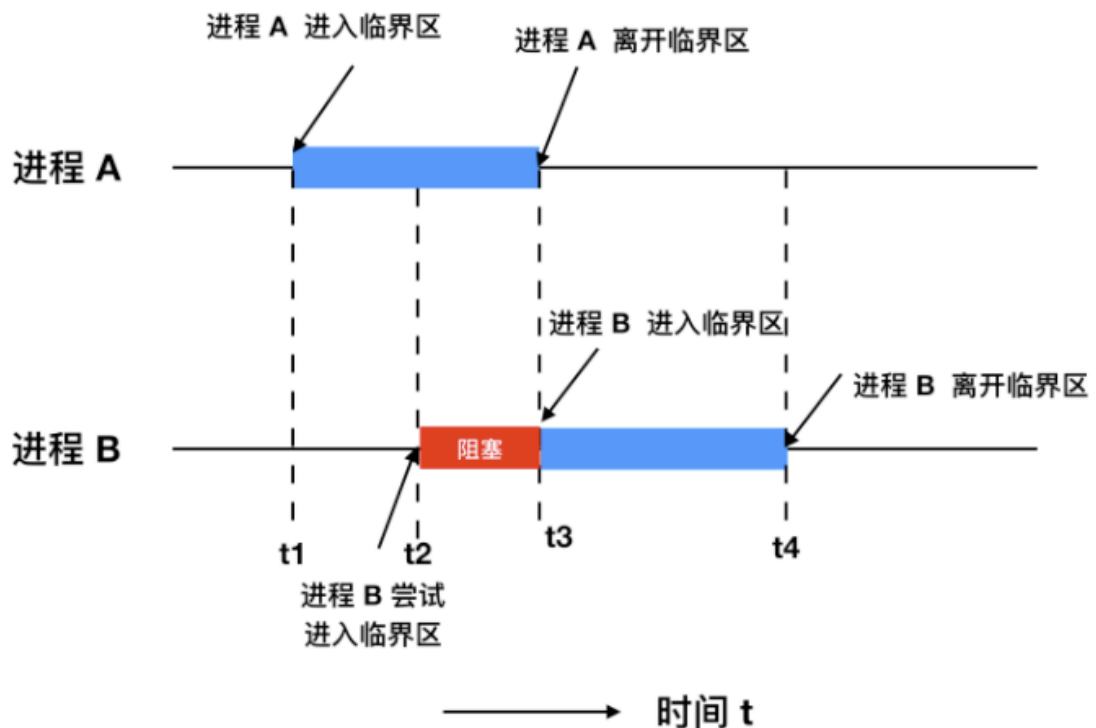
首先你需要理解下面这几个概念

- 竞态条件：即两个或多个线程同时对一共享数据进行修改，从而影响程序运行的正确性时，这种就被称为 竞态条件(race condition)。
- 临界区：不仅 共享资源 会造成竞态条件，事实上共享文件、共享内存也会造成竞态条件、那么该如何避免呢？或许一句话可以概括说明：**禁止一个或多个进程在同一时刻对共享资源（包括共享内存、共享文件等）进行读写**。换句话说，我们需要一种 互斥(mutual exclusion) 条件，这也就是说，如果一个进程在某种方式下使用共享变量和文件的话，除该进程之外的其他进程就禁止做这

种事（访问统一资源）。

一个好的解决方案，应该包含下面四种条件

1. 任何时候两个进程不能同时处于临界区
2. 不应 CPU 的速度和数量做任何假设
3. 位于临界区外的进程不得阻塞其他进程
4. 不能使任何进程无限等待进入临界区



- 忙等互斥：当一个进程在对资源进行修改时，其他进程必须进行等待，进程之间要具有互斥性，我们讨论的解决方案其实都是基于忙等互斥提出的。

进程间的通信用专业一点的术语来表示就是 `Inter Process Communication, IPC`，它主要有下面 7。

7种通信方式

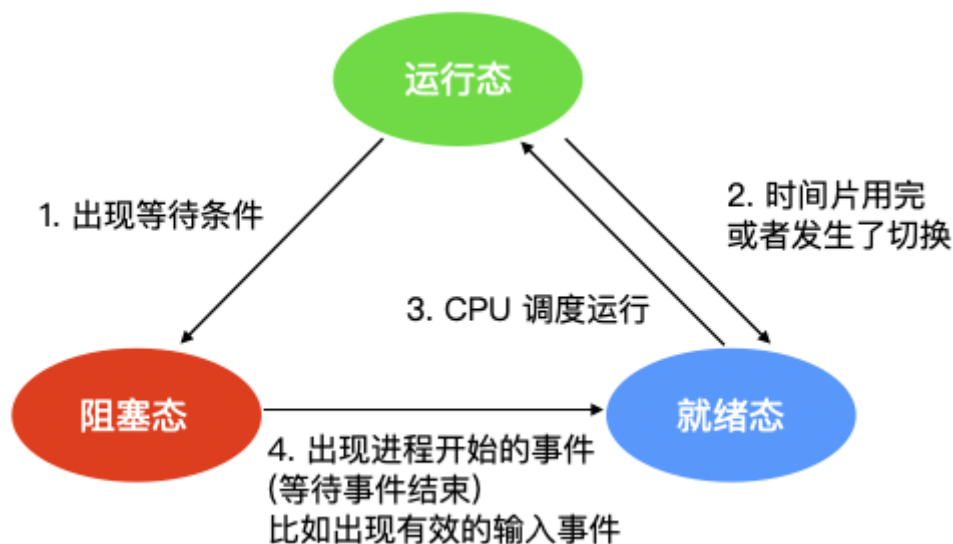


- **消息传递**：消息传递是进程间实现通信和同步等待的机制，使用消息传递，进程间的交流不需要共享变量，直接就可以进行通信；消息传递分为发送方和接收方
- **先进先出队列**：先进先出队列指的是两个不相关联进程间的通信，两个进程之间可以彼此相互进程通信，这是一种全双工通信方式
- **管道**：管道用于两个相关进程之间的通信，这是一种半双工的通信方式，如果需要全双工，需要另外一个管道。
- **直接通信**：在这种进程通信的方式中，进程与进程之间只存在一条链接，进程间要明确通信双方的命名。
- **间接通信**：间接通信是通信双方不会直接建立连接，而是找到一个中介者，这个中介者可能是个对象等等，进程可以在其中放置消息，并且可以从中删除消息，以此达到进程间通信的目的。
- **消息队列**：消息队列是内核中存储消息的链表，它由消息队列标识符进行标识，这种方式能够在不同的进程之间提供全双工的通信连接。
- **共享内存**：共享内存是使用所有进程之间的内存来建立连接，这种类型需要同步进程访问来相互保护。

聊聊：进程间状态模型

进程的三态模型

当一个进程开始运行时，它可能会经历下面这几种状态



图中会涉及三种状态

1. **运行态**：运行态指的就是进程实际占用 CPU 时间片运行时
2. **就绪态**：就绪态指的是可运行，但因为其他进程正在运行而处于就绪状态
3. **阻塞态**：阻塞态又被称为睡眠态，它指的是进程不具备运行条件，正在等待被 CPU 调度。

逻辑上来说，运行态和就绪态是很相似的。这两种情况下都表示进程可运行，但是第二种情况没有获得 CPU 时间片。第三种状态与前两种状态不同的原因是这个进程不能运行，CPU 空闲时也不能运行。

三种状态会涉及四种状态间的切换，在操作系统发现进程不能继续执行时会发生状态1的轮转，在某些系统中进程执行系统调用，例如 `pause`，来获取一个阻塞的状态。在其他系统中包括 UNIX，当进程从管道或特殊文件（例如终端）中读取没有可用的输入时，该进程会被自动终止。

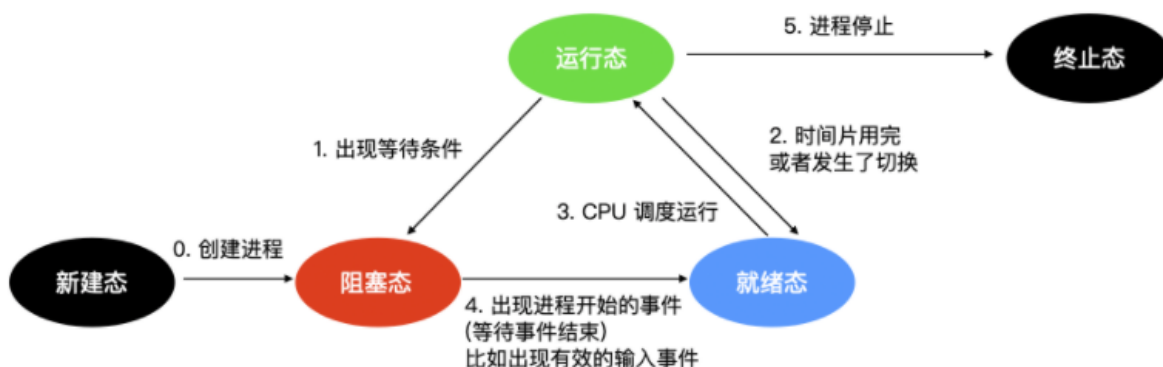
转换 2 和转换 3 都是由进程调度程序（操作系统的一部分）引起的，进程本身不知道调度程序的存在。转换 2 的出现说明进程调度器认定当前进程已经运行了足够长的时间，是时候让其他进程运行 CPU 时间片了。当所有其他进程都运行过后，这时候该是让第一个进程重新获得 CPU 时间片的时候了，就会发生转换 3。

程序调度指的是，决定哪个进程优先被运行和运行多久，这是很重要的一点。已经设计出许多算法来尝试平衡系统整体效率与各个流程之间的竞争需求。

当进程等待的一个外部事件发生时（如从外部输入一些数据后），则发生转换 4。如果此时没有其他进程在运行，则立刻触发转换 3，该进程便开始运行，否则该进程会处于就绪阶段，等待 CPU 空闲后再轮到它运行。

进程的五态模型

在三态模型的基础上，增加了两个状态，即 **新建** 和 **终止** 状态。



- **新建态**：进程的新建态就是进程刚创建出来的时候

创建进程需要两个步骤：即为新进程分配所需要的资源和空间，设置进程为就绪态，并等待调度执行。

- **终止态**：进程的终止态就是指进程执行完毕，到达结束点，或者因为错误而不得不中止进程。

终止一个进程需要两个步骤：

1. 先等待操作系统或相关的进程进行善后处理。
2. 然后回收占用的资源并被系统删除。

聊聊：什么是僵尸进程

僵尸进程是已完成且处于终止状态，但在进程表中却仍然存在的进程。僵尸进程通常发生在父子关系的进程中，由于父进程仍需要读取其子进程的退出状态所造成的。

聊聊：什么是守护、僵尸、孤儿进程

- **守护进程**：运行在后台的一种特殊进程，**独立于控制终端并周期性地执行某些任务**。
- **僵尸进程**：一个进程 fork 子进程，子进程退出，而父进程没有wait/waitpid子进程，那么**子进程的进程描述符仍保存在系统中**，这样的进程称为僵尸进程。
- **孤儿进程**：一个父进程退出，而它的一个或多个子进程还在运行，这些子进程称为孤儿进程。（孤儿进程将由 init 进程收养并对它们完成状态收集工作）

聊聊：Semaphore(信号量) Vs Mutex(互斥锁)

- 当用户创立多个线程 / 进程时，如果不同线程 / 进程同时读写相同的内容，则可能造成读写错误，或者数据不一致。此时，需要通过加锁的方式，控制临界区(critical section)的访问权限。对于 semaphore而言，在初始化变量的时候可以控制允许多少个线程 / 进程同时访问一个临界区，其他的线程 / 进程会被堵塞，直到有人解锁。
- Mutex相当于只允许一个线程 / 进程访问的semaphore。此外，根据实际需要，人们还实现了一种读写锁(read-write lock)，它允许同时存在多个读者(reader)，但任何时候至多只有一个写者(writer)，且不能于读者共存。

聊聊：进程调度策略有几种？

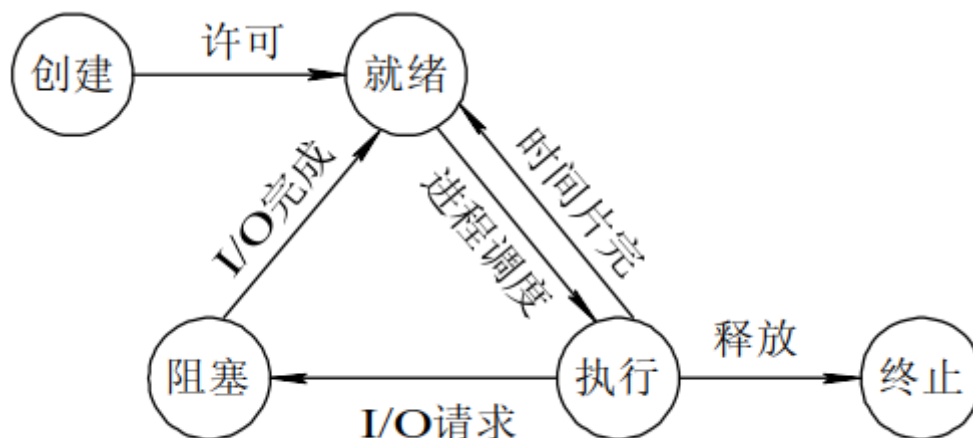
- **先来先服务**：非抢占式的调度算法，按照请求的顺序进行调度。有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行很长时间，造成了短作业等待时间过长。另外，对 I/O 密集型进程也不利，因为这种进程每次进行 I/O 操作之后又得重新排队。
- **短作业优先**：非抢占式的调度算法，按估计运行时间最短的顺序进行调度。长作业有可能会饿死，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到调度。
- **最短剩余时间优先**：最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待。
- **时间片轮转**：将所有就绪进程按 FCFS 的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。

时间片轮转算法的效率和时间片的大小有很大关系：因为进程切换都要保存进程的信息并且载入新进程的信息，如果时间片太小，会导致进程切换得太频繁，在进程切换上就会花过多时间。而如果时间片过长，那么实时性就不能得到保证。

- **优先级调度**：为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

聊聊：进程有哪些状态？

进程一共有 5 种状态，分别是创建、就绪、运行（执行）、终止、阻塞。



- 运行状态就是进程正在 CPU 上运行。在单处理机环境下，每一时刻最多只有一个进程处于运行状态。
- 就绪状态就是说进程已处于准备运行的状态，即进程获得了除 CPU 之外的一切所需资源，一旦得到 CPU 即可运行。
- 阻塞状态就是进程正在等待某一事件而暂停运行，比如等待某资源为可用或等待 I/O 完成。即使 CPU 空闲，该进程也不能运行。

运行态→阻塞态：往往是由于等待外设，等待主存等资源分配或等待人工干预而引起的。**阻塞态→就绪态**：则是等待的条件已满足，只需分配处理器后就能运行。**运行态→就绪态**：不是由于自身原因，而是由外界原因使运行状态的进程让出处理器，这时候就变成就绪态。例如时间片用完，或有更高优先级的进程来抢占处理器等。**就绪态→运行态**：系统按某种策略选中就绪队列中的一个进程占用处理器，此时就变成了运行态。

死锁篇

聊聊：死锁是什么

死锁，是指多个进程在运行过程中因争夺资源而造成的一种僵局，处于僵持状态，没有外力的情况下无法推动

聊聊：死锁产生条件

四个条件缺一不可

1. 互斥条件：进程对所需求的资源具有排他性，若有其他进程请求该资源，请求进程只能等待。
2. 不剥夺条件：进程在所获得的资源未释放前，不能被其他进程强行夺走，只能自己释放。
3. 请求和保持条件：进程当前所拥有的资源在进程请求其他新资源时，由该进程继续占有。
4. 循环等待条件：存在一种进程资源循环等待链，链中每个进程已获得的资源同时被链中下一个进程所请求。

聊聊：解决死锁的基本方法

预防死锁

避免死锁

检测死锁

解除死锁

聊聊：死锁产生的原因

死锁产生的原因大致有两个：资源竞争和程序执行顺序不当

聊聊：死锁产生的必要条件

资源死锁可能出现的情况主要有

- 互斥条件：每个资源都被分配给了一个进程或者资源是可用的
- 保持和等待条件：已经获取资源的进程被认为能够获取新的资源
- 不可抢占条件：分配给一个进程的资源不能强制的从其他进程抢占资源，它只能由占有它的进程显示释放
- 循环等待：死锁发生时，系统中一定有两个或者两个以上的进程组成一个循环，循环中的每个进程都在等待下一个进程释放的资源。

聊聊：死锁的恢复方式

所以针对检测出来的死锁，我们要对其进行恢复，下面我们会探讨几种死锁的恢复方式

通过抢占进行恢复

在某些情况下，可能会临时将某个资源从它的持有者转移到另一个进程。比如在不通知原进程的情况下，将某个资源从进程中强制取走给其他进程使用，使用完后又送回。这种恢复方式一般比较困难而且有些简单粗暴，并不可取。

通过回滚进行恢复

如果系统设计者和机器操作员知道有可能发生死锁，那么就可以定期检查流程。进程的检测点意味着进程的状态可以被写入到文件以便后面进行恢复。检测点不仅包含 存储映像(memory image)，还包含 资源状态(resource state)。一种更有效的解决方式是不要覆盖原有的检测点，而是每出现一个检测点都要把它写入到文件中，这样当进程执行时，就会有一系列的检查点文件被累积起来。

为了进行恢复，要从上一个较早的检查点上开始，这样所需要资源的进程会回滚到上一个时间点，在这个时间点上，死锁进程还没有获取所需要的资源，可以在此时对其进行资源分配。

杀死进程恢复

最简单有效的解决方案是直接杀死一个死锁进程。但是杀死一个进程可能照样行不通，这时候就需要杀死别的资源进行恢复。

另外一种方式是选择一个环外的进程作为牺牲品来释放进程资源。

聊聊：如何破坏死锁

和死锁产生的必要条件一样，如果要破坏死锁，也是从下面四种方式进行破坏。

破坏互斥条件

我们首先考虑的就是**破坏互斥使用条件**。如果资源不被一个进程独占，那么死锁肯定不会产生。如果两个打印机同时使用一个资源会造成混乱，打印机的解决方式是使用 假脱机打印机(spooling printer)，这项技术可以允许多个进程同时产生输出，在这种模型中，实际请求打印机的唯一进程是打印机守护进程，也称为后台进程。后台进程不会请求其他资源。我们可以消除打印机的死锁。

后台进程通常被编写为能够输出完整的文件后才能打印，假如两个进程都占用了假脱机空间的一半，而这两个进程都没有完成全部的输出，就会导致死锁。

因此，尽量做到尽可能少的进程可以请求资源。

破坏保持等待的条件

第二种方式是如果我们能阻止持有资源的进程请求其他资源，我们就能够消除死锁。一种实现方式是让所有的进程开始执行前请求全部的资源。如果所需的资源可用，进程会完成资源的分配并运行到结束。如果有任何一个资源处于频繁分配的情况，那么没有分配到资源的进程就会等待。

很多进程**无法在执行完成前就知道到底需要多少资源**，如果知道的话，就可以使用银行家算法；还有一个问题是这样**无法合理有效利用资源**。

还有一种方式是进程在请求其他资源时，先释放所占用的资源，然后再尝试一次获取全部的资源。

破坏不可抢占条件

破坏不可抢占条件也是可以的。可以通过 虚拟化 的方式来避免这种情况。

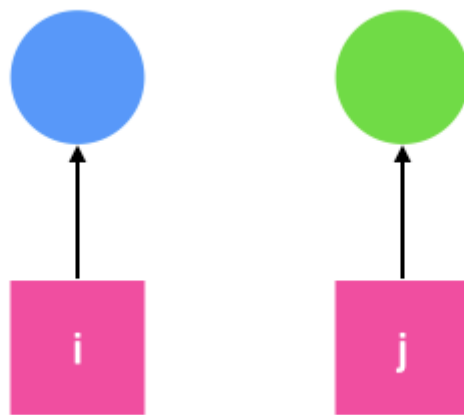
破坏循环等待条件

现在就剩最后一个条件了，循环等待条件可以通过多种方法来破坏。一种方式是制定一个标准，一个进程在任何时候只能使用一种资源。如果需要另外一种资源，必须释放当前资源。

另一种方式是将所有的资源统一编号，如下图所示

- 
1. 图像处理仪
 2. 打印机
 3. 绘图仪
 4. 磁带机
 5. 蓝光光驱

进程可以在任何时间提出请求，但是所有的请求都必须按照资源的顺序提出。如果按照此分配规则的话，那么资源分配之间不会出现环。



聊聊：死锁类型

两阶段加锁

虽然很多情况下死锁的避免和预防都能处理，但是效果并不好。随着时间的推移，提出了很多优秀的算法用来处理死锁。例如在数据库系统中，一个经常发生的操作是请求锁住一些记录，然后更新所有锁定的记录。当同时有多个进程运行时，就会有死锁的风险。

一种解决方式是使用 **两阶段提交(two-phase locking)**。顾名思义分为两个阶段，一阶段是进程尝试一次锁定它需要的所有记录。如果成功后，才会开始第二阶段，第二阶段是执行更新并释放锁。第一阶段并不做真正有意义的工作。

如果在第一阶段某个进程所需要的记录已经被加锁，那么该进程会释放所有锁定的记录并重新开始第一阶段。从某种意义上来说，这种方法类似于预先请求所有必需的资源或者是在进行一些不可逆的操作之前请求所有的资源。

不过在一般的应用场景中，两阶段加锁的策略并不通用。如果一个进程缺少资源就会半途中断并重新开始的方式是不可接受的。

通信死锁

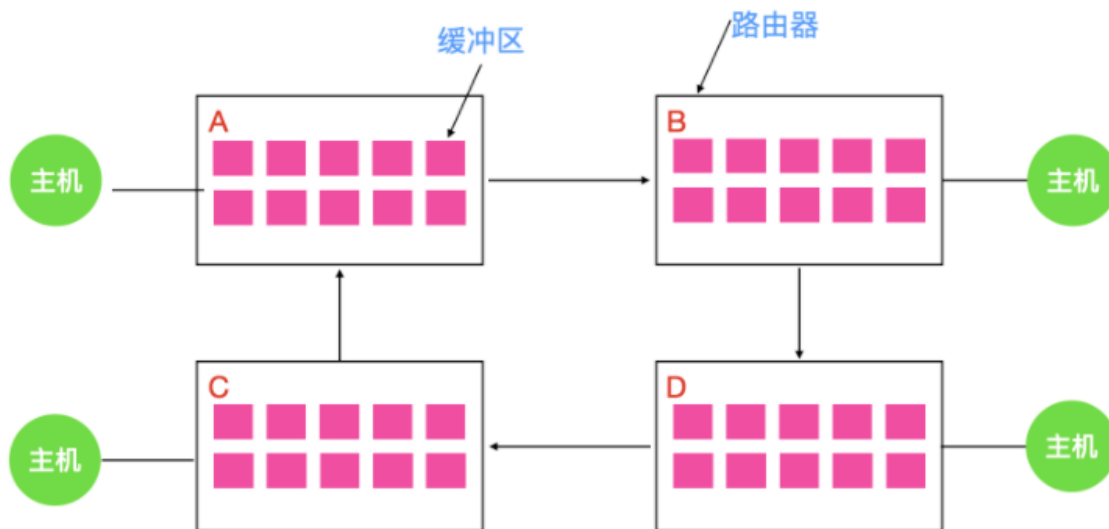
我们上面一直讨论的是资源死锁，资源死锁是一种死锁类型，但并不是唯一类型，还有通信死锁，也就是两个或多个进程在发送消息时出现的死锁。进程 A 给进程 B 发了一条消息，然后进程 A 阻塞直到进程 B 返回响应。假设请求消息丢失了，那么进程 A 在一直等着回复，进程 B 也会阻塞等待请求消息到来，这时候就产生 **死锁**。

尽管会产生死锁，但是这并不是一个资源死锁，因为 A 并没有占据 B 的资源。事实上，通信死锁并没有完全可见的资源。根据死锁的定义来说：每个进程因为等待其他进程引起的事件而产生阻塞，这就是一种死锁。相较于最常见的通信死锁，我们把上面这种情况称为 **通信死锁(communication deadlock)**。

通信死锁不能通过调度的方式来避免，但是可以使用通信中一个非常重要的概念来避免：**超时(timeout)**。在通信过程中，只要一个信息被发出后，发送者就会启动一个定时器，定时器会记录消息的超时时间，如果超时时间到了但是消息还没有返回，就会认为消息已经丢失并重新发送，通过这种方式，可以避免通信死锁。

但是并非所有网络通信发生的死锁都是通信死锁，也存在资源死锁，下面就是一个典型的资源死锁。

当一个数据包从主机进入路由器时，会被放入一个缓冲区，然后再传输到另外一个路由器，再到另一个，以此类推直到目的地。缓冲区都是资源并且数量有限。如下图所示，每个路由器都有 10 个缓冲区（实际上有很多）。



假如路由器 A 的所有数据需要发送到 B，B 的所有数据包需要发送到 D，然后 D 的所有数据包需要发送到 A。没有数据包可以移动，因为在另一端没有缓冲区可用，这就是一个典型的资源死锁。

活锁

某些情况下，当进程意识到它不能获取所需要的下一个锁时，就会尝试礼貌的释放已经获得的锁，然后等待非常短的时间再次尝试获取。可以想像一下这个场景：当两个人在狭路相逢的时候，都想给对方让路，相同的步调会导致双方都无法前进。

现在假想有一对并行的进程用到了两个资源。它们分别尝试获取另一个锁失败后，两个进程都会释放自己持有的锁，再次进行尝试，这个过程会一直进行重复。很明显，这个过程中没有进程阻塞，但是进程仍然不会向下执行，这种状况我们称之为 **活锁 (livelock)**。

饥饿

与死锁和活锁的一个非常相似的问题是 **饥饿 (starvation)**。想象一下你什么时候会饿？一段时间不吃东西是不是会饿？对于进程来讲，最重要的就是资源，如果一段时间没有获得资源，那么进程会产生饥饿，这些进程会永远得不到服务。

我们假设打印机的分配方案是每次都会分配给最小文件的进程，那么要打印大文件的进程会永远得不到服务，导致进程饥饿，进程会无限制的推后，虽然它没有阻塞。

聊聊：什么是临界区，如何解决冲突？

每个进程中访问临界资源的那段程序称为临界区，

一次仅允许一个进程使用的资源称为临界资源。

解决冲突的办法：

- 如果有若干进程要求进入空闲的临界区，**一次仅允许一个进程进入**，如已有进程进入自己的临界区，则其它所有试图进入临界区的进程必须等待；
- 进入临界区的进程要在**有限时间内退出**。
- 如果进程不能进入自己的临界区，则应**让出CPU**，避免进程出现“忙等”现象。

聊聊：什么是线程安全

如果多线程的程序运行结果是可预期的，而且与单线程的程序运行结果一样，那么说明是“线程安全”的。

聊聊：同步与异步

同步：

- 同步的定义：是指一个进程在执行某个请求的时候，若该请求需要一段时间才能返回信息，那么，这个进程将会一直等待下去，直到收到返回信息才继续执行下去。
- 特点：
 1. 同步是阻塞模式；
 2. 同步是按顺序执行，执行完一个再执行下一个，需要等待，协调运行；

异步：

- 是指进程不需要一直等下去，而是继续执行下面的操作，不管其他进程的状态。当有消息返回时系统会通知进程进行处理，这样可以提高执行的效率。
- 特点：
 1. 异步是非阻塞模式，无需等待；
 2. 异步是彼此独立，在等待某事件的过程中，继续做自己的事，不需要等待这一事件完成后再工作。线程是异步实现的一个方式。

聊聊：同步与异步的优缺点：

- 同步可以避免出现死锁，读脏数据的发生。一般共享某一资源的时候，如果每个人都有修改权限，同时修改一个文件，有可能使一个读取另一个人已经删除了内容，就会出错，同步就不会出错。但，同步需要等待资源访问结束，浪费时间，效率低。
- 异步可以提高效率，但，安全性较低。

基础知识：系统调用

系统调用概述

计算机系统的各种硬件资源是有限的，在现代多任务操作系统上同时运行的多个进程都需要访问这些资源，为了更好的管理这些资源进程是不允许直接操作的，所有对这些资源的访问都必须有操作系统控制。也就是说操作系统是使用这些资源的唯一入口，而这个入口就是操作系统提供的系统调用（System Call）。在 Linux 中系统调用是用户空间访问内核的唯一手段，除异常和陷入外，他们是内核唯一的合法入口。

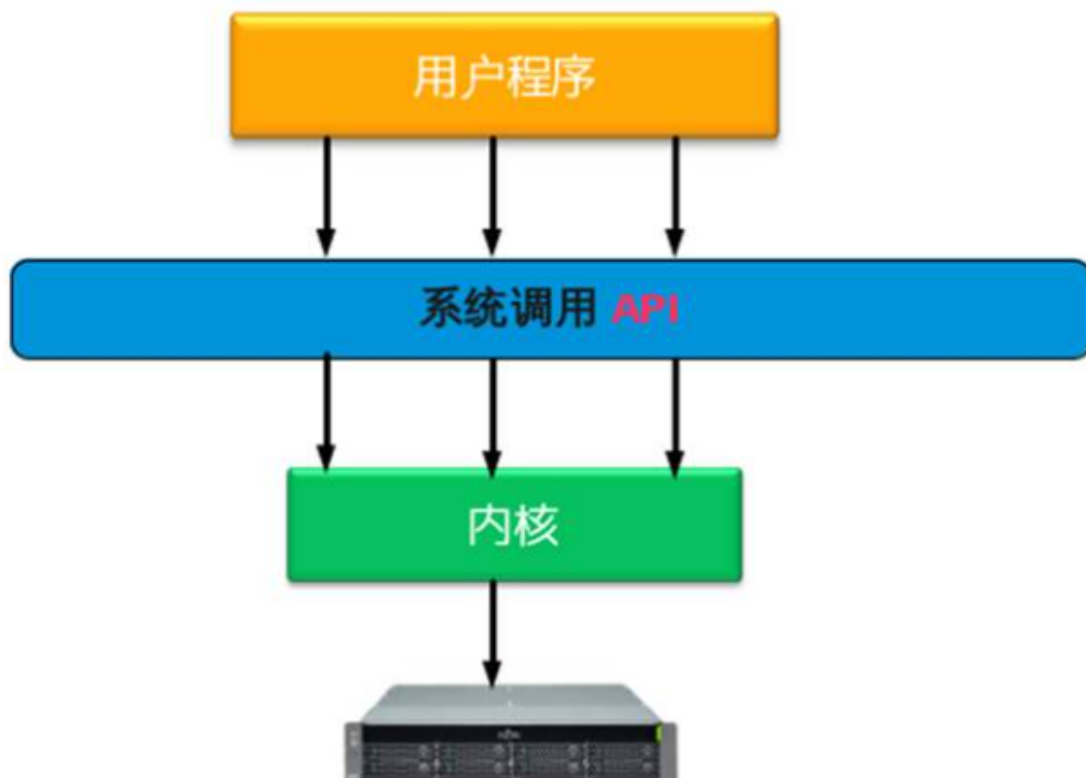
一般情况下应用程序通过应用编程接口 API，而不是直接通过系统调用来编程。在 Unix 世界，最流行的 API 是基于 POSIX 标准的。

操作系统一般是通过中断从用户态切换到内核态。中断就是一个硬件或软件请求，要求 CPU 暂停当前的工作，去处理更重要的事情。比如，在 x86 机器上可以通过 int 指令进行软件中断，而在磁盘完成读写操作后会向 CPU 发起硬件中断。

中断有两个重要的属性，中断号和中断处理程序。中断号用来标识不同的中断，不同的中断具有不同的中断处理程序。在操作系统内核中维护着一个中断向量表（Interrupt Vector Table），这个数组存储了所有中断处理程序的地址，而中断号就是相应中断在中断向量表中的偏移量。

一般地，系统调用都是通过软件中断实现的，x86 系统上的软件中断由 `int $0x80` 指令产生，而 128 号异常处理程序就是系统调用处理程序 `system_call()`，它与硬件体系有关，在 `entry.S` 中用汇编编写。接下来就来看一下 Linux 下系统调用具体的实现过程。

系统调用图如下图所示：



为什么需要系统调用

Linux 内核中设置了一组用于实现系统功能的子程序，称为系统调用。系统调用和普通库函数调用非常相似，只是系统调用由操作系统核心提供，运行于内核态，而普通的函数调用由函数库或用户自己提供，运行于用户态。

一般的，进程是不能访问内核的。它不能访问内核所占内存空间也不能调用内核函数。CPU 硬件决定了这些（这就是为什么它被称作“保护模式”）。

为了和用户空间上运行的进程进行交互，内核提供了一组接口。透过该接口，应用程序可以访问硬件设备和其他操作系统资源。这组接口在应用程序和内核之间扮演了使者的角色，应用程序发送各种请求，而内核负责满足这些请求(或者让应用程序暂时搁置)。实际上提供这组接口主要是为了保证系统稳定可靠，避免应用程序肆意妄行，惹出大麻烦。

系统调用在用户空间进程和硬件设备之间添加了一个中间层。该层主要作用有三个：

1. 它为用户空间提供了一种统一的硬件的抽象接口。比如当需要读些文件的时候，应用程序就可以不去管磁盘类型和介质，甚至不用去管文件所在的文件系统到底是哪种类型。
2. 系统调用保证了系统的稳定和安全。作为硬件设备和应用程序之间的中间人，内核可以基于权限和其他一些规则对需要进行的访问进行裁决。举例来说，这样可以避免应用程序不正确地使用硬件设备，窃取其他进程的资源，或做出其他什么危害系统的事情。
3. 每个进程都运行在虚拟系统中，而在用户空间和系统的其余部分提供这样一层公共接口，也是出于这种考虑。如果应用程序可以随意访问硬件而内核又对此一无所知的话，几乎就没法实现多任务和虚拟内存，当然也不可能实现良好的稳定性和安全性。在 Linux 中，系统调用是用户空间访问内核的惟一手段；除异常和中断外，它们是内核惟一的合法入口。

API/POSIX/C库的区别与联系

一般情况下，应用程序通过应用编程接口(API)而不是直接通过系统调用来编程。这点很重要，因为应用程序使用的这种编程接口实际上并不需要和内核提供的系统调用一一对应。

在 Unix 世界中，最流行的应用编程接口是基于 POSIX 标准的，其目标是提供一套大体上基于 Unix 的可移植操作系统标准。POSIX 是说明 API 和系统调用之间关系的一个极好例子。在大多数 Unix 系统上，根据 POSIX 而定义的 API 函数和系统调用之间有着直接关系。

Linux 的系统调用像大多数 Unix 系统一样，作为 C 库的一部分提供如下图所示。C 库实现了 Unix 系统的主要 API，包括标准 C 库函数和系统调用。所有的 C 程序都可以使用 C 库，而由于 C 语言本身的特点，其他语言也可以很方便地把它们封装起来使用。

从程序员的角度看，系统调用无关紧要，他们只需要跟API打交道就可以了。相反，内核只跟系统调用打交道；库函数及应用程序是怎么使用系统调用不是内核所关心的。

关于 Unix 的界面设计有一句通用的格言“提供机制而不是策略”。换句话说，Unix 的系统调用抽象出了用于完成某种确定目的的函数。至于这些函数怎么用完全不需要内核去关心。区别对待机制(mechanism)和策略(policy)是 Unix 设计中的一大亮点。大部分的编程问题都可以被切割成两个部分：“需要提供什么功能”(机制)和“怎样实现这些功能”(策略)。

区别

api 是函数的定义，规定了这个函数的功能，跟内核无直接关系。而系统调用是通过中断向内核发请求，实现内核提供的某些服务。

联系

一个 api 可能会需要一个或多个系统调用来完成特定功能。通俗点说就是如果这个 api 需要跟内核打交道就需要系统调用，否则不需要。程序员调用的是 API (API 函数)，然后通过系统调用共同完成函数的功能。因此，API 是一个提供给应用程序的接口，一组函数，是与程序员进行直接交互的。

系统调用则不与程序员进行交互的，它根据 API 函数，通过一个软中断机制向内核提交请求，以获取内核服务的接口。并不是所有的 API 函数都一一对应一个系统调用，有时，一个 API 函数会需要几个系统调用来共同完成函数的功能，甚至还有一些 API 函数不需要调用相应的系统调用（因此它所完成的不是内核提供的服务）。

系统调用的实现原理

基本机制

前文已经提到了 Linux 下的系统调用是通过 0x80 实现的，但是我们知道操作系统会有多个系统调用（Linux 下有 319 个系统调用），而对于同一个中断号是如何处理多个不同的系统调用的？最简单的方式对于不同的系统调用采用不同的中断号，但是中断号明显是一种稀缺资源，Linux 显然不会这么做；还有一个问题就是系统调用是需要提供参数，并且具有返回值的，这些参数又是怎么传递的？也就是说，对于系统调用我们要搞清楚两点：

1. 系统调用的函数名称转换。
2. 系统调用的参数传递。

首先看第一个问题。实际上，Linux 中每个系统调用都有相应的系统调用号作为唯一的标识，内核维护一张系统调用表，sys_call_table，表中的元素是系统调用函数的起始地址，而系统调用号就是系统调用在调用表的偏移量。在 x86 上，系统调用号是通过 eax 寄存器传递给内核的。比如 fork() 的实现。

用户空间的程序无法直接执行内核代码。它们不能直接调用内核空间中的函数，因为内核驻留在受保护的地址空间上。如果进程可以直接在内核的地址空间上读写的话，系统安全就会失去控制。所以，应用程序应该以某种方式通知系统，告诉内核自己需要执行一个系统调用，希望系统切换到内核态，这样内核就可以代表应用程序来执行该系统调用了。

通知内核的机制是靠软件中断实现的。首先，用户程序为系统调用设置参数。其中一个参数是系统调用编号。参数设置完成后，程序执行“系统调用”指令。x86系统上的软中断由int产生。这个指令会导致一个异常：产生一个事件，这个事件会致使处理器切换到内核态并跳转到一个新的地址，并开始执行那里的异常处理程序。此时的异常处理程序实际上就是系统调用处理程序。它与硬件体系结构紧密相关。

新地址的指令会保存程序的状态，计算出应该调用哪个系统调用，调用内核中实现那个系统调用的函数，恢复用户程序状态，然后将控制权返还给用户程序。系统调用是设备驱动程序中定义的函数最终被调用的一种方式。

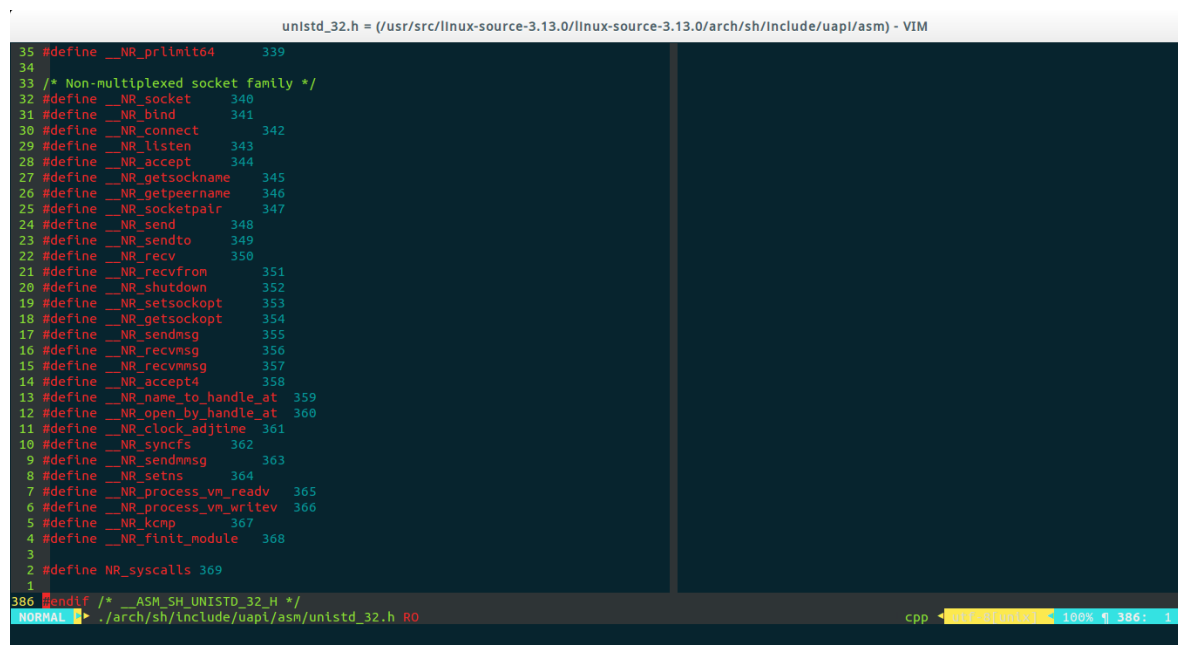
从系统分析的角度，linux的系统调用涉及 4 方面的问题。

响应函数sys_xxx

响应函数名以“sys_”开头，后跟该系统调用的名字。例如系统调用 fork() 的响应函数是 sys_fork(), exit() 的响应函数是 sys_exit()。

系统调用表与系统调用号==>数组与下标

文件 include/asm/unistd.h 为每个系统调用规定了唯一的编号。



```
35 #define __NR_prctl64 339
36
37 /* Non-multiplexed socket family */
38 #define __NR_socket 340
39 #define __NR_bind 341
40 #define __NR_connect 342
41 #define __NR_listen 343
42 #define __NR_accept 344
43 #define __NR_getsockname 345
44 #define __NR_getpeername 346
45 #define __NR_socketpair 347
46 #define __NR_send 348
47 #define __NR_sendto 349
48 #define __NR_recv 350
49 #define __NR_recvfrom 351
50 #define __NR_shutdown 352
51 #define __NR_setsockopt 353
52 #define __NR_getsockopt 354
53 #define __NR_sendmsg 355
54 #define __NR_recvmsg 356
55 #define __NR_recvmsg 357
56 #define __NR_accept4 358
57 #define __NR_name_to_handle_at 359
58 #define __NR_open_by_handle_at 360
59 #define __NR_clock_adjtime 361
60 #define __NR_syncfs 362
61 #define __NR_sendmmsg 363
62 #define __NR_setns 364
63 #define __NR_process_vm_readv 365
64 #define __NR_process_vm_writev 366
65 #define __NR_kcmp 367
66 #define __NR_finit_module 368
67
68 #define NR_syscalls 369
69
386 #endif /* __ASM_SH_UNISTD_32_H */
NORMAL . /arch/sh/include/uapi/asm/unistd_32.h RO cpp 100% 1 386: 1
```

假设用 name 表示系统调用的名称，那么系统调用号与系统调用响应函数的关系是：以系统调用号 `__NR_name` 作为下标，可找出系统调用表 `sys_call_table` 中对应表项的内容，它正好是该系统调用的响应函数 `sys_name` 的入口地址。

系统调用表 `sys_call_table` 记录了各 `sys_name` 函数在表中的位置，共 190 项。有了这张表，就很容易根据特定系统调用

```
syscalls_64.S = (/usr/src/linux-sour...ux-source-3.13.0/arch/sh/kernel) - VIM
15 /*
14 * System calls jump table
13 */
12 .globl sys_call_table
11 sys_call_table:
10 .long sys_restart_syscall /* 0 - old "setup()" system call */
9 .long sys_exit
8 .long sys_fork
7 .long sys_read
6 .long sys_write
5 .long sys_open /* 5 */
4 .long sys_close
3 .long sys_waitpid
2 .long sys_creat
1 .long sys_link
33 .long sys_unlink /* 10 */
1 .long sys_execve
2 .long sys_chdir
3 .long sys_time
4 .long sys_mknod
5 .long sys_chmod /* 15 */
6 .long sys_lchown16
7 .long sys_ni_syscall /* old break syscall holder */
NORMAL 0 > <kernel/syscalls_64.S R0 sys_call_table < asm <utf8[unix]> 8% ¶ 33: 4
```

在表中的偏移量，找到对应的系统调用响应函数的入口地址。系统调用表共 256 项，余下的项是可供用户自己添加的系统调用空间。

在 Linux 中，每个系统调用被赋予一个系统调用号。这样，通过这个独一无二的号就可以关联系统调用。当用户空间的进程执行一个系统调用的时候，这个系统调用号就被用来指明到底是要执行哪个系统调用。进程不会提及系统调用的名称。

系统调用号相当关键，一旦分配就不能再有任何变更，否则编译好的应用程序就会崩溃。Linux 有一个“未实现”系统调用 `sys_ni_syscall()`，它除了返回 `-ENOSYS` 外不做任何其他工作，这个错误号就是专门针对无效的系统调用而设的。

因为所有的系统调用陷入内核的方式都一样，所以仅仅是陷入内核空间是不够的。因此必须把系统调用号一并传给内核。在 x86 上，系统调用号是通过 `eax` 寄存器传递给内核的。在陷入内核之前，用户空间就把相应系统调用所对应的号放入 `eax` 中了。这样系统调用处理程序一旦运行，就可以从 `eax` 中得到数据。其他体系结构上的实现也都类似。

内核记录了系统调用表中的所有已注册过的系统调用的列表，存储在 `sys_call_table` 中。它与体系结构有关，一般在 `entry.s` 中定义。这个表中为每一个有效的系统调用指定了惟一的系统调用号。

`sys_call_table` 是一张由指向实现各种系统调用的内核函数的函数指针组成的表：

`system_call()` 函数通过将给定的系统调用号与 `NR_syscalls` 做比较来检查其有效性。如果它大于或者等于 `NR_syscalls`，该函数就返回 `-ENOSYS`。否则，就执行相应的系统调用。


```
entry_32.S = (/usr/src/linux-source-...x-source-3.13.0/arch/x86/kernel) - VIM
>> 11     ASM_CLAC
>> 10     movl %ebp,PT_EBP(%esp)
>> 9      _ASM_EXTABLE(1b,syscall_fault)
>> 8
>> 7      GET_THREAD_INFO(%ebp)
>> 6
>> 5      testl $ _TIF_WORK_SYSCALL_ENTRY, TI_flags(%ebp)
>> 4      jnz sysenter_audit
>> 3      sysenter_do_call:
>> 2      cmpl $(NR_syscalls), %eax
>> 1      jae sysenter_badsys
435     call *sys_call_table(,%eax,4)
>> 1      sysenter_after_call:
>> 2      movl %eax,PT_EAX(%esp)
>> 3      LOCKDEP_SYS_EXIT
>> 4      DISABLE_INTERRUPTS(CLBR_ANY)
>> 5      TRACE_IRQS_OFF
>> 6      movl TI_flags(%ebp), %ecx
>> 7      testl $ _TIF_ALLWORK_MASK, %ecx
>> 8      jne sysexit_audit
>> 9      sysexit_exit:
>> 10     /* if something modifies registers it must also disable sysexit */
>> 11     movl PT_EIP(%esp), %edx
NORMAL |> <nter_do_call < asm <utf-8[unix]< 29% ¶ 435: 5 < [Syntax: line:139 (763)]
```

进程的系统调用命令转换为INT 0x80中断的过程

宏定义 `_syscallN()` 见 (include/asm/unistd.h) 用于系统调用的格式转换和参数的传递。N 取 0~5 之间的整数。参数个数为 N 的系统调用由 `_syscallN()` 负责格式转换和参数传递。系统调用号放入 EAX 寄存器，启动 INT 0x80 后，规定返回值送 EAX 寄存器。

聊聊：进程调度算法了解多少

先来先服务、短作业优先、最短剩余时间优先、

- 先来先服务 first-come first-serverd (FCFS)：非抢占式，按照请求的顺序进行调度
优点：有利长作业
缺点：不利短作业，长作业需要执行很长时间，造成了短作业等待时间过长
- 短作业优先 shortest job first (SJF)：非抢占式，估计运行时间最短的顺序进行调度
缺点：长作业有可能会饿死，处于一直等待短作业执行完毕的状态。如果一直有短作业到来，那么长作业永远得不到调度
- 最短剩余时间优先 shortest remaining time next (SRTN)：最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度，当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较，如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待
- 时间片轮转：将所有就绪进程按 FCFS（先来先服务）的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。
时间片轮转算法的效率和时间片的大小有很大关系：因为进程切换都要保存进程的信息并且载入新进程的信息，如果时间片太小，会导致进程切换得太频繁，在进程切换上就会花过多时间。而如果时间片过长，那么实时性就不能得到保证。
- 优先级调度：为每个进程分配一个优先级，按优先级进行调度。
为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级
- 多级反馈队列
一个进程需要执行 100 个时间片，如果采用时间片轮转调度算法，那么需要交换 100 次。
多级队列是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列时间片大小都不同，例如 1,2,4,8,...。进程在第一个队列没执行完，就会被移到下一个队列。这种方式下，之前的进程只需要交换 7 次。每个队列优先权也不同，最上面的优先权最高。因此只有上一个队列

没有进程在排队，才能调度当前队列上的进程。可以将这种调度算法看成是时间片轮转调度算法和优先级调度算法的结合

聊聊：调度算法都有哪些

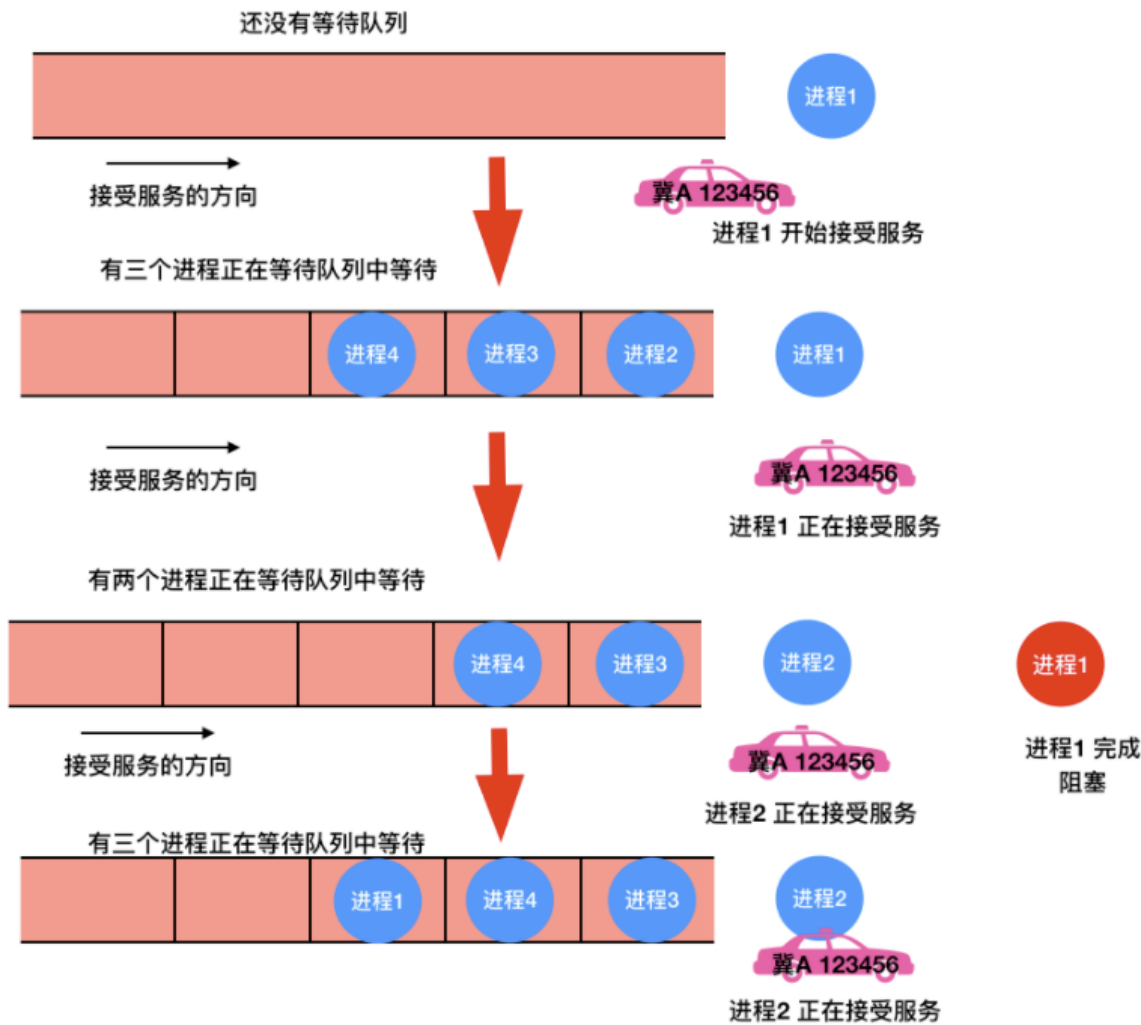
调度算法分为三大类：

- 批处理中的调度、
- 交互系统中的调度、
- 实时系统中的调度

批处理中的调度

先来先服务

很像是先到先得。。。可能最简单的非抢占式调度算法的设计就是 先来先服务(first-come, first-serverd)。使用此算法，将按照请求顺序为进程分配 CPU。最基本的，会有一个就绪进程的等待队列。当第一个任务从外部进入系统时，将会立即启动并允许运行任意长的时间。它不会因为运行时间太长而中断。当其他作业进入时，它们排到就绪队列尾部。当正在运行的进程阻塞，处于等待队列的第一个进程就开始运行。当一个阻塞的进程重新处于就绪态时，它会像一个新到达的任务，会排在队列的末尾，即排在所有进程最后。

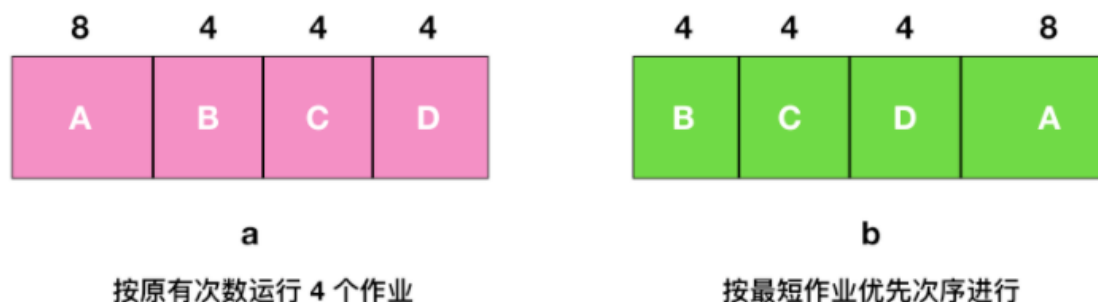


这个算法的强大之处在于易于理解和编程，在这个算法中，一个单链表记录了所有就绪进程。要选取一个进程运行，只要从该队列的头部移走一个进程即可；要添加一个新的作业或者阻塞一个进程，只要把这个作业或进程附加在队列的末尾即可。这是很简单的一种实现。

不过，先来先服务也是有缺点的，那就是没有优先级的关系，试想一下，如果有 100 个 I/O 进程正在排队，第 101 个是一个 CPU 密集型进程，那岂不是需要等 100 个 I/O 进程运行完毕才会等到一个 CPU 密集型进程运行，这在实际情况根本不可能，所以需要优先级或者抢占式进程的出现来优先选择重要的进程运行。

最短作业优先

批处理中，第二种调度算法是 **最短作业优先(Shortest Job First)**，我们假设运行时间已知。例如，一家保险公司，因为每天要做类似的工作，所以人们可以相当精确地预测处理 1000 个索赔的一批作业需要多长时间。当输入队列中有若干个同等重要的作业被启动时，调度程序应使用最短优先作业算法



如上图 a 所示，这里有 4 个作业 A、B、C、D，运行时间分别为 8、4、4、4 分钟。若按图中的次序运行，则 A 的周转时间为 8 分钟，B 为 12 分钟，C 为 16 分钟，D 为 20 分钟，平均时间内为 14 分钟。

现在考虑使用最短作业优先算法运行 4 个作业，如上图 b 所示，目前的周转时间分别为 4、8、12、20，平均为 11 分钟，可以证明最短作业优先是最优的。考虑有 4 个作业的情况，其运行时间分别为 a、b、c、d。第一个作业在时间 a 结束，第二个在时间 a + b 结束，以此类推。平均周转时间为 $(4a + 3b + 2c + d) / 4$ 。显然 a 对平均值的影响最大，所以 a 应该是最短优先作业，其次是 b，然后是 c，最后是 d 它就只能影响自己的周转时间了。

需要注意的是，在所有的进程都可以运行的情况下，最短作业优先的算法才是最优的。

最短剩余时间优先

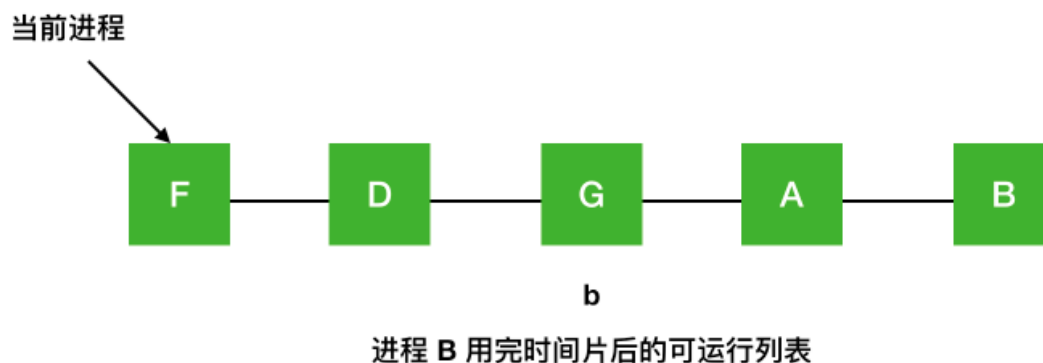
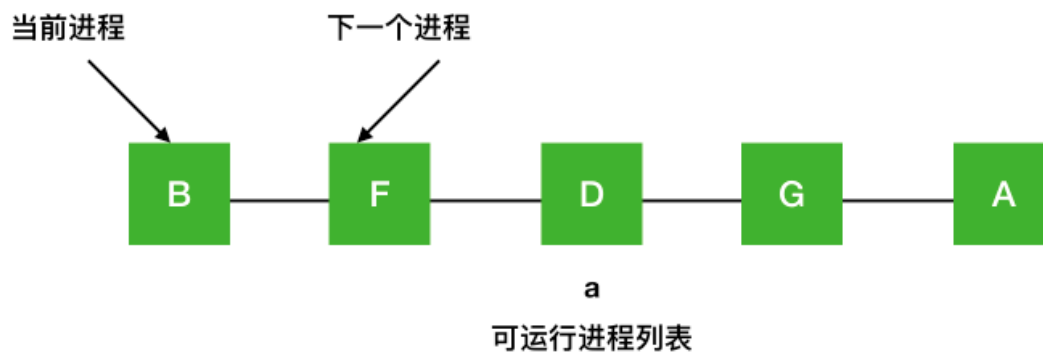
最短作业优先的抢占式版本被称之为 **最短剩余时间优先(Shortest Remaining Time Next)** 算法。使用这个算法，调度程序总是选择剩余运行时间最短的那个进程运行。当一个新作业到达时，其整个时间同当前进程的剩余时间做比较。如果新的进程比当前运行进程需要更少的时间，当前进程就被挂起，而运行新的进程。这种方式能够使短期作业获得良好的服务。

交互式系统中的调度

交互式系统中在个人计算机、服务器和其他系统中都是很常用的，所以有必要来探讨一下交互式调度

轮询调度

一种最古老、最简单、最公平并且最广泛使用的算法就是 **轮询算法(round-robin)**。每个进程都会被分配一个时间段，称为 **时间片(quantum)**，在这个时间片内允许进程运行。如果时间片结束时进程还在运行的话，则抢占一个 CPU 并将其分配给另一个进程。如果进程在时间片结束前阻塞或结束，则 CPU 立即进行切换。轮询算法比较容易实现。调度程序所做的就是维护一个可运行进程的列表，就像下图中的 a，当一个进程用完时间片后就被移到队列的末尾，就像下图的 b。



优先级调度

事实情况是不是所有的进程都是优先级相等的。例如，在一所大学中的等级制度，首先是院长，然后是教授、秘书、后勤人员，最后是学生。这种将外部情况考虑在内就实现了 `优先级调度(priority scheduling)`

某学院等级制度



院长 > 教授 > 秘书 > 后勤人员 > 学生

它的基本思想很明确，每个进程都被赋予一个优先级，优先级高的进程优先运行。

但是也不意味着高优先级的进程能够永远一直运行下去，调度程序会在每个时钟中断期间降低当前运行进程的优先级。如果此操作导致其优先级降低到下一个最高进程的优先级以下，则会发生进程切换。或者，可以为每个进程分配允许运行的最大时间间隔。当时间间隔用完后，下一个高优先级的进程会得到运行的机会。

最短进程优先

对于批处理系统而言，由于最短作业优先常常伴随着最短响应时间，一种方式是根据进程过去的行为进行推测，并执行估计运行时间最短的那一个。假设每个终端上每条命令的预估运行时间为 T_0 ，现在假设测量到其下一次运行时间为 T_1 ，可以用两个值的加权来改进估计时间，即 $aT_0 + (1 - a)T_1$ 。通过选择 a 的值，可以决定是尽快忘掉老的运行时间，还是在一段长时间内始终记住它们。当 $a = 1/2$ 时，可以得到下面这个序列

$$T_0, \quad \frac{T_0}{2} + \frac{T_1}{2}, \quad \frac{T_0}{4} + \frac{T_1}{4} + \frac{T_2}{2}, \quad \frac{T_0}{8} + \frac{T_1}{8} + \frac{T_2}{4} + \frac{T_3}{2}$$

可以看到，在三轮过后， T_0 在新的估计值中所占比重下降至 $1/8$ 。

有时把这种通过当前测量值和先前估计值进行加权平均从而得到下一个估计值的技术称作 **老化** (aging)。这种方法会使用很多预测值基于当前值的情况。

彩票调度

有一种既可以给出预测结果而又有一种比较简单的实现方式的算法，就是 **彩票调度** (lottery scheduling) 算法。他的基本思想为进程提供各种系统资源的 **彩票**。当做出一个调度决策的时候，就随机抽出一张彩票，拥有彩票的进程将获得资源。比如在 CPU 进行调度时，系统可以每秒持有 50 次抽奖，每个中奖进程会获得额外运行时间的奖励。

可以把彩票理解为 buff，这个 buff 有 15% 的几率能让你产生 **速度之靴** 的效果。

公平分享调度

如果用户 1 启动了 9 个进程，而用户 2 启动了一个进程，使用轮转或相同优先级调度算法，那么用户 1 将得到 90 % 的 CPU 时间，而用户 2 将之得到 10 % 的 CPU 时间。

为了阻止这种情况的出现，一些系统在调度前会把进程的拥有者考虑在内。在这种模型下，每个用户都会分配一些 CPU 时间，而调度程序会选择进程并强制执行。因此如果两个用户每个都会有 50% 的 CPU 时间片保证，那么无论一个用户有多少个进程，都将获得相同的 CPU 份额。

聊聊：影响调度程序的指标是什么

会有下面几个因素决定调度程序的好坏

- CPU 使用率：

CPU 正在执行任务（即不处于空闲状态）的时间百分比。

- 等待时间

这是进程轮流执行的时间，也就是进程切换的时间

- 吞吐量

单位时间内完成进程的数量

- 响应时间

这是从提交流程到获得有用输出所经过的时间。

- 周转时间

从提交流程到完成流程所经过的时间。

聊聊：什么是 RR 调度算法

RR (round-robin) 调度算法主要针对分时系统，RR 的调度算法会把时间片以相同的部分并循环的分配给每个进程，RR 调度算法没有优先级的概念。

这种算法的实现比较简单，而且每个线程都会占有时间片，并不存在线程饥饿的问题。

Copy-on-write（写时拷贝）

copy-on-write，写时拷贝，是计算机程序设计领域的一种优化策略，

其核心思想是，当有多个调用者都需要请求相同资源时，一开始资源只会有一份，多个调用者共同读取这一份资源，当某个调用者需要修改数据的时候，才会分配一块内存，将数据拷贝过去，供这个调用者使用，而其他调用者依然还是读取最原始的那份数据。

每次有调用者需要修改数据时，就会重复一次拷贝流程，供调用者修改使用。

使用 copy-on-write 可以避免或者减少数据的拷贝操作，极大的提高性能，其应用十分广泛，

例如 Linux 的 fork 调用，Linux 的文件管理系统，一些数据库服务，Java 中的 CopyOnWriteArrayList，C98/C03 中的 std::string 等等。

Linux中的fork()

Linux 在启动过程中，会初始化内核，而内核初始化的最后一步，是创建一个 PID 为 1 的超级进程，又叫做根进程。系统中所有的其他进程，都是由这个根进程直接或者间接产生的，而产生进程的方式，就是利用 fork 系统调用，fork 是类 Unix 操作系统上创建进程的主要方法。

fork() 的函数原型很简单：

```
pid_t fork();
```

我们来看一个简单的例子：

```
#include <unistd.h>
#include <stdio.h>
int main() {
    int pid = fork();
    if (pid == -1) {
        return -1;
    }
    if (pid > 0) {
        printf("Hi, father: %d\n", getpid());
        return 0;
    } else {
        printf("Hi, child: %d\n", getpid());
        return 0;
    }
}
```

通过 gcc 编译之后执行，输出：

```
Hi, father: 7562
Hi, child: 7563
```

从输出来看，if 和 else 居然都执行了，因为用 fork() 有个神奇的地方，一次调用，两次返回。

调用 fork() 之后，会出现两个进程，一个是子进程，一个是父进程，在子进程中，fork() 返回 0，在父进程中，fork() 返回新创建的子进程的进程 ID，我们可以通过 fork() 函数的返回值来判断当前进程是子进程还是父进程。两个进程都会从调用 fork() 的地方继续执行。

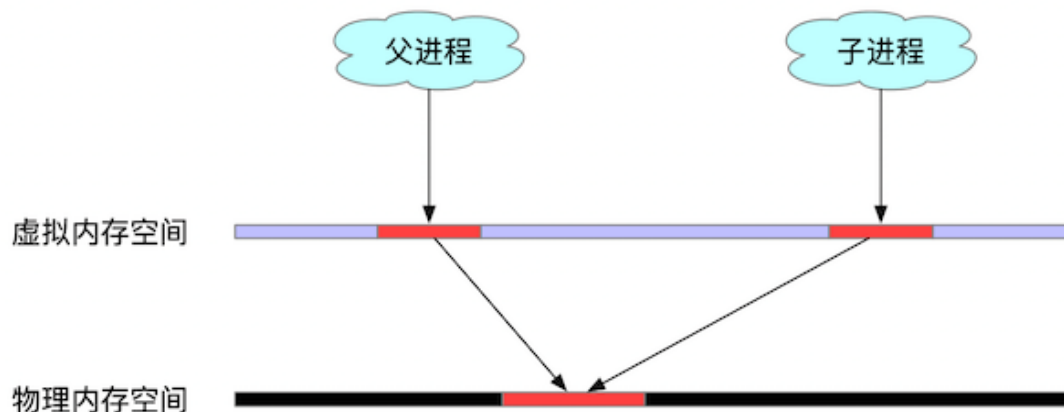
fork()中的copy-on-write

fork 进程之后，父进程中的数据怎么办？常规思路是，给予进程重新开辟一块物理内存，将父进程的数据拷贝到子进程中，拷贝完之后，父进程和子进程之间的数据段和堆栈是相互独立的。这样做会带来两个问题：

- 拷贝本身会有 CPU 和内存的开销；
- fork 出来的子进程在此后多会执行 exec() 系统调用。

也就是说，绝大部分情况下，fork 一个子进程会耗费 CPU 和内存资源，但是马上又被子进程抛弃不用了，那么资源的开销就显得毫无意义，于是出于效率考虑，Linux 引入了 copy-on-write 技术。

在 fork() 调用之后，只会给子进程分配虚拟内存地址，而父子进程的虚拟内存地址虽然不同，但是映射到物理内存上都是同一块区域，子进程的代码段、数据段、堆栈都是指向父进程的物理空间。



并且此时父进程中所有对应的内存页都会被标记为只读，父子进程都可以正常读取内存数据，当其中某个进程需要更新数据时，检测到内存页是 read-only 的，内存管理单元（MMU）便会抛出一个页面异常中断，（page-fault），在处理异常时，内核便会把触发异常的内存页拷贝一份（其他内存页还是共享的一份），让父子进程各自持有一份。

这样做的好处不言而喻，能极大的提高 fork 操作时的效率，但是坏处是，如果 fork 之后，两个进程各自频繁的更新数据，则会导致大量的分页错误，这样就得得不偿失了。

Java中的CopyOnWrite容器

Java 中有两个容器：CopyOnWriteArrayList 和 CopyOnWriteArraySet，从名字就可以看出，其实现思想也是参考了 copy-on-write 技术。

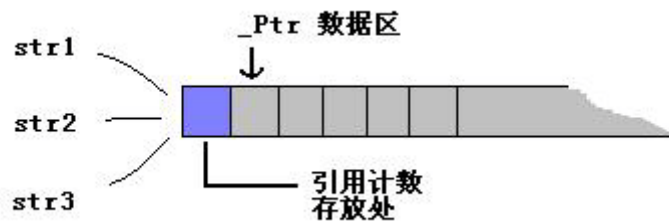
当我们往一个 CopyOnWrite 的容器中添加数据的时候，并不会直接添加到当前容器中，而是会拷贝出一个新的容器，然后往新的容器里添加数据，在添加过程中，所有的读操作都会指向旧的容器，添加操作完成之后，再将原容器的引用指向新的容器。为了避免同时有多个线程更新数据，从而拷贝出多个容器的副本，会在拷贝容器的时候进行加锁。

这样做的好处是对 CopyOnWrite 容器进行读操作的时候并不需要加锁，因为当前容器不会添加任何元素。所以 CopyOnWrite 容器也是一种读写分离的思想，读和写不同的容器。

C++中的std::string

C98/C03 中的 std::string 使用了 copy-on-write 技术，在 C++11 标准中为了提高并行性取消了这一策略。

C++ 在分配一个 string 对象时，会在数据区的前面多分配一点空间，用于存储 string 的引用计数。



当触发一个 string 的拷贝构造函数或者赋值函数时，便会对这个引用计数加一。需要修改内容时，如果引用计数不为零，表示有人在共享这块内存，那么自己需要先做一份拷贝，然后把引用计数减去一，再把数据拷贝过来。

Redis中的COW

Redis 中，执行 BGSAVE 命令，来生成 RDB 文件时，本质就是调用了 Linux 的系统调用 `fork()` 命令，Linux 下 `fork()` 系统调用，实现了 copy-on-write 写时复制。

Copy On Write技术实现原理

`fork()` 之后，kernel 把父进程中所有的内存页的权限都设为 read-only，然后子进程的地址空间指向父进程。当父子进程都只读内存时，相安无事。当其中某个进程写内存时，CPU 硬件检测到内存页是 read-only 的，于是触发页异常中断（page-fault），陷入 kernel 的一个中断例程。中断例程中，kernel 就会把触发的异常的页复制一份，于是父子进程各自持有独立的一份。

Copy On Write技术好处

COW 技术可减少分配和复制大量资源时带来的瞬间延时。

COW 技术可减少不必要的资源分配。比如 `fork` 进程时，并不是所有的页面都需要复制，父进程的代码段和只读数据段都不被允许修改，所以无需复制。

Copy On Write技术缺点

如果在 `fork()` 之后，父子进程都还需要继续进行写操作，那么会产生大量的分页错误(页异常中断 page-fault)，这样就得不偿失。

总结

`fork` 出的子进程共享父进程的物理空间，当父子进程有内存写入操作时，read-only 内存页发生中断，将触发的异常的内存页复制一份(其余的页还是共享父进程的)。

`fork` 出的子进程功能实现和父进程是一样的。如果有需要，我们会用 `exec()` 把当前进程映像替换成新的进程文件，完成自己想要实现的功能。

动态库与静态库区别

静态连接库就是把(lib)文件中用到的函数代码直接链接进目标程序，程序运行的时候不再需要其它的库文件；动态链接就是把调用的函数所在文件模块（DLL）和调用函数在文件中的位置等信息链接进目标程序，程序运行的时候再从DLL中寻找相应函数代码，因此需要相应 DLL 文件的支持。

动态库与静态库

通常情况下，对函数库的链接是放在编译时期（compile time）完成的。所有相关的对象文件（object file）与牵涉到的函数库（library）被链接合成一个可执行文件（executable file）。程序在运行时，与函数库再无瓜葛，因为所有需要的函数已拷贝到自己门下。

所以这些函数库被成为静态库（static library），通常文件名为“libxxx.a”的形式。其实，我们也可以把对一些库函数的链接载入推迟到程序运行的时期（runtime）。这就是如雷贯耳的动态链接库（dynamic link library）技术。

动态链接

动态链接方法：LoadLibrary()/GetProcAddress() 和 FreeLibrary()，使用这种方式的程序并不在一开始就完成动态链接，而是直到真正调用动态库代码时，载入程序才计算(被调用的那部分)动态代码的逻辑地址，然后等到某个时候，程序又需要调用另外某块动态代码时，载入程序又去计算这部分代码的逻辑地址，所以，这种方式使程序初始化时间较短，但运行期间的性能比不上静态链接的程序。

静态链接

静态链接方法：`#pragma comment(lib, "test.lib")`，静态链接的时候，载入代码就会把程序会用到的动态代码或动态代码的地址确定下来。

静态库的链接可以使用静态链接，动态链接库也可以使用这种方法链接导入库。

静态库和动态库的区别

在软件开发的过程中，大家经常会或多或少的使用别人编写的或者系统提供的动态库或静态库，但是究竟是使用静态库还是动态库呢？他们的适用条件是什么呢？

简单的说，静态库和应用程序编译在一起，在任何情况下都能运行，而动态库是动态链接，顾名思义就是在应用程序启动的时候才会链接，所以，当用户的系统上没有该动态库时，应用程序就会运行失败。再看它们的特点：

动态库

1. 类库的名字一般是 libxxx.so
2. 共享：多个应用程序可以使用同一个动态库，启动多个应用程序的时候，只需要将动态库加载到内存一次即可；
3. 开发模块好：要求设计者对功能划分的比较好。
4. 动态函数库的改变并不影响你的程序，所以动态函数库的升级比较方便。

静态库

1. 类库的名字一般是 libxxx.a
2. 代码的装载速度快，执行速度也比较快，因为编译时它只会把你需要的那部分链接进去。
3. 应用程序相对比较大，如果多个应用程序使用的话，会被装载多次，浪费内存。
4. 如果静态函数库改变了，那么你的程序必须重新编译。

如果你的系统上有多个应用程序都使用该库的话，就把它编译成动态库，这样虽然刚启动的时候加载比较慢，但是多任务的时候会节省内存；如果你的系统上只有一到两个应用使用该库，并且使用的API比较少的话，就编译成静态库吧，一般的静态库还可以进行裁剪编译，这样应用程序可能会比较大，但是启动的速度会大大提高。

静态库与动态库优缺点

静态链接库的优点

1. 代码装载速度快，执行速度略比动态链接库快；
2. 只需保证在开发者的计算机中有正确的 .LIB 文件，在以二进制形式发布程序时不需考虑在用户的计算机上 .LIB 文件是否存在及版本问题，可避免 DLL 地狱等问题。

动态链接库的优点

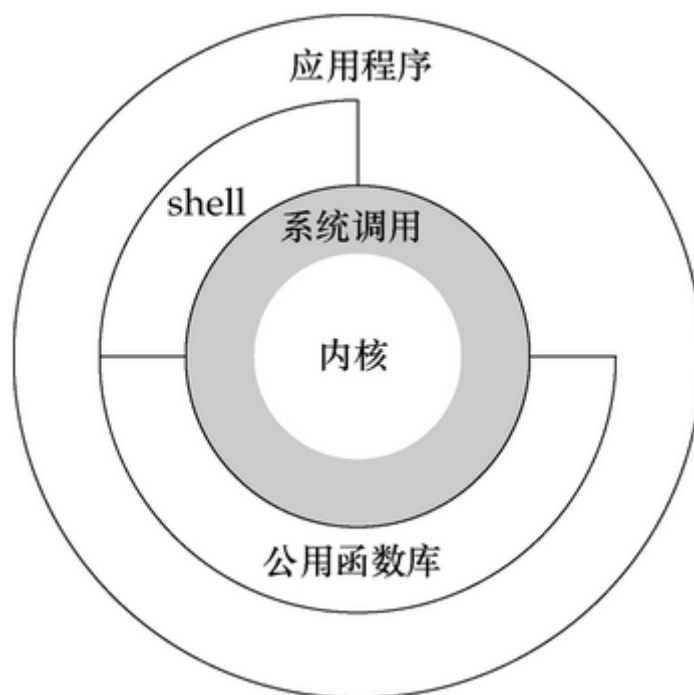
1. 更加节省内存并减少页面交换；
2. DLL 文件与 EXE 文件独立，只要输出接口不变（即名称、参数、返回值类型和调用约定不变），更换 DLL 文件不会对 EXE 文件造成任何影响，因而极大地提高了可维护性和可扩展性；
3. 不同编程语言编写的程序只要按照函数调用约定就可以调用同一个 DLL 函数；
4. 适用于大规模的软件开发，使开发过程独立、耦合度小，便于不同开发者和开发组织之间进行开发和测试。

不足之处

1. 使用静态链接生成的可执行文件体积较大，包含相同的公共代码，造成浪费；
2. 使用动态链接库的应用程序不是自完备的，它依赖的 DLL 模块也要存在，如果使用载入时动态链接，程序启动时发现 DLL 不存在，系统将终止程序并给出错误信息。

而使用运行时动态链接，系统不会终止，但由于 DLL 中的导出函数不可用，程序会加载失败；速度比静态链接慢。当某个模块更新后，如果新模块与旧的模块不兼容，那么那些需要该模块才能运行的软件，统统撕掉。这在早期 Windows 中很常见。

内核态与用户态



概念

Linux 的设计哲学之一就是：对不同的操作赋予不同的执行等级，就是所谓特权的概念，即与系统相关的一些特别关键的操作必须由最高特权的程序来完成。

Intel 的 X86 架构的 CPU 提供了 0 到 3 四个特权级，数字越小，特权越高，Linux 操作系统中主要采用了 0 和 3 两个特权级，分别对应的就是内核态(Kernel Mode)与用户态(User Mode)。

- **内核态：** CPU 可以访问内存所有数据,包括外围设备（硬盘、网卡），CPU 也可以将自己从一个程序切换到另一个程序；
- **用户态：** 只能受限的访问内存，且不允许访问外围设备，占用 CPU 的能力被剥夺，CPU 资源可以被其他程序获取；

Linux 中任何一个用户进程被创建时都包含 2 个栈：内核栈，用户栈，并且是进程私有的，从用户态开始运行。内核态和用户态分别对应内核空间与用户空间，内核空间中存放的是内核代码和数据，而进程的用户空间中存放的是用户程序的代码和数据。不管是内核空间还是用户空间，它们都处于虚拟空间中。

内核空间相关

- 内核空间：存放的是内核代码和数据，处于虚拟空间；
- 内核态：当进程执行系统调用而进入内核代码中执行时，称进程处于内核态，此时CPU处于特权级最高的0级内核代码中执行，当进程处于内核态时，执行的内核代码会使用当前进程的内核栈，每个进程都有自己的内核栈；
- CPU 堆栈指针寄存器指向：内核栈地址；
- 内核栈：进程处于内核态时使用的栈，存在于内核空间；
- 处于内核态进程的权利：处于内核态的进程，当它占有 CPU 的时候，可以访问内存所有数据和所有外设，比如硬盘，网卡等等；

用户空间相关

- 用户空间：存放的是用户程序的代码和数据，处于虚拟空间；
- 用户态：当进程在执行用户自己的代码（非系统调用之类的函数）时，则称其处于用户态，CPU 在特权级最低的3级用户代码中运行，当正在执行用户程序而突然被中断程序中断时，此时用户程序也可以象征性地称为处于进程的内核态，因为中断处理程序将使用当前进程的内核栈；
- CPU 堆栈指针寄存器指向：用户堆栈地址；
- 用户堆栈：进程处于用户态时使用的堆栈，存在于用户空间；
- 处于用户态进程的权利：处于用户态的进程，当它占有 CPU 的时候，只可以访问有限的内存，而且不允许访问外设，这里说的有限的内存其实就是用户空间，使用的是用户堆栈；

内核态和用户态的切换

系统调用

所有用户程序都是运行在用户态的，但是有时候程序确实需要做一些内核态的事情，例如从硬盘读取数据等。而唯一可以做这些事情的就是操作系统，所以此时程序就需要先操作系统请求以程序的名义来执行这些操作。这时需要一个这样的机制：用户态程序切换到内核态，但是不能控制在内核态中执行的指令。

这种机制叫系统调用，在 CPU 中的实现称之为陷阱指令(Trap Instruction)。

异常事件

当 CPU 正在执行运行在用户态的程序时，突然发生某些预先不可知的异常事件，这个时候就会触发从当前用户态执行的进程转向内核态执行相关的异常事件，典型的如缺页异常。

外围设备的中断

当外围设备完成用户的请求操作后，会向 CPU 发出中断信号，此时，CPU 就会暂停执行下一条即将要执行的指令，转而去执行中断信号对应的处理程序，如果先前执行的指令是在用户态下，则自然就发生从用户态到内核态的转换。

注意：系统调用的本质其实也是中断，相对于外围设备的硬中断，这种中断称为软中断，这是操作系统为用户特别开放的一种中断，如 Linux int 80h 中断。所以从触发方式和效果上来看，这三种切换方式是完全一样的，都相当于是执行了一个中断响应的过程。但是从触发的对象来看，系统调用是进程主动请求切换的，而异常和硬中断则是被动的。

用户态到内核态具体的切换步骤

1. 从当前进程的描述符中提取其内核栈的 ss0 及 esp0 信息。
2. 使用 ss0 和 esp0 指向的内核栈将当前进程的 cs, eip, eflags, ss, esp 信息保存起来，这个过程也完成了由用户栈到内核栈的切换过程，同时保存了被暂停执行的程序的下一条指令。
3. 将先前由中断向量检索得到的中断处理程序的 cs, eip 信息装入相应的寄存器，开始执行中断处理程序，这时就转到了内核态的程序执行了。

虚拟内存篇

虚拟内存教程

第一层理解

每个进程都有自己独立的 4G 内存空间，各个进程的内存空间具有类似的结构。

一个新进程建立的时候，将会建立起自己的内存空间，此进程的数据，代码等从磁盘拷贝到自己的进程空间，哪些数据在哪里，都由进程控制表中的 task_struct 记录，task_struct 中记录中一条链表，记录中内存空间的分配情况，哪些地址有数据，哪些地址无数据，哪些可读，哪些可写，都可以通过这个链表记录。

每个进程已经分配的内存空间，都与对应的磁盘空间映射，但是：

- 计算机明明没有那么多内存（n 个进程的话就需要 n*4G）内存
- 建立一个进程，就要把磁盘上的程序文件拷贝到进程对应的内存中去，对于一个程序对应的多个进程这种情况，浪费内存！

第二层理解

每个进程的 4G 内存空间只是虚拟内存空间，每次访问内存空间的某个地址，都需要把地址翻译为实际物理内存地址。

所有进程共享同一物理内存，每个进程只把自己目前需要的虚拟内存空间映射并存储到物理内存上。进程要知道哪些内存地址上的数据在物理内存上，哪些不在，还有在物理内存上的哪里，需要用页表来记录。

页表的每一个表项分两部分，第一部分记录此页是否在物理内存上，第二部分记录物理内存页的地址（如果在的话）。当进程访问某个虚拟地址，去看页表，如果发现对应的数据不在物理内存中，则缺页异常。

缺页异常的处理过程，就是把进程需要的数据从磁盘上拷贝到物理内存中，如果内存已经满了，没有空地方了，那就找一个页覆盖，当然如果被覆盖的页曾经被修改过，需要将此页写回磁盘

虚拟内存总结

既然每个进程的内存空间都是一致而且固定的，所以链接器在链接可执行文件时，可以设定内存地址，而不用去管这些数据最终实际的内存地址，这是有独立内存空间的好处。

当不同的进程使用同样的代码时，比如库文件中的代码，物理内存中可以只存储一份这样的代码，不同的进程只需要把自己的虚拟内存映射过去就可以了，节省内存。

在程序需要分配连续的内存空间的时候，只需要在虚拟内存空间分配连续空间，而不需要实际物理内存的连续空间，可以利用碎片。

另外，事实上，在每个进程创建加载时，内核只是为进程“创建”了虚拟内存的布局，具体就是初始化进程控制表中内存相关的链表，实际上并不立即就把虚拟内存对应位置的程序数据和代码（比如 .text .data 段）拷贝到物理内存中，只是建立好虚拟内存和磁盘文件之间的映射就好（叫做存储器映射），等到运行到对应的程序时，才会通过缺页异常，来拷贝数据。

还有进程运行过程中，要动态分配内存，比如 malloc 时，也只是分配了虚拟内存，即为这块虚拟内存对应的页表项做相应设置，当进程真正访问到此数据时，才引发缺页异常。

虚拟存储器

可以认为虚拟空间都被映射到了磁盘空间中，（事实上也是按需要映射到磁盘空间上，通过 mmap），并且由页表记录映射位置，当访问到某个地址的时候，通过页表中的有效位，可以得知此数据是否在内存中，如果不是，则通过缺页异常，将磁盘对应的数据拷贝到内存中，如果没有空闲内存，则选择牺牲页面，替换其他页面。

mmap 是用来建立从虚拟空间到磁盘空间的映射的，可以将一个虚拟空间地址映射到一个磁盘文件上，当不设置这个地址时，则由系统自动设置，函数返回对应的内存地址（虚拟地址），当访问这个地址的时候，就需要把磁盘上的内容拷贝到内存了，然后就可以读或者写，最后通过 munmap 可以将内存上的数据换回到磁盘，也就是解除虚拟空间和内存空间的映射，这也是一种读写磁盘文件的方法，也是一种进程共享数据的方法 共享内存

物理内存

在内核态申请内存比在用户态申请内存要更为直接，它没有采用用户态那种延迟分配内存技术。内核认为一旦有内核函数申请内存，那么就必须立刻满足该申请内存的请求，并且这个请求一定是正确合理的。相反，对于用户态申请内存的请求，内核总是尽量延后分配物理内存，用户进程总是先获得一个虚拟内存区的使用权，最终通过缺页异常获得一块真正的物理内存。

物理内存的内核映射

IA32 架构中内核虚拟地址空间只有 1GB 大小（从 3GB 到 4GB），因此可以直接将 1GB 大小的物理内存（即常规内存）映射到内核地址空间，但超出 1GB 大小的物理内存（即高端内存）就不能映射到内核空间。为此，内核采取了下面的方法使得内核可以使用所有的物理内存。

高端内存不能全部映射到内核空间，也就是说这些物理内存没有对应的线性地址。不过，内核为每个物理页框都分配了对应的页框描述符，所有的页框描述符都保存在 mem_map 数组中，因此每个页框描述符的线性地址都是固定存在的。内核此时可以使用 alloc_pages() 和 alloc_page() 来分配高端内存，因为这些函数返回页框描述符的线性地址。

内核地址空间的后 128MB 专门用于映射高端内存，否则，没有线性地址的高端内存不能被内核所访问。这些高端内存的内核映射显然是暂时映射的，否则也只能映射 128MB 的高端内存。当内核需要访问高端内存时就临时在这个区域进行地址映射，使用完毕之后再用来进行其他高端内存的映射。

由于要进行高端内存的内核映射，因此直接能够映射的物理内存大小只有 896MB，该值保存在 high_memory 中。内核地址空间的线性地址区间如下图所示

物理内存管理机制

基于物理内存存在内核空间中的映射原理，物理内存的管理方式也有所不同。内核中物理内存的管理机制主要有伙伴算法，slab 高速缓存和 vmalloc 机制。其中伙伴算法和slab高速缓存都在物理内存映射区分配物理内存，而 vmalloc 机制则在高端内存映射区分配物理内存。

非连续内存区内内存的分配

内核通过 vmalloc() 来申请非连续的物理内存，若申请成功，该函数返回连续内存区的起始地址，否则，返回NULL。

vmalloc() 和 kmalloc() 申请的内存有所不同，kmalloc() 所申请内存的线性地址与物理地址都是连续的，而 vmalloc() 所申请的内存线性地址连续而物理地址则是离散的，两个地址之间通过内核页表进行映射。

vmalloc() 的内存分配原理与用户态的内存分配相似，都是通过连续的虚拟内存来访问离散的物理内存，并且虚拟地址和物理地址之间是通过页表进行连接的，通过这种方式可以有效的使用物理内存。

但是应该注意的是，vmalloc() 申请物理内存时是立即分配的，因为内核认为这种内存分配请求是正当而且紧急的；

相反，用户态有内存请求时，内核总是尽可能的延后，毕竟用户态跟内核态不在一个特权级。

页面置换算法

操作系统中的页面置换算法主要包括最佳置换算法（OPT，Optimal）、先进先出置换算法（FIFO）、最近最久未使用置换算法（LRU，Least Recently Used）、时钟置换算法和改进型的时钟置换算法。

最佳置换算法（OPT，Optimal）

算法思想

每次选择淘汰的页面将是以后永不使用，或者在最长时间内不再被访问的页面，这样可以保证最低的缺页率。

举例说明

假设系统为进程分配了三个内存块，并考虑到有以下页面号引用串（会依次访问这些页面）：
7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1。

1. 第一个访问的是 7 号页，内存中没有此页，由缺页中断机构将 7 号页调入内存。此时有三个可用的内存块，不需要置换。即第一次(7)：7
2. 同理，第二个访问的是 0 号页，和第一次一样，第三次访问的是 1 号页，同样 1 号页也会被调入内存，1 号内被调入内存后，此时分配给该进程内存空间已占满。
 - 第二次(0)：7 0
 - 第三次(1)：7 0 1
3. 第四个访问的页是 2 号页，此时内存已经用完，需要将一个页调出内存，根据最佳置换算法，淘汰一个以后永不使用或最长时间不使用的，此时内存中的页有 7、0、1，查看待访问页号序列中这三个页号的先后位置，下图可以看到，0 号页和 1 号页在不久又会被访问到，而 7 号页需要被访问的时间最久。所以该算法会淘汰 7 号页。



按照此算法依次执行，最后的结果如下：

第一次(7) : 7
第二次(0): 7 0
第三次(1): 7 0 1
第四次(2): 0 1 2
第五次(0): 0 1 2 (命中)
第六次(3) : 0 3 1
第七次(0) : 0 3 1 (命中)
第八次(4) : 3 2 4
第九次(2) : 3 2 4 (命中)
第十次(3) : 3 2 4 (命中)
第十一次(0) : 3 2 0
第十二次(3) : 3 2 0 (命中)
.....

结果图：

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
内存块号1	7	7	7	2		2		2			2			2				7		
内存块号2		0	0	0		0		4			0			0				0		
内存块号3			1	1		3		3			3			1				1		
是否缺页	√	√	√	√		√		√			√			√				√		

整个过程缺页中断发生了 9 次，页面置换发生了 6 次。缺页率 = 9 / 20 = 45%。

注：缺页时未必发生页面置换，若还有可用的空闲内存空间就不用进行页面置换。

最佳置换算法可以保证最低的缺页率，但是实际上，只有进程执行的过程中才能知道接下来会访问到的是哪个页面。操作系统无法提前预判页面的访问序列。因此，最佳置换算法是无法实现的。

先进先出置换算法（FIFO）

算法思想

每次选择淘汰的页面是最早进入内存的页面。

举例说明

该算法很简单，每次淘汰最在内存中待时间最久的各个，下面分别给出系统为进程分为配三个内存块和四个内存块的执行情况图。访问序列为 3,2,1,0,3,2,4,3,2,1,0,4。

分配三个内存块的情况：

```
第一次(3) : 3
第二次(2) : 3 2
第三次(1) : 3 2 1
第四次(0) : 2 1 0
第五次(3) : 1 0 3
第六次(2) : 0 3 2
第七次(4) : 3 2 4
第八次(3) : 3 2 4 (命中)
第九次(2) : 3 2 4 (命中)
第十次(1) : 2 4 1
第十一次(0) : 4 1 0
第十二次(4) : 4 1 0 (命中)
```

分配三个内存块时，缺页次数：9 次。

分配四个内存块的情况：

```
第一次(3) : 3
第二次(2) : 3 2
第三次(1) : 3 2 1
第四次(0) : 3 2 1 0
第五次(3) : 3 2 1 0 (命中)
第六次(2) : 3 2 1 0 (命中)
第七次(4) : 2 1 0 4
第八次(3) : 1 0 4 3
第九次(2) : 0 4 3 2
第十次(1) : 4 3 2 1
第十一次(0) : 3 2 1 0
第十二次(4) : 2 1 0 4
```

分配四个内存块时，缺页次数：10 次。当为进程分配的物理块数增大时，缺页次数不减反增的异常现象称为贝莱迪 (Belay) 异常。

只有 FIFO 算法会产生 Belay 异常。另外，FIFO 算法虽然实现简单，但是该算法与进程实际运行时的规律不适应。因为先进入的页面也有可能最经常被访问。因此，算法性能差。

最近最久未使用置换算法 (LRU)

算法思想

每次淘汰的页面是最近最久未使用的页面。

实现方法

赋予每个页面对应的页表项中，用访问字段记录该页面自上次被访问以来所经历的时间 t 。当需要淘汰一个页面时，选择现有页面中 t 最大的页面，即最近最久未使用。

页号	内存块号	状态位	访问字段	修改位	外存地址
----	------	-----	------	-----	------

举例说明

加入某系统为某进程分配了四个内存块，并考虑到有以下页面号引用串：

1,8,1,7,8,2,7,2,1,8,3,8,2,1,3,1,7,1,3,7

这里先直接给出答案：

第一次(1) : 1
 第二次(8) : 1 8
 第三次(1) : 8 1 (命中) (由于1号页又被访问过了，所以放到最后)
 第四次(7) : 8 1 7
 第五次(8) : 1 7 8 (命中)
 第六次(2) : 1 7 8 2
 第七次(7) : 1 8 2 7 (命中)
 第八次(2) : 1 8 7 2 (命中)
 第九次(1) : 8 7 2 1 (命中)
 第十次(8) : 7 2 1 8 (命中)
 第十一次(3) : 2 1 8 3
 第十二次(8) : 2 1 3 8 (命中)
 第十三次(2) : 1 3 8 2 (命中)
 第十四次(1) : 3 8 2 1 (命中)
 第十五次(3) : 8 2 1 3 (命中)
 第十六次(1) : 8 2 3 1 (命中)
 第十七次(7) : 2 3 1 7

这里前 10 次都 1、8、7、2 这四个页，四个内存块号正好可以满足，当第 11 次要访问的 3 号页进入内存时，需要从 1、8、7、2 这四个页淘汰一个页，按照该算法，从页号为3的开始，从右往左一次找到这 4 个页第一次出现的地方，在最左边的就是最近最少使用的页。

如下图所示，所以该算法最终淘汰的是 7 号页。同时直接从第十次的访问结果 7 2 1 8 也可以直接看出，7 号页在最前面，是最久没有被访问过的，所以淘汰应该是 7 号页。



结果图

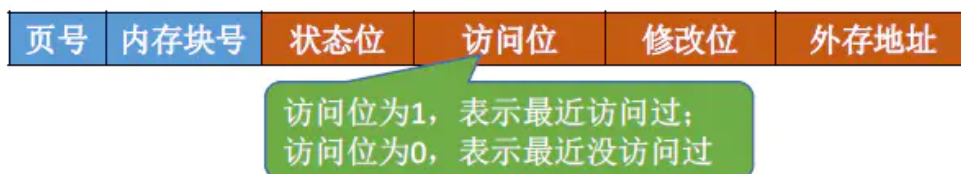
访问页面	1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
内存块号1	1	1		1		1					1						1			
内存块号2		8		8		8					8						7			
内存块号3				7		7					3						3			
内存块号4						2					2						2			
是否缺页	√	√		√		√					√						√			

时钟置换算法

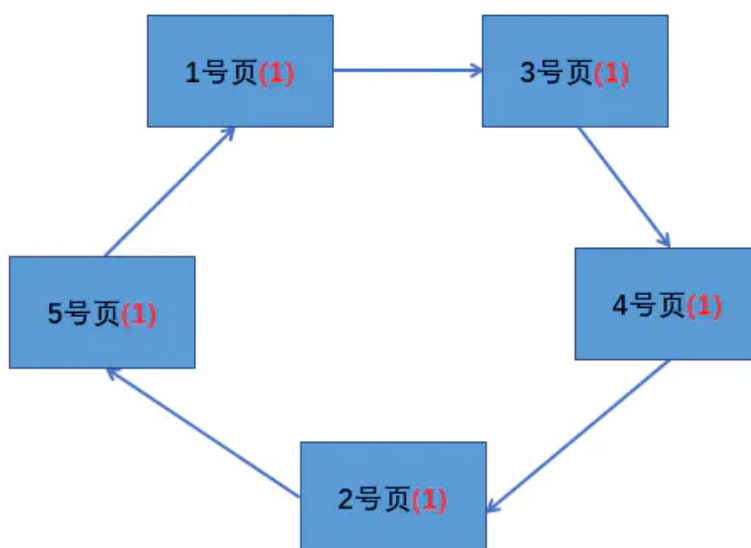
算法思想

最佳置换算法性能最好，但无法实现。先进先出置换算法实现简单，但是算法性能差。最近最久未使用置换算法性能好，是最接近 OPT 算法性能的，但是实现起来需要专门的硬件支持，算法开销大。时钟置换算法是一种性能和开销均平衡的算法。又称 CLOCK 算法，或最近未用算法（NRU，Not Recently Used）。

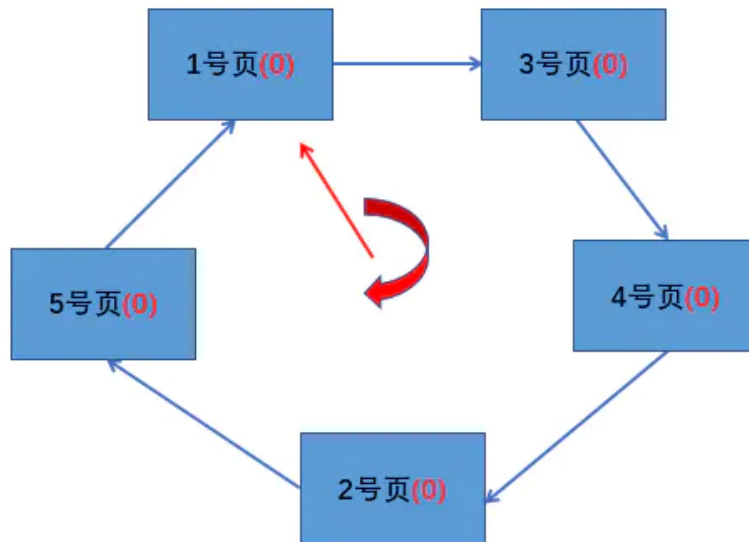
简单 CLOCK 算法算法思想：为每个页面设置一个访问位，再将内存中的页面都通过链接指针链接成一个循环队列。当某个页被访问时，其访问位置 1。当需要淘汰一个页面时，只需检查页的访问位。如果是 0，就选择该页换出；如果是 1，暂不换出，将访问位改为 0，继续检查下一个页面，若第一轮扫描中所有的页面都是 1，则将这些页面的访问位一次置为 0 后，再进行第二轮扫描（第二轮扫描中一定会有访问位为 0 的页面，因此简单的 CLOCK 算法选择一个淘汰页面最多会经过两轮扫描）。



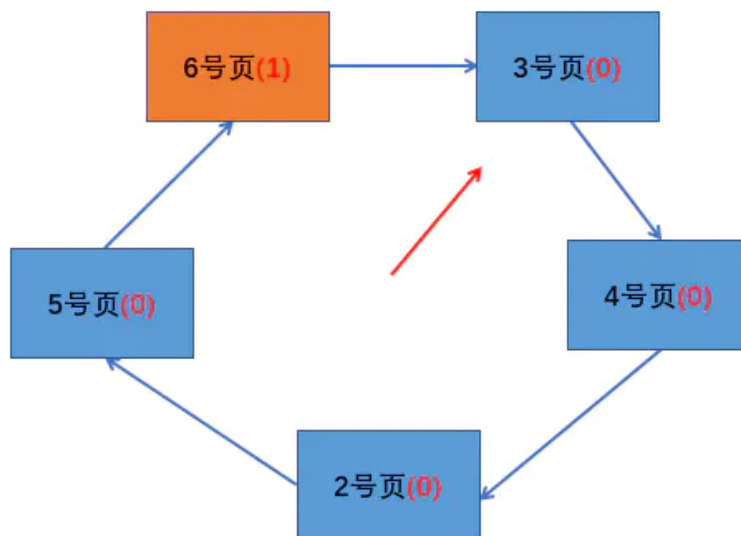
例如，假设某系统为某进程分配了五个内存块，并考虑有以下页面号引用串：1,3,4,2,5,6,3,4,7。刚开始访问前 5 个页面，由于都是刚刚被访问所以它们的访问位都是 1，在内存的页面如下图所示：



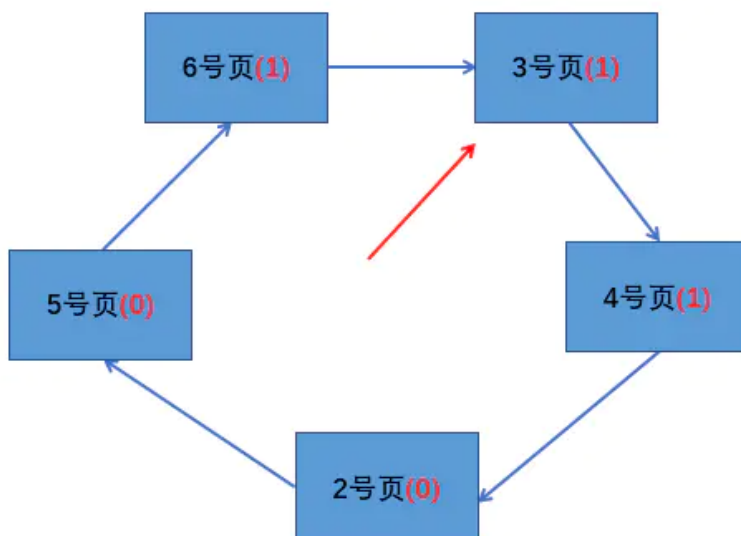
此时页面 6 需要进入内存，那么需要从中淘汰一个页面，于是从循环队列的队首（1 号页）开始扫描，尝试找到访问位为 0 的页面。经过一轮扫描发现所有的访问位都是 1，经过一轮扫描后，需要将所有的页面标志位设置为 0，如下图：



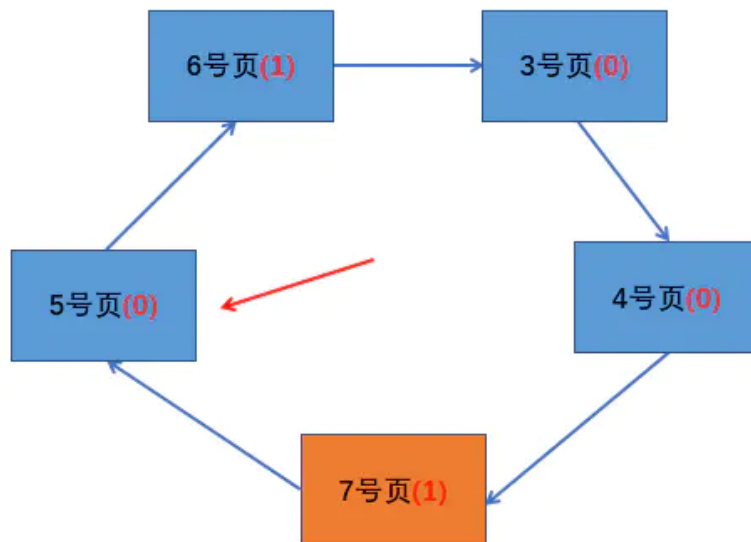
之后进行第二轮扫描，发现 1 号页的访问位为 0，所以换出 1 号页，同时指针指向下一页，如下图：



接下来是访问 3 号页和 4 号页，这两个页都在内存中，直接访问，并将访问位改为 1。在访问 3 号页和 4 号页时指针不需要动，指针只有在缺页置换时才移动下一页。如下图：



最后，访问 7 号页，此时从 3 号页开始扫描循环队列，扫描过程中将访问位为 1 的页的访问位改为 0，并找到第一个访问位为 0 的页，即 2 号页，将 2 号页置换为 7 号页，最后将指针指向 7 号页的下一页，即 5 号页。如下图：



这个算法指针在扫描的过程就像时钟一样转圈，才被称为时钟置换算法。

改进型的时钟置换算法

算法思想

简单的时钟置换算法仅考虑到了一个页面最近是否被访问过。事实上，如果淘汰的页面没有被修改过，就不需要执行 I/O 操作写回外存。只有淘汰的页面被修改过时，才需要写回外存。因此，除了考虑一个页面最近有没有被访问过之外，操作系统还需要考虑页面有没有被修改过。

改进型时钟置换算法的算法思想：在其他条件相同时，应该优先淘汰没有被修改过的页面，从而来避免 I/O 操作。为了方便讨论，用（访问位，修改位）的形式表示各页面的状态。如（1,1）表示一个页面近期被访问过，且被修改过。

算法规则

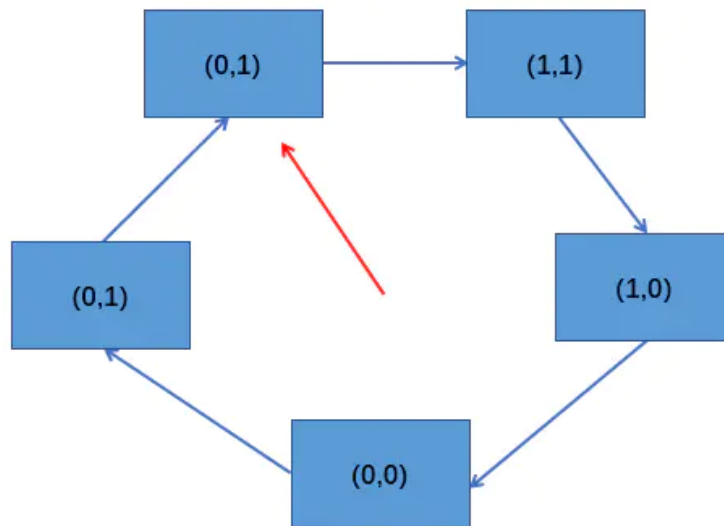
将所有可能被置换的页面排成一个循环队列：

1. 第一轮：从当前位置开始扫描第一个（0,0）的页用于替换，本轮扫描不修改任何标志位。
2. 第二轮：若第一轮扫描失败，则重新扫描，查找第一个（0,1）的页用于替换。本轮将所有扫描过的页访问位设为 0。
3. 第三轮：若第二轮扫描失败，则重新扫描，查找第一个（0,0）的页用于替换。本轮扫描不修改任何标志位。
4. 第四轮：若第三轮扫描失败，则重新扫描，查找第一个（0,1）的页用于替换。

由于第二轮已将所有的页的访问位都设为 0，因此第三轮、第四轮扫描一定会选中一个页，因此改进型 CLOCK 置换算法最多会进行四轮扫描。

第一轮就找到替换的页的情况

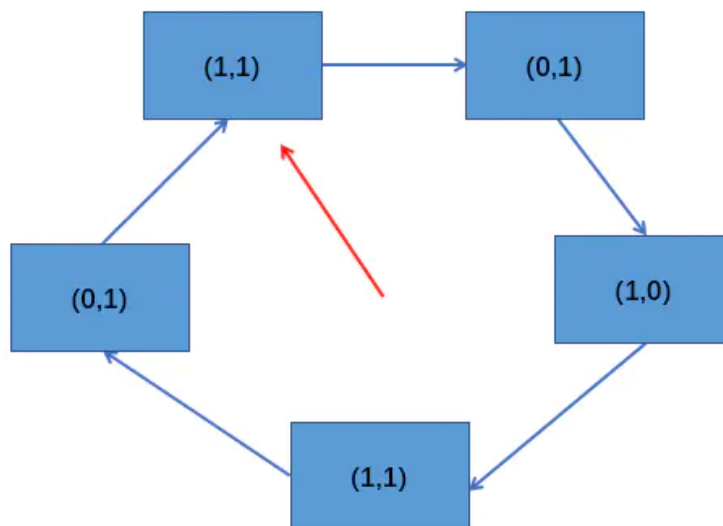
假设系统为进程分配了 5 个内存块，某时刻，各个页的状态如下图：



如果此时有新的页要进入内存，开始第一轮扫描就找到了要替换的页，即最下面的状态为 $(0,0)$ 的页。

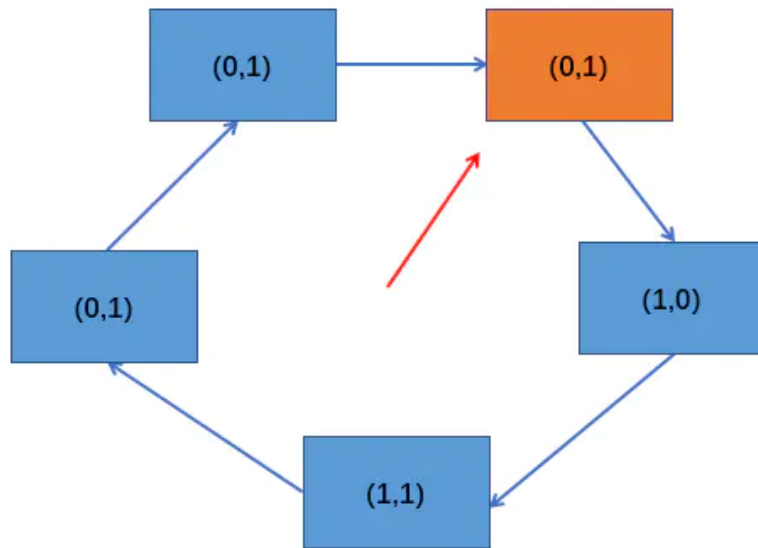
第二轮就找到替换的页的情况

某一时刻页面状态如下：



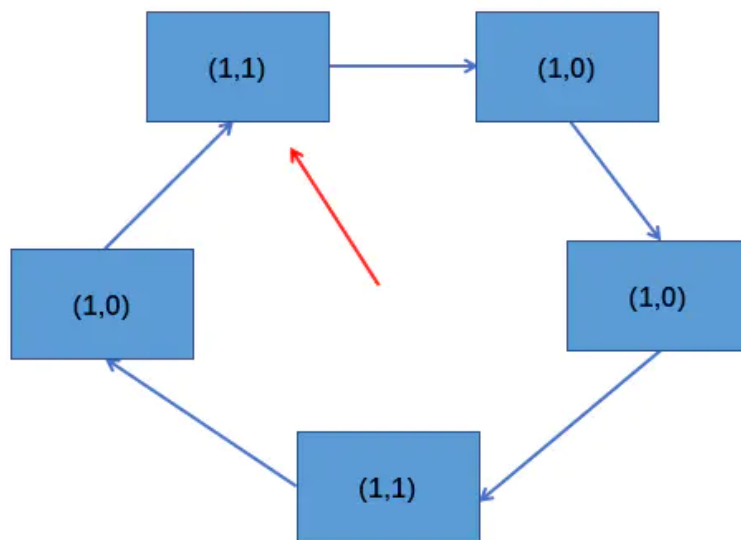
如果此时有新的页要进入内存，开始第一轮扫描就发现没有状态为 $(0,0)$ 的页，第一轮扫描后不修改任何标志位。所以各个页状态和上图一样。

然后开始第二轮扫描，尝试找到状态为 $(0,1)$ 的页，并将扫描过后的页的访问位设为 0，第二轮扫描找到了要替换的页。



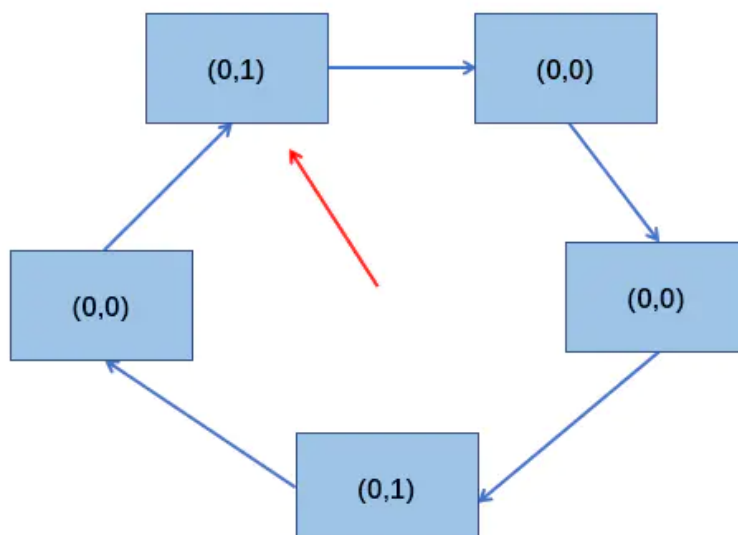
第三轮就找到替换的页的情况

某一时刻页面状态如下:

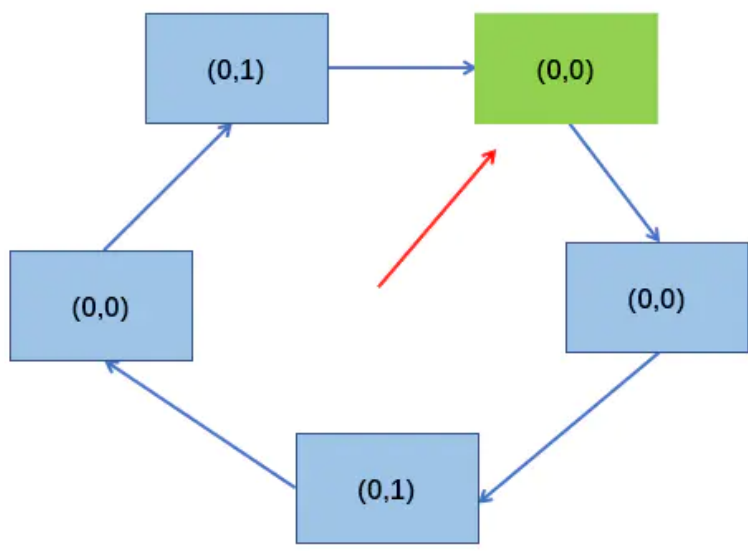


第一轮扫描没有找到状态为 (0,0) 的页，且第一轮扫描不修改任何标志位，所以第一轮扫描后状态和上图一致。

然后开始第二轮扫描，尝试找状态为 (0,1) 的页，也没有找到，第二轮扫描需要将访问位设为 1，第二轮扫描后，状态为下图：

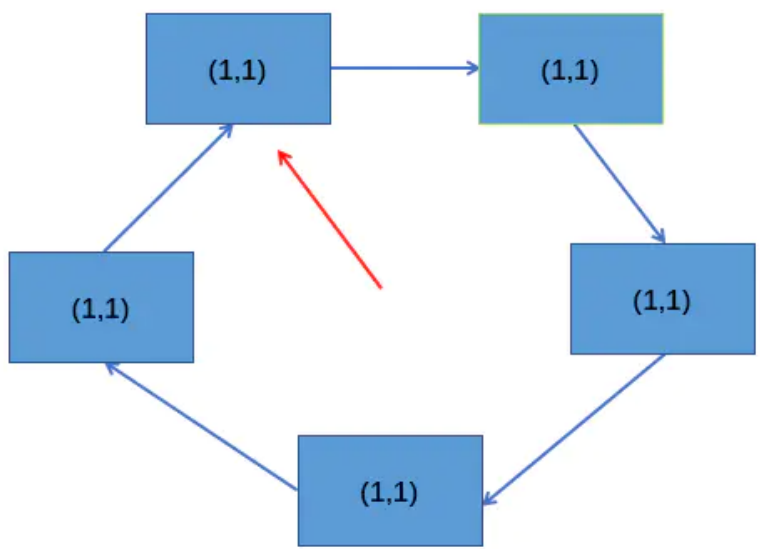


接着开始第三轮扫描，尝试找状态为 (0, 0) 的页，此轮扫描不修改标志位，第三轮扫描就找到了要替换的页。

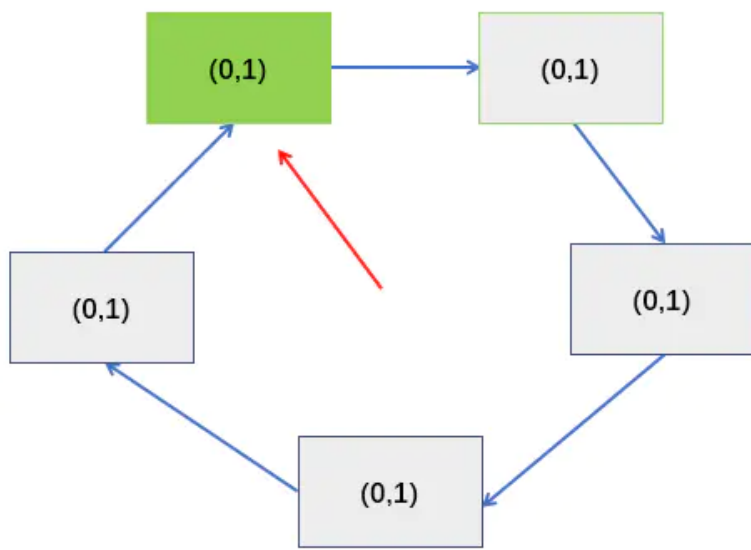


第四轮就找到替换的页的情况

某一时刻页面状态如下：



具体的扫描过程和上面相同，这里只给出最后的结果，如下图：



所以，改进型的 CLOCK 置换算法最多需要四轮扫描确定要置换的页。从上面的分析可以看出，改进型的 CLOCK 置换算法：

- 1. 第一优先级淘汰的是最近没有访问且没有修改的页面。
- 2. 第二优先级淘汰的是最近没有访问但修改的页面。
- 3. 第三优先级淘汰的是最近访问但没有修改的页面。
- 4. 第四优先级淘汰的是最近访问且修改的页面。

第三、四优先级为什么是访问过的？因为如果到了第三轮扫描，所有页的访问位都在第二轮扫描被设置为了 0，如果访问位不是 0 的话，也达到不了第三轮扫描，前两轮就会被淘汰。

所以到了第三轮，第四轮淘汰的页都是最近被访问过的。

总结

算法	算法规则	优缺点
OPT	优先淘汰最长时间不会被访问的页面	缺页率低，性能最好，但是无法实现。
FIFO	优先淘汰最先进入内存的页面	实现简单，但性能差，可能会出现 贝莱迪 (Belay) 异常。
LRU	优先淘汰最近最久未访问的页面	性能很好，但是需要硬件支持，算法开销大。
CLOCK (NRU)	循环扫描个页面 第一轮淘汰访问位 = 0 的，并将扫描过的页面访问位改为 0。若第一轮没有选中在，则进行第二轮扫描。	实现简单，算法开销小，但未考虑页面是否被修改过。
改进CLOCK	若用 (访问位,修改位) 的形式表述，则 第一轮：淘汰 (0,0) 第二轮：淘汰 (0,1)，并将扫描过的页访问位置为 0 第三轮：淘汰 (0,0) 第四轮：淘汰 (0,1)	算法开销小，性能也不错。

软中断与硬中断

中断的基本概念

中断是指计算机在执行期间，系统内发生任何非寻常的或非预期的急需处理事件，使得 CPU 暂时中断当前正在执行的程序而转去执行相应的事件处理程序，待处理完毕后又返回原来被中断处继续执行或调度新的进程执行的过程。引起中断发生的事件被称为中断源。中断源向 CPU 发出的请求中断处理信号称为中断请求，而 CPU 收到中断请求后转到相应的事件处理程序称为中断响应。

在有些情况下，尽管产生了中断源和发出了中断请求，但 CPU 内部的处理器状态字 PSW 的中断允许位已被清除，从而不允许 CPU 响应中断。这种情况称为禁止中断。CPU 禁止中断后只有等到 PSW 的中断允许位被重新设置后才能接收中断。禁止中断也称为关中断，PSW 的中断允许位的设置也被称为开中断。开中断和关中断是为了保证某段程序执行的原子性。

还有一个比较常用的概念是中断屏蔽。中断屏蔽是指在中断请求产生之后，系统有选择地封锁一部分中断而允许另一部分中断仍能得到响应。不过，有些中断请求是不能屏蔽甚至不能禁止的，也就是说，这些中断具有最高优先级，只要这些中断请求一旦提出，CPU 必须立即响应。例如，电源掉电事件所引起的中断就是不可禁止和不可屏蔽的。

中断的分类与优先级

根据系统对中断处理的需要，操作系统一般对中断进行分类并对不同的中断赋予不同的处理优先级，以便在不同的中断同时发生时，按轻重缓急进行处理。

根据中断源产生的条件，可把中断分为外中断和内中断。外中断是指来自处理器和内存外部的中断，包括 IO 设备发出的 IO 中断、外部信号中断(例如用户键入 ESC 键)。各种定时器引起的时钟中断以及调试程序中设置的断点等引起的调试中断等。外中断在狭义上一般被称为中断。

内中断主要指在处理器和内存内部产生的中断。内中断一般称为陷阱(trap)或异常。它包括程序运算引起的各种错误，如地址非法、校验错、页面失效、存取访问控制错、算术操作溢出、数据格式非法、除数为零、非法指令、用户程序执行特权指令、分时系统中的时间片中断以及从用户态到核心态的切换等都是陷阱的例子。

为了按中断源的轻重缓急处理响应中断，操作系统为不同的中断赋予不同的优先级。例如在 UNIX 系统中，外中断和陷阱的优先级共分为 8 级。为了禁止中断或屏蔽中断，CPU 的处理器状态字 PSW 中也设有相应的优先级。如果中断源的优先级高于 PSW 的优先级，则 CPU 响应该中断源的请求；反之，CPU 屏蔽该中断源的中断请求。

各中断源的优先级在系统设计时给定，在系统运行时是固定的。而处理器的优先级则根据执行情况由系统程序动态设定。

除了在优先级的设置方面有区别之外，中断和陷阱还有如下主要区别：

- 陷阱通常由处理器正在执行的现行指令引起，而中断则是由与现行指令无关的中断源引起的。陷阱处理程序提供的服务为当前进程所用，而中断处理程序提供的服务则不是为了当前进程的。
- CPU 执行完一条指令之后，下一条指令开始之前响应中断，而在一条指令执行中也可以响应陷阱。例如执行指令非法时，尽管被执行的非法指令不能执行结束，但 CPU 仍可对其进行处理。

硬中断

1. 硬中断是由硬件产生的，比如，像磁盘，网卡，键盘，时钟等。每个设备或设备集都有它自己的 IRQ（中断请求）。基于 IRQ，CPU 可以将相应的请求分发到对应的硬件驱动上（注：硬件驱动通常是内核中的一个子程序，而不是一个独立的进程）。
2. 处理中断的驱动是需要运行在 CPU 上的，因此，当中断产生的时候，CPU 会中断当前正在运行的任务，来处理中断。在有多核心的系统上，一个中断通常只能中断一颗 CPU（也有一种特殊情况，就是在大型主机上是有硬件通道的，它可以在没有主 CPU 的支持下，可以同时处理多个中断。）。
3. 硬中断可以直接中断 CPU。它会引起内核中相关的代码被触发。对于那些需要花费一些时间去处理的进程，中断代码本身也可以被其他的硬中断中断。
4. 对于时钟中断，内核调度代码会将当前正在运行的进程挂起，从而让其他的进程来运行。它的存在是为了让调度代码（或称为调度器）可以调度多任务。

软中断

1. 软中断的处理非常像硬中断。然而，它们仅仅是由当前正在运行的进程所产生的。
2. 通常，软中断是一些对 I/O 的请求。这些请求会调用内核中可以调度 I/O 发生的程序。对于某些设备，I/O 请求需要被立即处理，而磁盘 I/O 请求通常可以排队并且可以稍后处理。根据 I/O 模型的不同，进程或许会被挂起直到 I/O 完成，此时内核调度器就会选择另一个进程去运行。I/O 可以在进程之间产生并且调度过程通常和磁盘 I/O 的方式是相同。
3. 软中断仅与内核相联系。而内核主要负责对需要运行的任何其他进程进行调度。一些内核允许设备驱动的一些部分存在于用户空间，并且当需要的时候内核也会调度这个进程去运行。
4. 软中断并不会直接中断 CPU。也只有当前正在运行的代码（或进程）才会产生软中断。这种中断是一种需要内核为正在运行的进程去做一些事情（通常为 I/O）的请求。有一个特殊的软中断是

Yield 调用，它的作用是请求内核调度器去查看是否有一些其他的进程可以运行。

硬中断与软中断之区别与联系

1. 硬中断是有外设硬件发出的，需要有中断控制器之参与。其过程是外设检测到变化，告知中断控制器，中断控制器通过 CPU 或内存的中断脚通知 CPU，然后硬件进行程序计数器及堆栈寄存器之现场保存工作（引发上下文切换），并根据中断向量调用硬中断处理程序进行中断处理。
2. 软中断则通常是由硬中断处理程序或者进程调度程序等软件程序发出的中断信号，无需中断控制器之参与，直接以一个 CPU 指令之形式指示 CPU 进行程序计数器及堆栈寄存器之现场保存工作（亦会引发上下文切换），并调用相应的软中断处理程序进行中断处理（即我们通常所言之系统调用）。
3. 硬中断直接以硬件的方式引发，处理速度快。软中断以软件指令之方式适合于对响应速度要求不是特别严格的场景。
4. 硬中断通过设置 CPU 的屏蔽位可进行屏蔽，软中断则由于是指令之方式给出，不能屏蔽。
5. 硬中断发生后，通常会在硬中断处理程序中调用一个软中断来进行后续工作的处理。
6. 硬中断和软中断均会引起上下文切换（进程/线程之切换），进程切换的过程是差不多的。

中断处理过程

一旦 CPU 响应中断，转入中断处理程序，系统就开始进行中断处理。下面对中断处理过程进行详细说明：

1. CPU 检查响应中断的条件是否满足。CPU 响应中断的条件是：有来自于中断源的中断请求、CPU 允许中断。如果中断响应条件不满足，则中断处理无法进行。
2. 如果 CPU 响应中断，则 CPU 关中断，使其进入不可再次响应中断的状态。
3. 保存被中断进程现场。为了在中断处理结束后能使进程正确地返回到中断点，系统必须保存当前处理器状态字 PSW 和程序计数器 PC 等的值。这些值一般保存在特定堆栈或硬件寄存器中。
4. 分析中断原因，调用中断处理子程序。在多个中断请求同时发生时，处理优先级最高的中断源发出的中断请求。在系统中，为了处理上的方便，通常都是针对不同的中断源编制有不同的中断处理子程序（陷阱处理子程序）。这些子程序的入口地址（或陷阱指令的入口地址）存放在内存的特定单元中。

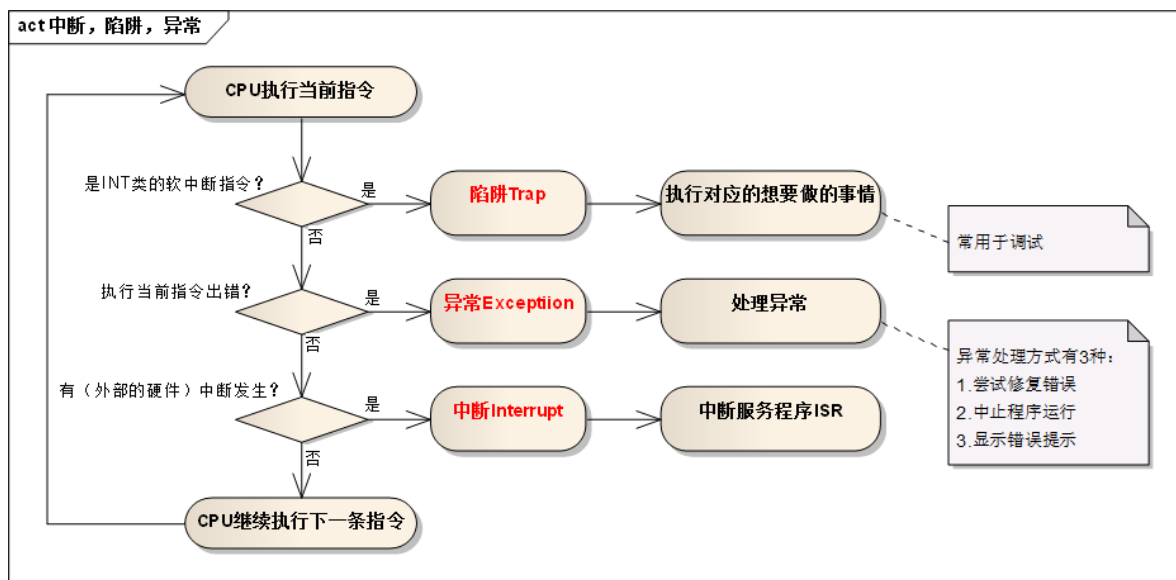
再者，不同的中断源也对应着不同的处理器状态字 PSW。这些不同的 PSW 被放在相应的内存单元中，与中断处理子程序入口地址一起构成中断向量。显然，根据中断或陷阱的种类，系统可由中断向量表迅速地找到该中断响应的优先级、中断处理子程序（或陷阱指令）的入口地址和对应的 PSW。

1. 执行中断处理子程序。对陷阱来说，在有些系统中则是通过陷阱指令向当前执行进程发出软中断信号后调用对应的处理子程序执行。
2. 退出中断，恢复被中断进程的现场或调度新进程占据处理器。
3. 开中断，CPU 继续执行。

中断与异常

在操作系统中引入核心态和用户态这两种工作状态后，就需要考虑这两种状态之间如何切换。操作系统内核工作在核心态，而用户程序工作在用户态。但系统不允许用户程序实现核心态的功能，而它们又必须使用这些功能。因此，需要在核心态建立一些“门”，实现从用户态进入核心态。

在实际操作系统中，CPU 运行上层程序时唯一能进入这些“门”的途径就是通过中断或异常。当中断或异常发生时，运行用户态的 CPU 会立即进入核心态，这是通过硬件实现的（例如，用一个特殊寄存器的一位来表示 CPU 所处的工作状态，0 表示核心态，1 表示用户态。若要进入核心态，只需将该位置 0 即可）。中断是操作系统中非常重要的一个概念，对一个运行在计算机上的实用操作系统而言，缺少了中断机制，将是不可想象的。



中断

中断(Interruption)，也称外中断，指来自 CPU 执行指令以外的事件的发生，如设备发出的 I/O 结束中断，表示设备输入/输出处理已经完成，希望处理机能够向设备发下一个输入 / 输出请求，同时让完成输入/输出后的程序继续运行。时钟中断，表示一个固定的时间片已到，让处理机处理计时、启动定时运行的任务等。这一类中断通常是与当前程序运行无关的事件，即它们与当前处理机运行的程序无关。

异常

异常(Exception)，也称内中断、例外或陷入(Trap)，指源自 CPU 执行指令内部的事件，如程序的非法操作码、地址越界、算术溢出、虚存系统的缺页以及专门的陷入指令等引起的事件。对异常的处理一般要依赖于当前程序的运行现场，而且异常不能被屏蔽，一旦出现应立即处理。关于内中断和外中断的联系与区别如下图所示：



中断面试题

聊聊：外中断和异常的区别

- 外中断是指由CPU 执行指令以外的事件引起，如 I/O 完成中断（设备输入/输出处理已经完成，处理器能够发送下一个输入/输出请求）、时钟中断、控制台中断等
- 异常是由CPU 执行指令的内部事件引起，如非法操作码、地址越界、算术溢出等

相同点：

最后都是由CPU发送给内核，由内核去处理，处理程序的流程设计上相似的

不同点：

1. 产生源不相同，异常是由CPU产生的，而中断是由硬件设备产生的
2. 内核需要根据是异常还是中断调用不同的处理程序

3. 中断不是时钟同步的，这意味着中断可能随时到来；异常由于是CPU产生的，所以它是时钟同步的
4. 当处理中断时，处于中断上下文中；处理异常时，处于进程上下文中

内存的管理策略

当允许进程动态增长时，操作系统必须对内存进行更有效的管理，

操作系统使用如下两种方法之一来得知内存的使用情况，分别为

- 1)位图(bitmap)
- 2)链表

使用位图，将内存划为多个大小相等的块，比如一个32K的内存1K一块可以划为32块，则需要32位（4字节）来表示其使用情况，使用位图将已经使用的块标为1，位使用的标为0.而使用链表，则将内存按使用或未使用分为多个段进行链接，这个概念如图4所示。

使用位图表示内存简单明了，但一个问题是当分配内存时必须在内存中搜索大量的连续0的空间，这是十分消耗资源的操作。

相比之下，使用链表进行此操作将会更胜一筹。还有一些操作系统会使用双向链表，因为当进程销毁时，邻接的往往是空内存或是另外的进程。使用双向链表使得链表之间的融合变得更加容易。

还有，当利用链表管理内存的情况下，创建进程时分配什么样的空闲空间也是个问题。

通常情况下有如下几种算法来对进程创建时的空间进行分配。

临近适应算法(**Next fit**)—从当前位置开始，搜索第一个能满足进程要求的内存空间
最佳适应算法(**Best fit**)—搜索整个链表，找到能满足进程要求最小内存的内存空间
最大适应算法(**wrost fit**)—找到当前内存中最大的空闲空间
首次适应算法(**First fit**) —从链表的第一个开始，找到第一个能满足进程要求的内存空间

聊聊：中断的处理过程？

1. 保护现场：将当前执行程序的相关数据保存在寄存器中，然后入栈。
2. 开中断：以便执行中断时能响应较高级别的中断请求。
3. 中断处理
4. 关中断：保证恢复现场时不被新中断打扰
5. 恢复现场：从堆栈中按序取出程序数据，恢复中断前的执行状态。

聊聊：中断和轮询有什么区别？

- 轮询：CPU对**特定设备**轮流询问。中断：通过**特定事件**提醒CPU。
- 轮询：效率低等待时间长，CPU利用率不高。中断：容易遗漏问题，CPU利用率不高。

虚拟内存(Virtual Memory)

虚拟内存是现代操作系统普遍使用的一种技术。

很多情况下，现有内存无法满足仅仅一个大进程的内存要求(比如很多游戏，都是10G+的级别)。

在早期的操作系统曾使用覆盖(overlays)来解决这个问题，将一个程序分为多个块，基本思想是先将块0加入内存，块0执行完后，将块1加入内存。

依次往复，这个解决方案最大的问题是需要程序员去程序进行分块，这是一个费时费力让人痛苦不堪的过程。

后来这个解决方案的修正版就是虚拟内存。

虚拟内存的基本思想是，每个进程有用独立的逻辑地址空间，内存被分为大小相等的多个块,称为页(Page)。

每个页都是一段连续的地址。对于进程来看,逻辑上貌似有很多内存空间，其中一部分对应物理内存上的一块(称为页框，通常页和页框大小相等)，还有一些没加载在内存中的对应硬盘上。

而虚拟内存和物理内存的匹配是通过页表实现，页表存在MMU中，页表中每个项通常为32位，既4byte,除了存储虚拟地址和页框地址之外，还会存储一些标志位，比如是否缺页，是否修改过，写保护等。

可以把MMU想象成一个接收虚拟地址项返回物理地址的方法。

因为页表中每个条目是4字节，现在的32位操作系统虚拟地址空间会是2的32次方，即使每页分为4K，也需要2的20次方*4字节=4M的空间，为每个进程建立一个4M的页表并不明智。

因此在页表的概念上进行推广，产生二级页表,二级页表每个对应4M的虚拟地址，而一级页表去索引这些二级页表，因此32位的系统需要1024个二级页表，虽然页表条目没有减少，但内存中可以仅仅存放需要使用的二级页表和一级页表，大大减少了内存的使用。

#

操作系统快表

什么是快表

快表 (TLB - translation lookaside buffer)，直译为旁路快表缓冲，也可以理解为页表缓冲，地址变换高速缓存。

由于页表存放在主存中，因此程序每次访存至少需要两次：一次访存获取物理地址，第二次访存才获得数据。提高访存性能的关键在于依靠页表的访问局部性。当一个转换的虚拟页号被使用时，它可能在不久的将来再次被使用到。

TLB 是一种高速缓存，内存管理硬件使用它来改善虚拟地址到物理地址的转换速度。当前所有的个人桌面，笔记本和服务处理器都使用 TLB 来进行虚拟地址到物理地址的映射。使用 TLB 内核可以快速的找到虚拟地址指向物理地址，而不需要请求 RAM 内存获取虚拟地址到物理地址的映射关系。这与 data cache 和 instruction caches 有很大的相似之处。

快表 (TLB) 原理

当 cpu 要访问一个虚拟地址/线性地址时，CPU 会首先根据虚拟地址的高 20 位 (20 是 x86 特定的，不同架构有不同的值) 在 TLB 中查找。如果是表中没有相应的表项，称为 TLB miss，需要通过访问慢速 RAM 中的页表计算出相应的物理地址。同时，物理地址被存放在一个 TLB 表项中，以后对同一线性地址的访问，直接从 TLB 表项中获取物理地址即可，称为 TLB hit。

想像一下 x86_32 架构下没有 TLB 的存在时的情况，对线性地址的访问，首先从 PGD 中获取 PTE (第一次内存访问)，在 PTE 中获取页框地址 (第二次内存访问)，最后访问物理地址，总共需要 3 次 RAM 的访问。如果有 TLB 存在，并且 TLB hit，那么只需要一次 RAM 访问即可。

TLB表项

TLB 内部存放的基本单位是页表条目，对应着 RAM 中存放的页表条目。页表条目的大小固定不变的，所以 TLB 容量越大，所能存放的页表条目越多，TLB hit 的几率也越大。但是 TLB 容量毕竟是有限的，因此 RAM 页表和 TLB 页表条目无法做到一一对应。因此 CPU 收到一个线性地址，那么必须快速做两个判断：

1. 所需的也表示否已经缓存在 TLB 内部 (TLB miss 或者 TLB hit)
2. 所需的页表在 TLB 的哪个条目内

为了尽量减少 CPU 做出这些判断所需的时间，那么就必须在 TLB 页表条目和内存页表条目之间的对应方式做足功夫。

全相连 - full associative

在这种组织方式下，TLB cache 中的表项和线性地址之间没有任何关系，也就是说，一个 TLB 表项可以和任意线性地址的页表项关联。这种关联方式使得 TLB 表项空间的利用率最大。但是延迟也可能相当的大，因为每次 CPU 请求，TLB 硬件都把线性地址和 TLB 的表项逐一比较，直到 TLB hit 或者所有 TLB 表项比较完成。特别是随着 CPU 缓存越来越大，需要比较大量的 TLB 表项，所以这种组织方式只适合小容量 TLB。

直接匹配

每一个线性地址块都可通过模运算对应到唯一的 TLB 表项，这样只需进行一次比较，降低了 TLB 内比较的延迟。但是这个方式产生冲突的几率非常高，导致 TLB miss 的发生，降低了命中率。

比如，我们假定 TLB cache 共包含 16 个表项，CPU 顺序访问以下线性地址块：1, 17, 1, 33。当 CPU 访问地址块 1 时， $1 \bmod 16 = 1$ ，TLB 查看它的第一个页表项是否包含指定的线性地址块 1，包含则命中，否则从 RAM 装入；然后 CPU 访问地址块 17， $17 \bmod 16 = 1$ ，TLB 发现它的第一个页表项对应的不是线性地址块 17，TLB miss 发生，TLB 访问 RAM 把地址块 17 的页表项装入 TLB；CPU 接下来访问地址块 1，此时又发生了 miss，TLB 只好访问 RAM 重新装入地址块 1 对应的页表项。因此在某些特定访问模式下，直接匹配的性能差到了极点。

组相连 - set-associative

为了解决全相连内部比较效率低和直接匹配的冲突，引入了组相连。这种方式把所有的 TLB 表项分成多个组，每个线性地址块对应的不再是一个 TLB 表项，而是一个 TLB 表项组。CPU 做地址转换时，首先计算线性地址块对应哪个 TLB 表项组，然后在这个 TLB 表项组顺序比对。按照组长度，我们可以称之为 2 路，4 路，8 路。

经过长期的工程实践，发现 8 路组相连是一个性能分界点。8 路组相连的命中率几乎和全相连命中率几乎一样，超过 8 路，组内对比延迟带来的缺点就超过命中率提高带来的好处了。

这三种方式各有优缺点，组相连是个折衷的选择，适合大部分应用环境。当然针对不同的领域，也可以采用其他的 cache 组织形式。

TLB表项更新

TLB 表项更新可以有 TLB 硬件自动发起，也可以有软件主动更新：

1. TLB miss 发生后，CPU 从 RAM 获取页表项，会自动更新 TLB 表项
2. TLB 中的表项在某些情况下是无效的，比如进程切换，更改内核页表等，此时 CPU 硬件不知道哪些 TLB 表项是无效的，只能由软件在这些场景下，刷新 TLB。

在 Linux kernel 软件层，提供了丰富的 TLB 表项刷新方法，但是不同的体系结构提供的硬件接口不同。比如 x86_32 仅提供了两种硬件接口来刷新 TLB 表项：

1. 向 cr3 寄存器写入值时，会导致处理器自动刷新非全局页的 TLB 表项
2. 在 Pentium Pro 以后，invlpg 汇编指令用来无效指定线性地址的单个 TLB 表项无效

操作系统页表

什么是页表

页表是内存管理系统中的数据结构，用于向每个进程提供一致的虚拟地址空间，每个页表项保存的是虚拟地址到物理地址的映射以及一些管理标志。应用进程只能访问虚拟地址，内核必须借助页表和硬件把虚拟地址翻译为对物理地址的访问。

页表作用

在使用虚拟地址空间的 Linux 操作系统上，每一个进程都工作在一个 4G 的地址空间上，其中 0~3G 是应用进程可以访问的 user 地址空间，是这个进程独有的，其他进程看不到也无法操作这个地址空间；3G~4G 是 kernel 地址空间，所有进程共享这部分地址空间。

由于每个进程都有 3G 的私有进程空间，所以系统的物理内存无法对这些地址空间进行一一映射，因此 kernel 需要一种机制，把进程地址空间映射到物理内存上。当一个进程请求访问内存时，操作系统通过存储在 kernel 中的进程页表把这个虚拟地址映射到物理地址，如果还没有为这个地址建立页表项，那么操作系统就为这个访问的地址建立页表项。最基本的映射单位是 page，对应的是页表项 PTE。

页表项和物理地址是多对一的关系，即多个页表项可以对应一个物理页面，因而支持共享内存的实现（几个进程同时共享物理内存）。

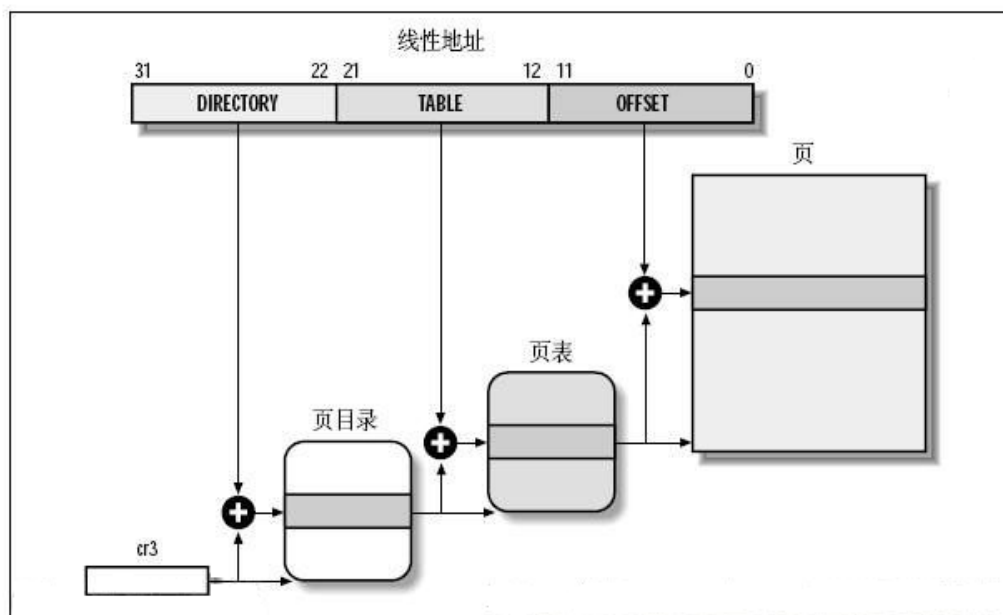
页表的实现

实现虚拟地址到物理地址转换最容易想到的方法是使用数组，对虚拟地址空间的每一个页，都分配一个数组项。但是有一个问题，考虑 IA32 体系结构下，页面大小为 4KB，整个虚拟地址空间为 4GB，则需要包含 1M 个页表项，这还只是一个进程，因为每个进程都有自己独立的页表。因此，系统所有的内存都来存放页表项恐怕都不够。

相像一下进程的虚拟地址空间，实际上大部分是空闲的，真正映射的区域几乎是汪洋大海中的小岛，因此我们可以考虑使用多级页表，可以减少页表内存使用量。实际上多级页表也是各种体系结构支持的，没有硬件支持，我们是没办法实现页表转换的。

为了减少页表的大小并忽略未做实际映射的区域，计算机体系结构的设计都会考虑将虚拟地址划分为多个部分。具体的体系结构划分方式不同，比如 ARM7 和 IA32 就有不同的划分，在这里我们不讨论这部分内容。

Linux 操作系统使用 4 级页表：



图中 CR3 保存着进程页目录 PGD 的地址，不同的进程有不同的页目录地址。进程切换时，操作系统负责把页目录地址装入 CR3 寄存器。

地址翻译过程如下

1. 对于给定的线性地址，根据线性地址的 bit22 ~ bit31 作为页目录项索引值，在 CR3 所指向的页目录中找到一个页目录项。
2. 找到的页目录项对应着页表，根据线性地址的 bit12 ~ bit21 作为页表项索引值，在页表中找到一个页表项。
3. 找到的页表项中包含着一个页面的地址，线性地址的 bit0 ~ bit11 作为页内偏移值和找到的页确定线性地址对应的物理地址。

这个地址翻译过程完全是由硬件完成的。

页表转化失败

在地址转换过程中，有两种情况会导致失败发生。

1. 要访问的地址不存在，这通常意味着由于编程错误访问了无效的虚拟地址，操作系统必须采取某种措施来处理这种情况，对于现代操作系统，发送一个段错误给程序；或者要访问的页面还没有被映射进来，此时操作系统要为此线性地址分配相应的物理页面，并更新页表。
2. 要查找的页不在物理内存中，比如页已经交换出物理内存。在这种情况下需要把页从磁盘交换回物理内存。

TLB

CPU 的 Memory management unit(MMU) cache 了最近使用的页面映射。我们称之为 translation lookaside buffer(TLB)。TLB 是一个组相连的 cache。当一个虚拟地址需要转换成物理地址时，首先搜索 TLB。如果发现了匹配 (TLB命中)，那么直接返回物理地址并访问。然而，如果没有匹配项 (TLB miss)，那么就要从页表中查找匹配项，如果存在也要把结果写回 TLB。

页表格式

页目录项和页表项大小都是 32bit(4 bytes)，由于 4KB 地址对齐的原因，页目录项和页表项只有 bit12 ~ bit31 用于地址，剩余的低 12bits 则用来描述页有关的附加信息。尽管这些位是特定于 CPU 的，下列位在 Linux 内核支持的大部分 CPU 都能找到：

Present

页目录项和页表项都包含这个位。

虚拟地址对应的物理页面不在内存中，比如页被交换出去，此时页表项的其他部分通常会代表不同的含义，因为不需要描述页在物理内存中的地址，相反，需要信息来找到换出的页。

如果页目录或者页表项的 Present 位为 0，那么 CPU 分页单元把虚拟地址存储到 CR2 中，然后生成一个异常 14：page fault 异常。

Accessed

每次分页单元访问页面时，都会自动设置 Accessed 位，内核会定期检查该位，以便确定页的活跃程度，内核会选择不活跃的页面 swapout 到交换空间。注意分页单元只负责置位，清除位操作要内核自己执行。

Dirty

仅仅存在于页表项，每当向页帧写入数据分页单元都会设置 dirty 标志，swap 进程可以通过这个位来决定是否选择这个页面进行交换。记住，分页单元不会清除这个标记，所以必须由操作系统来清除这个标记。

Read/Write

包含了页面的读写权限，如果设置为 0，那么只有读权限；设置为 1，则有读写权限。

User/Supervisor

User 允许用户空间代码访问该页；Supervisor 只有内核才可以访问。

Exec

在较新的 64 bit 处理器上，分页单元支持 No eXec 位，因此 2.6.11 内核开始加入了这个标志。

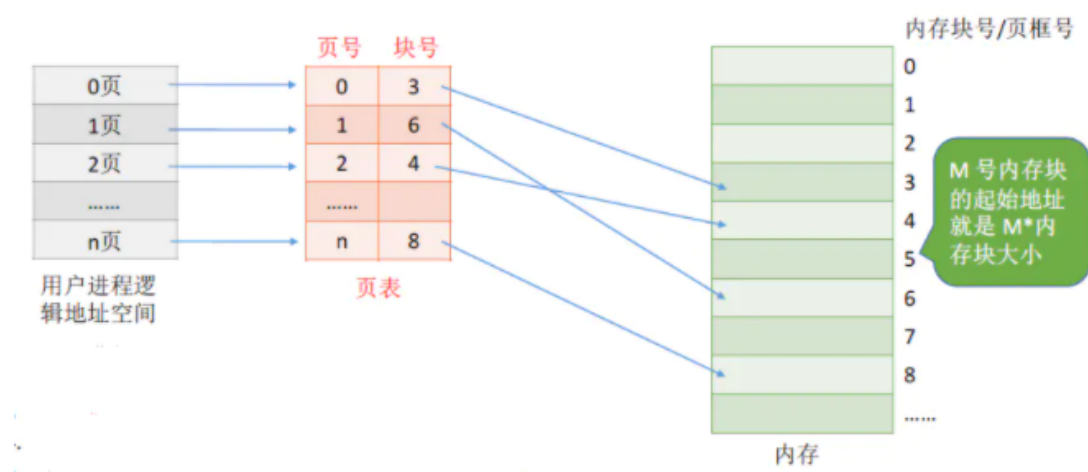
页表项的创建和操作

所有体系结构都要实现下面的页表项创建，释放和操作函数，以便于内存管理代码创建和销毁页表：

函数	描述
mk_pte	创建一个页表项，必须将page实列和所需的访问权限作为参数传入
pte_page	获得页表项描述的页对应的page实列地址
pgd_alloc	分配并初始化可容纳一个完整目录表的内存（不是一个表项）
pud_alloc	
pmd_alloc	
pte_alloc	
pgd_free	释放目录表占用的内存
pud_free	
pmd_free	
pte_free	
set_pgd	设置项目录项中某项的值
set_pud	
set_pmd	
set_pte	

多级页表

单级页表存在的问题



假设某计算机系统按字节寻址，支持 32 位逻辑地址，采用分页存储管理，页面大小为 4KB，页表项长度为 4B。4KB = 212 B，因此页内地址要用 12 位表示，剩余 20 位表示页号。

因此，该系统中用户进程最多有 220 页。相应的，一个进程的页表中，最多会有 220 个页表项，所以一个页表最大需要 $220 * 4B = 222B$ 。一个页框（内存块）大小为 4B，所以需要 $222/212 = 210$ 个页框存储该页表。而页表的存储是需要连续存储的，因为根据页号查询页表的方法： K 号页对应的页表项的位置 = 页表起始地址 + $K * 4B$ （页表项长度），所以这就要求页表的存储必须是连续的。

回想一下，当初为什么使用页表，就是要将进程划分为一个个页面可以不用连续的存放在内存中，但是此时页表就需要 1024 个连续的页框，似乎和当时的目标有点背道而驰了...

此外，根据局部性原理可知，很多时候，进程在一段时间内只需要访问某几个页面就可以正常运行了。因此也没有必要让整个页面都常驻内存。所以，单级页表存在以上两个问题。

两级页表

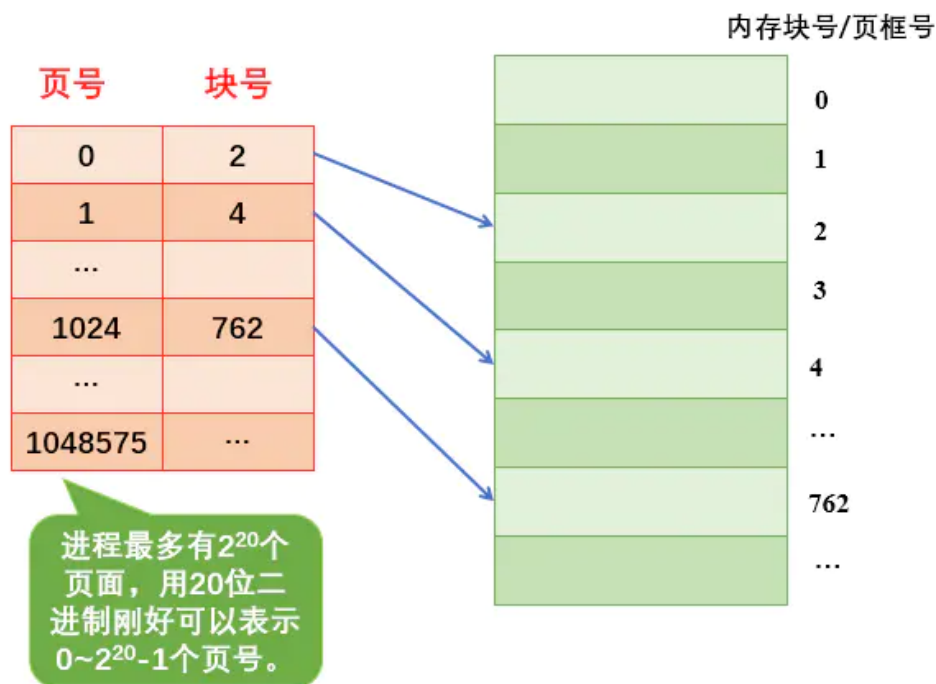
如何解决页表过大需要连续存储的问题呢？这个问题可以参考进程太大需要连续存储的答案。因为页表必须连续存放，所以可以将页表再分页。

解决方案：可以将长长的页表进行分组，使每个页面中刚好可以放下一个分组（如上面的例子中，页面的大小 4KB），每个页表项 4B，所以每个页面中可以存放 1K 个（1024）个页表项，因此每 1K 个连续的页表项为一组，每组刚好占一个页面，再讲各组离散的放在各个内存块中）。这样就需要为离散的页表再建立一张页表，称为页目录表，或外层页表，或顶层页表。

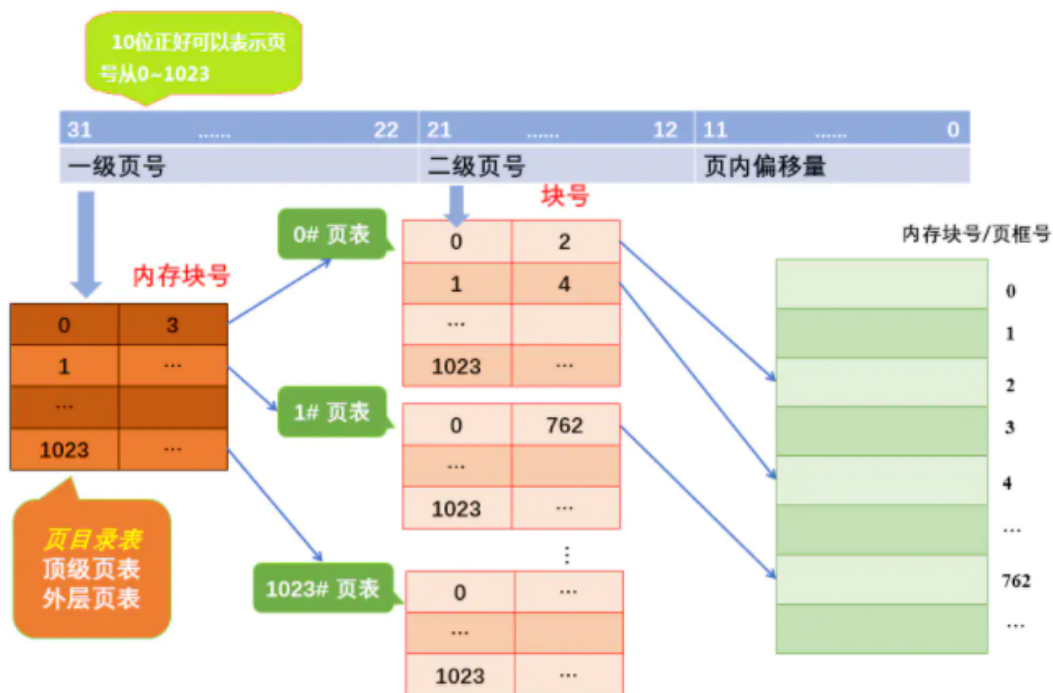
还是上面的例子，32 位的逻辑地址空间，页表项大小为 4B，页面大小 4KB，则页内地址占 12 位，单级页表结构逻辑结构图如下图所示：

31		12	11		0
页号					
页内偏移量					

使用单级页表的情况：



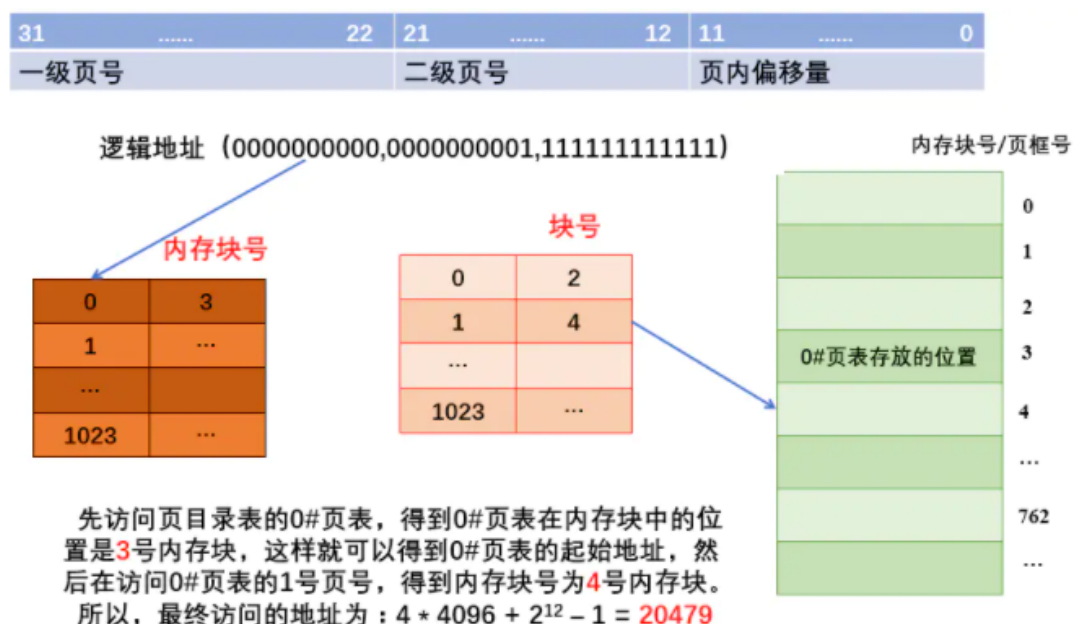
将页表分为分为 1024 个表，每个表中包含 1024 个页表项，形成二级页表。二级页表结构的逻辑地址结构如下图：



两级页表如何实现地址转换：

1. 按照地址结构将逻辑地址拆成三个部分。
2. 从 PCB 中读取页目录起始地址，再根据一级页号查页目录表，找到下一级页表在内存中存放位置。
3. 根据二级页号查表，找到最终想要访问的内存块号。
4. 结合页内偏移量得到物理地址。

下面以一个逻辑地址为例。将逻辑地址 (0000000000,0000000001,1111111111) 转换为物理地址的过程。



虚拟存储技术

在解决了页必须连续存放的问题后，再看如何第二个问题：没有必要让整个页表常驻内存，因为进程一段时间内可能只需要访问某几个特定的页面。

解决方案：可以在需要访问页面时才把页面调入内存——虚拟存储技术（后面再说）。可以在页表中增加一个标示位，用于表示该页表是否已经调入内存。

一级页号	内存块号	是否在内存中
0	3	是
1	无	否
.....		
1023	...	

二级页号	内存块号	是否在内存中
0	2	是
1	4	是
.....		
1023	...	

若想访问的页面不在内存中，则产生缺页中断（内中断），然后将目标页面从外存调入内存

几个问题

1. 若采用多级页表机制，则各级页表的大小不能超过一个页面。

举例说明，某系统按字节编址，采用 40 位逻辑地址，页面大小为 4KB，页表项大小为 4B，假设采用纯页式存储，则要采用（）级页表，页内偏移量为（）位？

页面大小 = 4KB，按字节编址，因此页内偏移量为 12 位。

页号 = $40 - 12 = 28$ 位。

页面大小 = 4KB，页表项大小 = 4B，则每个页面可存放 1024 个页表项。因此各级页表最多包含 1024 个页表项，需要 10 个二进制位才能映射到 1024 个页表项，因此每级页表对应的页号应为 10 位二进制。共 28 位的页号至少要分为 3 级。

逻辑地址：	页号 28 位		页内偏移量 12 位
逻辑地址：	一级页号 8 位	二级页号 10 位	三级页号 10 位
			页内偏移量 12 位

1. 两级页表的访问次数分析（假设没有页表）：

1. 第一次访问：访问内存中的页目录表。
2. 访问内存中的二级目录。
3. 访问目标内存单元。

从上面可以看出，两级页表虽然解决了页表需要连续存储的问题，但是同时也增加了内存的访问次数。

使用二级页表的优势

1. 使用多级页表可以使得页表在内存中离散存储。多级页表实际上是增加了索引，有了索引就可以定位到具体的项。举个例子：比如虚拟地址空间大小为 4G，每个页大小依然为 4K，如果使用一级页表的话，共有 2^{20} 个页表项，如果每一个页表项占 4B，那么存放所有页表项需要 4M，为了能够随机访问，那么就需要连续 4M 的内存空间来存放所有的页表项。

随着虚拟地址空间的增大，存放页表所需要的连续空间也会增大，在操作系统内存紧张或者内存碎片较多时，这无疑会带来额外的开销。但是如果使用多级页表，我们可以使用一页来存放页目录项，页表项存放在内存中的其他位置，不用保证页目录项和页表项连续。

2. 使用多级页表可以节省页表内存。使用一级页表，需要连续的内存空间来存放所有的页表项。多级页表通过只为进程实际使用的那些虚拟地址内存区请求页表来减少内存使用量。举个例子：一个进程的虚拟地址空间是 4GB，假如进程只使用 4MB 内存空间。对于一级页表，我们需要 4M 空间来存放这 4GB 虚拟地址空间对应的页表，然后可以找到进程真正使用的 4M 内存空间。也就是说，

虽然进程实际上只使用了 4MB 的内存空间，但是为了访问它们我们需要为所有的虚拟地址空间建立页表。

但是如果使用二级页表的话，一个页目录项可以定位 4M 内存空间，存放一个页目录项占 4K，还需要一页用于存放进程使用的 4M ($4M=1024*4K$ ，也就是用 1024 个页表项可以映射 4M 内存空间) 内存空间对应的页表，总共需要 4K (页表) + 4K (页目录) = 8K 来存放进程使用的这 4M 内存空间对应页表和页目录项，这比使用一级页表节省了很多内存空间。

当然，在这种情况下，使用多级页表确实是可以节省内存的。但是，我们需要注意另一种情况，如果进程的虚拟地址空间是 4GB，而进程真正使用的内存也是 4GB，如果是使用一级页表，则只需要 4MB 连续的内存空间存放页表，我们就可以寻址这 4GB 内存空间。而如果使用的是二级页表的话，我们需要 4MB 内存存放页表，还需要 4KB 内存来存放页目录项，此时多级页表反倒是多占用了内存空间。注意在大多数情况都是进程的 4GB 虚拟地址空间都是没有使用的，实际使用的都是小于 4GB 的，所以我们说多级页表可以节省页表内存。

那么使用多级页表比使用一级页表有没有什么劣势呢？

当然是有的。比如：使用一级页表时，读取内存中一页内容需要 2 次访问内存，第一次是访问页表项，第二次是访问要读取的一页数据。但如果是使用二级页表的话，就需要 3 次访问内存了，第一次访问页目录项，第二次访问页表项，第三次访问要读取的一页数据。访存次数的增加也就意味着访问数据所花费的总时间增加。

总结

多级页表优势：

1. 可以离散存储页表。
2. 在某种意义上节省页表内存空间。

多级页表劣势：

1. 增加寻址次数，从而延长访存时间。

局部性原理

什么是局部性原理

虚拟存储器的核心思路是根据程序运行时的局部性原理：一个程序运行时，在一小段时间内，只会用到程序和数据的很小一部分，仅把这部分程序和数据装入主存即可，更多的部分可以在需要用到时随时从辅存调入主存。在操作系统和相应硬件的支持下，数据在辅存和主存之间按程序运行的需要自动成批量地完成交换。

局部性原理是虚拟内存技术的基础，正是因为程序运行具有局部性原理，才可以只装入部分程序到内存就开始运行。早在 1968 年的时候，就有人指出我们的程序在执行的时候往往呈现局部性规律，也就是说在某个较短的时间段内，程序执行局限于某一小部分，程序访问的存储空间也局限于某个区域。

局部性原理表现在以下两个方面：

时间局部性：如果程序中的某条指令一旦执行，不久以后该指令很可能再次执行；如果某数据被访问过，不久以后该数据很可能再次被访问。产生时间局部性的典型原因，是由于在程序中存在着大量的循环操作。时间局部性是通过将近来使用的指令和数据保存到高速缓存存储器中，并使用高速缓存的层次结构实现。

空间局部性：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问，即程序在一段时间内所访问的地址，可能集中在一定的范围之内，这是因为指令通常是顺序存放、顺序执行的，数据也一般是以向量、数组、表等形式簇聚存储的。空间局部性通常是使用较大的高速缓存，并将预取机制集成到高速缓存控制逻辑中实现。虚拟内存技术实际上就是建立了“内存-外存”的两级存储器的结构，利用局部性原理实现高速缓存。

基于局部性原理，在程序装入时，可以将程序的一部分装入内存，而将其他部分留在外存，就可以启动程序执行。由于外存往往比内存大很多，所以我们运行的软件的内存大小实际上是可以比计算机系统实际的内存大小大的。在程序执行过程中，当所访问的信息不在内存时，由操作系统将所需要的部分调入内存，然后继续执行程序。另一方面，操作系统将内存中暂时不使用的内容换到外存上，从而腾出空间存放将要调入内存的信息。

可见，内-外存交换技术本质是一种时间换空间的策略，你用 CPU 的计算时间，页的调入调出花费的时间，换来了一个虚拟的更大的空间来支持程序的运行。

示例

上面是通过理论来说明的，下面我们通过一段代码来看看局部性原理：

```
public int sum(int[] array) {  
    int sum = 0;  
    for (int i = 0; i < array.length; i++) {  
        sum = sum + array[i];  
    }  
    return sum;  
}
```

从上面的这段代码来看，就是一个很简单的数组元素求和，这里我们主要看 sum 和 array 两个变量，我们可以看到 sum 在每次循环中都会用到，另外它只是一个简单变量，所以我们可以看到，sum 是符合我们上面提到的时间局部性，再访问一次后还会被继续访问到，但是它不存在我们所说的空间局部性了。

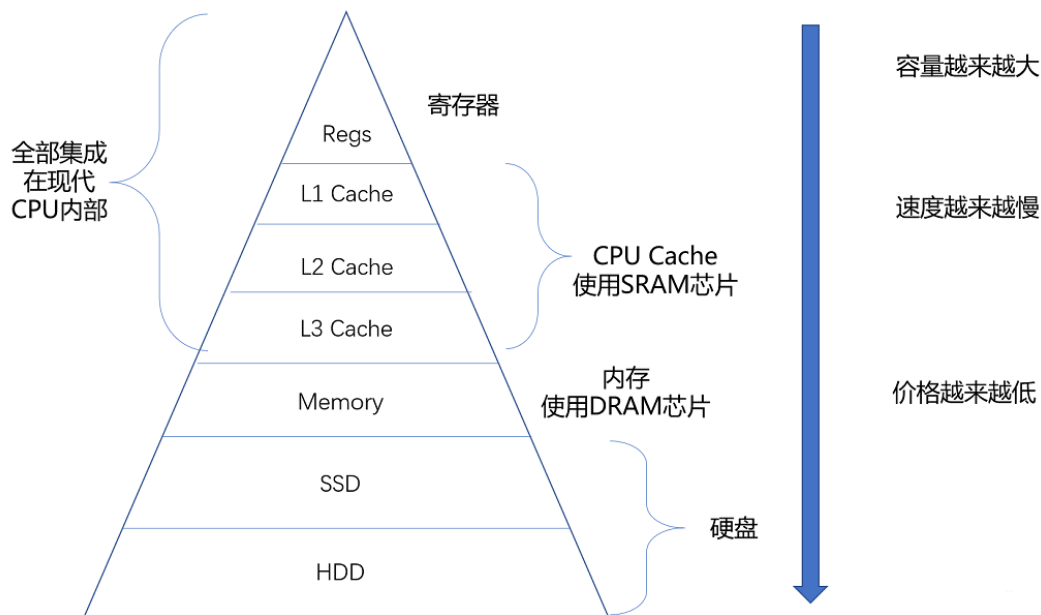
相反的，array 数组中的每个元素只访问一次，另外数组底层的存储是连续的，所以 array 变量符合我们上面提到的空间局部性，但是不符合时间局部性。

这只是局部性原理的简单示例，对于局部性原理还有很多地方会用到，我们如果能熟练的掌握和使用，对我们的帮助会很大的。

相关应用

CPU缓存

上面的示例其实很简单，相信大家都能理解，另外局部性原理其实在我们日常使用的软件中随处可见，并且在操作系统中也少不了。我们知道 CPU 的速度是非常快的，而且 CPU 与内存之间有多级缓存，如下图：



为了充分的利用 CPU，操作系统会利用局部性原理，将高频的数据从内存中加载的缓存中，从而加快 CPU 的处理速度。

广义局部性

其实我们的局部性原理不单单是上面提到的狭义性的局部性，还可以是广义的局部性。我们系统里面的热点数据，CDN 数据，微博的热点流量等等这些都利用了局部性原理。只是我们可能没有意识到而已，实际上已经在使用了。我们会通过 Redis 缓存热点数据，会通过 CDN 提前加载图片或者视频资源，等等，都是因为这些数据本身就符合局部性原理，合理的利用局部性可以得到了能效、成本上的提升。

利弊结合

任何事情都是多面性的，局部性原理虽然我们使用起来很不错，可以提高系统性能，但是在有些场景下，我们是需要避免局部性原理的出现的。或者说出现了这种情况，我们需要人工处理。我们可以试想一下，如果在我们的一个大数据处理平台上，由于局部性原理的存在，导致我们部分节点数据庞大运算吃力，部分节点数据量小十分空闲，这种情况自然是不合理，我们就需要把数据按照业务场景进行重新分配，以达到整个集群的最大利用。

分段与分页机制

分段机制

什么是分段机制

分段机制就是把虚拟地址空间中的虚拟内存组织成一些长度可变的称为段的内存块单元。

什么是段

每个段由三个参数定义：段基地址、段限长和段属性。段的基地址、段限长以及段的保护属性存储在一个称为段描述符的结构项中。

段的作用

段可以用来存放程序的代码、数据和堆栈，或者用来存放系统数据结构。

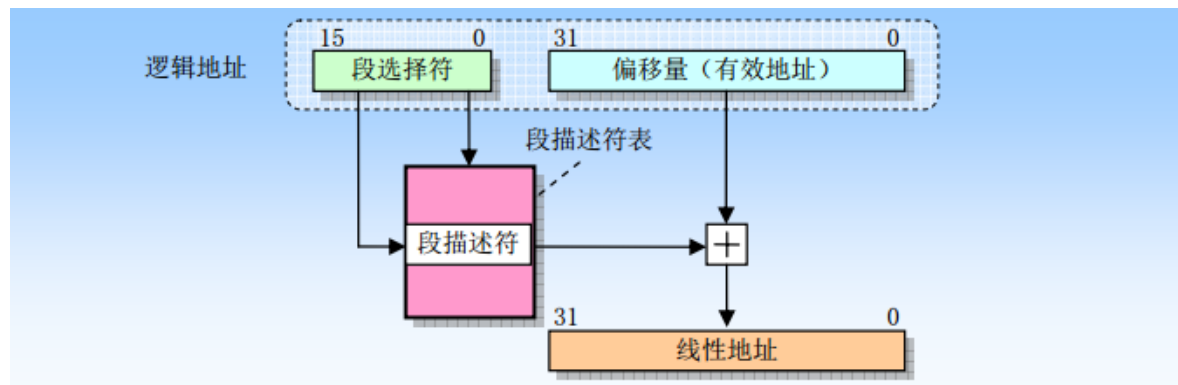
段的存储地址

系统中所有使用的段都包含在处理器线性地址空间中。

段选择符

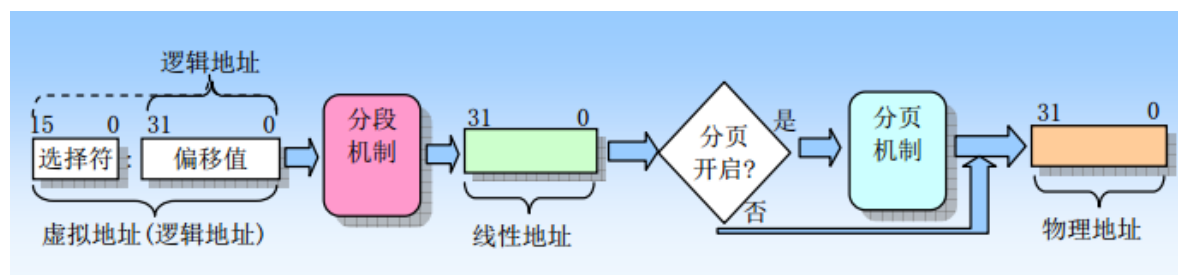
逻辑地址包括一个段选择符或一个偏移量，段选择符是一个段的唯一标识，提供了段描述符表，段描述符表指明段的大小、访问权限和段的特权级、段类型以及段的第一个字节在线性地址空间中的位置（称为段的基地址）。逻辑地址的偏移量部分到段的基地址上就可以定位段中某个字节的位置。因此基地址加上偏移量就形成了处理器线性地址空间中的地址。

逻辑地址到线性地址的变换过程



如果没有开启分页，那么处理器直接把线性地址映射到物理地址，即线性地址被送到处理器地址总线上；如果对线性地址空间进行了分页处理，那么就会使用二级地址转换把线性地址转换成物理地址。

虚拟地址到物理地址的变换过程



分页机制

什么是分页机制

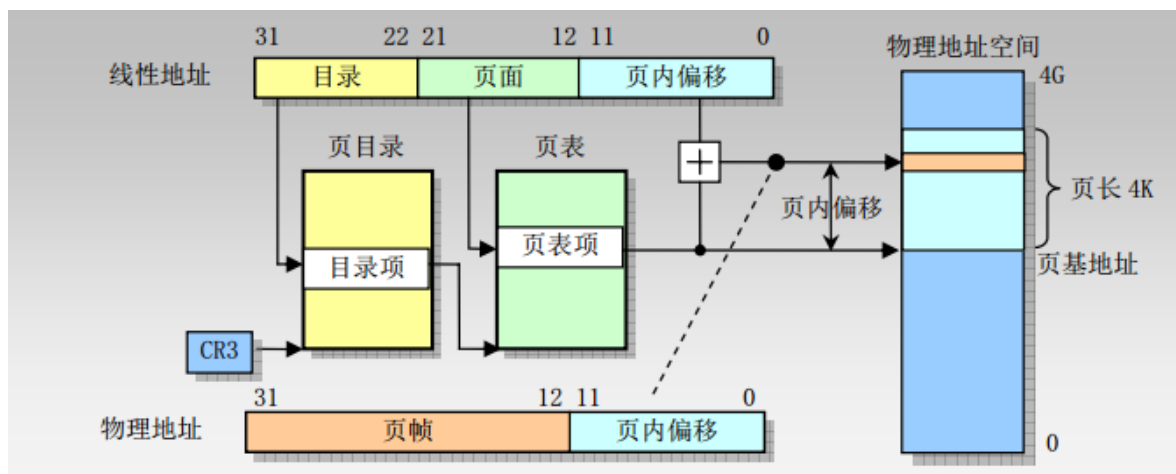
分页机制在段机制之后进行的，它进一步将线性地址转换为物理地址。

分页机制的存储

分页机制支持虚拟存储技术，在使用虚拟存储的环境中，大容量的线性地址空间需要使用小块的物理内存（RAM 或 ROM）以及某些外部存储空间来模拟。当使用分页时，每个段被划分成页面（通常每页为 4K 大小），页面会被存储于物理内存中或硬盘中。

操作系统通过维护一个页目录和一些页表来留意这些页面。当程序（或任务）试图访问线性地址空间中的一个地址位置时，处理器就会使用页目录和页表把线性地址转换成一个物理地址，然后在该内存位置上执行所要求的操作。

线性地址和物理地址之间的变换过程



聊聊：分段机制和分页机制的区别

1. 分页机制会使用大小固定的内存块，而分段管理则使用了大小可变的块来管理内存。
2. 分页使用固定大小的块更为适合管理物理内存，分段机制使用大小可变的块更适合处理复杂系统的逻辑分区。
3. 段表存储在线性地址空间，而页表则保存在物理地址空间。

聊聊：分段分页优缺点

优点

1. 它减少了内存使用量。
2. 分页表大小受到分段大小的限制。
3. 分段表只有一个对应于一个实际分段的条目。
4. 外部碎片不存在。
5. 它简化了内存分配。

缺点

1. 内部碎片将在那里。
2. 与分页相比，分段复杂度要高得多。
3. 分页表需要连续存储在内存中。

聊聊：什么是逻辑地址/线性地址/物理地址

逻辑地址 (Logical Address)

是指由程序产生的与段相关的偏移地址部分。例如，你在进行 C 语言 指针编程中，可以读取指针变量本身值(& 操作)，实际上这个值就是逻辑地址，它是相对于你当前进程数据段的地址，不和绝对物理地址相干。

只有在 Intel 实模式下，逻辑地址才和物理地址相等（因为实模式没有分段或分页机制,Cpu 不进行自动地址转换）；逻辑也就是在 Intel 保护模式下程序执行代码段限长内的偏移地址（假定代码段、数据段如果完全一样）。应用程序员仅需与逻辑地址打交道，而分段和分页机制对您来说是完全透明的，仅由系统编程人员涉及。应用程序员虽然自己可以直接操作内存，那也只能在操作系统给你分配的内存段操作。

线性地址 (Linear Address)

线性地址（Linear Address）是逻辑地址到物理地址变换之间的中间层。程序代码会产生逻辑地址，或者说是段中的偏移地址，加上相应段的基地址就生成了一个线性地址。如果启用了分页机制，那么线性地址可以再经变换以产生一个物理地址。若没有启用分页机制，那么线性地址直接就是物理地址。Intel 80386 的线性地址空间容量为 4G（2 的 32 次方即 32 根地址总线寻址）。

物理地址（Physical Address）

是指出现在 CPU 外部地址总线上的寻址物理内存的地址信号，是地址变换的最终结果地址。如果启用了分页机制，那么线性地址会使用页目录和页表中的项变换成物理地址。如果没有启用分页机制，那么线性地址就直接成为物理地址了。

虚拟内存（Virtual Memory）

是指计算机呈现出要比实际拥有的内存大得多的内存量。因此它允许程序员编制并运行比实际系统拥有的内存大得多的程序。这使得许多大型项目也能够具有有限内存资源的系统上实现。一个很恰当的比喻是：你不需要很长的轨道就可以让一列火车从上海开到北京。你只需要足够长的铁轨（比如说 3 公里）就可以完成这个任务。

采取的方法是把后面的铁轨立刻铺到火车的前面，只要你的操作足够快并能满足要求，列车就能象在一条完整的轨道上运行。这也就是虚拟内存管理需要完成的任务。在 Linux 0.11 内核中，给每个程序（进程）都划分了总容量为 64MB 的虚拟内存空间。因此程序的逻辑地址范围是 0x0000000 到 0x4000000。

有时我们也把逻辑地址称为虚拟地址。因为与虚拟内存空间的概念类似，逻辑地址也是与实际物理内存容量无关的。逻辑地址与物理地址的“差距”是 0xC0000000，是由于虚拟地址->线性地址->物理地址映射正好差这个值。这个值是由操作系统指定的。

虚拟地址到物理地址的转化方法是与体系结构相关的。一般来说有分段、分页两种方式。以现在的 x86 cpu 为例，分段分页都是支持的。Memory Management Unit 负责从虚拟地址到物理地址的转化。逻辑地址是段标识+段内偏移量的形式，MMU 通过查询段表，可以把逻辑地址转化为线性地址。如果 cpu 没有开启分页功能，那么线性地址就是物理地址；如果 cpu 开启了分页功能，MMU 还需要查询页表来将线性地址转化为物理地址：

逻辑地址 ----（段表）----> 线性地址 - （页表）-> 物理地址

不同的逻辑地址可以映射到同一个线性地址上；不同的线性地址也可以映射到同一个物理地址上；所以是多对一的关系。另外，同一个线性地址，在发生换页以后，也可能被重新装载到另外一个物理地址上。所以这种多对一的映射关系也会随时间发生变化。

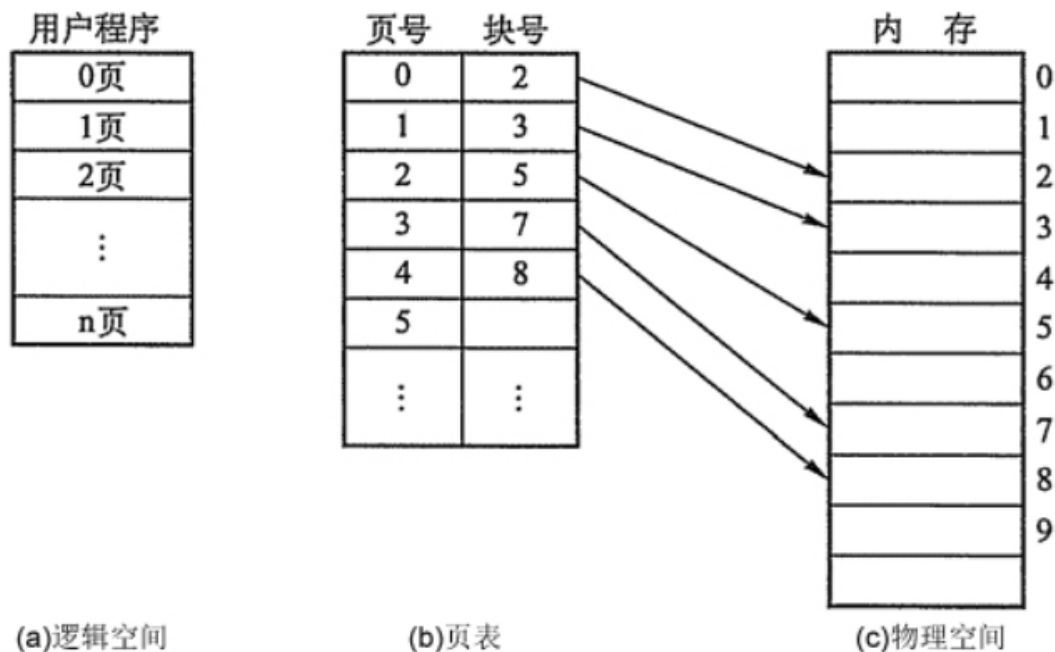
聊聊：什么是虚拟内存？

虚拟内存就是说，让物理内存扩充成更大的逻辑内存，从而让程序获得更多的可用内存。虚拟内存使用部分加载的技术，让一个进程或者资源的某些页面加载进内存，从而能够加载更多，甚至能加载比内存大的进程，这样看起来好像内存变大了，这部分内存其实包含了磁盘或者硬盘，并且就叫做虚拟内存。

聊聊：什么是分页？

把内存空间划分为**大小相等且固定的块**，作为主存的基本单位。因为程序数据存储在不同的页面中，而页面又离散的分布在内存中，**因此需要一个页表来记录映射关系，以实现从页号到物理块号的映射。**

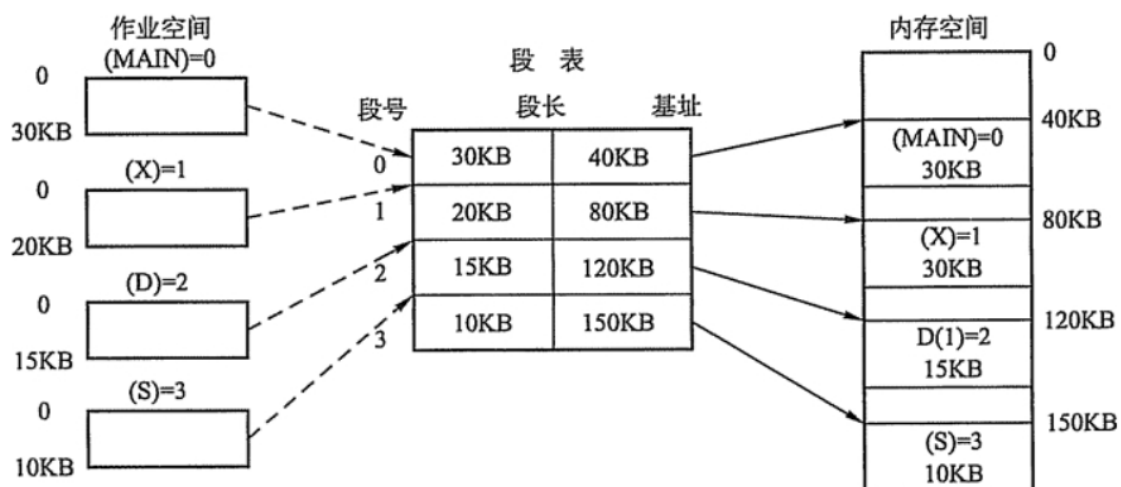
访问分页系统中内存数据需要**两次的内存访问**（一次是从内存中访问页表，从中找到指定的物理块号，加上页内偏移得到实际物理地址；第二次就是根据第一次得到的物理地址访问内存取出数据）。



聊聊：什么是分段？

分页是为了提高内存利用率，而分段是为了满足程序员在编写代码的时候的一些逻辑需求(比如数据共享，数据保护，动态链接等)。

分段内存管理当中，地址是二维的，一维是段号，二维是段内地址；其中每个段的长度是不一样的，而且每个段内部都是从0开始编址的。由于分段管理中，每个段内部是连续内存分配，但是段和段之间是离散分配的，因此也存在一个逻辑地址到物理地址的映射关系，相应的就是段表机制。



聊聊：分页和分段有什么区别？

- 分页对程序员是透明的，但是分段需要程序员显式划分每个段。
- 分页的地址空间是一维地址空间，分段是二维的。
- 页的大小不可变，段的大小可以动态改变。
- 分页主要用于实现虚拟内存，从而获得更大的地址空间；分段主要是为了使程序和数据可以被划分为逻辑上独立的地址空间并且有助于共享和保护。

聊聊：为什么使用两级页表

假设每个进程都占用了4G的线性地址空间，页表共含1M个表项，每个表项占4个字节，那么每个进程的页表要占据4M的内存空间。为了节省页表占用的空间，我们使用两级页表。每个进程都会被分配一个页目录，但是只有被实际使用页表才会被分配到内存里面。一级页表需要一次分配所有页表空间，两级页表则可以在需要的时候再分配页表空间。

两级表结构的第一级称为页目录，存储在一个4K字节的页面中。页目录表共有1K个表项，每个表项为4个字节，并指向第二级表。线性地址的最高10位(即位31~位22)用来产生第一级的索引，由索引得到的表项中，指定并选择了1K个二级表中的一个表。

两级表结构的第二级称为页表，也刚好存储在一个4K字节的页面中，包含1K个字节的表项，每个表项包含一个页的物理基地址。第二级页表由线性地址的中间10位(即位21~位12)进行索引，以获得包含页的物理地址的页表项，这个物理地址的高20位与线性地址的低12位形成了最后的物理地址，也就是页转化过程输出的物理地址。

聊聊：地址变换中，有快表和没快表的区别

区别	地址变换过程	访问一个逻辑地址的访存次数
无快表	①算页号、页内偏移量 ②检查页号合法性 ③查页表，找到页面存放的内存块号 ④根据内存块号与页内偏移量得到物理地址 ⑤访问目标内存单元	两次访存
具有快表的地址	①算页号、页内偏移量 ②检查页号合法性 ③查快表。若命中，即可知道页面存放的内存块号，可直接进行⑤;若未命中则进行④， ④查页表，找到页面存放的内存块号，并且将页表项复制到快表中 ⑤根据内存块号与页内偏移量得到物理地址 ⑥访问目标内存单元	快表命中，只需一次访存 快表未命中

聊聊：动态分区分配算法的了解

- 首次适应算法：从低地址开始查找，找到第一个能满足大小的空闲分区。
原理：空闲分区以地址递增的次序排列，每次分配内存时顺序查找
- 最佳适应算法：由于动态分区分配是一种连续分配方式，为各进程分配的空间必须是连续的一整片区域。
因此为了保证当“大进程”到来时能有连续的大片空间，可以尽可能多地留下大片的空闲区,即，优先使用更小的空闲区。原理：空闲分区按容量递增次序链接。每次分配内存时顺序查找
- 最坏适应算法：为了解决最佳适应算法的问题—即留下太多难以利用的小碎片，可以在每次分配时优先使用最大的连续空闲区，这样分配后剩余的空闲区就不会太小。原理：空闲分区按容量递减次序链接，每次分配内存时顺序查找

- 邻近适应算法：首次适应算法每次都从链头开始查找的。这可能会导致低地址部分出现很多小的空闲分区，而每次分配查找时，都要经过这些分区，因此也增加了查找的开销。如果每次都从上次查找结束的位置开始检索，就能解决上述问题。原理：空闲分区以地址递增的顺序排列(可排成一个循环链表)。每次分配内存时从上次查找结束的位置开始查找空闲分区链(或空闲分区表)，找到大小能满足要求的第一个空闲分区

总结：

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找合适分区	空闲分区以地址递增次序排列	综合看性能最好，算法开销小，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片;算法开销大，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小片	大分区容易被用完，不利于大进程，算法开销大(原因同上)
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列(可排列成循环链表)	不用每次都从低地址的小分区开始检索。算法开销小(原因同首次适应算法)	会使高地址的大分区也被用完

聊聊：几种典型的锁

- 读写锁
(可以同时进行读)
写者必须互斥(只允许一个写者写，也不能读者写者同时进行)
写者优先于读者
- 互斥锁
一次只能一个线程拥有互斥锁，其他线程只有等待
互斥锁是在抢锁失败的情况下主动放弃CPU进入睡眠状态直到锁的状态改变时再唤醒，而操作系统负责线程调度，为了实现锁的状态发生改变时唤醒阻塞的线程或者进程，需要把锁交给操作系统管理，所以互斥锁在加锁操作时涉及上下文的切换
- 条件变量(同步)
互斥锁一个明显的缺点是他只有两种状态：锁定和非锁定
条件变量通过允许线程阻塞和等待另一个线程发送信号的方法弥补了互斥锁的不足，他常和互斥锁一起使用，以免出现竞态条件。当条件不满足时，线程往往解开相应的互斥锁并阻塞线程然后等待条件发生变化。一旦其他的某个线程改变了条件变量，他将通知相应的条件变量唤醒一个或多个正被此条件变量阻塞的线程。总的来说互斥锁是线程间互斥的机制，条件变量则是同步机制。

- 自旋锁

如果进线程无法取得锁，进线程不会立刻放弃CPU时间片，而是一直循环尝试获取锁，直到获取为止

如果别的线程长时期占有锁，那么自旋就是在浪费CPU做无用功，但是自旋锁一般应用于加锁时间很短的场景，这个时候效率比较高

聊聊：常见的几种磁盘调度算法

磁盘块的时间的影响因素有：

- 旋转时间（主轴转动盘面，使得磁头移动到适当的扇区上）
- 寻道时间（制动手臂移动，使得磁头移动到适当的磁道上）
- 实际的数据传输时间

其中，寻道时间最长，因此磁盘调度的主要目标是使磁盘的平均寻道时间最短。

1. 先来先服务：按照磁盘请求的顺序进行调度。

优点：公平

缺点：未对寻道做任何优化，使平均寻道时间可能较长。

2. 最短寻道时间优先

优先调度与当前磁头所在磁道距离最近的磁道。

缺点：不够公平。如果新到达的磁道请求总是比一个在等待的磁道请求近，那么在等待的磁道请求会一直等待下去，也就是出现饥饿现象。具体来说，两端的磁道请求更容易出现饥饿现象

3. 电梯扫描算法

电梯总是保持一个方向运行，直到该方向没有请求为止，然后改变运行方向。

电梯算法（扫描算法）和电梯的运行过程类似，总是按一个方向来进行磁盘调度，直到该方向上没有未完成的磁盘请求，然后改变方向。

因为考虑了移动方向，因此所有的磁盘请求都会被满足，解决了 SSTF 的饥饿问题

聊聊：什么叫抖动

如果对一个进程未分配它所要求的全部页面，有时就会出现分配的页面数增多但缺页率反而提高的异常现象。通俗的说刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出外存，这种频繁的页面调度行为称为抖动，或颠簸。

原因：进程频繁访问的页面数目高于可用的物理块数(分配给进程的物理块不够)

为进程分配的物理块太少，会使进程发生抖动现象。为进程分配的物理块太多，又会降低系统整体的并发度，降低某些资源的利用率

为了研究为应该为每个进程分配多少个物理块，Denning 提出了“进程工作集”的概念

聊聊：页面置换算法的了解

- 最佳置换法(OPT)：每次选择淘汰的页面将是以后永不使用，或者在最长时间内不再被访问的页面，这样可以保证最低的缺页率

最佳置换算法可以保证最低的缺页率，但实际上，只有在进程执行的过程中才能知道接下来会访问到的是哪个页面。操作系统无法提前预判页面访问序列。因此，最佳置换算法是无法实现的

- 先进先出置换算法(FIFO)：每次选择淘汰的页面是最早进入内存的页面

把调入内存的页面根据调入的先后顺序排成一个队列，需要换出页面时选择队头页面队列，它最大长度取决于系统为进程分配了多少个内存块

缺点：先进入的页面也有可能最经常被访问。因此，算法性能差。在FIFO算法下被反复调入和调出，并且有抖动现象

- 最近最久未使用置换算法(LRU)：每次淘汰的页面是最近最久未使用的页面
赋予每个页面对应的页表项中，用访问字段记录该页面自上次被访问以来所经历的时间 t (该算法的实现需要专门的硬件支持，虽然算法性能好，但是实现困难，开销大)。当需要淘汰一个页面时，选择现有页面中 t 值最大的，即最近最久未使用的页面。算法开销比较大
- 时钟置换算法(CLOCK)或者叫做或最近未用算法：循环扫描缓冲区像时钟一样转动
为每个页面设置一个访问位，再将内存中的页面都通过链接指针链接成一个循环队列。当某页被访问时，其访问位置为1。当需要淘汰一个页面时，只需检查页的访问位。如果是0，就选择该页换出；如果是1，则将它置为0，暂不换出，继续检查下一个页面，若第 n 轮扫描中所有页面都是1，则将这些页面的访问位依次置为0后，再进行第二轮扫描(第二轮扫描中一定会有访问位为0的页面，因此简单的CLOCK算法选择一个淘汰页面最多会经过两轮扫描)
- 改进的时钟置换算法：使用访问位和修改位来判断是否置换该页面
1类($A=0, M=0$)：表示该页面最近既未被访问，又未被修改，是最佳淘汰页。
2类($A=0, M=1$)：表示该页面最近未被访问，但已被修改，并不是很好的淘汰页。
3类($A=1, M=0$)：表示该页面最近已被访问，但未被修改，该页有可能再被访问。
4类($A=1, M=1$)：表示该页最近已被访问且被修改，该页可能再被访问。

总结：

1. 最佳置换算法性OPT能最好，但无法实现；
2. 先进先出置换算法实现简单，但算法性能差；
3. 最近最久未使用置换算法性能好，是最接近OPT算法性能的，但是实现起来需要专门的硬件支持，算法开销大
4. CLOCK循环扫描各页面 第一轮淘汰访问位=0的，并将扫描过的页面访问位改为1。若第二轮没选中，则进行第二轮扫描。实现简单，算法开销小；但未考虑页面是否被修改过。
5. 改进的clock：若用(访问位，修改位)的形式表述，则 第一轮:淘汰(0,0)，第二轮:淘汰(0,1)，并将扫描过的页面访问位都置为0，第三轮:淘汰(0,0)，第四轮:淘汰(0,1)。算法开销较小，性能也不错

聊聊：页面替换算法有哪些？

在程序运行过程中，如果要访问的页面不在内存中，就发生缺页中断从而将该页调入内存中。此时如果内存已无空闲空间，系统必须从内存中调出一个页面到磁盘对换区中来腾出空间。

包括以下算法：

- **最佳算法**：所选择的被换出的页面将是最长时间内不再被访问，通常可以保证获得最低的缺页率。这是一种理论上的算法，因为无法知道一个页面多长时间不再被访问。
- **先进先出**：选择换出的页面是最先进入的页面。该算法将那些经常被访问的页面也被换出，从而使缺页率升高。
- **LRU**：虽然无法知道将来要使用的页面情况，但是可以知道过去使用页面的情况。LRU 将最近最久未使用的页面换出。为了实现 LRU，需要在内存中维护一个所有页面的链表。当一个页面被访问时，将这个页面移到链表表头。这样就能保证链表表尾的页面是最近最久未访问的。因为每次访问都需要更新链表，因此这种方式实现的 LRU 代价很高。
- **时钟算法**：时钟算法使用环形链表将页面连接起来，再使用一个指针指向最老的页面。它将整个环形链表的每一个页面做一个标记，如果标记是 0，那么暂时就不会被替换，然后时钟算法遍历整个环，遇到标记为 1 的就替换，否则将标记为 0 的标记为 1。

聊聊：页面置换算法都有哪些

在地址映射过程中，如果在页面中发现所要访问的页面不在内存中，那么就会产生一条缺页中断。

当发生缺页中断时，如果操作系统内存中没有空闲页面，那么操作系统必须在内存选择一个页面将其移出内存，以便为即将调入的页面让出空间。而用来选择淘汰哪一页的规则叫做页面置换算法。

下面我汇总的这些页面置换算法比较齐全，只给出简单介绍，算法具体的实现和原理读者可以自行了解。

算法	注释
最优算法	不可实现，但可以用作基准
NRU(最近未使用) 算法	和 LRU 算法很相似
FIFO(先进先出) 算法	有可能会抛弃重要的页面
第二次机会算法	比 FIFO 有较大的改善
时钟算法	实际使用
LRU(最近最少)算法	比较优秀，但是很难实现
NFU(最不经常食用)算法	和 LRU 很类似
老化算法	近似 LRU 的高效算法
工作集算法	实施起来开销很大
工作集时钟算法	比较有效的算法

- **最优算法** 在当前页面中置换最后要访问的页面。不幸的是，没有办法来判定哪个页面是最后一个要访问的，因此实际上该算法不能使用。然而，它可以作为衡量其他算法的标准。
- **NRU** 算法根据 R 位和 M 位的状态将页面氛围四类。从编号最小的类别中随机选择一个页面。NRU 算法易于实现，但是性能不是很好。存在更好的算法。
- **FIFO** 会跟踪页面加载进入内存中的顺序，并把页面放入一个链表中。有可能删除存在时间最长但是还在使用的页面，因此这个算法也不是一个很好的选择。
- **第二次机会** 算法是对 FIFO 的一个修改，它会在删除页面之前检查这个页面是否仍在使用。如果页面正在使用，就会进行保留。这个改进大大提高了性能。
- **时钟** 算法是第二次机会算法的另外一种实现形式，时钟算法和第二次算法的性能差不多，但是会花费更少的时间来执行算法。
- **LRU** 算法是一个非常优秀的算法，但是没有特殊的硬件(TLB)很难实现。如果没有硬件，就不能使用 LRU 算法。
- **NFU** 算法是一种近似于 LRU 的算法，它的性能不是非常好。
- **老化** 算法是一种更接近 LRU 算法的实现，并且可以更好的实现，因此是一个很好的选择
- 最后两种算法都使用了工作集算法。工作集算法提供了合理的性能开销，但是它的实现比较复杂。**wsclock** 是另外一种变体，它不仅能够提供良好的性能，而且可以高效地实现。

最好的算法是老化算法和WSClock算法。他们分别是基于 LRU 和工作集算法。他们都具有良好的性能并且能够被有效的实现。还存在其他一些好的算法，但实际上这两个可能是最重要的。

聊聊：什么是按需分页

在操作系统中，进程是以页为单位加载到内存中的，按需分页是一种 虚拟内存 的管理方式。在使用请求分页的系统中，只有在尝试访问页面所在的磁盘并且该页面尚未在内存中时，也就发生了 缺页异常，操作系统才会将磁盘页面复制到内存中。

聊聊：什么是虚拟内存

虚拟内存 是一种内存分配方案，是一项可以用来辅助内存分配的机制。

我们知道，应用程序是按页装载进内存中的。

但并不是所有的页都会装载到内存中，计算机中的硬件和软件会将数据从 RAM 临时传输到磁盘中来弥补内存的不足。

如果没有虚拟内存的话，

一旦你将计算机内存填满后，计算机会对你说

呃，不，**对不起，您无法再加载任何应用程序，请关闭另一个应用程序以加载新的应用程序。**

对于虚拟内存，计算机可以执行操作是查看内存中最近未使用过的区域，然后将其复制到硬盘上。

虚拟内存通过复制技术实现了。

复制是自动进行的，你无法感知到它的存在。

聊聊：虚拟内存的实现方式

虚拟内存中，允许将一个作业分多次调入内存。采用连续分配方式时，会使相当一部分内存空间都处于暂时或永久 的空闲状态，造成内存资源的严重浪费，而且也无法从逻辑上扩大内存容量。因此，虚拟内存的实需要建立在离散分配的内存管理方式的基础上。

虚拟内存的实现有以下三种方式：

- 请求分页存储管理。
- 请求分段存储管理。
- 请求段页式存储管理。

不管哪种方式，都需要有一定的硬件支持。一般需要的支持有以下几个方面：

- 一定容量的内存和外存。
- 页表机制（或段表机制），作为主要的数据结构。
- 中断机构，当用户程序要访问的部分尚未调入内存，则产生中断。
- 地址变换机构，逻辑地址到物理地址的变换。

聊聊：内存为什么要分段

内存是随机访问设备，对于内存来说，不需要从头开始查找，只需要直接给出地址即可。内存的分段是从 8086 CPU 开始的，8086 的 CPU 还是 16 位的寄存器宽，16 位的寄存器可以存储的数字范围是 2^{16} 的 16 次方，即 64 KB，8086 的 CPU 还没有 虚拟地址，只有物理地址，也就是说，如果两个相同的程序编译出来的地址相同，那么这两个程序是无法同时运行的。为了解决这个问题，操作系统设计人员提出了让 CPU 使用 段基址 + 段内偏移 的方式来访问任意内存。这样的好处是让程序可以 重定位，这也是内存为什么要分段的第一个原因。

那么什么是重定位呢？

简单来说就是将程序中的指令地址改为另一个地址，地址处存储的内容还是原来的。

CPU 采用段基址 + 段内偏移地址的形式访问内存，就需要提供专门的寄存器，这些专门的寄存器就是 CS、DS、ES 等

也就是说，程序中需要用到哪块内存，就需要先加载合适的段到段基址寄存器中，再给出相对于该段基址的段偏移地址即可。

CPU 中的地址加法器会将这两个地址进行合并，从地址总线送入内存。

8086 的 CPU 有 20 根地址总线，最大的寻址能力是 1MB，而段基址所在的寄存器宽度只有 16 位，最大为你 64 KB 的寻址能力，64 KB 显然不能满足 1MB 的最大寻址范围，所以就要把内存分段，每个段的最大寻址能力是 64KB，但是仍旧不能达到最大 1 MB 的寻址能力，所以这时候就需要 **偏移地址** 的辅助，偏移地址也存入寄存器，同样为 64 KB 的寻址能力，这么一看还是不能满足 1MB 的寻址，所以 CPU 的设计者对地址单元动了手脚，将段基址左移 4 位，然后再和 16 位的段内偏移地址相加，就达到了 1MB 的寻址能力。

所以内存分段的第二个目的就是能够访问到所有内存。

聊聊：物理地址、逻辑地址、有效地址、线性地址、虚拟地址的区别

物理地址就是内存中真正的地址，它就相当于是你家的门牌号，你家就肯定有这个门牌号，具有唯一性。

不管哪种地址，最终都会映射为物理地址。

在实模式下，段基址 + 段内偏移经过地址加法器的处理，经过地址总线传输，最终也会转换为物理地址。

但是在保护模式下，段基址 + 段内偏移被称为线性地址，不过此时的段基址不能称为真正的地址，而是会被称作为一个选择子的东西，选择子就是个索引，相当于数组的下标，通过这个索引能够在 GDT 中找到相应的段描述符，段描述符记录了**段的起始、段的大小**等信息，这样便得到了基地址。

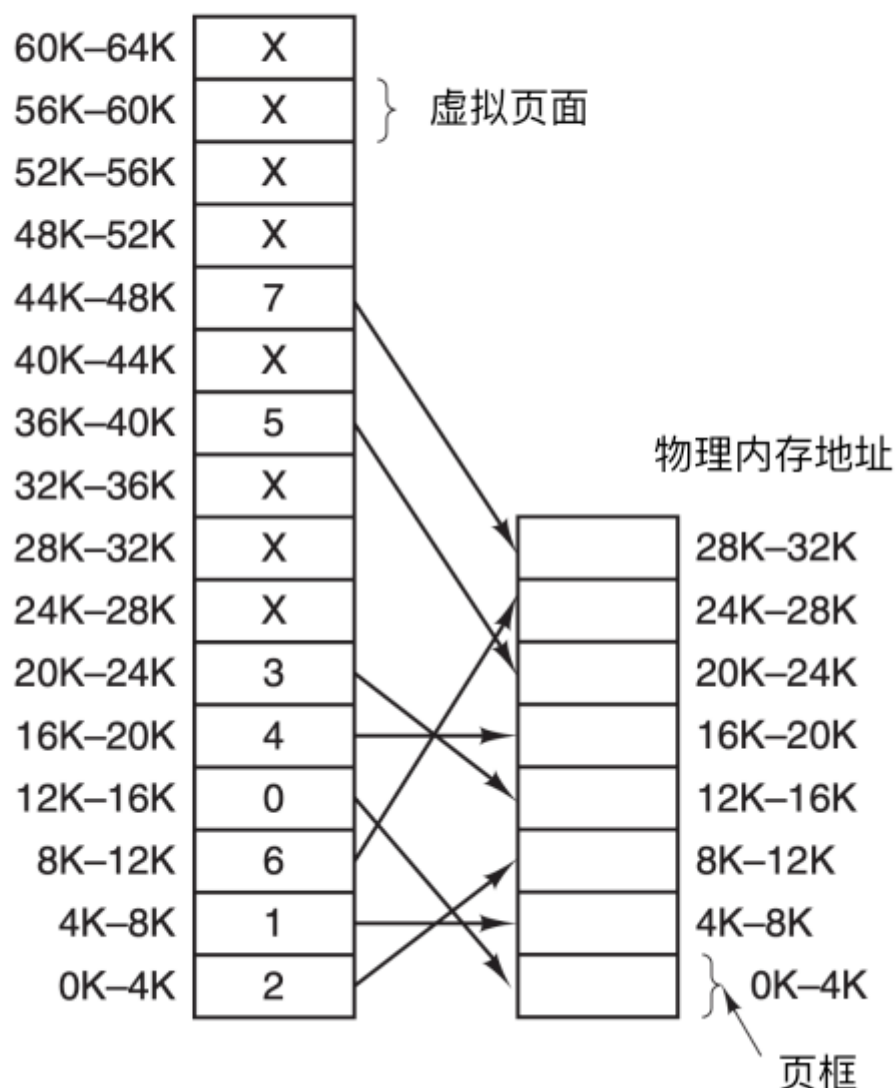
如果此时没有开启内存分页功能，那么这个线性地址可以直接当做物理地址来使用，直接访问内存。如果开启了分页功能，那么这个线性地址又多了一个名字，这个名字就是虚拟地址。

不论在实模式还是保护模式下，段内偏移地址都叫做有效地址。有效地址也是逻辑地址。

线性地址可以看作是虚拟地址，虚拟地址不是真正的物理地址，但是虚拟地址会最终被映射为物理地址。

下面是虚拟地址 -> 物理地址的映射。

虚拟地址空间



聊聊：空闲内存管理的方式

操作系统在动态分配内存时（malloc, new），需要对空间内存进行管理。一般采用了两种方式：位图和空闲链表。

使用位图进行管理

使用位图方法时，内存可能被划分为小到几个字或大到几千字节的分配单元。每个分配单元对应于位图中的一位，0 表示空闲，1 表示占用（或者相反）。一块内存区域和其对应的位图如下

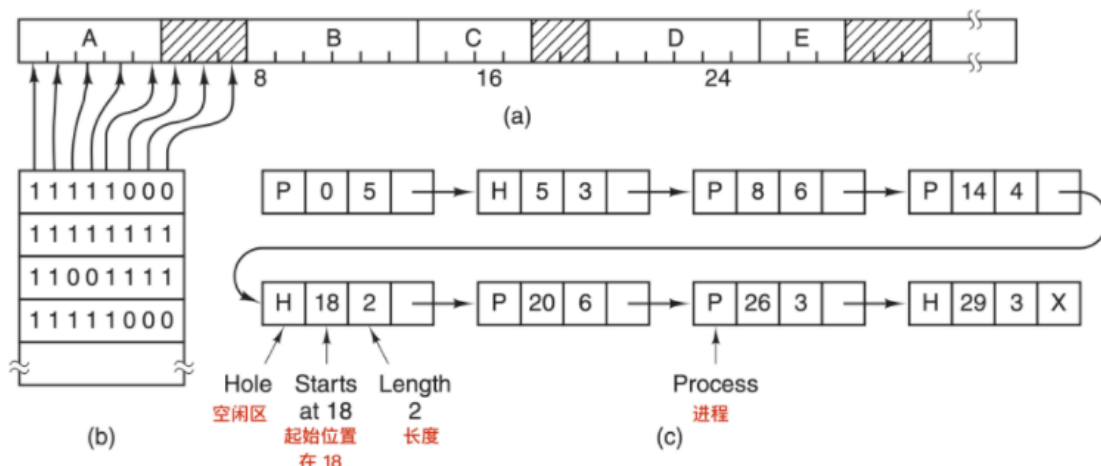


图 a 表示一段有 5 个进程和 3 个空闲区的内存，刻度为内存分配单元，阴影区表示空闲（在位图中用 0 表示）；图 b 表示对应的位图；图 c 表示用链表表示同样的信息

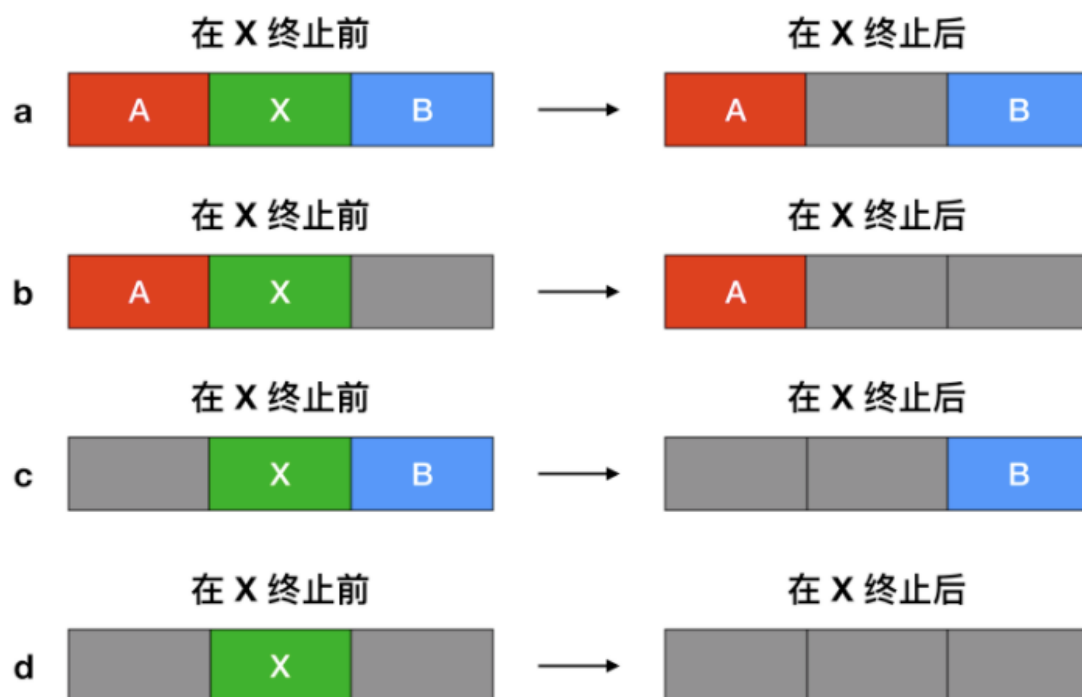
分配单元的大小是一个重要的设计因素，分配单位越小，位图越大。然而，即使只有 4 字节的分配单元，32 位的内存也仅仅只需要位图中的 1 位。32n 位的内存需要 n 位的位图，所以**1 个位图只占用了 1/32 的内存**。如果选择更大的内存单元，位图应该要更小。如果进程的大小不是分配单元的整数倍，那么在最后一个分配单元中会有大量的内存被浪费。

位图提供了一种简单的方法在固定大小的内存中跟踪内存的使用情况，因为**位图的大小取决于内存和分配单元的大小**。这种方法有一个问题，当决定为把具有 k 个分配单元的进程放入内存时，**内容管理器** (memory manager) 必须搜索位图，在位图中找出能够运行 k 个连续 0 位的串。在位图中找出制定长度的连续 0 串是一个很耗时的操作，这是位图的缺点。（可以简单理解为在杂乱无章的数组中，找出具有一大长串空闲的数组单元）

使用空闲链表

另一种记录内存使用情况的方法是，维护一个记录已分配内存段和空闲内存段的链表，段会包含进程或者是两个进程的空闲区域。可用上面的图 c **来表示内存的使用情况**。链表中的每一项都可以代表一个**空闲区(H)**或者是**进程(P)**的起始标志，长度和下一个链表项的位置。

在这个例子中，**段链表(segment list)**是按照地址排序的。这种方式的优点是，当进程终止或被交换时，更新列表很简单。一个终止进程通常有两个邻居（除了内存的顶部和底部外）。相邻的可能是进程也可能是空闲区，它们有四种组合方式。



当按照地址顺序在链表中存放进程和空闲区时，有几种算法可以为创建的进程（或者从磁盘中换入的进程）分配内存。

- 首次适配算法：在链表中进行搜索，直到找到最初的一个足够大的空闲区，将其分配。除非进程大小和空闲区大小恰好相同，否则会将空闲区分为两部分，一部分为进程使用，一部分成为新的空闲区。该方法是速度很快的算法，因为索引链表结点的个数较少。
- 下次适配算法：工作方式与首次适配算法相同，但每次找到新的空闲区位置后都记录当前位置，下次寻找空闲区从上次结束的地方开始搜索，而不是与首次适配放一样从头开始；
- 最佳适配算法：搜索整个链表，找出能够容纳进程分配的最小的空闲区。这样存在的问题是，尽管可以保证为进程找到一个最为合适的空闲区进行分配，但大多数情况下，这样的空闲区被分为两部分，一部分用于进程分配，一部分会生成很小的空闲区，而这样的空闲区很难再被进行利用。

- 最差适配算法：与最佳适配算法相反，每次分配搜索最大的空闲区进行分配，从而可以使得空闲区拆分得到的新的空闲区可以更好的被进行利用。

聊聊：什么是交换空间？

操作系统把物理内存(physical RAM)分成一块一块的小内存，每一块内存被称为**页(page)**。当内存资源不足时，**Linux把某些页的内容转移至硬盘上的一块空间上，以释放内存空间**。硬盘上的那块空间叫做**交换空间(swap space)**,而这一过程被称为交换(swapping)。物理内存和交换空间的总容量就是**虚拟内存的可用容量**。

用途：

- 物理内存不足时一些不常用的页可以被交换出去，腾给系统。
- 程序启动时很多内存页被用来初始化，之后便不再需要，可以交换出去。

聊聊：什么是缓冲区溢出？有什么危害？

缓冲区溢出是指当计算机向缓冲区填充数据时超出了缓冲区本身的容量，溢出的数据覆盖在合法数据上。

危害有以下两点：

- 程序崩溃，导致拒绝额服务
- 跳转并且执行一段恶意代码

造成缓冲区溢出的主要原因是程序中没有仔细检查用户输入。

聊聊：用户态（模式）与内核态（模式）

用户态和系统态是操作系统的两种运行状态：

内核态：内核态运行的程序可以访问计算机的任何数据和资源，不受限制，包括外围设备，比如网卡、硬盘等。处于内核态的 CPU 可以从一个程序切换到另外一个程序，并且占用 CPU 不会发生抢占情况。切换进程，拥有最高权限。

用户态：用户态运行的程序只能受限地访问内存，只能直接读取用户程序的数据，并且不允许访问外围设备，用户态下的 CPU 不允许独占，也就是说 CPU 能够被其他程序获取。**大部分用户直接面对的程序都是运行在用户态**

将操作系统的运行状态分为用户态和内核态，主要是为了对访问能力进行限制，防止随意进行一些比较危险的操作导致系统的崩溃，比如设置时钟、内存清理，这些都需要在内核态下完成。

用户模式和内核模式最根本区别就是是否拥有对硬件的控制权。

如果用户模式想操作硬件，这时操作系统可以暴露一些借口给我们，比如创建销毁进程，让用户分配更多内存等操作。等几百个API供用户模式使用。

聊聊：用户态切换到内核态的3种方式

1. 系统调用

这是用户态进程主动要求切换到内核态的一种方式，用户态进程通过系统调用申请使用操作系统提供的服务程序完成工作，比如前例中fork()实际上就是执行了一个创建新进程的系统调用。而系统调用的机制其核心还是使用了操作系统为用户特别开放的一个中断来实现，例如Linux的int 80h中断。

2. 外围设备的中断

硬件中断，进程执行过程中，好比说用户点击了什么按钮，触发了按键中断，要赶紧去处理这个中断啊，保存进程上下文，切换到中断处理流程，处理完了，恢复进程上下文，返回用户态（返回之前可能会进行进程调度，选择一个更值得运行的进程投入运行态），进程继续执行

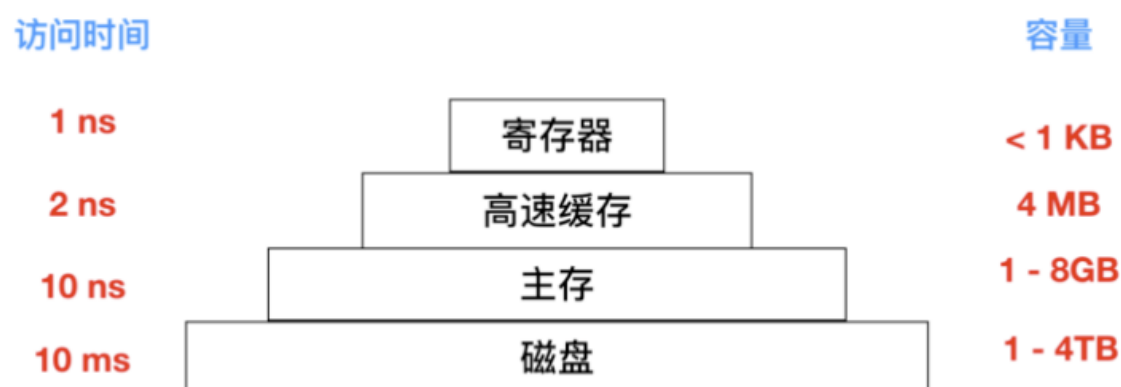
3. 异常

当CPU在执行运行在用户态下的程序时，发生了某些事先不可知的异常，这时会触发由当前运行进程切换到处理此异常的内核相关程序中，也就转到了内核态，比如缺页异常。

文件系统篇

聊聊：如何提高文件系统性能的方式

访问磁盘的效率要比内存慢很多，具体如下图



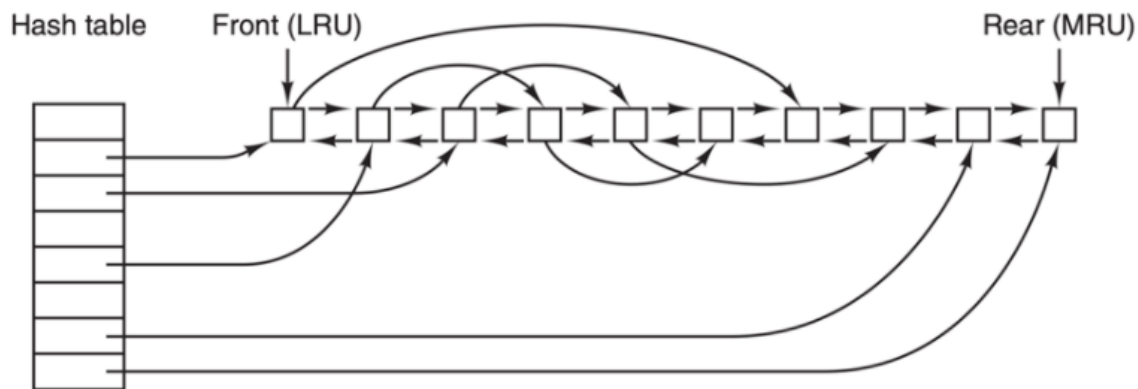
所以磁盘优化是很有必要的，下面我们会讨论几种优化方式

高速缓存

最常用的减少磁盘访问次数的技术是使用 块高速缓存(block cache) 或者 缓冲区高速缓存(buffer cache)。高速缓存指的是一系列的块，它们在逻辑上属于磁盘，但实际上基于性能的考虑被保存在内存中。

管理高速缓存有不同的算法，常用的算法是：检查全部的读请求，查看在高速缓存中是否有所需要的块。如果存在，可执行读操作而无须访问磁盘。如果检查块不再高速缓存中，那么首先把它读入高速缓存，再复制到所需的地方。之后，对同一个块的请求都通过 高速缓存 来完成。

高速缓存的操作如下图所示



由于在高速缓存中有许多块，所以需要某种方法快速确定所需的块是否存在。常用方法是将设备和磁盘地址进行散列操作。然后在散列表中查找结果。具有相同散列值的块在一个链表中连接在一起（这个数据结构是不是很像 HashMap?），这样就可以沿着冲突链查找其他块。

如果高速缓存 **已满**，此时需要调入新的块，则要把原来的某一块调出高速缓存，如果要调出的块在上次调入后已经被修改过，则需要把它写回磁盘。这种情况与分页非常相似。

块提前读

第二个明显提高文件系统的性能是**在需要用到块之前试图提前将其写入高速缓存从而提高命中率**。许多文件都是 **顺序读取**。如果请求文件系统在某个文件中生成块 k ，文件系统执行相关操作并且在完成之后，会检查高速缓存，以便确定块 $k + 1$ 是否已经在高速缓存。如果不在，文件系统会为 $k + 1$ 安排一个预读取，因为文件希望在用到该块的时候能够直接从高速缓存中读取。

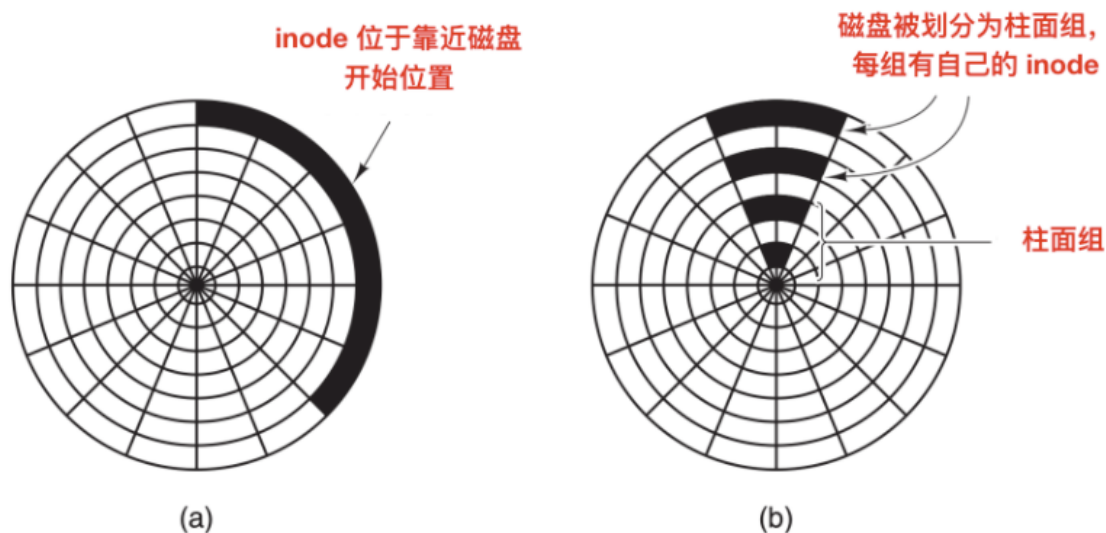
当然，块提前读取策略只适用于实际顺序读取的文件。对随机访问的文件，提前读丝毫不起作用。甚至还会造成阻碍。

减少磁盘臂运动

高速缓存和块提前读并不是提高文件系统性能的唯一方法。另一种重要的技术是**把有可能顺序访问的块放在一起，当然最好是在同一个柱面上，从而减少磁盘臂的移动次数**。当写一个输出文件时，文件系统就必须按照要求一次又一次地分配磁盘块。如果用位图来记录空闲块，并且整个位图在内存中，那么选择与前一块最近的空闲块是很容易的。如果用空闲表，并且链表的一部分存在磁盘上，要分配紧邻的空闲块就会困难很多。

不过，即使采用空闲表，也可以使用 **块簇** 技术。即不用块而用连续块簇来跟踪磁盘存储区。如果一个扇区有 512 个字节，有可能系统采用 1 KB 的块（2 个扇区），但却按每 2 块（4 个扇区）一个单位来分配磁盘存储区。这和 2 KB 的磁盘块并不相同，因为在高速缓存中它仍然使用 1 KB 的块，磁盘与内存数据之间传送也是以 1 KB 进行，但在一个空闲的系统上顺序读取这些文件，寻道的次数可以减少一半，从而使文件系统的性能大大改善。若考虑旋转定位则可以得到这类方法的变体。在分配块时，系统尽量把一个文件中的连续块存放在同一个柱面上。

在使用 inode 或任何类似 inode 的系统中，另一个性能瓶颈是，读取一个很短的文件也需要两次磁盘访问：**一次是访问 inode，一次是访问块**。通常情况下，inode 的放置如下图所示



a) inode 放在磁盘开始位置；b) 磁盘分为柱面组，每组有自己的块和 inode

其中，全部 inode 放在靠近磁盘开始位置，所以 inode 和它所指向的块之间的平均距离是柱面组的一半，这将会需要较长时间的寻道时间。

一个简单的改进方法是，在磁盘中部而不是开始处存放 inode，此时，在 inode 和第一个块之间的寻道时间减为原来的一半。另一种做法是：将磁盘分成多个柱面组，每个柱面组有自己的 inode，数据块和空闲表，如上图 b 所示。

当然，只有在磁盘中装有磁盘臂的情况下，讨论寻道时间和旋转时间才是有意义的。现在越来越多的电脑使用 **固态硬盘(SSD)**，对于这些硬盘，由于采用了和闪存同样的制造技术，使得随机访问和顺序访问在传输速度上已经较为相近，传统硬盘的许多问题就消失了。但是也引发了新的问题。

磁盘碎片整理

在初始安装操作系统后，文件就会被不断的创建和清除，于是磁盘会产生很多的碎片，在创建一个文件时，它使用的块会散布在整个磁盘上，降低性能。删除文件后，回收磁盘块，可能会造成空穴。

磁盘性能可以通过如下方式恢复：移动文件使它们相互挨着，并把所有的至少是大部分的空闲空间放在一个或多个大的连续区域内。Windows 有一个程序 **defrag** 就是做这个事儿的。Windows 用户会经常使用它，SSD 除外。

磁盘碎片整理程序会在让文件系统上很好地运行。Linux 文件系统（特别是 ext2 和 ext3）由于其选择磁盘块的方式，在磁盘碎片整理上一般不会像 Windows 一样困难，因此很少需要手动的磁盘碎片整理。而且，固态硬盘并不受磁盘碎片的影响，事实上，在固态硬盘上做磁盘碎片整理反而是多此一举，不仅没有提高性能，反而磨损了固态硬盘。所以碎片整理只会缩短固态硬盘的寿命。

聊聊：磁盘臂调度算法

一般情况下，影响磁盘快读写的时间由下面几个因素决定

- 寻道时间 - 寻道时间指的就是将磁盘臂移动到需要读取磁盘块上的时间
- 旋转延迟 - 等待合适的扇区旋转到磁头下所需的时间
- 实际数据的读取或者写入时间

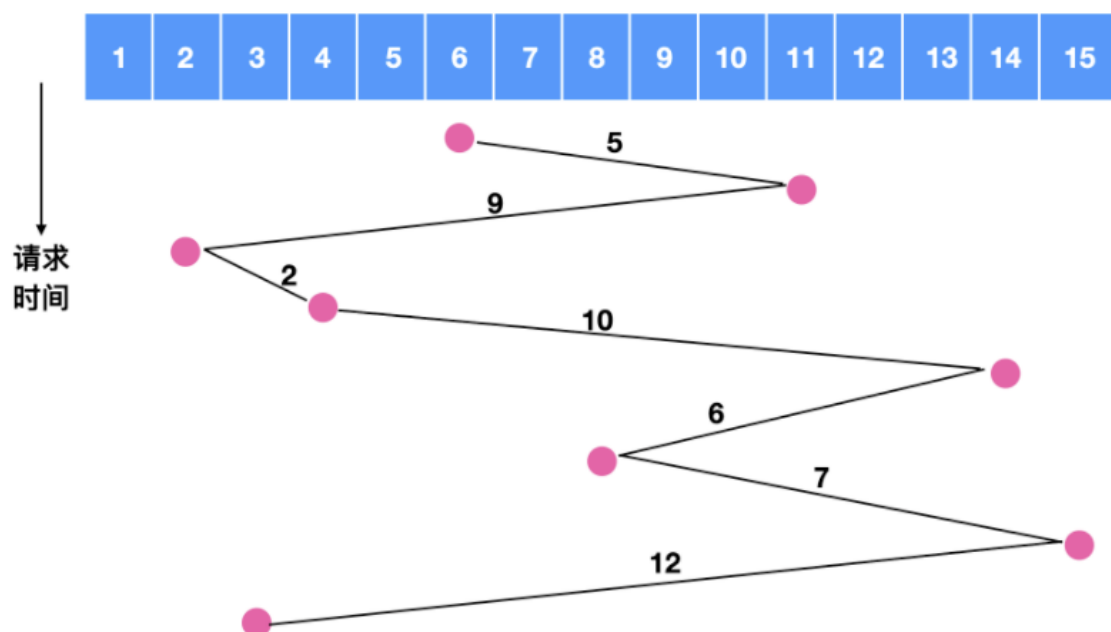
这三种时间参数也是磁盘寻道的过程。一般情况下，寻道时间对总时间的影响最大，所以，有效的降低寻道时间能够提高磁盘的读取速度。

如果磁盘驱动程序每次接收一个请求并按照接收顺序完成请求，这种处理方式也就是 **先来先服务 (First-Come, First-served, FCFS)**，这种方式很难优化寻道时间。因为每次都会按照顺序处理，不管顺序如何，有可能这次读完后需要等待一个磁盘旋转一周才能继续读取，而其他柱面能够马上进行读取，这种情况下每次请求也会排队。

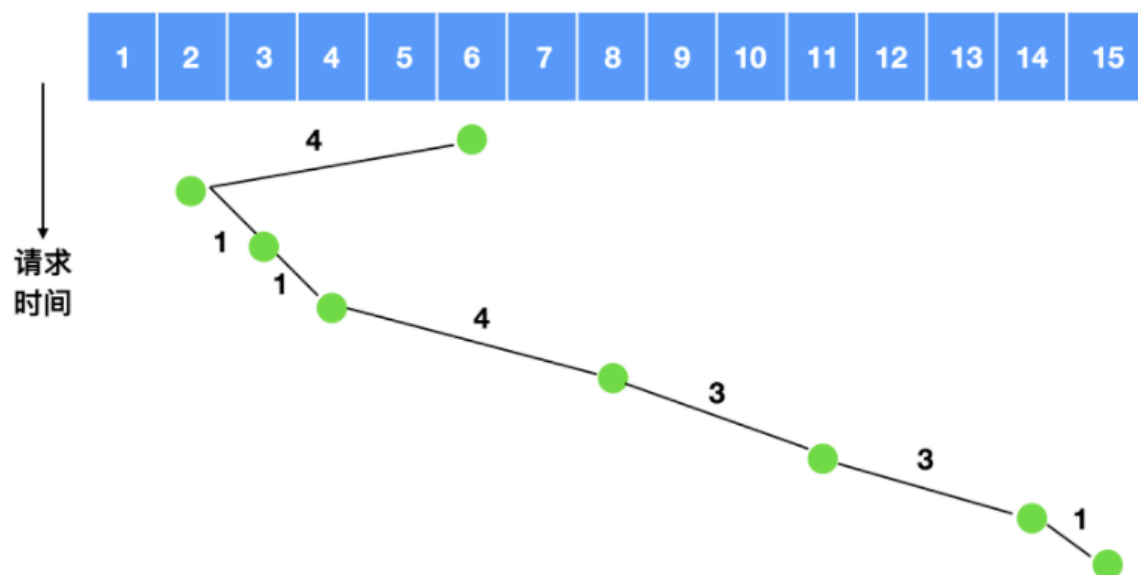
通常情况下，磁盘在进行寻道时，其他进程会产生其他的磁盘请求。磁盘驱动程序会维护一张表，表中会记录着柱面号当作索引，每个柱面未完成的请求会形成链表，链表头存放在表的相应表项中。

一种对先来先服务的算法改良的方案是使用 **最短路径优先(SSF)** 算法，下面描述了这个算法。

假如我们在对磁道 6 号进行寻址时，同时发生了对 11, 2, 4, 14, 8, 15, 3 的请求，如果采用先来先服务的原则，如下图所示



我们可以计算一下磁盘臂所跨越的磁盘数量为 $5 + 9 + 2 + 10 + 6 + 7 + 12 = 51$ ，相当于是跨越了 51 次盘面，如果使用最短路径优先，我们来计算一下跨越的盘面



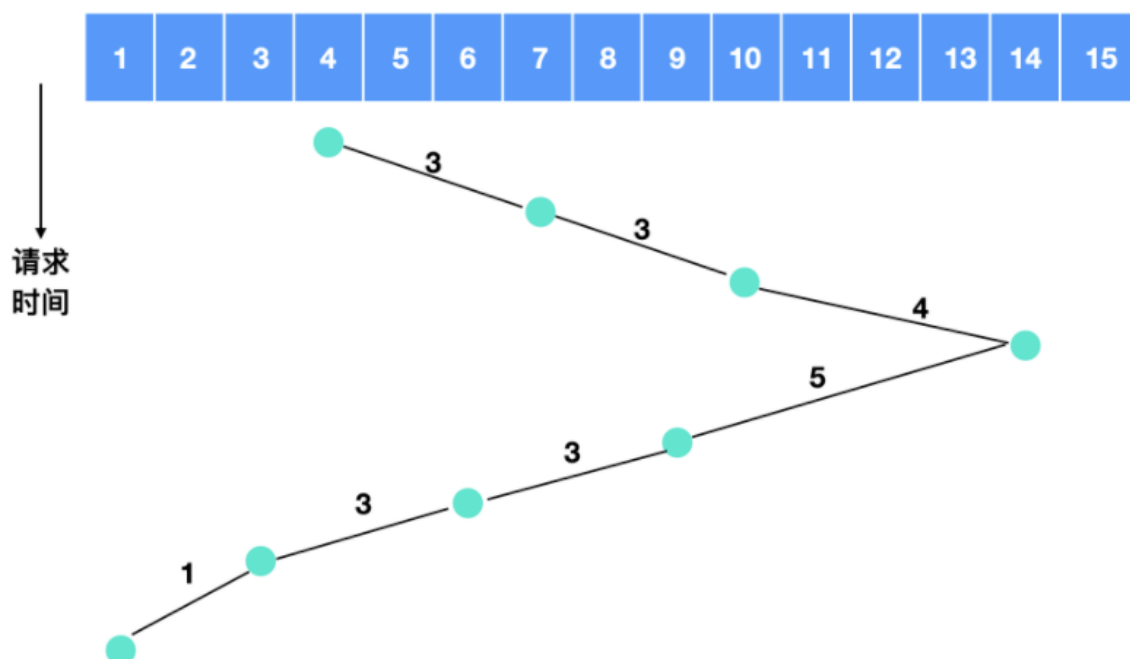
跨越的磁盘数量为 $4 + 1 + 1 + 4 + 3 + 3 + 1 = 17$ ，相比 51 足足省了两倍的时间。

但是，最短路径优先的算法也不是完美无缺的，这种算法照样存在问题，那就是 **优先级** 问题，

这里有一个原型可以参考就是我们日常生活中的电梯，电梯使用一种 **电梯算法(elevator algorithm)** 来进行调度，从而满足协调效率和公平性这两个相互冲突的目标。电梯一般会保持向一个方向移动，直到在那个方向上没有请求为止，然后改变方向。

电梯算法需要维护一个二进制位，也就是当前的方向位：UP(向上)或者是DOWN(向下)。当一个请求处理完成后，磁盘或电梯的驱动程序会检查该位，如果此位是UP位，磁盘臂或者电梯仓移到下一个更高阶未完成的请求。如果高位没有未完成的请求，则取相反方向。当方向位是DOWN时，同时存在一个低位的请求，磁盘臂会转向该点。如果不存在的话，那么它只是停止并等待。

我们举个例子来描述一下电梯算法，比如各个柱面得到服务的顺序是4, 7, 10, 14, 9, 6, 3, 1，那么它的流程图如下



所以电梯算法需要跨越的盘面数量是 $3 + 3 + 4 + 5 + 3 + 3 + 1 = 22$

电梯算法通常情况下不如 SSF 算法。

一些磁盘控制器为软件提供了一种检查磁头下方当前扇区号的方法，使用这样的控制器，能够进行另一种优化。如果对一个相同的柱面有两个或者多个请求正等待处理，驱动程序可以发出请求读写下一次要通过磁头的扇区。

这里需要注意一点，当一个柱面有多条磁道时，相继的请求可能针对不同的磁道，这种选择没有代价，因为选择磁头不需要移动磁盘臂也没有旋转延迟。

对于磁盘来说，最影响性能的就是寻道时间和旋转延迟，所以一次只读取一个或两个扇区的效率是非常低的。出于这个原因，许多磁盘控制器总是读出多个扇区并进行高速缓存，即使只请求一个扇区时也是这样。一般情况下读取一个扇区的同时会读取该扇区所在的磁道或者是所有剩余的扇区被读出，读出扇区的数量取决于控制器的高速缓存中有多少可用的空间。

磁盘控制器的高速缓存和操作系统的高速缓存有一些不同，磁盘控制器的高速缓存用于缓存没有实际被请求的块，而操作系统维护的高速缓存由显示地读出的块组成，并且操作系统会认为这些块在近期仍然会频繁使用。

当同一个控制器上有多个驱动器时，操作系统应该为每个驱动器都单独的维护一个未完成的请求表。一旦有某个驱动器闲置时，就应该发出一个寻道请求来将磁盘臂移到下一个被请求的柱面。如果下一个寻道请求到来时恰好没有磁盘臂处于正确的位置，那么驱动程序会在刚刚完成传输的驱动器上发出一个新的寻道命令并等待，等待下一次中断到来时检查哪个驱动器处于闲置状态。

聊聊：RAID 的不同级别

RAID 称为 磁盘冗余阵列，简称 磁盘阵列。利用虚拟化技术把多个硬盘结合在一起，成为一个或多个磁盘阵列组，目的是提升性能或数据冗余。

RAID 有不同的级别

- RAID 0 - 无容错的条带化磁盘阵列
- RAID 1 - 镜像和双工
- RAID 2 - 内存式纠错码
- RAID 3 - 比特交错奇偶校验
- RAID 4 - 块交错奇偶校验
- RAID 5 - 块交错分布式奇偶校验
- RAID 6 - P + Q 冗余

10 篇

聊聊：操作系统中的时钟是什么

时钟(Clocks) 也被称为 定时器(timers)，时钟/定时器对任何程序系统来说都是必不可少的。

时钟负责维护时间、防止一个进程长期占用 CPU 时间等其他功能。

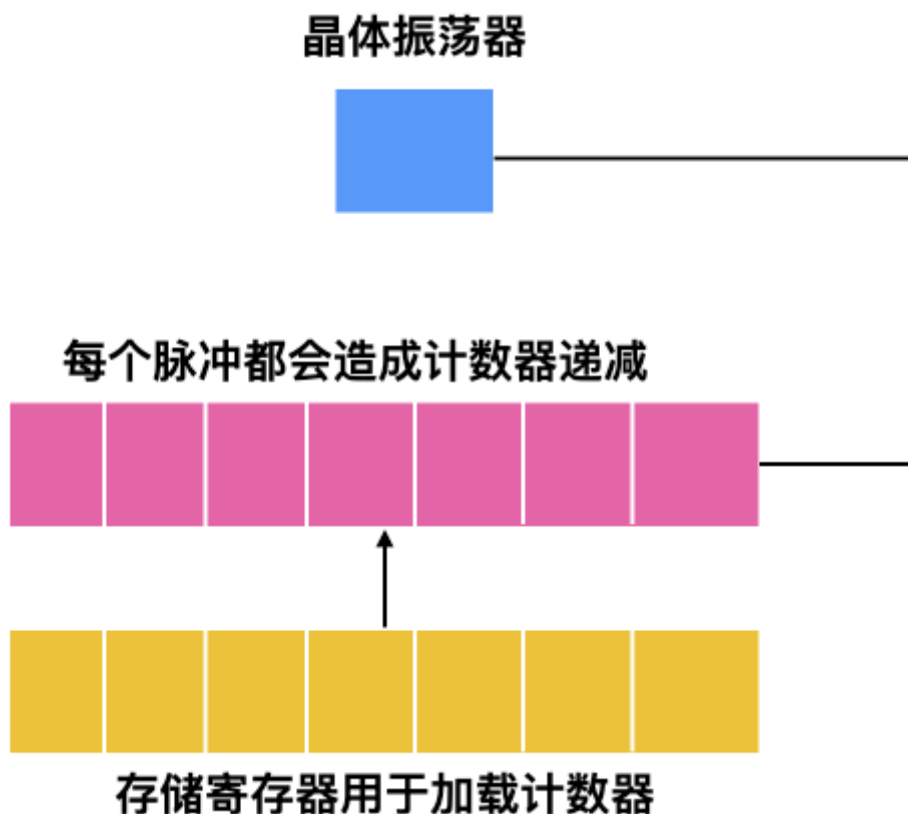
时钟软件(clock software) 也是一种设备驱动的方式。

下面我们就来对时钟进行介绍，一般都是先讨论硬件再介绍软件，采用由下到上的方式，也是告诉你，底层是最重要的。

时钟硬件

在计算机中有两种类型的时钟，这些时钟与现实生活中使用的时钟完全不一样。

- 比较简单的一种时钟被连接到 110 V 或 220 V 的电源线上，这样每个 电压周期 会产生一个中断，大概是 50 - 60 HZ。这些时钟过去一直占据支配地位。
- 另外的一种时钟由晶体振荡器、计数器和寄存器组成，示意图如下所示



这种时钟称为 可编程时钟，可编程时钟有两种模式，

一种是 **一键式(one-shot mode)**，当时钟启动时，会把存储器中的值复制到计数器中，然后，每次晶体的振荡器的脉冲都会使计数器 -1。当计数器变为 0 时，会产生一个中断，并停止工作，直到软件再一次显示启动。

还有一种模式是 **方波(square-wave mode)** 模式，在这种模式下，当计数器变为 0 并产生中断后，存储寄存器的值会自动复制到计数器中，这种周期性的中断称为一个时钟周期。

聊聊：设备控制器的主要功能

设备控制器是一个 **可编址** 的设备，当它仅控制一个设备时，它只有一个唯一的设备地址；

如果设备控制器控制多个可连接设备时，则应含有多个设备地址，并使每一个设备地址对应一个设备。

设备控制器主要分为两种：字符设备和块设备

设备控制器的主要功能有下面这些

- **接收和识别命令**：设备控制器可以接受来自 CPU 的指令，并进行识别。设备控制器内部也会有寄存器，用来存放指令和参数
- **进行数据交换**：CPU、控制器和设备之间会进行数据的交换，CPU 通过总线把指令发送给控制器，或从控制器中并行地读出数据；控制器将数据写入指定设备。
- **地址识别**：每个硬件设备都有自己的地址，设备控制器能够识别这些不同的地址，来达到控制硬件的目的，此外，为使 CPU 能向寄存器中写入或者读取数据，这些寄存器都应具有唯一的地址。
- **差错检测**：设备控制器还具有对设备传递过来的数据进行检测的功能。

聊聊：中断处理过程

中断处理方案有很多种，下面是《ARM System Developer's Guide

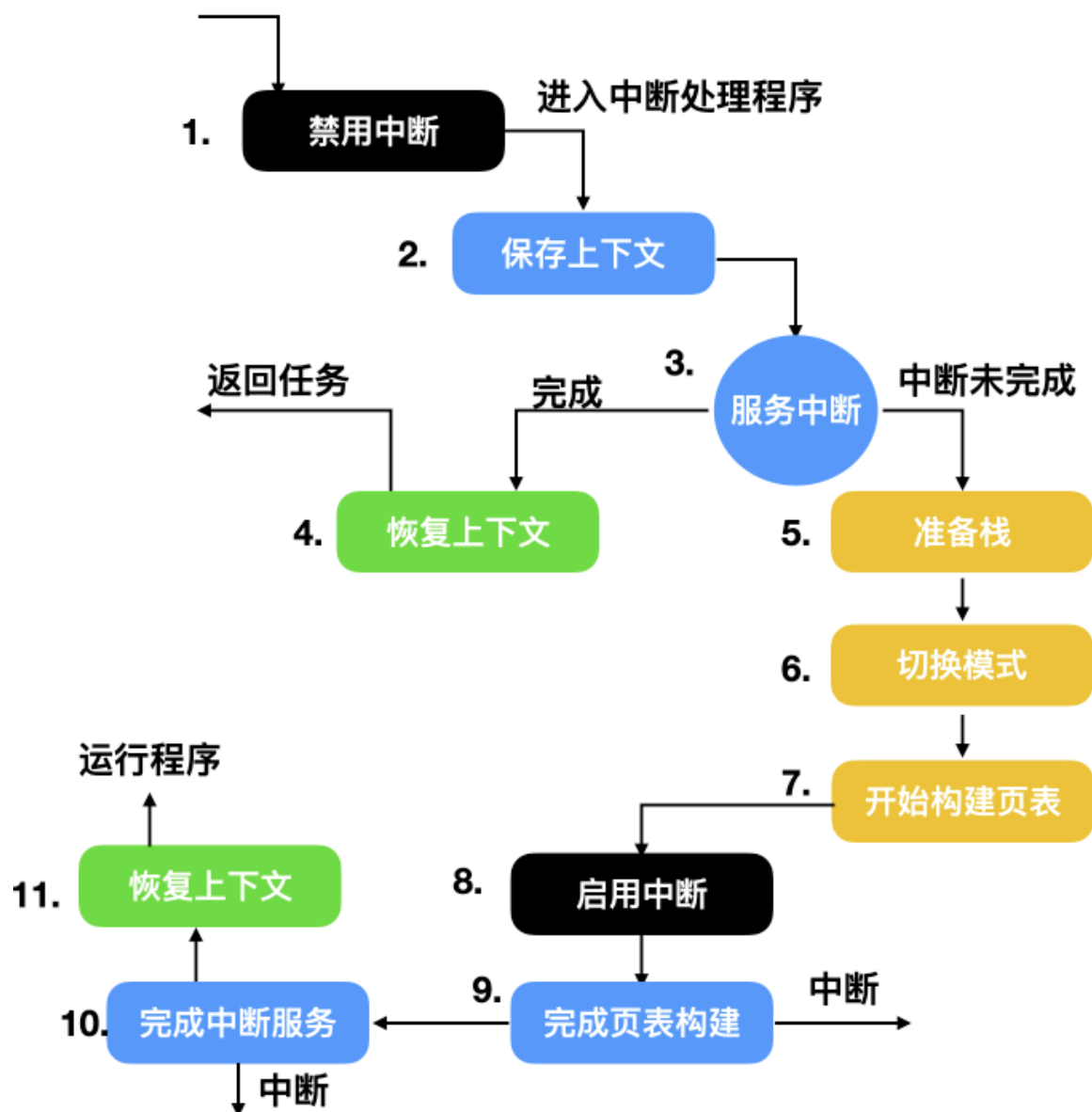
Designing and Optimizing System Software》列出来的一些方案

- **非嵌套** 的中断处理程序按照顺序处理各个中断，非嵌套的中断处理程序也是最简单的中断处理
- **嵌套** 的中断处理程序会处理多个中断而无需分配优先级
- **可重入** 的中断处理程序可使用优先级处理多个中断
- **简单优先级** 中断处理程序可处理简单的中断
- **标准优先级** 中断处理程序比低优先级的中断处理程序在更短的时间能够处理优先级更高的中断
- **高优先级** 中断处理程序在短时间能够处理优先级更高的任务，并直接进入特定的服务例程。
- **优先级分组** 中断处理程序能够处理不同优先级的中断任务

下面是一些通用的中断处理程序的步骤，不同的操作系统实现细节不一样

- 保存所有没有被中断硬件保存的寄存器
- 为中断服务程序设置上下文环境，可能包括设置 **TLB**、**MMU** 和页表，如果不太了解这三个概念，请参考另外一篇文章
- 为中断服务程序设置栈
- 对中断控制器作出响应，如果不存在集中的中断控制器，则继续响应中断
- 把寄存器从保存它的地方拷贝到进程表中
- 运行中断服务程序，它会从发出中断的设备控制器的寄存器中提取信息
- 操作系统会选择一个合适的进程来运行。如果中断造成了一些优先级更高的进程变为就绪态，则选择运行这些优先级高的进程
- 为进程设置 MMU 上下文，可能也会需要 TLB，根据实际情况决定
- 加载进程的寄存器，包括 PSW 寄存器
- 开始运行新的进程

上面我们罗列了一些大致的中断步骤，不同性质的操作系统和中断处理程序能够处理的中断步骤和细节也不尽相同，下面是一个嵌套中断的具体运行步骤



聊聊：什么是设备驱动程序

在计算机中，设备驱动程序是一种计算机程序，它能够控制或者操作连接到计算机的特定设备。驱动程序提供了与硬件进行交互的软件接口，使操作系统和其他计算机程序能够访问特定设备，而不需要了解其硬件的具体构造。

聊聊：什么是 DMA

DMA 的中文名称是直接内存访问，它意味着 CPU 授予 I/O 模块权限在不涉及 CPU 的情况下读取或写入内存。也就是 DMA 可以不需要 CPU 的参与。

这个过程由称为 DMA 控制器（DMAC）的芯片管理。由于 DMA 设备可以直接在内存之间传输数据，而不是使用 CPU 作为中介，因此可以缓解总线上的拥塞。

DMA 通过允许 CPU 执行任务，同时 DMA 系统通过系统和内存总线传输数据来提高系统并发性。

聊聊：直接内存访问的特点

DMA 方式有如下特点：

- 数据传送以数据块为基本单位

- 所传送的数据从设备直接送入主存，或者从主存直接输出到设备上
- 仅在传送一个或多个数据块的开始和结束时才需 CPU 的干预，而整块数据的传送则是在控制器的控制下完成。

DMA 方式和中断驱动控制方式相比，减少了 CPU 对 I/O 操作的干预，进一步提高了 CPU 与 I/O 设备的并行操作程度。

DMA 方式的线路简单、价格低廉，适合高速设备与主存之间的成批数据传送，小型、微型机中的快速设备均采用这种方式，但其功能较差，不能满足复杂的 I/O 要求。

聊聊：IO多路复用？

IO多路复用是指内核一旦发现进程指定的一个或者多个IO条件准备读取，它就通知该进程。IO多路复用适用如下场合：

- 当客户处理多个描述字时（一般是交互式输入和网络套接口），必须使用I/O复用。
- 当一个客户同时处理多个套接口时，而这种情况是可能的，但很少出现。
- 如果一个TCP服务器既要处理监听套接口，又要处理已连接套接口，一般也要用到I/O复用。
- 如果一个服务器即要处理TCP，又要处理UDP，一般要使用I/O复用。
- 如果一个服务器要处理多个服务或多个协议，一般要使用I/O复用。
- 与多进程和多线程技术相比，I/O多路复用技术的最大优势是系统开销小，系统不必创建进程/线程，也不必维护这些进程/线程，从而大大减小了系统的开销。

聊聊：硬链接和软链接有什么区别？

- 硬链接就是在目录下创建一个条目，记录着文件名与 `inode` 编号，这个 `inode` 就是源文件的 `inode`。删除任意一个条目，文件还是存在，只要引用数量不为 0。但是硬链接有限制，它不能跨越文件系统，也不能对目录进行链接。
- 符号链接文件保存着源文件所在的绝对路径，在读取时会定位到源文件上，可以理解为 `windows` 的快捷方式。当源文件被删除了，链接文件就打不开了。因为记录的是路径，所以可以为目录建立符号链接。

聊聊：大小端模式

大端模式（Big-Endian）：指的是数据的低位保存在内存的高地址中，而数据的高位保存在内存的低地址中。

小端模式（Little-Endian）：指的是数据的低位保存在内存的低地址中，而数据的高位保存在内存的高地址中

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：

<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



The poster is for the 'Nin Java Hardcore Architecture Class' (尼恩java 硬核架构班). It features a dark red background with gold text and decorative elements like a tiger and circular motifs. The main title is '尼恩java 硬核架构班'. Below it, the pricing is listed as '定价19999 / 早鸟 3999' and '即将涨价 4999'. A status bar indicates '已经发布' (Already Released). The course content is listed in a series of bullet points, each preceded by a gold star icon. The topics include high-performance RPC, distributed systems, Redis, RocketMQ, and Netty, with specific focus on practical implementation and high-concurrency scenarios.

尼恩java 硬核架构班

定价19999 / 早鸟 3999

即将涨价 4999

已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- ★ 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023春招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）——架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》

规划中

《彻底穿透：skywalking 源码（代表链路跟踪）+ Java agent + bytebuddy 探针》

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

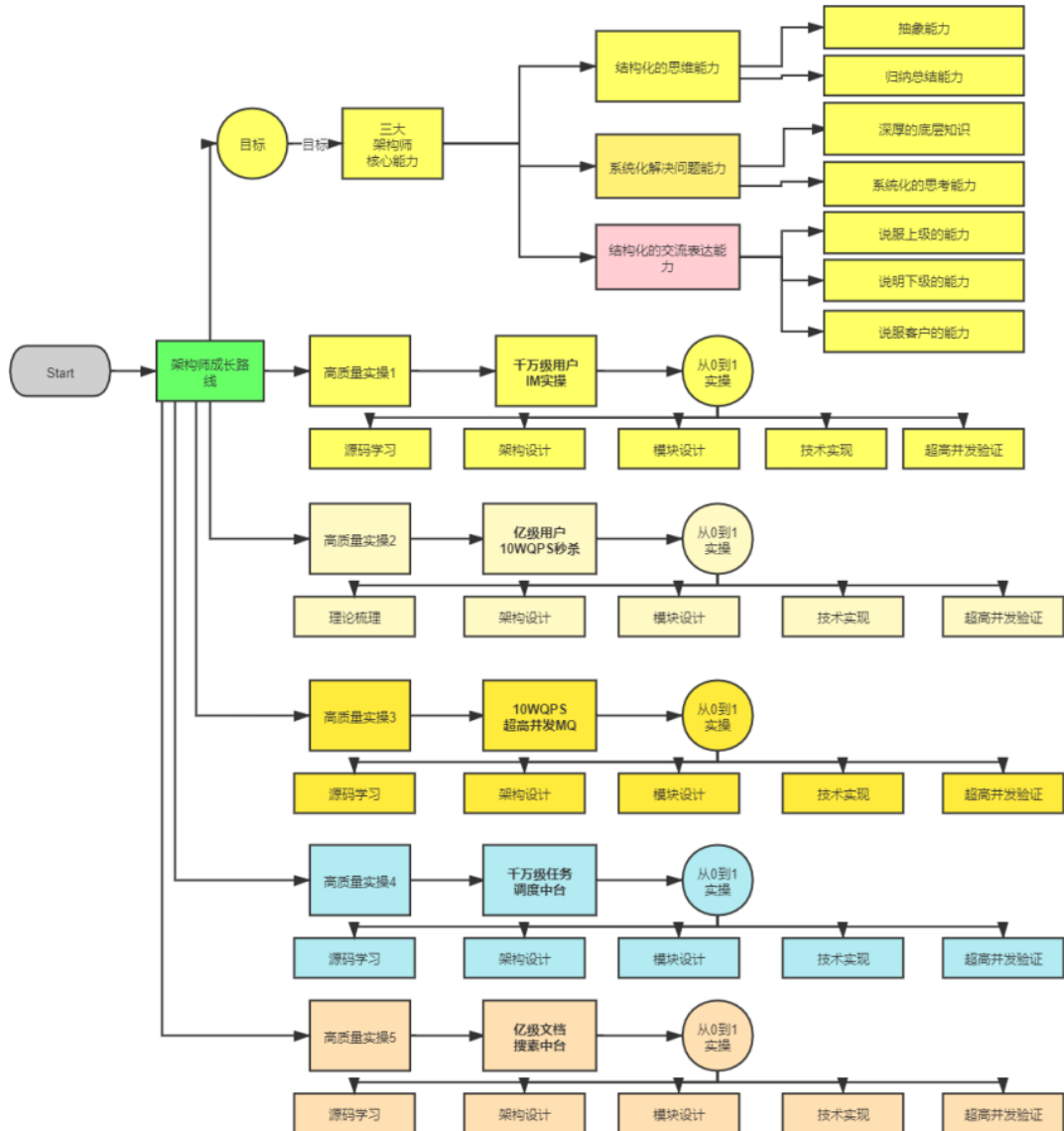
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷3月成高手，狠卷3年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

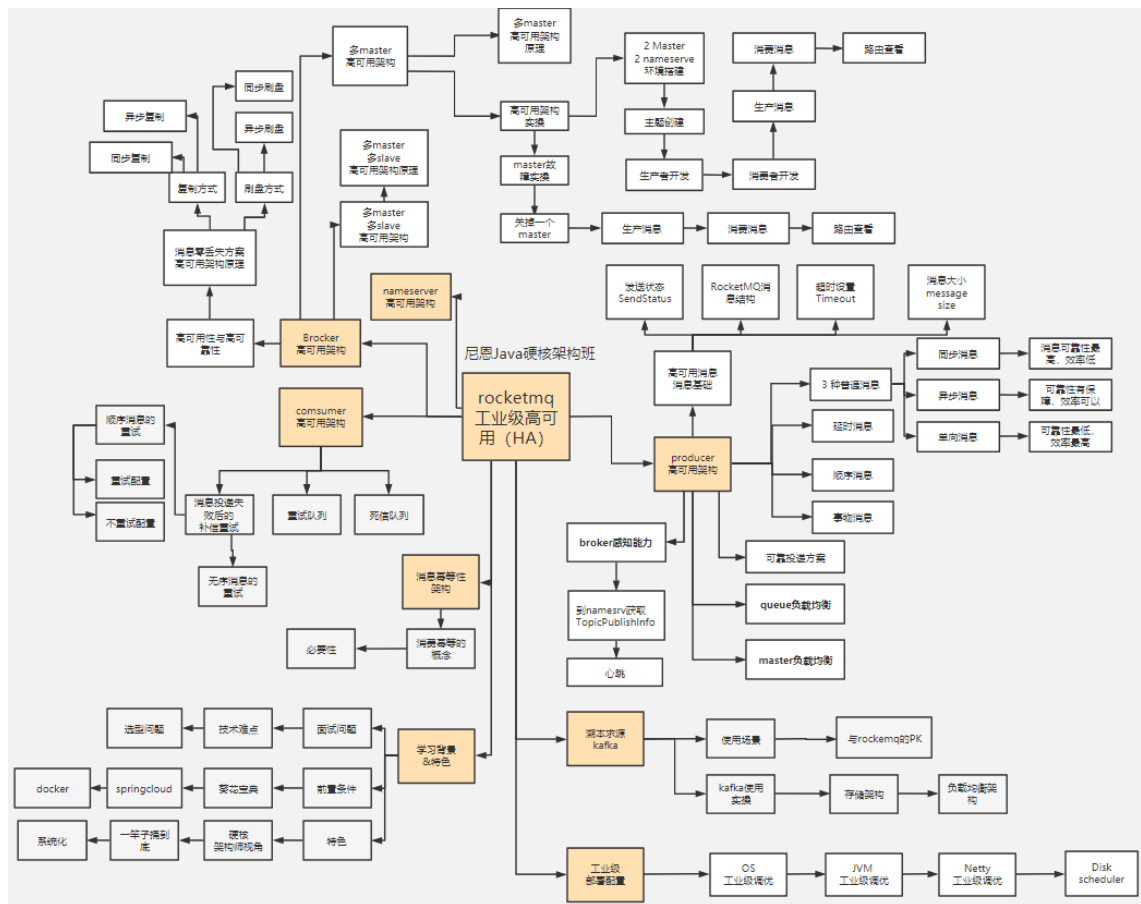
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

[最新](#) [最后发表](#) [热门](#) [精华](#)

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例：[7号卷王] 热烈祝贺小伙伴涨薪120%

① 卷王1号 超级版主 2022-8-13

成功案例：[134号卷王] 大三小伙卷1年，斩获顶级央企Offer，成功逆袭

① 卷王1号 超级版主 2022-7-6

成功案例：[1008号卷王] 5年经验卷王收42W offer，月涨8000，可喜可贺

① 卷王1号 超级版主 2022-5-30

成功案例：[453号卷王] 非全日制 6年卷王喜提3 offer，年薪30W，可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例：[924号卷王] 6年卷王喜提4 offer，最高涨薪9000，可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例：[15号卷王] 4年卷王入职 微软，涨薪50%，可喜可贺

① 卷王1号 超级版主 2022-5-12

成功案例：[527号卷王] 4年卷王喜提2 offer，涨薪50%，可喜可贺

① 卷王1号 超级版主 2022-5-13

成功案例：[788号卷王] 3年卷王喜提优质Offer，涨薪60%

① 卷王1号 超级版主 2022-5-11

成功案例：热烈祝贺：非全日制卷王，喜提2个心仪offer，面3家过2家

① 卷王1号 超级版主 2022-4-21

成功案例：[693号卷王] 二线城市6年卷王喜提4大优质Offer，含央企offer，最高薪酬35W

① 卷王1号 超级版主 2022-4-16

成功案例：[85号卷王] 双非2本小伙，春招大捷，喜提9个offer，最高薪酬近30万

① 卷王1号 超级版主 2022-4-14

成功案例：[741号卷王] 卷王逆袭！6年小伙从很少面试机会到搞定35K*14薪Offer

① 卷王1号 超级版主 2022-4-12

成功案例：[642号卷王] 热烈祝贺，6年卷王喜提优质国企offer

① 卷王1号 超级版主 2022-4-7

成功案例：[796号卷王] 热烈祝贺，36岁卷王喜提52万优质offer

① 卷王1号 超级版主 2022-3-25

成功案列: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

1 卷王1号 超级版主 2022-3-24

成功案列: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

1 卷王1号 超级版主 2022-3-21

成功案列: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

1 卷王1号 超级版主 2022-3-20

成功案列: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

1 卷王1号 超级版主 2022-3-16

简历案列: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

1 卷王1号 超级版主 2022-3-10

成功案列: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

1 卷王1号 超级版主 2022-3-10

成功案列: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

1 卷王1号 超级版主 2022-3-10

成功案列: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

尼恩 超级版主 2022-3-5

成功案列: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

1 卷王1号 超级版主 2022-2-27

成功案列: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

1 卷王1号 超级版主 2022-2-27

成功案列: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

1 卷王1号 超级版主 2022-2-27

成功案列: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

1 卷王1号 超级版主 2022-2-27

成功案列: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

1 卷王1号 超级版主 2022-2-27

成功案列: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂 Offer

1 卷王1号 超级版主 2022-2-27

成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

1 卷王1号 超级版主 2022-2-27

成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

1 卷王1号 超级版主 2022-2-27

成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

1 卷王1号 超级版主 2022-2-27

成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

1 卷王1号 超级版主 2022-2-10

成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

1 卷王1号 超级版主 2022-2-10

成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

1 卷王1号 超级版主 2022-2-10

成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

1 卷王1号 超级版主 2022-2-10

成功案例: [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

1 卷王1号 超级版主 2022-2-10

成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

1 卷王1号 超级版主 2022-2-27

成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

1 卷王1号 超级版主 2022-2-27

成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

1 卷王1号 超级版主 2022-2-25

成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

1 卷王1号 超级版主 2022-2-25

成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

1 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+狠卷3高

以此为样
大家狠卷卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+狠卷卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+狠卷卷

卷王逆袭成功案例

4年卷王入收2个offer，涨50%

3月23日改简历

3月23日 周日 下午 14:50 (周四)

VPS27

DEVELOPER

你是来面试的简历你看了吗，有什么建议？

OK

咱们开始吧

工作经历 这里不需要再描述

下面请你说说 写简历需要注意什么

这个不用怕

都是一样，写错了反悔没有效果

**张磊法宝：
改简历 + 狠狠卷**

5月12日晒offer

5月12日 周日 下午 14:50 (周四)

德盛源 VPS27

offer 决策圈
(主要是 n+1 电商 esp、部门成熟、人数多、加班少、年终奖 n+2、股权激励和内部关系、新部门、人少、晋升与多)

又收到两个 offer 和邀请

哦~

能确定是两个 offer，不通知

现在很多小伙伴面试机会都没有

14:52

多了多少钱

准备在商量

深圳

14:53

有 50% 的样子

已经工作几年啦，大概工作几年啦

[illegible]

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

在一个项目中用过

面试的大局观了

和它改了一下

对比一下，感觉如何？

嗯，感觉还行，挺顺的

那就好，继续加油

嗯，谢谢

你改了一个项目，你看看，有没有问题

学到了

4月21日晒offer

面试了几个offer呀

面试了三家，都是自己比较喜欢的，然后收到两个offer

OK

继续加油

一定一定，继续加油

面试三家，收到2个offer，已经录用

恭喜呀，恭喜你

面试了包发布的面试题库，对面试有帮助

是的，如果面试一个知识体系去考，会更容易

是的，知识体系的知识体系也很重要

面试法宝：
改简历 + 面试题

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

简历改了20225 docx

辛苦了！

好的，谢谢

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

9月13日晒offer

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

秘诀：
改简历 + 狠狠卷

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

你的简历，大概多少呢？

大概100吧

OK，感觉应该好

你的情况，我大概了解了

接下来咱们开始面试

面试的话，感觉怎么样？

感觉还行，挺顺的

那就好，继续加油

嗯，谢谢

你改了一个项目，你看看，有没有问题

学到了

5月11日晒offer

面试了几个offer呀

面试了三家，都是自己比较喜欢的，然后收到两个offer

OK

继续加油

一定一定，继续加油

面试三家，收到2个offer，已经录用

恭喜呀，恭喜你

面试了包发布的面试题库，对面试有帮助

是的，如果面试一个知识体系去考，会更容易

是的，知识体系的知识体系也很重要

涨薪法宝：
改简历 + 狠狠卷

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

简历改了20225 docx

辛苦了！

好的，谢谢

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

4月13日晒offer

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试官说20225 docx

辛苦了！

面试法宝：
改简历 + IM实操

9大offer 最高年薪30万

序号	公司	部门	职位	薪资结构	福利
1	美团	美团	美团	美团	美团
2	美团	美团	美团	美团	美团
3	美团	美团	美团	美团	美团
4	美团	美团	美团	美团	美团
5	美团	美团	美团	美团	美团
6	美团	美团	美团	美团	美团
7	美团	美团	美团	美团	美团
8	美团	美团	美团	美团	美团
9	美团	美团	美团	美团	美团

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>